

THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : MSTII - Mathématiques, Sciences et technologies de l'information, Informatique

Spécialité : Mathématiques et Informatique

Unité de recherche : Laboratoire Jean Kuntzmann

Du son à l'action: apprentissage profond pour la localisation et la navigation robotique basées sur l'audio

From Sound to Action: Deep Learning for Audio-Based Localization and Navigation in Robotics

Présentée par :

Gaétan Lepage

Direction de thèse :

Xavier ALAMEDA-PINEDA

DIRECTEUR DE RECHERCHE, Centre Inria de l'Université Grenoble Alpes

Laurent GIRIN

PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA

Chris REINKE

DOCTEUR EN SCIENCES, Commissariat à l'Énergie Atomique, Grenoble

Directeur de thèse

Co-directeur de thèse

Co-encadrant de thèse

Rapporteurs :

Patrick DANÈS

PROFESSEUR DES UNIVERSITES, Université Paul Sabatier, Toulouse

Romain SERIZEL

MAITRE DE CONFERENCES, HDR, Université de Lorraine

Thèse soutenue publiquement le **15 Juillet 2025**, devant le jury composé de :

Didier SCHWAB

PROFESSEUR DES UNIVERSITES, Université Grenoble-Alpes

Elisa RICCI

PROFESSEURE DES UNIVERSITES, Université de Trento

Patrick DANÈS

PROFESSEUR DES UNIVERSITES, Université Paul Sabatier, Toulouse

Romain SERIZEL

MAITRE DE CONFERENCES, HDR, Université de Lorraine

Xavier ALAMEDA-PINEDA

DIRECTEUR DE RECHERCHE, Centre Inria de l'Université Grenoble Alpes

Laurent GIRIN

PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA

Président

Examinatrice

Rapporteur

Rapporteur

Directeur de thèse

Co-directeur de thèse

Invités :

Chris REINKE

DOCTEUR EN SCIENCES, Commissariat à l'Énergie Atomique, Grenoble

Co-encadrant de thèse

Abstract

Social robotics is a diverse, multidisciplinary research field aiming to design capable robotic agents. In particular, it addresses the challenges of human-robot interaction, in which robots operate not in isolated, controlled environments but alongside humans in performing dynamic and complex tasks. This domain poses numerous challenges, including multi-modal perception and action planning. This thesis explores a selection of essential tasks involved in building effective social robots, with a particular focus on their acoustic dimension. Specifically, we investigate Sound Source Localization (SSL) in both static and active contexts, as well as perceptually motivated robot navigation.

To support this investigation, we first introduce a flexible and powerful acoustic simulation environment. It enables the virtual recording of signals in reverberant rooms, with the ability to model various microphone arrays, room geometries, and sound source configurations. Built upon state-of-the-art sound rendering libraries, this environment serves as a feature-rich sandbox for training and evaluating novel auditory algorithms. Since deep learning has enabled major advances in robotics but requires large volumes of data, often costly or impractical to collect with physical systems, simulation plays a key role in enabling scalable data generation.

Within this simulated framework, we focus on auditory perception as a foundational capability. Among its various facets, SSL is especially critical for social robots. SSL involves identifying the location of one or more active speakers and is grounded in a long-standing body of research. We examine this problem from multiple perspectives and introduce a series of deep-learning-based methods to address its main challenges. After presenting a solution for the single-source case, we propose a more advanced multi-source localizer. Both models are thoroughly evaluated through extensive experiments conducted under varied conditions.

However, in real-world social settings, robots rarely remain stationary. Their movement introduces new dimensions to the localization problem. To address this, we extend our SSL approach to dynamic scenarios, where motion is explicitly considered. We propose a novel method for aggregating predictions from our static multi-source localizer over time. This framework leverages the robot’s motion to refine its estimates of speaker positions. To support this, we extend our simulator to synthesize complete trajectories in virtual environments, enabling effective training and evaluation of this dynamic localization model.

Having addressed perception, we then turn to action, the second pillar of robotic capability. We explore how modern Deep Reinforcement Learning (DRL) techniques can be applied to robot navigation, specifically to enhance auditory perception through movement. Recent Automatic Speech Recognition (ASR) models enable robots to transcribe human speech with high accuracy, but their performance can degrade significantly in reverberant environments. To mitigate this, we introduce a perceptually motivated navigation task in which a robot learns to position itself to minimize speech recognition errors. The agent’s decisions are based solely on audio captured by its microphone array. This final contribution builds directly upon our

previous work in acoustic simulation and SSL, integrating perception and action to advance embodied auditory intelligence.

Keywords: deep learning, robotic auditory perception, sound source localization, acoustic simulation, robot navigation, deep reinforcement learning,

Résumé

La robotique sociale est un domaine de recherche vaste et multidisciplinaire, visant à concevoir des agents robotiques capables d’interagir efficacement. Elle traite notamment des défis posés par les interactions homme-robot, où les robots ne sont plus confinés à des environnements isolés, mais évoluent aux côtés des humains dans des contextes dynamiques et complexes. Ce domaine soulève de nombreux enjeux, notamment en perception multimodale et planification d’actions. Cette thèse explore un ensemble de tâches essentielles à la conception de robots sociaux efficaces, avec un accent particulier sur leur dimension acoustique. Plus précisément, nous étudions la localisation de sources sonores dans des contextes statiques et dynamiques, ainsi qu’une tâche de navigation robotique guidée par la perception auditive.

Pour cela, nous introduisons un environnement de simulation acoustique flexible et performant. Il permet d’enregistrer virtuellement de signaux dans des pièces réverbérantes, en modélisant différentes configurations de microphones, de géométries de pièces et de sources sonores. Basé sur des bibliothèques de synthèse sonore de pointe, cet environnement constitue un espace d’expérimentation riche pour l’apprentissage et l’évaluation de nouveaux algorithmes auditifs. L’apprentissage profond a permis des avancées majeures en robotique, mais ces méthodes nécessitent de grandes quantités de données, souvent coûteuses ou difficiles à collecter dans des environnements physiques. La simulation joue ainsi un rôle clé dans la génération de données à grande échelle.

Dans ce cadre virtuel, nous nous concentrons sur la perception auditive, considérée comme une capacité fondamentale. Parmi ses composantes, la localisation de sources sonores est essentielle pour les robots sociaux. Elle consiste à estimer la position d’un ou plusieurs locuteurs actifs, et repose sur une littérature scientifique riche et mature. Nous abordons ce problème sous plusieurs angles et proposons une série de méthodes fondées sur l’apprentissage profond pour en relever les principaux défis. Après une solution pour le cas à source unique, nous proposons un localisateur multi-sources plus avancé. Les deux modèles sont rigoureusement évalués au moyen d’expériences approfondies dans des conditions variées.

Dans des contextes sociaux réels, les robots sont rarement immobiles. Leur mouvement introduit de nouvelles particularités dans le problème de localisation. Pour y répondre, nous étendons notre approche à des scénarios dynamiques, en tenant explicitement compte du déplacement du robot. Nous proposons une méthode d’agrégation temporelle des prédictions issues du localisateur multi-sources. Ce cadre exploite les déplacements du robot pour affiner les estimations de position des locuteurs. Le simulateur a été étendu pour générer des trajectoires complètes dans des environnements virtuels, facilitant un apprentissage et une évaluation efficaces de nos algorithmes dans ce contexte dynamique.

Enfin, nous abordons l’action, second pilier des capacités robotiques. Nous appliquons des techniques d’apprentissage par renforcement profond à la navigation, afin d’améliorer la perception auditive par le mouvement. Les modèles récents de reconnaissance automatique de la parole permettent aux robots de transcrire la parole humaine avec précision, mais

leurs performances peuvent se dégrader en présence de réverbération. Pour y remédier, nous introduisons une tâche de navigation guidée par la perception auditive, dans laquelle le robot apprend à se positionner pour minimiser les erreurs de reconnaissance. Les décisions de l'agent reposent uniquement sur les signaux audio captés par sa matrice de microphones. Cette dernière contribution s'appuie sur nos travaux précédents en simulation acoustique et en localisation, intégrant perception et action pour faire progresser l'intelligence auditive embarquée.

Mots-clés: apprentissage profond, perception auditive en robotique, localisation de sources sonores, simulation acoustique, navigation robotique, apprentissage par renforcement profond.

Acknowledgements

Pursuing a PhD is choosing to embark on an adventure filled with uncertainties and unexpected turns. The investment is considerable, but the journey and its rewards are just as significant. Despite its solitary nature, one should not underestimate the people who support a candidate along the way. I was fortunate to be surrounded by remarkable individuals who were instrumental throughout this journey.

I want to start by deeply thanking my three supervisors, Xavier Alameda-Pineda, Laurent Girin and Chris Reinke. They trusted me from the very beginning by selecting me among many competitive applicants, and they accompanied me with unfaltering support until the very end. My PhD certainly did not spare us scientific difficulties or setbacks, yet they always managed to keep the hope and vision alive when I sometimes felt like losing confidence. Their complementary expertise, patience, and kindness made every discussion immensely valuable. The hours we spent understanding problems, clarifying ideas, and exploring new directions helped me grow both scientifically and personally. Xavi, Laurent, and Chris will remain models of dedication to me.

Besides my great supervisors, I had the honor of working with inspiring collaborators. The RobotLearn team brings together young, talented, and driven individuals from diverse cultures, backgrounds, and experiences to tackle challenging research projects. It was a blast to exchange ideas on specialized and novel subjects with such curious and brilliant minds. Experiencing the rapid progress of AI together over these five years was deeply stimulating. Our reading group sessions offered great opportunities to explore cutting-edge research. Everyone was encouraged to ask questions and share their views, creating a supportive and enriching atmosphere. I was grateful to share my successes, failures, and doubts with my teammates, who were always keen to offer their support. Some of my fondest memories come from our daily meals at the “*Champ de l’Étoile*” enjoying Patrick’s desserts, our coffee breaks, and many excursions.

Xiaoyu Lin started her PhD at the same time as I did. I have rarely known someone as exceptionally hard-working and dedicated. I have the utmost respect for her discretion, kindness, and maturity. She will undoubtedly have a successful and inspiring career that I will follow closely. Andres, Louis, Alex, Anand, Kiru, Victor, and Lorenzo quickly became close friends, consistently cheering me up and making every shared moment enjoyable. Radu, Soraya and Nicolas represent the roots and enduring spirit of the team, and hearing from their experience in research was always a pleasure. I would also like to warmly thank the many other members of RobotLearn I met during these years, including Samir, Ahamed, Stéphane, Matthieu, Tim, Luis, Guillaume, Yihong, Xiaoyu Bie, Wen, Maxime, David, Ghazi, Zhiqi, Daniel, Natanaël, Alexandre and Saghar. I must not forget Nathalie, our team assistant, without whom this bunch of nerds would definitely not get very far!

Now, let me briefly go back in time. My academic path did not begin with this PhD, and the reason I was able to join such a talented team lies in the excellent professors I met and the

diverse environments I experienced. My years in the *classes préparatoires* of *La Martinière Monplaisir* gave me solid scientific foundations. There, I discovered the beauty of mathematics, physics, and engineering thanks to exceptionally dedicated professors such as Viviane Gaggioli, Laurence Lechuga, Cédric Gamelon, Xavier Pessoles, and Delphine Sembély, whose high expectations taught me rigor and hard work. My time at Ensimag then opened my eyes to the world of computer science, mathematics, and artificial intelligence, surrounded by like-minded students eager to explore intricate topics. Looking back, I am deeply grateful for the ten fulfilling years spent in such inspiring academic environments, culminating in this PhD.

I am grateful for the fantastic friends with whom I shared so many warm and carefree moments. Our dinners, long discussions, hikes, and board games were a wonderful escape from the daily routine. Lucas, Isabelle, Benjamin, and Thomas Girerd in Lyon, as well as Olivier Lim, Thomas Bauvent and Michael from Grenoble — thank you for all these moments. Last but not least, my fellow flatmates Antoine, Olivier Brun, and Yangtao shared the PhD journey with me day to day. Living with these cheerful guys and sharing our victories and doubts made this period lighter and more enjoyable.

I have practiced rock climbing for many years, and Grenoble offered an exceptional environment to progress. Beyond its physical and mental benefits, climbing was essential to my well-being, helping me disconnect from research and surround myself with wonderful people. The Grenoble climbing community is full of talented and welcoming climbers, and I met numerous partners over the past five years. Rémy, Marjorie, Simon, Basile, Nicolas, Lionel, Thomas, Hugo, Gabin, Noé, Lilou, Coraline, Merlin, Manu, and all the other *Requins du CSU* were incredible people to train and travel with. Our trips to Chamonix, Fontainebleau, Céüse, and especially Ailefroide remain unforgettable. The more experienced *adeptes de la force droite*, including Nico, Théo, Roman, and Mathieu, were strong, dedicated athletes I continue to look up to. I also owe a lot to my coach, Adrien Pirolo, whose support and trust pushed me far beyond what I thought possible. His guidance helped me grow steadily and confidently. Let's say he was my fourth supervisor.

My parents, Anne-Catherine and Francis, as well as my brother Tanguy, have provided me with unconditional and loving support. They were always by my side during the difficult periods of this adventure, reminding me that life is not only about academic success and complex equations. I was exceptionally lucky to have them around, and I truly realized how crucial a loving family is. They deserve a large share of this success, as I would probably have given up without them.

I would like to express my sincere gratitude to the jury members, Prof. Didier Schwab, Prof. Elisa Ricci, Prof. Patrick Danès, and Dr. Romain Serizel, who agreed to review my work and participate in my defense. Their insightful and challenging questions made the discussion both stimulating and enriching. Patrick and Didier went through a careful and thorough review of my manuscript and provided thoughtful and precise feedback.

Last but not least, I want to thank all the people who attended my defense. I was delighted to be surrounded by so many friendly faces on such an important day. Sharing this success with all of them was the best celebration I could have hoped for.

Contents

1	Introduction	1
1.1	Motivation	1
1.1.1	Social Robotics & Embodied AI	1
1.1.2	Acoustic Perception in Robotics	2
1.1.3	Learning Robot Policies with Reinforcement Learning	2
1.1.4	Acoustic Localization and Navigation to Enhance Perception	3
1.2	Outline and Contributions	4
1.2.1	A Modular Simulation Platform for Robotic Acoustic Learning	4
1.2.2	Learning-Based Sound Source Localization in Static Settings	4
1.2.3	Active Sound Source Localization through Temporal Aggregation	5
1.2.4	Perceptually Guided Navigation via Deep Reinforcement Learning	5
1.2.5	Scope, Methodology, and Implementation Context	6
1.3	Thesis Structure	6
1.4	Note on Writing Tools	7
2	Acoustic Robot Simulator	9
2.1	Introduction	9
2.2	Acoustic Simulation of Reverberant Environments	13
2.2.1	Background	13
2.2.2	Overview of Acoustic Reverberation Simulation Methods	23
2.2.3	RIR Simulation Libraries	27
2.3	Custom Acoustic Simulation Pipeline	29
2.3.1	Overview	29
2.3.2	Components	31
2.3.3	Simulation of Dynamic Scenarios	38
2.3.4	Benchmarking and Performance Analysis	39
2.3.5	Conclusion	42
2.4	Conclusion	42
3	Sound Source Localization	44
3.1	Background	44
3.1.1	Original Problem	44
3.1.2	Variations in the Sound Source Localization Task	46
3.1.3	Classical Approaches	48
3.1.4	Deep Learning Methods for Sound Source Localization (SSL)	49
3.1.5	Sound Source Localization in Robotics	53
3.1.6	Conclusion	54
3.2	Single-source localization	55
3.2.1	Problem Statement	55
3.2.2	Method	56

3.2.3 Experiments	63
3.2.4 Conclusion	69
3.3 Multi-Source Localization	70
3.3.1 Method	70
3.3.2 Experiments	75
3.3.3 Discussion	89
3.4 Conclusion	90
4 Active Sound Source Localization	92
4.1 Background	93
4.2 Problem Formulation	96
4.3 Method	97
4.3.1 Pipeline Overview	97
4.3.2 Egocentric SSL Maps	98
4.3.3 Aggregation Strategies	99
4.3.4 Clustering for Detection Extraction	103
4.4 Experiments	104
4.4.1 Metrics	104
4.4.2 Dataset Collection	105
4.4.3 Performance Study	106
4.5 Conclusion and Discussion	115
5 Deep Reinforcement Learning for Sound-Driven Navigation	117
5.1 Introduction to Reinforcement Learning	117
5.1.1 History	117
5.1.2 Core Notions	119
5.1.3 Deep Reinforcement Learning	121
5.1.4 Reinforcement Learning for Robotics	122
5.1.5 Policy Gradient Algorithms	124
5.2 Sound-Driven Robot Navigation: Problem Statement	130
5.2.1 Motivation	130
5.2.2 Background	131
5.2.3 Problem Formulation	133
5.3 Proposed Approach	138
5.3.1 Acoustic Pipeline	138
5.3.2 WER as a Reward Signal	138
5.3.3 Deep Neural Agent	143
5.3.4 PPO Implementation and Training Strategy	143
5.4 Experiments	146
5.4.1 ASR Performance in a Reverberant Room	146
5.4.2 Reward Design	147
5.4.3 Agent Performance on the Navigation Task	150
5.4.4 Importance of Localization Feature Extraction	152
5.4.5 Alternative Cost Maps	154

5.5 Conclusion	156
6 Conclusion	158
6.1 Summary of Contributions	158
6.2 Challenges and Limitations	159
6.3 Prospective Research Directions	160
Bibliography	162

List of Figures

Figure 1.1	The ARI humanoid robot.	2
Figure 1.2	Overview of the thesis organization.	6
Figure 2.1	Schematic representation of an RIR.	15
Figure 2.2	Illustration of the different ways sound interacts with surfaces.	15
Figure 2.3	Plots of an impulse response and time-energy response recorded in a room. .	16
Figure 2.4	Power spectrogram of a two-second-long clean speech recording.	19
Figure 2.5	Two-microphone configuration in the near-field and far-field cases.	21
Figure 2.6	Illustration of time-frequency representations of a synthesized speech signal.	23
Figure 2.7	Representation of the first-order virtual image sources.	25
Figure 2.8	Overview of the simulator architecture.	29
Figure 2.9	Schematic view of the audio processing pipeline.	31
Figure 2.10	Examples of microphone arrays available in the simulator.	34
Figure 2.11	Typical simulator execution workflow.	36
Figure 2.12	Screenshot of the simulator renderer.	38
Figure 2.13	Illustration of the pseudo-continuous simulation process.	39
Figure 2.14	Profiling results for both available RIR simulation back-ends.	40
Figure 3.1	Schema of the SSL task.	55
Figure 3.2	Statistics of ground-truth label pairs (θ, D) in the generated dataset.	57
Figure 3.3	Illustration of the most common microphone polar patterns.	58
Figure 3.4	Deep convolutional network architecture for SSL.	61
Figure 3.5	Plot of the angular loss.	62
Figure 3.6	Evolution of the training and validation DoA loss $\mathcal{L}_{\text{DoA}}$ for different input features.	67
Figure 3.7	Evolution of the mean absolute error on the distance estimation MAE_D during training.	69
Figure 3.8	Reverberation impact on SSL performance.	69
Figure 3.9	DoA spectrum encoding of three sources.	74
Figure 3.10	Deep neural network architecture for multi-source SSL.	75
Figure 3.11	Illustrative example of a poor network prediction and the corresponding ground truth spectrum.	78
Figure 3.12	Evolution of the norm of the network output and loss during training.	78
Figure 3.13	A visual comparison of Batch and Layer Normalization.	82
Figure 3.14	Training and validation accuracies during training for different normalization schemes.	83
Figure 3.15	Example of a sequence processing result.	85
Figure 3.16	Distribution of $\Delta\theta_{\min}$ for different numbers of active sources.	88
Figure 3.17	Distribution of $\Delta\theta_{\min}$ for different values of τ_{DoA}	89
Figure 4.1	Active-SSL pipeline.	98
Figure 4.2	Example of a DoA map.	99

Figure 4.3	Shifting process for local DoA maps.	99
Figure 4.4	Ground truth encoding $\mathcal{M}_t^*$ of the localization map.	102
Figure 4.5	Deep neural network architecture for 2D localization map aggregation.	102
Figure 4.6	Agent position initialization for the dataset collection	105
Figure 4.7	Effect of the clipping threshold on the aggregated maps.	107
Figure 4.8	Number of points remaining after the filtering operation with respect to $\delta_{\min}$	109
Figure 4.9	Comparison of ground-truth and predicted DoA spectra	109
Figure 4.10	Visual comparison of the two aggregation methods.	110
Figure 4.11	Illustration of the DoA spectrum transformation.	111
Figure 4.12	Effect of DoA spectrum amplification on the local maps.	112
Figure 5.1	Reinforcement Learning framework.	119
Figure 5.2	Neural network architecture for the Deep Reinforcement Learning (DRL) agent.	143
Figure 5.3	Distribution of the 12 possible source positions.	144
Figure 5.4	WER cost maps for different reverberation levels.	146
Figure 5.5	Example of a directional WER cost map.	147
Figure 5.6	Plot of the main reward component as a function of the cost C_t	147
Figure 5.7	Evolution of the training metrics demonstrating the policy collapsing phenomenon.	150
Figure 5.8	Examples of agent trajectories for two distinct source positions.	150
Figure 5.9	Example of a directional analytical cost map.	154

List of Tables

Table 2.1	Relative performance of the simulator back-end RIR libraries.	40
Table 3.1	SSL performance for different microphone arrays.	65
Table 3.2	Reverberation impact on SSL performance.	66
Table 3.3	SSL performance depending on the input features.	66
Table 3.4	SSL performance when estimating the source-microphone distance.	68
Table 3.5	Reverberation impact on SSL performance.	69
Table 3.6	SSL performance when trained with different number of sources.	80
Table 3.7	SSL performance depending on the number of active sources.	80
Table 3.8	Performance of the SSL model trained with the ε -loss.	81
Table 3.9	Final performance of SSL for different normalization/evaluation schemes.	84
Table 3.10	SSL performance depending on the input duration.	86
Table 3.11	SSL performance for different values of τ_{DoA}	88
Table 4.1	Comparison of the ASSL performance for different clipping threshold ($\delta_{\min}$) values.	107
Table 4.2	Final performance of both aggregation methods.	110
Table 4.3	Comparison of the ASSL performance for different DoA threshold (τ_o) values.	112
Table 4.4	Evolution of localization performance for different horizon values.	114
Table 4.5	Comparison of the ASSL performance for different FoV values.	114
Table 4.6	Comparison of the ASSL performance for different pixel resolutions.	115
Table 5.1	Comparison of three ASR models provided by <i>SpeechBrain</i>	141
Table 5.2	Final hyperparameter values used to train the PPO algorithm.	144
Table 5.3	Benchmark of various policies' performance on the navigation task.	152
Table 5.4	Comparison of the agent's performance for different feature extractor training strategies.	153
Table 5.5	Navigation performance when using different cost functions.	156

List of Acronyms

AI:	Artificial Intelligence
API:	Application Programming Interface
AR:	Augmented Reality
EM:	expectation-maximization
GPU:	Graphics Processing Unit
GT:	Ground Truth
HMM:	Hidden Markov Model
HRI:	Human-Robot Interaction
LIDAR:	light detection and ranging
LTi:	linear time-invariant
LUT:	Look Up Table
MCTS:	Monte Carlo tree search
MKF:	mixture Kalman filter
NLP:	Natural Language Processing
RMSE:	Root Mean Square Error
SLAM:	Simultaneous Localization And Mapping
BatchNorm:	Batch Normalization
CNN:	Convolutional Neural Network
CRNN:	Convolutional Recurrent Neural Network
DL:	deep learning
DNN:	deep neural network
GRU:	Gated Recurrent Unit
LLM:	Large Language Model
LayerNorm:	Layer Normalization
LSTM:	Long-Short Term Memory
RNNLM:	Recurrent Neural Network Language Model
MSE:	Mean Squared Error
MLP:	Multi-Layer Perceptron
ReLU:	Rectified Linear Unit
RNN:	Recurrent Neural Network
SGD:	stochastic gradient descent
ASR:	Automatic Speech Recognition

LM:	Language Model
CTC:	Connectionist Temporal Classification
WER:	Word Error Rate
DCASE:	Detection and Classification of Acoustic Scenes and Events
DoA:	Direction of Arrival
SSL:	Sound Source Localization
SELD:	Sound Event Localization and Detection
SSLR:	Sound Source Localization for Robots
SRP-PHAT:	Steered Response Power with Phase Transform
ACC:	Accuracy
MAE:	Mean Absolute Error
MUSIC:	Multiple emitter location and signal parameter estimation
ESPRIT:	Estimation of Signal Parameters via Rotational Invariance Technique
GMM:	Gaussian Mixture Model
VAD:	Voice Activity Detector
ASSL:	Active Sound Source Localization
FoV:	Field of View
ATF:	Acoustic Transfer Function
BEM:	Boundary Element Method
BINAHR:	BINaural Active Audition for Humanoid Robots
BST:	Bidirectional Sound Transport
DFT:	Discrete Fourier Transform
FDTD:	Finite-Difference Time-Domain
FEM:	Finite-Element Method
FFT:	Fast Fourier Transform
FIR:	finite impulse response
FOA:	first-order Ambisonics
GA:	Geometrical Acoustics
GCC:	Generalized Cross-Correlation
GCC-PHAT:	Generalized Cross-Correlation with Phase Transform
HRTF:	head-related transfer function
ILD:	Interaural Level Difference
IPD:	Interaural Phase Difference
ITD:	Interaural Time Difference
ISM:	Image Source Method

MFCC:	Mel-Frequency Cepstral Coefficients
PHAT:	Phase Transform
RIR:	Room Impulse Response
RTF:	Related Transfer Function
SED:	Sound Event Detection
SNR:	Signal to Noise Ratio
STFT:	Short-Term Fourier Transform
TDoA:	time difference of arrival
ULA:	Uniform Linear Array
CASP:	Critical Assessment of protein Structure Prediction
DDPG:	deep deterministic policy gradient
DQN:	Deep Q Learning
DRL:	Deep Reinforcement Learning
GAE:	Generalized Advantage Estimation
KL:	Kullback-Leibler
LQR:	Linear–quadratic regulator
MDP:	Markov Decision Process
MFAE:	mean final angle error
MFD:	mean final distance
MFC:	mean final cost
MPC:	Model Predictive Control
PESQ:	Perceptual Evaluation of Speech Quality
PG:	Policy Gradient
POMDP:	partially observable Markov Decision Process
PID:	Proportional-Integral-Derivative
PPO:	Proximal Policy Optimization
RL:	Reinforcement Learning
RLHF:	Reinforcement Learning from Human Feedback
SAC:	soft actor-critic
TD:	Temporal-difference
TRPO:	Trust Region Policy Optimization

Chapter 1

Introduction

1.1 Motivation

1.1.1 Social Robotics & Embodied AI

Recent years have seen rapid and remarkable progress in Artificial Intelligence (AI). AI systems have advanced dramatically in a relatively short period. Subsequently, these techniques have been used in numerous domains, ranging from highly specialized applications to widely spread end-consumer products. These technologies have had a profound impact on daily life, society, and the global economy. Large Language Models (LLMs) are the latest and most striking illustration of this revolution. Since the invention of the attention mechanism and the transformer architecture in 2017 [1], larger and larger models could be trained on constantly growing datasets. The capabilities of LLMs have pushed the boundaries of artificial reasoning and natural language processing. Similarly, diffusion models have been equally successful for image generation. While interactions with these systems remain largely text-based, academic and industrial actors invest substantial resources in making them more natural. More precisely, AI agents aim to interact with humans directly through speech and vision. For instance, KyutAI's Moshi [2] can engage in real-time two-way spoken interactions with humans. Speech-based interaction is a central research challenge in Human-Robot Interaction (HRI). More generally, the design of embodied agents is at the core of social robotics. Social robots operate in complex, dynamic, and often crowded environments. An example of such an embodied social agent is shown in [Figure 1.1](#). They differ from traditional robots, such as the ones used in industrial settings to perform repetitive, narrow-scope actions. These manufacturing-oriented robots do not need to adapt and interact in an unpredictable environment. Their policy is often deterministic and programmed in advance. Also, they are not expected to interact naturally with humans. Bridging the gap to capable social robots remains a significant challenge. As Hans Moravec observed in the 1980s [3], tasks that are easy for humans—such as perception and motor coordination—are surprisingly difficult for robots. This paradox highlights a central challenge of social robotics: while computers can outperform humans in abstract reasoning tasks, they still struggle with sensorimotor skills that even infants master early in life. His observation, from 1988, remains highly relevant today as multimodal perception and action remain key challenges in AI and robotics.



Figure 1.1 – A photo of the ARI robot, a humanoid robotic platform developed by PAL Robotics.

1.1.2 Acoustic Perception in Robotics

Multimodal perception refers to a robot's ability to sense and understand its environment using multiple types of sensory input. Robots can be equipped with various sensors depending on their objectives, nature, and budget constraints. For example, LIDAR sensors allow for estimating the distance to surrounding objects and are widely used in autonomous vehicles. Cameras and microphones are cost-effective and let robots sense their environment like humans. The goal is to derive actionable knowledge from raw sensory input. For instance, object detection involves localizing and classifying semantic objects in the captured frames. Multi-object tracking adds a temporal aspect to the task by tracking each identified object across time. Depth estimation attempts to infer how far each pixel is in an image. Regarding auditory perception, Automatic Speech Recognition (ASR) aims to enable machines to “understand” human speech by transcribing audio input into a readable format. This task is essential in social robotics as speech is often the primary communication channel between humans and robots. SSL involves estimating the direction or position of the active audio sources in an environment. SSL helps robots be more spatially aware by complementing visual features. Also, it allows them to know which person is currently speaking and where they are located. While not sufficient on its own, SSL enhances a robot's ability to interpret and respond to human interactions and can complement other modalities in navigation tasks.

1.1.3 Learning Robot Policies with Reinforcement Learning

Reinforcement Learning (RL) is a flexible framework for solving multi-step decision problems. RL has a long history of research and has allowed numerous successes in various application domains. Since the rise of deep learning methods, RL has benefited from more effective and capable models and function approximators. DRL methods make it possible to train deep neural networks with high representational capacity. This shift from traditional RL methods to modern neural-based algorithms has significantly broadened the range of solvable problems. Traditional RL was limited to low-dimensional observation spaces. The use of deep neural

networks (DNNs) as function approximators expanded RL’s applicability to high-dimensional, unstructured inputs. DRL methods have led to breakthroughs in gaming environments like Atari 2600 [4] and Go [5], and more recently in complex embodied tasks such as legged locomotion [6] and multi-agent cooperation [7]. Such fundamental progress clearly impacted robotics, which is well suited to RL’s sequential problem formulation. Indeed, relevant robot behaviors are complex and cannot be modeled explicitly, especially in social robotics. This limits the applicability of traditional rule-based approaches, which require explicit behavior modeling. RL proposes an alternative solution to the problem of robot policy design. It relies on a reward function that provides feedback to the agent about the effectiveness of its actions [8]. The agent will then progressively optimize its policy by interacting with the environment to maximize the collected reward. Therefore, it is not required to model the environment explicitly; it is implicitly learned during training. DRL allows learning from high-dimensional sensory inputs, such as camera frames or recorded audio signals. However, these approaches are extremely data-intensive and require considerable interaction with the environment to learn. DRL rapidly achieved impressive results on entirely virtual tasks where interacting with the environment is cost-effective and scalable. Some recent works report agents interacting with environments for the equivalent of years of simulated time—OpenAI’s Five agent trained on over 180 years of experience [9], while common DRL benchmarks like Atari require millions of frames to achieve good performance [4]. By contrast, robots are embodied agents that must interact with the physical world, where each action has a cost. This mismatch gives rise to the well-known sim-to-real gap—the challenge of transferring policies trained in simulation to real-world hardware. Overall, DRL is a powerful framework for robotics, with many successful applications. However, it still faces key limitations, including poor generalization, sample inefficiency, and limited interpretability.

1.1.4 Acoustic Localization and Navigation to Enhance Perception

Perceptually-motivated navigation is a canonical example of a complex multimodal robotics task. It can be framed in various ways, depending on the context and the targeted application. Robots must often perform complex tasks that require gathering information and planning actions with foresight. Moreover, robots rarely have full observability. Sensor limitations and occlusions often obscure parts of the environment. Hence, a robot must willingly explore its environment to collect the necessary information and enhance its overall perception. For example, the *Move2Hear* task [10] requires a mobile robot to navigate in a realistic environment to enhance its ability to separate distinct speech sources. Other objectives could similarly motivate an agent’s exploration strategy. In this thesis, we narrow our focus to auditory perception as the sole sensory modality. This constraint makes the task more challenging—since audio offers far less spatial information than vision—but also more fundamental. It forces the robot to reason about its environment based on subtle acoustic cues, such as reverberation, directionality, and spectral variation. The remainder of this thesis investigates how deep learning can support audio-based spatial perception and action in simulated environments. This line of inquiry not only deepens our understanding of auditory intelligence but also contributes toward building more autonomous, resilient, and efficient robotic systems that can operate under minimal sensory assumptions.

1.2 Outline and Contributions

This thesis explores how deep learning can be leveraged to enhance acoustic perception in robotics, with a focus on sound source localization and navigation tasks driven by auditory input. The contributions are centered around the development of tools and learning-based methods that enable social robots to operate more effectively in complex, reverberant environments using audio alone. The research is articulated across four technical chapters, each addressing a distinct sub-problem: from simulation and signal modeling to supervised perception and reinforcement learning. All contributions are supported by original software implementations and extensive experimentation.

1.2.1 A Modular Simulation Platform for Robotic Acoustic Learning

The first core contribution is the development of a custom acoustic simulation platform ([Chapter 2](#)). We introduce a flexible and feature-rich simulation framework for acoustic robotics, developed as a foundational tool for the research conducted in this thesis. Modern deep learning methods—particularly in SSL and DRL—require large-scale data that is difficult to acquire through physical experimentation. To address this, we designed a modular simulator that generates realistic multi-channel audio recordings in virtual rooms. The simulator supports customizable microphone arrays, varied source types (speech, noise), and dynamic trajectories for both sources and receivers. It integrates two back-end RIR simulation engines: the feature-rich but slower *Pyroomacoustics* [11], and the high-performance GPU-accelerated *gpuRIR* [12] library. Additional layers of abstraction were developed to model robot movement, generate egocentric acoustic features, and simulate realistic interaction scenarios in a step-based pipeline. A novel method was introduced to bootstrap reverberation across short simulation steps, allowing accurate modeling of reverberant tails during dynamic agent movement. This simulator, released as open-source research software, is used throughout the thesis for dataset generation, training supervision, and interactive policy evaluation.

1.2.2 Learning-Based Sound Source Localization in Static Settings

Static sound source localization is the focus of [Chapter 3](#), where we examine how deep learning can address this fundamental challenge in speech-based auditory perception. We begin with a focused literature review of this fundamental signal processing task, covering both classical methods and recent deep-learning-based approaches. The chapter then explores two deep-learning-based localization pipelines developed and tested using synthetic datasets generated by our simulator.

In the single-source setting, a convolutional neural network is trained to estimate the Direction of Arrival (DoA) of a speech source based on interaural features (ILD and IPD). We evaluate the model across several microphone array layouts and signal representations, showing how design choices—such as array geometry, microphone directivity, and feature pre-processing—impact performance. This analysis offers practical insights into building accurate, low-latency SSL models using compact hardware setups.

The multi-source formulation extends the problem to scenarios with an arbitrary number of active speakers. An angular spectrum representation is adopted, framing SSL as a continuous regression task. A dedicated convolutional architecture is implemented and trained on a large synthetic dataset, with performance evaluated using both fixed and variable source counts. Inspired in part by prior work [13], the pipeline was reimplemented, adapted, and integrated into our experimental framework. Additional contributions include a detailed ablation study on source proximity, context length, and alternative loss formulations.

Together, these experiments establish robust and reproducible baselines for SSL in complex environments, forming a foundation for the dynamic and interactive extensions developed in the next chapters.

1.2.3 Active Sound Source Localization through Temporal Aggregation

Chapter 4 addresses the task of Active Sound Source Localization (ASSL), in which a robot accumulates auditory observations over time as it moves through an environment. Building on the multi-source SSL model developed in **Chapter 3**, we propose a method that aggregates directional information from short audio segments recorded at different positions along a trajectory. The key idea is to construct and refine an egocentric 2D likelihood map of source positions by combining SSL predictions from multiple time steps. Two aggregation strategies are explored: a naive averaging approach and a learned blending network based on a U-Net architecture. The final source positions are extracted via a spatial clustering algorithm (DBSCAN), enabling the system to handle an arbitrary number of concurrent speakers. Experiments reveal the critical role of trajectory length (horizon), feature resolution, and the quality of the upstream SSL model in determining localization accuracy. While the method is constrained to static sources, it demonstrates that spatial reasoning can be significantly improved by leveraging motion, even in the absence of ground-truth robot positions or multi-modal sensing.

1.2.4 Perceptually Guided Navigation via Deep Reinforcement Learning

Chapter 5 shifts focus from passive perception to active decision-making, exploring how DRL can be used to optimize robotic behavior in acoustically complex environments. The chapter begins with a comprehensive introduction to reinforcement learning, outlining its conceptual foundations, historical development, and mathematical formulation. It provides detailed explanations of policy gradient methods and focuses on the Proximal Policy Optimization (PPO) algorithm used throughout the work. This overview is designed to make the chapter self-contained and accessible to readers with varying levels of prior experience in RL.

Building on this theoretical grounding, we introduce a novel sound-driven navigation task in which a robot must reposition itself in a reverberant environment to optimize ASR performance. The task is formalized as a partially observable Markov Decision Process (POMDP), where the agent receives only multi-channel audio input and is rewarded based on the Word Error Rate (WER) achieved by a downstream ASR model. A full training pipeline is implemented using the PPO algorithm, with a custom neural architecture and an environment

built on our simulator. The agent’s perception module reuses a convolutional feature extractor pretrained in a supervised setting on the single-source localization task introduced in [Chapter 3](#). This enables it to ground its navigation policy in spatial auditory cues. Observations are encoded as interaural spectral features (ILD and IPD), and the reward signal is provided by a pre-computed WER map evaluated on real ASR transcripts.

Experiments show that the learned policy reliably improves ASR performance by navigating toward acoustically favorable positions, even without access to visual or geometric cues. The contribution demonstrates how auditory perception can be cast as a reward optimization problem and illustrates the potential of DRL to produce interpretable, perception-driven navigation strategies. It also establishes a flexible and reproducible framework for future work on learning-based auditory control.

1.2.5 Scope, Methodology, and Implementation Context

A deliberate constraint throughout the thesis is the exclusive use of acoustic sensing. While audio is often used in combination with vision or depth sensing in robotics, this work focuses solely on microphone-based input to explore how far auditory perception alone can be pushed. This makes the learning tasks harder but also more focused, and aligns with biological inspiration—many animals rely heavily on hearing for navigation and interaction. This constraint also emphasizes the contribution of the acoustic simulation platform, which enables the creation of large, diverse datasets without relying on costly visual modeling.

The entire research pipeline—simulator, datasets, models, training routines, and evaluation tools—was developed from scratch as part of this thesis. No pre-existing codebase was forked or extended. Instead, each module was built to fit within a coherent ecosystem. This reflects a dual contribution: a set of scientific results in deep learning for robotic audition, and a reproducible software foundation for future research in the field.

1.3 Thesis Structure

[Figure 1.2](#) provides a graphical overview of the thesis organization, which is structured around four technical chapters, each corresponding to a specific contribution.

[Chapter 2](#) introduces key acoustic principles and reviews the state of the art in room acoustic simulation. It presents the custom simulation library developed for this thesis, detailing its architecture, capabilities, and performance. This simulator supports both static and dynamic scenarios and serves as the foundation for all subsequent experiments.

[Chapter 3](#) addresses the task of SSL. After reviewing relevant literature, we present deep-learning-based models for both single- and multi-source localization. These models are evaluated within our simulated environment, with a focus on their robustness and limitations.

[Chapter 4](#) extends the SSL problem to dynamic settings. We define an active localization task in which the robot moves to improve spatial awareness. A deep-learning-based pipeline is

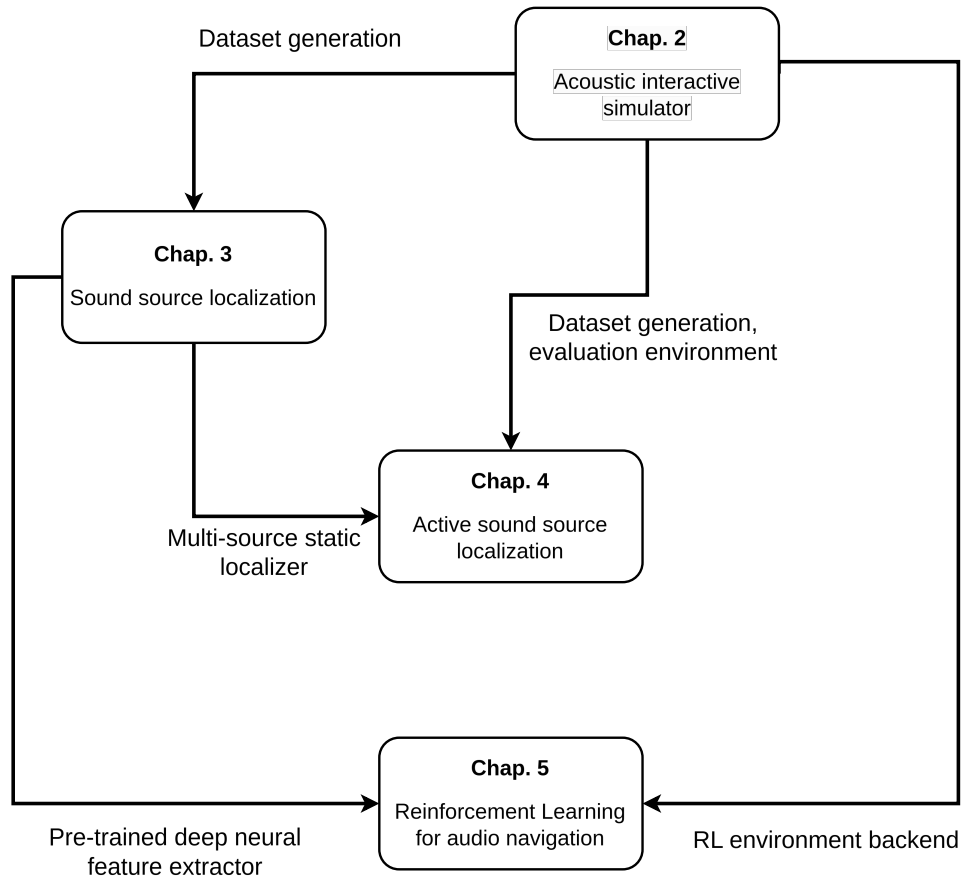


Figure 1.2 – Overview of the thesis organization. Each chapter, represented by a block, outlines a specific contribution.

introduced to accumulate localization evidence over time, leveraging motion as a source of information.

Chapter 5 applies Deep Reinforcement Learning (DRL) to a perceptually motivated navigation task. Here, the robot learns to reposition itself in order to improve ASR performance, using only acoustic feedback. We describe the task design, learning setup, and policy evaluation.

Chapter 6 concludes the thesis by summarizing the main contributions, highlighting limitations, and proposing future research directions.

This structure reflects a gradual progression: from simulation tools (**Chapter 2**) to perception (**Chapter 3** and **Chapter 4**) and finally, learned control and behavior (**Chapter 5**). Each stage builds upon the previous one to address the overarching challenge of embodied auditory intelligence in robotics.

1.4 Note on Writing Tools

During the writing of this thesis, I occasionally used large language models, such as ChatGPT, to help refine the style and clarity of the introduction and conclusion sections. These tools

were used solely for language polishing. All scientific ideas, formulations, and arguments are entirely my own, and I have independently verified the accuracy and integrity of the content. No such tools were used in the technical chapters.

Chapter 2

Acoustic Robot Simulator

2.1 Introduction

HRI investigates the challenges of bringing capable humanoid robots into real-world environments. These robots are embodied agents that must operate in complex, often unpredictable settings involving humans. This context introduces multiple challenges: social robotics experiments are subject to material costs, safety constraints, and logistical complexity. As a result, applying data-intensive, trial-and-error learning approaches to physical robots is difficult to scale. Simulation provides an effective alternative to physical interaction for bootstrapping and designing intelligent robot behaviors. However, it also introduces its own limitations, which must be carefully considered when transferring algorithms to the real world. The following explores the role of simulation in robotics and why it is pivotal in addressing many of the field's practical and scientific challenges.

Simulation offers an alternative tool to experiment with new techniques. Since its first operation in the late 1940s, the Monte Carlo algorithm has been widely used in most scientific fields [14], [15]. In 1950, a team led by John von Neumann and Jule Charney used the ENIAC computer to produce the first weather forecast by an electronic computer [16]. Although their results carried important numerical errors, this work led to the foundation of modern meteorology. This is an example of numerically replicating a physical phenomenon by implementing and solving the corresponding equations. NASA also used simulations early in its space programs, such as Apollo. The primary goal of their enterprise was to build a training setup for astronauts to practice specific skills. Indeed, in high-stakes enterprises, the crew's accommodation to their environment and tools is essential for the mission's success. Between 1963 and 1972, the Apollo flight crew trained for 30,000 hours on different simulation devices. Using simulators as sandbox environments to perform training is a significant use case of such systems. It can be applied to human training, algorithm testing, and data collection. Once built, simulation systems can be dramatically valuable as cheap replications of physical settings that are either expensive or difficult to access. Multiple techniques or discoveries would have been impossible without the scaling offered by simulation. For instance, the drug discovery field has benefited considerably from molecular computer simulation [17], [18]. Building on this general overview of simulation, we now turn to its specific applications in deep learning and robotics.

Simulation for Deep (Reinforcement) Learning. Deep learning techniques have shown impressive results on various tasks ranging from computer vision to natural language processing. At their core resides the processing of substantial amounts of data. Collecting datasets of sufficient size and quality is a significant obstacle in many concrete applications of deep learning techniques. Simulation provides an alternative way to gather massive amounts of data and often allows for automatic annotation. However, building an effective simulator can be hard or even impossible. Also, simulated features frequently deviate from real-world data, which can heavily hinder the performance of a system trained with simulated data. Reinforcement learning, for instance, considers an agent interacting with an environment and improving its action policy from trial and error. Since deep neural networks have been employed in this field, the interest in simulating the targeted environments has grown significantly. The success of DRL in applications such as board games and video games came early because simulating them is trivial. Large artificial neural networks could then be trained on massive amounts of data. Nonetheless, the design of realistic simulated environments has allowed the deployment of DRL in more complex and practical scenarios.

For instance, industrial and academic actors have used simulated environments as a first step toward achieving fully autonomous driving. The availability of driving simulators allows more modest research teams to contribute to this field without requiring them to handle data collection. Sallab et al. [19] and Osiński et al. [20] proposed simulation-based DRL techniques for autonomous driving. [21] and [22] further survey the landscape of available techniques, datasets, and simulators on this topic. Additionally, a team from Google DeepMind has proposed a novel approach for controlling the magnetic field of tokamak plasmas using DRL [23]. A critical challenge in this work has been the scarcity of access to a real-world fusion reactor and, thus, to the training data. To get around this, they collaborated with physicists to build a fast and accurate JAX [24] simulator modeling the plasma core, TORAX [25]. Finally, games have been among the most widespread application domains for DRL as they are inherently virtual [4], [5], [7], [26]. No effort is necessary to provide a simulated version of those environments.

Robotics. Returning to HRI and robotics in general, simulators have also been shown to be essential for developing and testing novel algorithms. Experimenting with uncertain methods remains cheaper and safer in a virtual environment than in the physical world. Involving real hardware brings extra cost and the need for sufficient safety measures. Simulation for robotics is fundamentally a multidisciplinary field. Liu et al. [27] discuss the role of physics-based simulators in robotics. The authors highlight the necessity of relying on software simulation to compensate for real-world robotic systems' challenges. Naturally, most existing toolboxes commonly feature the simulation of the various physics phenomena involved in a robotic platform. Depending on the richness of the targeted environment, modeling the necessary behaviors may be difficult. The diversity of existing solutions is highlighted by Collins et al. [28]. They list and compare the available offerings for a selection of robotics applications (medical, marine, aerial, and soft robotics). In addition to the underlying physics, a simulator should also model the behavior of different sensors. Indeed, in the natural world, robots use sensors to build a faithful representation of their surroundings. The accuracy of simulators in this regard conditions the algorithm's performance in practical use cases. Furthermore, performance is a crucial metric when developing a robotics simulator. Solving complex

equations that model multi-physics phenomena is often computationally expensive. Keeping runtime low enough requires simulator designers to make tradeoffs in their implementation choices. In their study on opportunities and challenges of robotics simulation [29], Choi et al. notably highlight the difficulty of gauging the right level of model complexity. They also suggest that the speed limitations of current solutions are an obstacle to the broader adoption of simulation in this field.

Simulation for dataset collection. Most DRL research projects directly learn from a robotics simulator by running their policy within it to train. Simulators have also been leveraged to gather substantial synthetic datasets for offline training. The branch of RL that operates on fixed interaction data collected a priori is known as offline reinforcement learning. Despite its formulation deviating from the traditional, interaction-based framework of RL, offline RL offers sensible advantages. Gathering data remains a costly operation in any RL training process, so delegating this task allows one to focus exclusively on algorithm development. It separates the expensive effort of gathering data and lets diverse research actors attempt to solve the underlying decision problem. For instance, Walke et al. [30] introduced the *BridgeData V2* dataset. It contains 60,096 trajectories collected across 24 robot manipulation environments. In this case, they used a real robot, not a simulator, bringing additional fidelity. However, they acknowledge that such endeavors are both expensive and time-consuming. D4RL [31] and its successor D54L [32] were released to become the de facto standards of offline RL robotics benchmarks. They comprise a variety of environments relying on existing simulation platforms such as Mujoco [33] or CARLA [34]. The high costs of gathering and labeling datasets also limit supervised learning applications. Synthetic datasets can be generated by simulators that also offer automatic sample labeling. The Kubric [35] dataset generator leverages the *PyBullet* [36] simulator and allows for synthesizing massive amounts of photorealistic data.

Human-robot interactions. Human-robot interactions are another dynamic field of study [37]. This broad field encompasses all the tasks that require robots to operate near or directly with humans. Such scenarios bring additional difficulties, such as safety implications or social acceptance. Zachari et al. [38] conducted an extensive survey on the multiple facets of safety in HRI. They enumerate the numerous remaining open problems in this field. For instance, adding machine learning techniques in social robotics introduces new possibilities and safety difficulties. Simulation is, therefore, compelling to researchers working in HRI. Abeyruwan et al. [39] propose a framework to train RL policies for human-interaction tasks. Specifically, they train a robot arm to play table tennis against a human adversary. The learning process iteratively switches from a simulated environment to a physical system involving a human player. Kaur et al. [40] insist on the importance of modeling human behaviors in HRI simulators. They reviewed the existing simulators for mobile robot navigation in pedestrian-rich environments. They identified the key missing features of available solutions. For instance, the richness of human behavior models is not satisfying as they often lack realism and diversity. Sprague et al. [41] proposed *SocialGym 2.0*, a multi-agent navigation simulator that models robot-robot and human-robot interactions. Safety is another crucial challenge when humans are added to the loop.

Sim2Real challenges. Despite offering substantial advantages for learning robust policies using DRL, simulators should be handled carefully. Indeed, the end objective of robotics is to design systems that can interact with the real world. The performance of policies learned in simulated environments is not guaranteed to transfer to real scenarios. Hence, an entire segment of the robotics DRL community targets the problem of sim-to-real [42], [43], [44]. This area encompasses the challenge of leveraging simulation while ensuring appropriate behavior and performance of the target physical system. These discrepancies are primarily due to differences between the simulator and the real-world environment. They can be modeling limitations, numerical imprecision, unrealistic assumptions, etc. In conclusion, the many benefits of simulation come at some cost, which must be carefully accounted for when deploying RL policies in the real world.

Acoustic simulation. Thanks to the widespread availability of camera sensors and significant advancements in image processing techniques, vision has emerged as one of the most predominant modalities in robotics. Nevertheless, other cues have also been employed, either in conjunction with or in place of vision (light detection and ranging (LIDAR) [45], [46], haptic feedback [47], audio [10], [48], [49], etc.). In this thesis, the focus will be on acoustic applications in robotics. Simulators targeting audio simulation are scarcer, especially the ones dedicated to robotics. In their review, Kaur et al. [40] highlight the absence of ambient sound modeling for testing sound-based navigation algorithms. The video game industry, among others, has motivated the development of acoustic rendering engines. Zhao et al. [42] enhanced a vision-based RL pipeline for robotic grasping by implementing audio perception. To conduct their experiments, they use the *ThreeDWorld* [50] multi-modal simulator, which can synthesize impact sounds. The proposed grasping pipeline performs better using both vision and audio cues than with vision only. Grauman et al. conducted a line of work on auditory-based robot navigation tasks [10], [48]. They used a simulated environment modeling both visual and auditory cues to train their deep neural networks. Srivastava’s doctoral thesis [51] investigates the functioning of acoustic simulation in great detail. It provides a precise overview of the functioning of Room Impulse Response (RIR) estimation techniques and related concepts. Srivastava’s main contribution is training a deep neural network to estimate the room acoustic parameters. The acoustic pipeline has been developed explicitly for this project and is built around the *Bidirectional Sound Transport* algorithm by Cao et al. [52]. This chapter will detail the literature on acoustic reverberation simulators and provide an overview of existing software solutions.

In conclusion, simulation is a key component in many scientific domains, especially in robotics and deep reinforcement learning. It allows for scaling experiments and iterating rapidly on research ideas without relying on costly and time-consuming physical infrastructure. However, simulation has trade-offs that should be carefully considered when developing software platforms. For instance, RL policies learned in simulated environments do not always translate to performing convincingly in real-world scenarios. Also, no satisfying simulation solution exists for sound-based robot navigation. For those reasons, we contributed an original implementation of a flexible and feature-rich acoustic simulator for robotics, *Acoustix*. Its goal is to provide a practical and convenient sandbox for machine learning and robotics scientists to experiment with various sound-related problems. The obtained software ecosystem has been the core framework for the experimental work conducted in the present thesis. This

chapter will introduce the fundamental notions of sound propagation and audio processing to understand the phenomenon of acoustic reverberation. Also, various state-of-the-art methods for acoustic simulation will be presented along with existing implementations. Finally, we will detail the core design of our simulator and highlight its several features.

2.2 Acoustic Simulation of Reverberant Environments

This section explains the core concepts of acoustic reverberation and the relevant notions in audio processing. The key principles of acoustic reverberation are outlined in the following paragraphs to establish a foundation. Additionally, we present some essential works regarding the problem of realistically simulating sound propagation in reverberant environments. Finally, we provide an overview of the available software ecosystem.

2.2.1 Background

This section introduces key concepts in acoustics and audio signal processing that underpin the scientific developments presented throughout the thesis. It begins with a review of the physical modeling of acoustic signals, followed by a discussion of reverberation, a recurring theme in the robotic scenarios explored in this work. Since our simulator explicitly accounts for reverberant environments, understanding this phenomenon is essential. The section concludes with an overview of spectral representations, which form the basis for many of the audio features used in subsequent chapters.

2.2.1.1 Fundamentals of Sound Propagation

Sound is a mechanical wave phenomenon. Sound waves propagate in various media, such as air, water, and solids. Naturally, sound is represented as a real-valued temporal signal $x(t)$. Physical recording devices operate in the analog domain and capture continuous signals, which are typically quantized and sampled to produce a discrete signal $x[n]$. Discrete representations allow the numerical processing and saving of sound signals.

While many modern algorithms, especially DNN-based solutions, process sound as purely statistical data, the physical reality of acoustic phenomena is a central aspect of this thesis. Indeed, this work aims to tackle various audio-related tasks in reverberant environments. Thus, it is necessary to highlight the impact of the physical reality on recorded signals to study how they will affect the performance of the proposed methods later.

Let us first consider the ideal case of a single receiver in the free field. Free field denotes an idealized environment that can be considered anechoic. Hence, no sound reflections are considered; thus, the reverberation phenomenon is ignored.

The signal x received by the microphone can be expressed as a function of the source signal s by the following equation ([53] - Chapter 3, [54]):

$$x(t) = \frac{1}{\sqrt{4\pi d}} s\left(t - \frac{d}{c}\right), \quad (2.1)$$

where

- d is the source-to-microphone distance (in m)
- $c \approx 343$ is the speed of sound (in m/s at 20°C)
- $\frac{d}{c}$ is the time of arrival (in s)

In this simple case, the received signal corresponds to a delayed and attenuated version of the source signal.

When considering digital signals, Equation (2.1) becomes

$$x[n] = \frac{1}{\sqrt{4\pi d}} s\left[n - \frac{d}{c} f_s\right], \quad (2.2)$$

where f_s is the sampling rate (in Hz), neglecting sampling and quantization issues.

The microphone signal can be rewritten as

$$x[n] = (h * s)[n] \quad (2.3)$$

where $h[n] = \frac{1}{\sqrt{4\pi d}} \delta\left[n - \frac{d}{c} f_s\right]$ characterizes the acoustic path from the source to the microphone. δ denotes the Dirac delta function.

2.2.1.2 Acoustic Reverberation

Most realistic scenarios do not behave so simply, and other physical phenomena must be modeled. In a closed environment, such as an indoor room, sound will reflect off the walls and cause reverberation. A reverberant room can be modeled as an linear time-invariant (LTI) causal system by neglecting secondary effects such as temperature and pressure changes. Hence, expressing the listened signal as a convolution remains possible, similarly to Equation (2.3):

$$x[n] = (h_r * s)[n], \quad (2.4)$$

The filter h_r is called the **Room Impulse Response (RIR)**. It depends on the positions of the microphone and source, the walls' acoustic properties, and the room's dimensions. Hence, the RIR is defined for each source-microphone pair. It characterizes how the sound travels between their positions and is thus sufficient to reconstruct the listened signal. As its name suggests, the RIR depicts the room's acoustic response to an impulse, modeled by a Dirac delta function:

$$x[n] = (h_r * \delta)[n] = h_r[n]. \quad (2.5)$$

The RIR encodes the multiple propagation paths between the source and the microphone. Each path has a specific delay and attenuation factor.

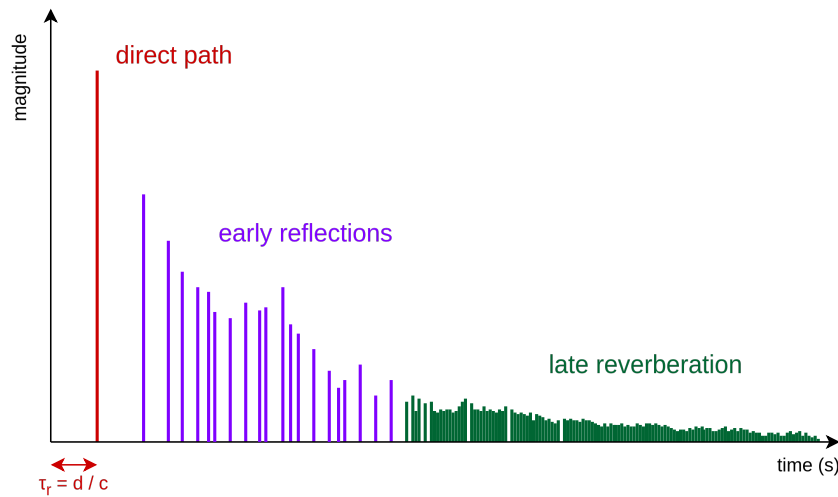


Figure 2.1 – Schematic representation of an RIR. It can be decomposed in three sections: **direct path**, **early reflections** and **late reverberation**.

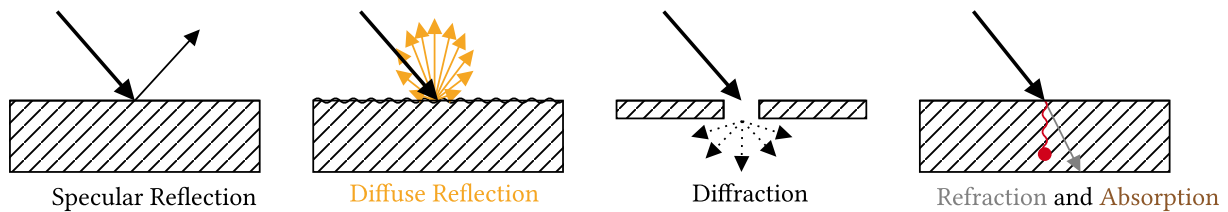


Figure 2.2 – Illustration of the different ways sound interacts with surfaces [55].

Figure 2.1 gives a schematic illustration of an RIR. It can be decomposed into three sections. The plot of a real RIR recording is visible in Figure 2.3. The first is the *direct path*, which corresponds to the signal reaching the microphone directly from the source without reflecting on any surface. This corresponds to Equation (2.1) describing sound propagation in an anechoic room. The delay amounts to $\tau_r = \frac{d}{c}$. Secondly, follow the *early echoes*, which are the first reflections on the walls. Finally, the late reflections correspond to the echoes bouncing several times before reaching the microphone. They form the long and dense tail of the RIR. The boundary between early echoes and late reflections is called the *mixing time* and depends on the room's acoustic characteristics.

The RIR model does not account for all the existing reflection phenomena. Sound will interact in various, potentially simultaneous, ways with the surface it encounters (Figure 2.2). It might reflect from the surface, leading to reverberation, but it can also be partly diffracted (in the presence of a small aperture), refracted, or absorbed. Besides, the reflection can be of two kinds. On the one hand, smooth surfaces lead to specular reflections that behave similarly to light reflecting on a mirror. This is the only phenomenon modeled by the RIR approach. On the other hand, diffuse reflections occur when the surface is imperfect or rough. In this case, the trajectories of the reflected waves are entirely unpredictable. The RIR paradigm solely accounts for specular reflections.

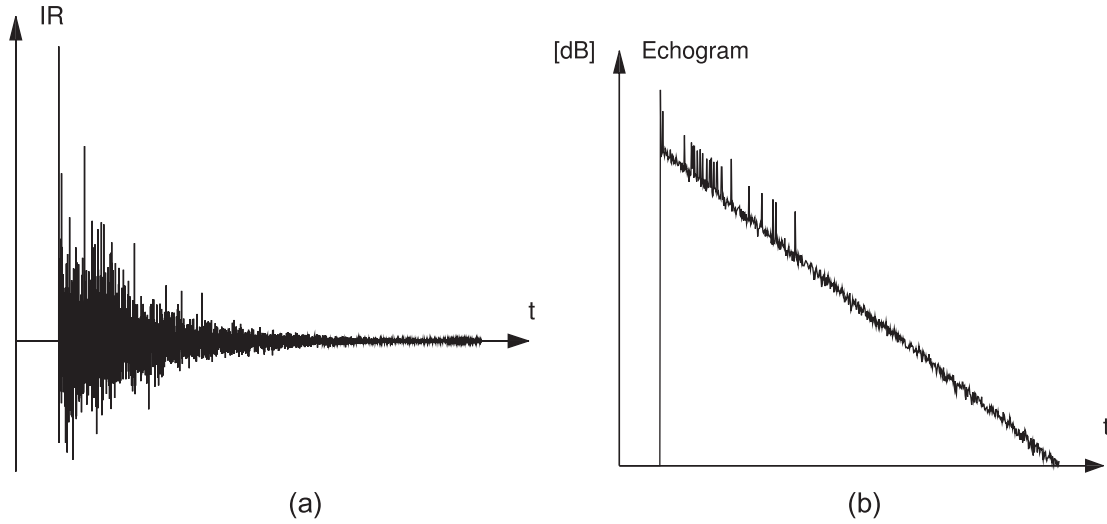


Figure 2.3 – RIR plot. (a) An impulse response recorded in a room. It represents the propagation of sound pressure from the sound source to the receiver. (b) The associated time-energy response plots the propagation of sound energy. [56]

When several sources are active in the room, the sound received by a microphone is the sum of each source's contribution. This formulation is known as a mixture model ([53] - Chapter 3). These individual contributions are the convolution between each source signal s_i and the corresponding RIR h_i . The final received signal can be expressed as:

$$x[n] = \sum_{i=1}^{n_s} (h_i * s_i)[n]. \quad (2.6)$$

Equation (2.6) can be straightly generalized to multiple microphones $\{m_1, \dots, m_{n_m}\}$. The signal recorded by the k -th receiver is:

$$m_j[n] = \sum_{i=1}^{n_s} (h_{i,j} * s_i)[n] \quad \forall j \in \llbracket 1, n_m \rrbracket, \quad (2.7)$$

where $h_{i,j}$ is the RIR corresponding to the pair of positions of source s_i and microphone m_j . Hence, $n_s \times n_m$ RIRs must be computed for a scene involving n_s active sources and n_m receivers.

2.2.1.3 Characterizing Reverberant Rooms

This section introduces essential quantities that depict the reverberation properties of a room. They will be essential in the interface of our simulator as they allow specifying how the environment should behave acoustically.

The reverberation time noted T_{60} or RT60 denotes the time before the sound pressure decreases by 60 dB after the source signal is abruptly stopped.

It can be estimated from the room's dimensions and has been empirically expressed by Wallace Clement Sabine [57] as:

$$T_{60} = \frac{24 \ln(10)}{c} \frac{V}{A} \approx 0.16 \frac{V}{A}, \quad (2.8)$$

where c is the speed of sound in the air at 20°C, V is the volume of the room (in m^3), and A is the *equivalent absorption surface* (in sabins). The latter can be obtained by summing the weighted surface area of each wall (including floor and ceiling):

$$A = \sum_{i=1}^6 e_{\text{abs},i} s_i, \quad (2.9)$$

where $e_{\text{abs},i}$ and s_i are each surface's absorption coefficient and area, respectively. The Sabine equation can be inverted to infer the overall absorption coefficient e_{abs} of the room from a reverberation time T_{60} :

$$e_{\text{abs}} = \frac{24 \ln(10) \times V}{c \times S \times T_{60}}. \quad (2.10)$$

Here, all surfaces are assumed to behave the same, and the *equivalent absorption surface* A may then be written as $A = S \times e_{\text{abs}}$, with S being the total surface area. Srivastava's PhD thesis [58] investigates the role of acoustic parameters in simulation and how to estimate them.

2.2.1.4 Spectral Representations of Audio Signals

Fourier Transform, from Continuous to Discrete

Although the waveform rendering of an audio signal is a raw and natural representation of the information, the acoustic literature has studied several alternative higher-level transforms. A popular way of representing audio signals is to project the temporal signal in the Fourier domain. The Fourier transform stands as the core concept of this category of encoding. Several resources, such as Oppenheim et al. [59] (Chapter 2) and Bracewell [60] (Chapter 2), propose extensive descriptions of the Fourier Transform and its properties.

As acoustic signals are stored and processed numerically, the continuous framing of the Fourier transform cannot be directly employed. In contrast, the signal processing community has turned to the Discrete Fourier Transform (DFT) to process discrete signals (see Smith [61] Chapter 8). The Short-Term Fourier Transform (STFT) is a tool for representing the temporal real-valued signal as a two-dimensional complex form. This target domain is often referenced as the time-frequency plan. It consists of computing the signal's DFT on short overlapping smoothed windows. The STFT has been partly motivated by the non-stationarity characteristic of speech. The signal can be assumed to be locally stationary when using short analysis frames.

Let us first define an STFT frame for a given time index $m \in \mathbb{Z}$:

$$x_m[n] = x[n + mH]w[n], \quad (2.11)$$

where

- $x[n]$ is any sampled real-valued signal ($n \in \mathbb{Z}$);
- $w[n]$ denotes the analysis window with compact support $\llbracket 0, N - 1 \rrbracket$, where N is assumed to be even. Several choices can be employed for the shape of w . While, in theory, any function with compact support can be employed, it significantly impacts the result. Popular choices include the rectangular, Hamming, and Hann window functions.
- H is an increment, also called hop size. It should remain lower than N to ensure a non-zero overlap $N - H$ between successive frames;

The support of the STFT frame $x[n]$ is also $\llbracket 0, N - 1 \rrbracket$. The discrete STFT of the signal x is defined as the set of DFTs of the frames x_m , $m \in \mathbb{Z}$:

$$X[m, k] = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x_m[n] e^{-2i\pi \frac{kn}{N}}, \quad (2.12)$$

where k denotes the frequency index in $\llbracket -\frac{N}{2}, \frac{N}{2} - 1 \rrbracket$. In this definition, both the time and frequency indices are integers. In practice, the signal has a finite length $L \gg N$, leading to approximately $M = \lceil \frac{L}{H} \rceil$ total STFT frames. Hence, a finite-length signal's STFT is a complex-valued matrix of size $M \times N$. As practical signals are real-valued, the STFT is symmetric, and only positive frequencies ($k \in \llbracket 0, \frac{N}{2} \rrbracket$) are considered.

The time-frequency resolution is a tradeoff directly impacted by the choice of the window size N . Low values of N will produce a wide-band spectrogram with a high time resolution at the cost of a lower frequency resolution. On the contrary, high values of N will give narrow-band spectrograms with a low time resolution but a high-frequency resolution.

Spectrogram

The spectrogram of a signal is a 2D real-valued representation of its STFT. It displays the magnitude, phase, or power of the complex STFT. The spectrogram allows for visualizing a signal as an image. In addition to this practical property, it quantifies the intensity of the signal at each time frame and frequency bin. Different types of spectrograms can be defined:

- The magnitude spectrogram is the modulus of the STFT:

$$\text{spectrogram}\{x\}[m, k] = |X[m, k]|. \quad (2.13)$$

- The power spectrogram, or power spectral density, is its squared modulus:

$$X_{\text{power}}[m, k] = |X[m, k]|^2. \quad (2.14)$$

- The power spectrogram can also be expressed in decibels (dB):

$$X_{\text{power, dB}}[m, k] = 20 \log_{10} |X[m, k]|. \quad (2.15)$$

Figure 2.4 is an example of a power spectrogram computed from a speech signal.

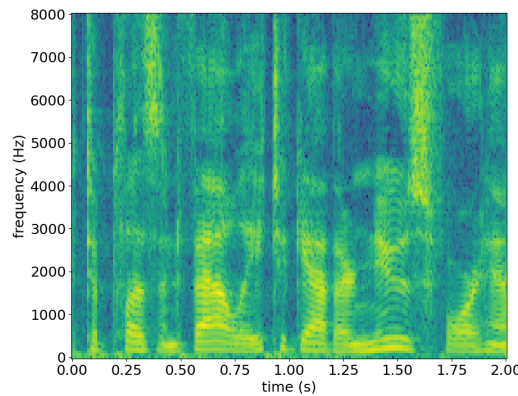


Figure 2.4 – Power spectrogram of a two-second-long clean speech recording.

- The phase spectrogram is its argument:

$$X_{\text{phase}}[m, k] = \arg(X[m, k]). \quad (2.16)$$

The term spectrogram can also refer directly to the complex-valued result of the STFT.

Motivation for Using STFT for Reverberant Signals

The convolution theorem (Oppenheim et al. [59] Section 2.9.6) grants one of the Fourier transform's fundamental properties. It states that the Fourier transform of a convolution is the product of the Fourier transforms:

$$\mathcal{F}(f * g) = \mathcal{F}(f) \times \mathcal{F}(g). \quad (2.17)$$

Additionally, the transform of a product is the convolution of the transforms:

$$\mathcal{F}(f \times g) = \mathcal{F}(f) * \mathcal{F}(g). \quad (2.18)$$

This result gives an intuitively compelling argument for using Fourier representations in problems involving reverberant environments. Indeed, the reverberation phenomenon can be modeled in the time domain as the convolution of a source signal with the RIR. In the Fourier domain, this operation becomes a multiplication, which simplifies the analysis and processing of reverberant signals by separating the effects of the environment from the original source. However, it must be noted that the convolution theorem does not hold for the STFT. It can be verified for the general DFT under certain conditions, but it is wrong when using short-term frames. In practice, the length of the STFT window function is often significantly shorter than the length of the convolution.

Despite this theoretical result not transferring to the STFT-based representations, they are still widely used in the literature when dealing with reverberating phenomena [62], [63], [64]. This assumption of multiplicative transfer functions is known as the narrow-band assumption. It is made in various domains of audio processing despite the error being often large.

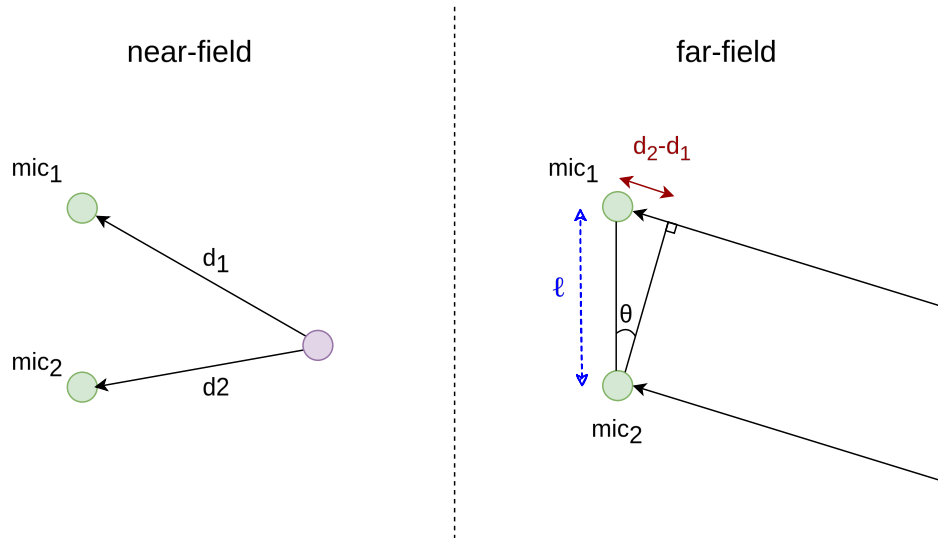


Figure 2.5 – Two-microphone configuration in the near-field and far-field cases. In the far-field approximation, sound propagation directions can be assumed to be parallel.

2.2.1.5 Multi-Microphones and Interaural Cues

Motivation

Using multiple microphones opens a wide range of possibilities in audio processing tasks. Humans rely on their two ears to exploit the spatiality of their auditory environment. Similarly, the signal processing community has exploited microphone arrays and developed algorithms to usefully aggregate signals from several sensors. Array processing spans several downstream tasks, such as speech enhancement [65], dereverberation [66], [67], sound source localization [68], [69], [70], separation [71] or acoustic scene analysis [72]. Disposing of more than one microphone allows computing the time difference of arrival (TDoA)

Many array configurations have been experimented with. Both the geometry (linear, polygons, spheres, or more complex arrangements) and the number of microphones (from two to several thousand) vary widely across applications. Two families of arrays have emerged in the community. On the one hand, researchers have increased the number of receivers in a single array to capture as much geometric information as possible. The ambisonic format is an example of an approach that leverages high microphone-count arrays [70], [73]. On the other hand, the binaural setup is one of the most studied configurations because it attempts to model human hearing. In this case, specific signal representations have been proposed to ease the extraction of relevant information. Some data representations have been introduced for the specific case of binaural devices. This section focuses on motivating and deriving those representations.

Sound Propagation within the Multi-Microphone Setting

For simplicity, the following section maintains the free-field assumption introduced in [Section 2.2.1.1](#). By adapting the single-microphone case ([Equation \(2.2\)](#)), the signal received by microphone i can be expressed as (Vincent et al. [53] Chapter 3 or Gustafsson et al. [74]):

$$x_i[n] = \frac{1}{\sqrt{4\pi d_i}} s \left[n - \frac{d_i}{c} f_s \right]. \quad (2.19)$$

where d_i is the distance from microphone i to the source (see [Figure 2.5](#)).

One can write the signal x_2 , recorded by microphone 2, as a function of the one recorded by microphone 1 by combining their expressions:

$$x_2[n] = \frac{d_1}{d_2} x_1 \left[n - \frac{d_2 - d_1}{c} f_s \right]. \quad (2.20)$$

$\frac{d_1}{d_2}$ is called the level ratio, while $\frac{d_2 - d_1}{c}$ is the time difference of arrival (TDoA). Those quantities encode relative information on the source position.

The situation is described as far-field when the source is significantly far from the microphone array [75]. The far-field situation is when the source-to-microphone distances d_i are large compared to the inter-receiver distance l . In this case, and assuming there is no reverberation, the level ratio is almost equal to 1:

$$\frac{d_1}{d_2} \approx 1. \quad (2.21)$$

Also, the far-field assumption implies that the TDoA is fully determined by the DoA and its corresponding angle θ :

$$\frac{d_2 - d_1}{c} = \frac{l}{c} \cos(\theta). \quad (2.22)$$

Hence, theoretically, a measure of the TDoA can be sufficient to infer the value of θ . Increasing the number of microphones brings redundancy and, thus, robustness when the measures of the TDoA are noisy.

Relative Transfer Function

When considering a pair of microphones, defining interaural features becomes possible. Those quantities and techniques have been introduced to help understand and model binaural hearing and beamforming in general.

In practice, the aforementioned geometrical observations are leveraged by computing specific features from the input signal. Then, one computes the Related Transfer Function (RTF) to express the interchannel information. It corresponds to the ratio between two microphones' Acoustic Transfer Functions (ATFs) [62]. The value of the RTF at a given time can be obtained by computing the ratio of the STFT received by two microphones: the interaural spectrogram.

More precisely, the RTF of the i -th microphone is obtained by dividing the STFT of its received signal by that of a reference microphone (e.g., the first one):

$$\text{RTF}_i[m, k] = \frac{X_i[m, k]}{X_1[m, k]} \in \mathbb{C}, \quad (2.23)$$

where m and k are the time and frequency indices, respectively. By construction, we have $\text{RTF}_1[m, k] = 1$.

Cohen [64] is developing a way to use speech signals to identify an acoustic system's RTF. Markovich-Golan et al. [76] thoroughly evaluate two statistical estimators for the RTF. Li et al. [77] propose an estimation method based on segmental power spectral density matrix subtraction. In a later work, Li et al. [63] explore methods to accurately estimate the RTF to enhance SSL methods.

In the specific case of binaural hearing on a humanoid robot platform, the device's physical head will introduce additional acoustic perturbations between the two microphones. The simple propagation model derived in Equation (2.19) - (2.22) is not valid anymore. Here, the ratio of ATFs is called the head-related transfer function (HRTF) (Vincent et al. [53] Chapter 4). The modeling and simulation of HRTFs are outside the scope of this thesis but are nevertheless central to the field of acoustic robotics.

The RTF is a complex-valued function. It is often projected to real quantities, namely its argument and phase, for practical use. Let us consider a binaural array with microphones m_1 and m_2 .

- The modulus of the RTF, expressed in decibels, is called the Interaural Level Difference (ILD)

$$\text{ILD}[m, k] = 20 \log_{10} |\text{RTF}[m, k]| = 20 \log_{10} \left| \frac{X_2[m, k]}{X_1[m, k]} \right|. \quad (2.24)$$

- The phase is called the Interaural Phase Difference (IPD):

$$\text{IPD}[m, k] = \arg(\text{RTF}[m, k]) = \arg\left(\frac{X_2[m, k]}{X_1[m, k]}\right). \quad (2.25)$$

Interaural features, especially IPD, have been successfully used in SSL as they directly relate to the DoA. As Equation (2.20) illustrates, the times at which each microphone of the array receives the signal differ by some short delay $\tau = \frac{d_2 - d_1}{c}$. Under ideal circumstances, meaning in the absence of reverberation and perturbations such as noise, the phase of the interaural spectrogram is an explicit and deterministic function of the TDoA τ . Mandel et al. [78] derived a probability model to estimate the IPD from binaural recordings. They test their framework on an SSL task and compare it against an approach based on the Generalized Cross-Correlation with Phase Transform (GCC-PHAT) [79] estimator. GCC-PHAT provides an estimator of the TDoA. The proposed approach achieves better results than the baseline and shows robustness to noise and reverberation. By monitoring brain activity in rats, Urugun et al. [80] studied how neurons are sensitive to ILD functions. They discovered that rats use ILD as a critical cue

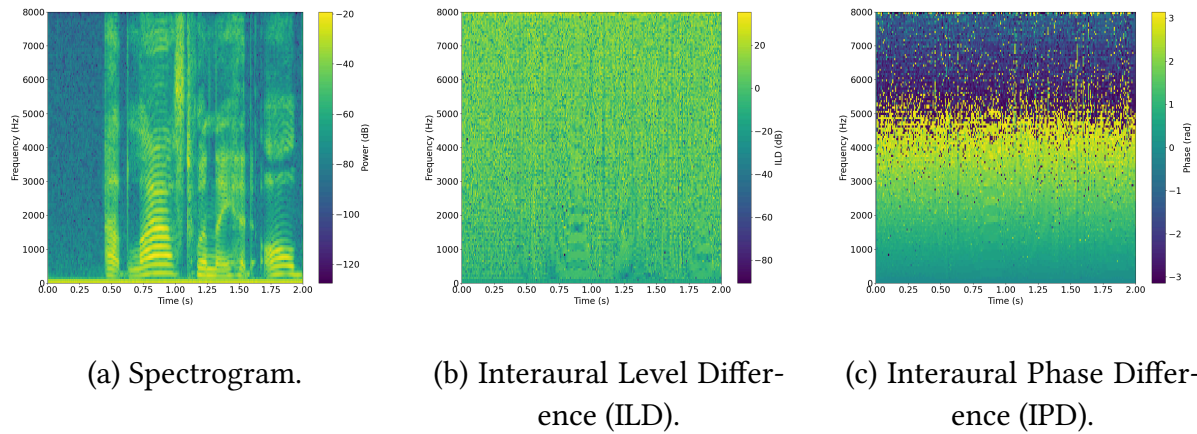


Figure 2.6 – Illustration of time-frequency representations of a synthesized speech signal. (a) shows a mono-aural spectrogram obtained from the left microphone. (b) and (c) are interaural representations and use both left and right channels. Recordings have been generated in the acoustic simulator.

to localize sounds in space, highlighting the biological motivation of interaural features and confirming their relevance.

Figure 2.6 provides an example of each aforementioned spectral representation. A binaural array and a speech source have been placed in a simulated room. The omnidirectional source plays a 1-second section of speech recording sampled from the *LibriSpeech* [81] corpus. The features were then extracted from the inferred microphone signal:

- Figure 2.6a shows the power spectrogram of the signal received by the left microphone: $20 \log |X_1[m, k]|$.
- Figure 2.6b and Figure 2.6c show the binaural cues, computed from Equation (2.24) and Equation (2.25) respectively.

2.2.2 Overview of Acoustic Reverberation Simulation Methods

Simulating the acoustic reverberation phenomenon has a broad range of applications. Reverberation must be considered when a scene happens indoors to obtain a realistic render. Thus, diverse communities have explored this problem, from physicists and roboticists to the video game industry and the signal processing community. This task has been referred to as computational room acoustic modeling or auralization. The first works mentioning the acoustic rendering concept originate from the 1960s [82]. This second term was coined in 1993 by Kleiner et al. [83], inspired by the concept of visualization for visual rendering. In 1992, Takala et [84] proposed a thorough description of a general methodology for sound rendering. Savioja et al. [85] highlight the diversity of the existing methods for sound rendering despite the field's relative youth.

Several approaches have been employed to tackle the auralization task. They differ on which underlying physical model and equation they use. Hence, a taxonomy can be derived from those differences. We may first consider rigorous methods that explicitly attempt to solve the

acoustic wave equation using numerical methods. Indeed, closed-form analytic solutions do not exist in the general case. The different existing practical strategies are presented later in this section.

Geometrical acoustics denotes a family of methods that adopt another approach to this problem. Instead of adhering to strict physical modeling of the sound propagation phenomenon, they assume that sound behaves as rays. This choice implies the neglect of the wave nature of sound and is, therefore, less accurate. Nonetheless, the efficiency of geometrical acoustics methods has led them to be widely adopted.

2.2.2.1 Numerical Simulation of Sound Propagation

A natural approach to acoustic rendering is numerically solving the acoustic wave equation. The wave equation is a second-order partial differential equation characterizing the propagation of acoustic wave in a given medium (Feynman [86], Volume I, Chapter 47):

$$\nabla^2 \mathbf{p} = \frac{1}{c^2} \frac{\partial^2 \mathbf{p}}{\partial t^2}, \quad (2.26)$$

where $\mathbf{p} : \mathbb{R}^3 \times \mathbb{R}_+ \rightarrow \mathbb{R}$ is the acoustic pressure as a function of position $\mathbf{x} \in \mathbb{R}^3$ and time $t \in \mathbb{R}_+$. $\nabla^2 = \frac{\partial}{\partial x^2} + \frac{\partial}{\partial y^2} + \frac{\partial}{\partial z^2}$ denotes the Laplacian operator in 3D space. This strategy is the most faithful to the physical reality. Theoretically, it accurately represents complex mechanisms such as diffraction, interference, scattering, or modal resonances [52]. However, tackling this second-order partial differential equation is challenging. No closed-form solution is readily available, and one must fall back on approximating a numerical solution. Botteldooren [87] has proposed using the Finite-Difference Time-Domain (FDTD) method to obtain a practical solution to the acoustic wave equation. In general, methods from this family discretize space and time to apply numerical integration techniques. Kirkup [88] proposed an in-depth investigation of applying the Boundary Element Method (BEM) to acoustics. It enables the solution of the Helmholtz equation by reformulating it as a boundary integral equation. The BEM method is less computationally expensive than the volume-based formulation. Additionally, Thompson [89] provides an overview of other Finite-Element Methods (FEMs) for solving the Helmholtz equation.

Raghuvanshi et al. [90] developed a new method to lower the computing cost of numerical simulation. They introduce an adaptive rectangular description of 3D scenes, unlocking 100-fold speedups compared to traditional techniques. This approach allows for rendering complex acoustic 3D scenes without requiring more than commodity hardware. They can model large and challenging environments such as a cathedral. However, high-frequency support is limited as they simulate only up to 1 to 2 kHz. Finally, their FDTD scheme is not adapted to moving objects.

Rosen et al. [91] tackle the problem of rendering acoustic scenes as efficiently as possible. Also, they target dynamic situations where the geometry can evolve over time. To achieve this, they restrict the solving to a 2D slice of the environment at the height of the listener's head. They use a second-order FDTD scheme, operating at a frequency of 275Hz. This approach shows compelling results in terms of performance and supported features. The ability to run

on a limited computing budget, such as a single CPU core, demonstrates its relevance for applications such as video games or virtual reality. Limiting the model to two dimensions grants a significant performance uplift. It must be noted that this approximation can affect the reproduction accuracy of certain phenomena, such as reflections and reverberations. The C++ implementation is available as the *Planeverb* open-source library [92].

2.2.2.2 Geometrical Acoustics

Geometrical Acoustics (GA) denotes a broad class of methods partly motivated to circumvent the initial limitations of the wave-based methods. More particularly, they aimed at providing more computationally efficient alternatives. Its main principle resides in considering that sound propagates as rays [56]. This is an immediate parallel of the ray-based rendering techniques used for light (ray tracing). Each ray travels in a straight line in the air until it hits a wall. It then deviates by following a simplistic specular reflection model. The assumption that all reflections are specular boils down to supposing the walls' surface to be ideally rigid. This implies that all sound wave properties are neglected, and thus, diffraction effects are entirely neglected. There are two main categories of GA methods: the image source techniques and the ray tracing ones. Rindel [93] gives a clear overview of those two families of methods, along with their respective advantages and drawbacks. Both categories are further explained in the following two paragraphs. Siltanen et al. [94] propose a general formulation for the GA approach to sound rendering. They introduce the *room acoustic rendering equation*, which aims to provide a unifying framework for the diverse GA techniques (beam tracing, ray tracing, image source, etc.). They can then all be expressed as particular cases of this general rendering equation. They draw their inspiration from the optics and computer graphics literature. Thanks to its generality, this formulation can model both specular and diffuse reflections.

Image source methods. The image source method is among the most notable and popular GA approaches. It was also one of the first methods to simulate sound propagation in reverberant environments. Initially introduced by Lothar Cremer in 1948 [95], this concept still stands today as an efficient way of modeling the reverberation phenomenon. It constructs specular reflections by mirroring each sound source in the plane of the reflecting surface. This process can be repeated recursively to model reflection up to a given order. [Figure 2.7](#) shows the virtual image sources for an example setup. According to Savioja [56], Gibbs et al. [96] were the first to propose an implementation for computing image-source positions and sound pressure levels in a rectangular room. Borish et al. [97] extended the model from rectangular *shoe-box* rooms to arbitrary polyhedra. Allen and Berkley significantly expanded this idea back in 1979 by developing a more comprehensive and computationally effective version of the Image Source Method (ISM) [98]. Their paper came with a FORTRAN implementation of the proposed algorithm. The significant popularity of the ISM is partly due to its simplicity and efficiency.

Ray tracing methods. The second family of GA methods consists of the ray tracing approaches. This paradigm considers that sound behaves similarly to light particles. More precisely, sound is supposed to travel as rays along straight paths. As soon as a ray hits a wall, its direction is updated according to Snell's law from geometrical optics. Hence, ray tracing

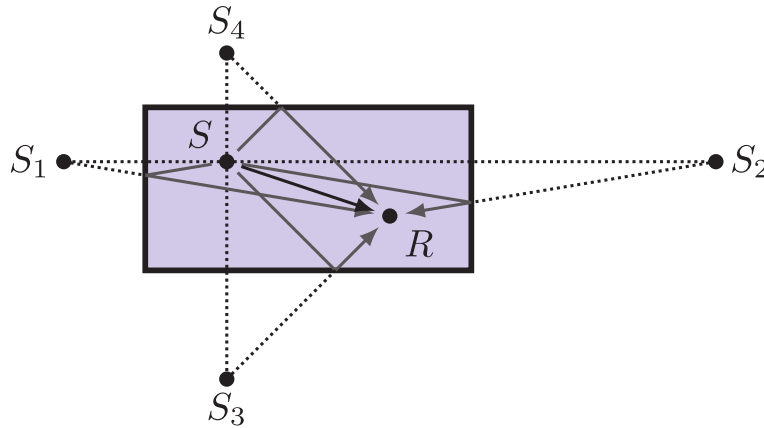


Figure 2.7 – Representation of the first-order virtual image sources [99]. The physical source S leads to 4 virtual sources S_i , i.e. one per reflective surface. The black arrow represents the direct path from the source to the receiver. The grey arrows depict the paths corresponding to the first-order reflections.

methods historically only account for specular reflections. A small volume is delimited around the receiver to obtain the resulting signal. Then, all rays traversing this volume will contribute to the final response. Ray tracing differs from ISM as it is a stochastic method. The overall simulation accuracy (including the maximum viable sampling rate) significantly depends on the number of rays used in the simulation [100]. Krokstad et al. [101] overview the early history of ray tracing methods in acoustics. Cao et al. [52] propose an advanced algorithm based on bidirectional path tracing: Bidirectional Sound Transport (BST). It addresses the drawbacks of existing ray-tracing GA methods. They reformulate Silken’s *room acoustic rendering equation* [94]. Schissler et al. [102] propose an innovative hybrid algorithm handling complex scenes with numerous sources. It combines a ray-tracing approach with sound source clustering to accurately model both indoor and outdoor environments. The clustering of distant sound sources allows their algorithm to efficiently scale with the number of sources while keeping the processing time low. Tang et al. [103] combine a ray-tracing algorithm with a deep neural network to render dynamic acoustic scenes at high refresh rates. The dataset to train the network was generated using an accurate wave-based method. This work is motivated to provide a hybrid approach combining high fidelity and responsiveness. Their approach can represent multiple reflection patterns (specular and diffuse reflections and diffraction). During training, the network learns to predict the scattering fields of objects. At inference, the network achieves real-time performance and can handle dynamic scenes where objects are moving.

2.2.2.3 Simulation of Dynamic Environments

The original methods for acoustic simulation were limited to static scenes where neither sources nor microphones move across time. This ideal situation is not representative of real-world scenarios. Especially in robotics, modeling moving humans and agents is an essential requirement. Historical techniques such as the ISM solely account for a given static layout of the sources and sensors. The result of the ISM process is a set of RIRs, one for each

microphone-source pair. The resulting signal is computed by convolving the source's input signals with those filters. However, the RIRs must be recomputed whenever a source or microphone moves. Chen et al. [48] pre-compute the RIRs for each attainable configuration of their simulated environment. This upfront computation moves the substantial simulation effort from training to a prior pre-processing step. Specific works have focused on improving existing simulators by supporting more and more dynamic scenarios. Diaz-Guerra et al. [12] leverage the performance of their ISM Graphics Processing Unit (GPU) implementation to pre-compute the RIR of every position in the trajectory. The simulated signal is obtained by filtering the source recordings with the RIRs using the overlap-add method. In [104], Zang et al. propose a method to simulate dynamic acoustic environments where both the sources and microphones can move. Similarly to [12], their approach also relies on the discretization of the trajectory. Cao et al.'s BST algorithm [105] can model dynamic environments in real time and leverage caching schemes to speed up computations. The efficient and accurate simulation of complex environments where multiple sources and microphones interact dynamically remains an active research area.

2.2.3 RIR Simulation Libraries

This section presents an overview of implementations of room acoustics simulation methods. As for simulation techniques, the software landscape is broad. Implementations can be motivated by different reasons. Some serve the only purpose of demonstrating a research idea. Others may be industry-grade libraries integrated into commercial products. This diversity in the state-of-the-art for auralization reflects the numerous directions investigated by the scientific communities in this domain.

The popularity of the ISM makes it the most common implementation approach. First and foremost, the very first computer programs for simulating room acoustics were leveraging ISM techniques. In 2005, Campbell et al. published the popular *RoomSim* MATLAB implementation of the original ISM method proposed by Allen et al. [98]. It aimed to provide researchers with a simple yet efficient method to simulate sound propagation in rectangular rooms. Schimmel et al. [99] iterated on their approach in 2009, implementing the *RoomSim* program in C. This new code was claimed to be faster and thus address the main limitation of the MATLAB version, computational performance. They evaluated their library against the latter and obtained a 40x improvement in terms of speed. Also, they implemented the diffuse rain algorithm to better model the high-order and diffuse reflections. *Pyroomacoustics* [11] is a modern and feature-rich implementation by Scheibler et al. It also relies on the ISM method to compute the RIRs. Their formulation allows for handling arbitrary polyhedral rooms, which were not handled by the classic ISM implementations. The core simulation algorithms are written in C to achieve good performance. Yet, they also provide a user-friendly Python API to make the library easy to learn and use. In addition to its primary RIR computation capability, this project ships various additional features such as beamforming algorithms, direction finding, and adaptive filtering. Rathnayake et al. [106] proposed a 3D room simulation library with an additional *OpenGL* visualization feature. It allows observing all the reflection paths from the source to the receiver. Also, the library permits plotting the RIR graph, similar to [Figure 2.1](#) and [Figure 2.3](#).

The community has also explored using GPUs to speed up the computation of the RIRs. In their 2011 paper, Savioja et al. [107] explored the potential of GPUss for performing several audio signal processing tasks. Significant speedups are identified in various algorithms and computations. For example, computing a two-million-point Fast Fourier Transform (FFT) in real-time has been achieved on a GPU. Furthermore, the accelerator substantially accelerated operations such as time-domain convolution or multichannel finite impulse responses (FIRs). Some early works on room acoustic modeling are mentioned in this article. They rely primarily on wave-based methods and adapt GPU implementation of differential equation solvers. To our knowledge, Fu et al. [108] were the first to parallelize the ISM algorithm on numerous GPU cores. Their performance benchmarks identified a 20 to 120-fold improvement over the CPU implementation. This effort was broadened by Diaz-Guerra et al. with the *gpuRIR* library [12]. This implementation provides performant custom CUDA kernels for parallelizing the ISM technique on the GPU. This open-source implementation, in particular, leverages the mixed precision capabilities of modern GPU accelerators. Also, this method innovates by introducing Look Up Tables (LUTs) to avoid unnecessarily re-running costly trigonometric functions. While not as feature-rich as *Pyroomacoustics*, *gpuRIR* provides easy-to-use Python bindings to the CUDA code which helps embed it in a more complete pipeline. The authors have conducted performance testing and found that the GPU code was 100x faster than alternative CPU implementations. This efficient library is thus efficient enough to be used in real-time applications such as virtual or augmented reality.

Although ISM-based methods represent a large share of available room acoustic simulation implementations, other approaches have also been used. Rosen et al. [91] have employed a wave-based modeling technique in their *planeverb* [92] library. Their approach has been described previously in more detail (see [Section 2.2.2.1](#)). They relaxed the 3D simulation problem to a planar 2D domain, allowing for significant performance gains. This method indeed targets real-time applications and is, therefore, implemented in C++. The authors demonstrate the software's capabilities in challenging, static, and dynamic scenes. They highlight an eventual porting of the code to GPU accelerators as an interesting direction for future work. The BST algorithm [52] by Cao et al. illustrates how ray tracing GA techniques can also achieve good real-world performance. The exhaustive testing of their library seems to demonstrate fast and accurate rendering of complex acoustic scenes. It can also handle dynamic scenes and geometries that are more complex than limited rectangular rooms. Unfortunately, the implementation is not open-source.

Finally, the ecosystem of room acoustic simulation libraries is vibrant. Several alternatives coexist and provide various tradeoffs. Some solutions target the research community and attempt to provide easy-to-use software tools for conducting experiments. Others focus on more integrated scenarios where performance is crucial. In this regard, multiple works investigate adapting classical algorithms to modern GPU hardware accelerators to achieve state-of-the-art simulation speeds.

2.3 Custom Acoustic Simulation Pipeline

The motivation to build a simulator from the ground up was to benefit from a capable yet flexible virtual platform for acoustic-based HRI tasks. Indeed, while existing audio simulation libraries offer a wide range of functionalities, they are generally designed as focused components dedicated to sound propagation, and are often integrated into larger ecosystems such as game engines (see [Section 2.2.3](#)). Their scope is understandably limited to audio rendering rather than interaction modeling. In contrast, the design of a custom, extensible platform for simulating diverse interaction scenarios, with support for varied audio sensors and acoustic representations, constitutes a central contribution of this work. Its goal is to allow the simulation of various interaction scenarios involving a diverse array of audio sensors and acoustic representations.

Also, its implementation has evolved throughout the project, leading to an organic development process. The feature set reflects the various downstream use cases encountered across this thesis. The Python code for this pipeline has been published under an open-source license.

In this section, we provide an overview of our simulator *Acoustix*, its main functionalities, and several use-case examples.

2.3.1 Overview

The acoustic simulator is composed of several blocks that interact with each other. [Figure 2.8](#) offers a global overview of the pipeline architecture.

The core of the platform resides in the *RIR simulation library*. The latter performs the actual computation of the several required RIRs. However, those libraries work at a low level and only account for a given static scene involving different unrelated microphones and sources. Also, not all options handle acoustic signals. Some only provide the RIRs.

Thus, the *room* component offers a higher abstraction for the static acoustic simulation. Its main feature is to leverage the *RIR simulation library* to compute the listened signal of each microphone.

Finally, the *simulator* adds multiple features relating to dynamic scenarios. Also, it operates more advanced audio objects such as multi-microphone arrays and sound sources of different kinds. This block provides our pipeline with the most important external Application Programming Interface (API). It allows for defining a room, an agent (i.e. microphone array) and a set of sound sources with a very limited amount of code. The user can then move the different audio objects with convenient movement primitives and fetch the resulting audio and geometric data in diverse forms.

[Listing 2.1](#) demonstrates a basic example of how our library can be operated. The following section will present the software blocks required to run this type of operation.

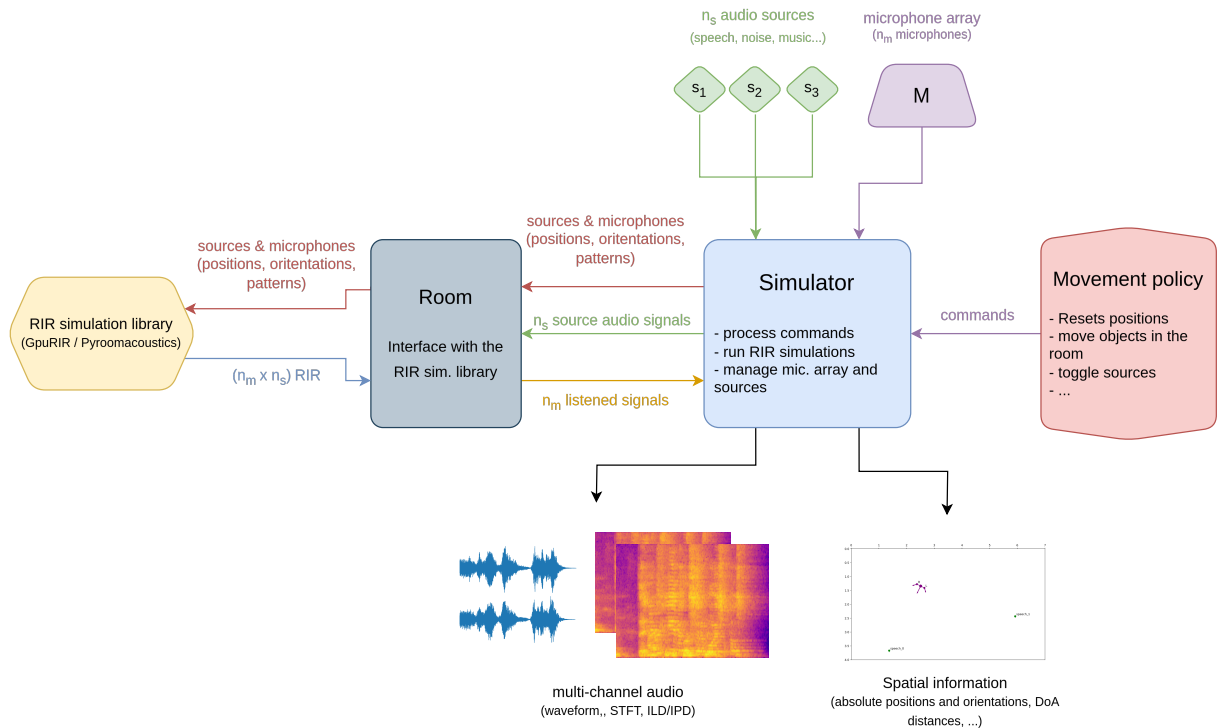


Figure 2.8 – Overview of the simulator architecture. The principal elements involved in the simulator are depicted. The *room* wraps the *RIR simulation library* and renders the acoustic signals. The *simulator* provides an additional abstraction and operates the sources and microphones' movements across time.

```
from acoustix.audio_simulator import AudioSimulator
from acoustix.room import GpuRirRoom
from acoustix.microphone_arrays import SquareArray

# Initialization
room = GpuRirRoom(size_x=4, size_y=7, rt_60=0.3)
mic_array = SquareArray(
    position=np.array([3.0, 3.0, 1.0]),
    orientation=np.array([-1.0, 1.0, 0.0]),
)
audio_simulator = AudioSimulator(room, mic_array, n_speech_sources=3)

# Load speech signals and perform simulation
audio_simulator.step()

# (4, F, T) complex tensor
stft = audio_simulator.get_agent_stft()

# Compute the DoA with respect to the "speech_1" source
doa_source_1 = audio_simulator.get_doa("speech_1")
```

Listing 2.1 – Example of basic usage of the simulator.

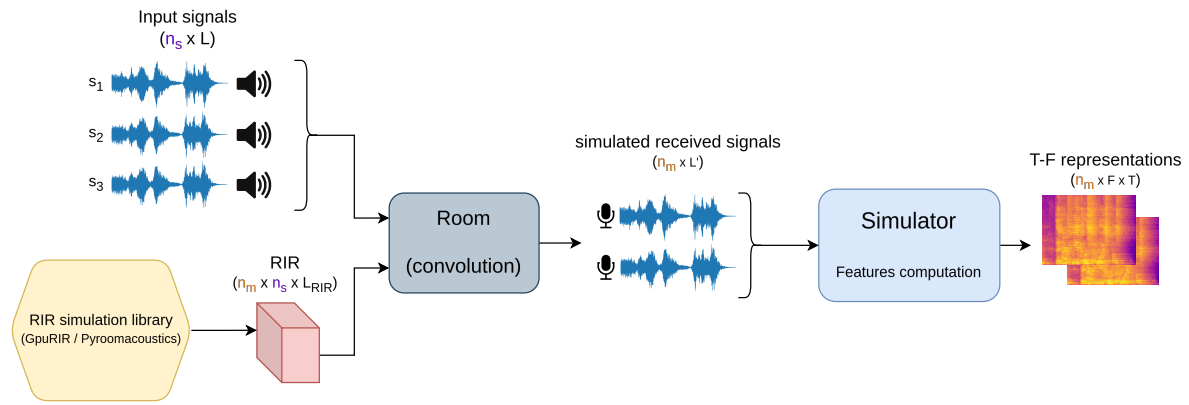


Figure 2.9 – Schematic view of the audio processing pipeline.

2.3.2 Components

In this part, each component briefly introduced in [Section 2.3.1](#) will be detailed and motivated. This offers a more in-depth description of the pipeline’s inner workings.

2.3.2.1 Low Level Static Simulation

First, the acoustic simulation aspect of the library will be explored. This constitutes the starting point of the library and is responsible for its central feature: computing listened signals in a reverberant environment.

RIR Simulation

The core component around which the simulation pipeline revolves is the RIR simulation library. We have chosen the RIR framework for its simplicity and prevalence in the scientific literature. [Section 2.2.2](#) introduced this approach’s main concepts and fundamental aspects.

[Figure 2.9](#) illustrates how the audio processing occurs within the pipeline. The different steps of this procedure will be detailed in the following sections. The role of the RIR simulation library consists in inferring, given the localization and properties of a set of sound sources and receivers (microphones), the pairwise RIRs.

Choice of the back-end library. Developing such a library was out of the scope of this work. Members of the acoustic simulation community have conducted several notable efforts. Multiple existing libraries have been enumerated in [Section 2.2.3](#). Motivated by a seamless operation within our Python code base, we have focused on the libraries providing bindings for this language. Two libraries appeared mature and able to satisfy the requirements of the pipeline. *Pyroomacoustics* [11] implements the ISM and adds numerous advanced features (see [Section 2.2.3](#) for more details). Its design has inspired the architecture of our pipeline, most specifically its own Room object. *Pyroomacoustics* is the original back-end library that we have been using for sound propagation simulation. As we envisioned more intensive workloads where processing time could limit the simulator’s usability, the recent *gpuRIR* implementation by Diaz-Guerra et al. [12] has been integrated. As seen in [Section 2.2.3](#), the authors of this library have focused on achieving the best performance in RIR estimation. Our solution offers

its users the choice between those two libraries for the backbone of the simulation. Allowing this flexibility has required designing the Room module in an abstract manner. This would allow for using other RIR simulators while keeping the rest of the pipeline working accordingly.

To operate those libraries, the positions, patterns and orientations (when appropriate) of each source and microphone are provided. Also, the room's dimensions and acoustic properties belong to the necessary parameters. Both libraries offer a way to compute the wall's reflection coefficients from a target T_{60} value by leveraging the inverse-Sabine estimator for the reverberation time (see Equation (2.10)). The image source order in the i -th axis N_i counts the number of virtual rooms that must be considered in this direction. Those reflected rooms, stacked up, form a diamond with the real room in its center. One way to approximate N_i is to measure the distance that sound can travel in T_{60} seconds:

$$N_i = \left\lceil \frac{2T_{60} \times c}{L_i} \right\rceil, \quad (2.27)$$

where L_i is the room's length in the i -th direction. All reflections happening closer than $T_{60} \times c$ meters *GpuRIR* computes three distinct number of images, according to Equation (2.27).

On the other hand, *Pyroomacoustics* settles for a single maximum order $N_{\max}$:

$$N_{\max} = \left\lceil \frac{T_{60} \times c}{R_{\min}} - 1 \right\rceil, \quad (2.28)$$

where

$$R_{\min} = \min_{\substack{i,j \in \llbracket 1,3 \rrbracket \\ i \neq j}} \frac{L_i \times L_j}{\sqrt{L_i^2 + L_j^2}} \quad (2.29)$$

is the radius of the largest sphere fitting in the diamond of virtual rooms. The higher the image source order, the more detailed the simulation will be.

Finally, a sampling frequency must be provided. This parameter affects the computed RIR's time resolution and, thus, its ability to capture high-frequency phenomena. A higher sampling frequency will increase the computational cost of the simulation. In most cases, using the same frequency as that of the audio signals involved is a safe and practical choice.

Room, Static Acoustic Simulation

The *Room* module is a wrapper around the RIR simulation library. It brings some additional features, among which is the computation of listened signals. Although originally inspired from *Pyroomacoustics* [11], our *Room* module has been grown for improved interoperability with the rest of our pipeline.

We consider a *shoebox* room defined by its dimensions (L_x , L_y and L_z) as well as the desired reflection time T_{60} and sampling frequency f_s . Individual point-wise sources and microphones can then be positioned in the 3D space. Each change in the audio objects' localization leads

to a new simulation that updates the $(n_m \times n_s \times L_{\text{RIR}})$ RIR tensor. Our implementation ensures the caching of the RIR and solely re-computes it when necessary. As stated in the previous [Section 2.3.2.1](#), the Room can use both the *Pyroomacoustics* and *gpuRIR* back-ends interchangeably.

To actually compute the signal received at each microphone, the input signal from all active sources is gathered. Then, the listened signals are estimated by convolving the sources' signals with the corresponding RIR vector as described in [Equation \(2.7\)](#).

Additionally, the *Room* module provides a convenient interface for dynamically adding and positioning both sources and microphones. It ensures the validity of all locations at any time. The different audio objects have the ability to be pinned to a grid of arbitrary resolution. Such a feature permits, for instance, the ability to effortlessly compute the listened signal at all possible positions in the room. Contrarily, one could fix the microphone position and collect the received audio for evenly distributed source localizations.

Lastly, a plotting helper has been implemented to handily visualize the state of the room and its content.

2.3.2.2 Ergonomic Simulation of Complex Scenarios

Although our *room* abstraction extends the capabilities of the core RIR simulation library, it still lacks abstraction power to allow for conveniently experimenting complex dynamic scenarios. Providing this user experience required introducing more powerful objects.

Sound Sources

The core motivation for developing this acoustic pipeline was experimenting with HRI scenarios. In this sense, the most important type of sound source to consider was speech sources, mimicking humans speaking in the room. However, other kinds of sources have also been implemented, such as music sources or white noise sources.

All sources have a position in the room, a polar pattern (see [Figure 3.3](#)) that affect their directivity and most importantly the ability to generate an audio signal. This latter property differs across source types.

For white noise sources, the waveform is randomly generated from a standard normal distribution. To adjust the gain relevantly, some speech signal as well as a target Signal to Noise Ratio (SNR) value can be provided. This allows for conducting experiments in the presence of adversarial noise sources and controlling precisely the SNR parameter.

Speech sources pull their signal from the *LibriSpeech* [81] dataset. The latter is an ASR corpus of 1000 hours worth of audiobooks, sampled at 16 kHz. Each time a speech source is required to produce a signal, a random sample is pulled from *LibriSpeech* and output.

Microphone Arrays

Microphone arrays provide a convenient abstraction to use pre-defined multiple microphone arrays in the environment as well as defining custom ones.

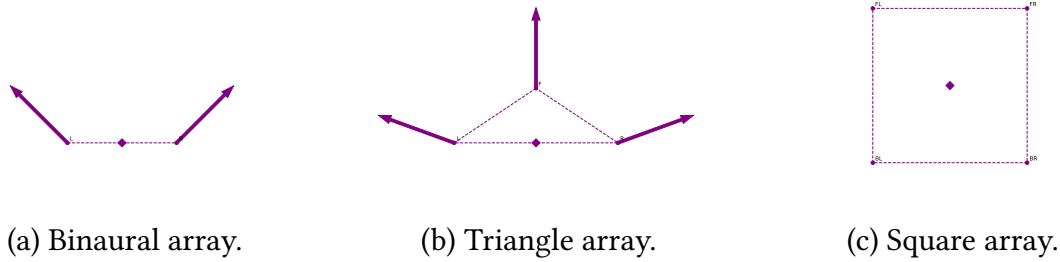


Figure 2.10 – Examples of microphone arrays available in the simulator. All dimensions and angle values of the displayed geometries can be customized.

They offer an upgrade from the limited Room API, which only considers individual microphones independently. Each array variation defines its geometry and the properties of each of its microphones. Grouping microphones as such makes the process of experimenting with different array configurations fast and convenient for the user. The simulator can access the array’s position, orientation and footprint, but more importantly can move it as a single entity, without the need for considering each of its microphones. The array’s position and orientation are expressed in the room’s reference frame. The array maintains its own geometrical consistency when being translated or rotated.

Figure 2.10 illustrates some currently included arrays. They all offer a degree of configurability. The pattern of the microphones can be adjusted, especially. Also, the relative distances and orientations of the microphones are alterable.

The arrows depict each microphone’s orientation. The position of the array, although virtual, is represented by a diamond (◆). Naturally, a single-microphone array is also provided.

Users of this library can easily implement their own microphone array and benefit from all the simulator’s features.

The Simulator Interface

The simulator constitutes the centerpiece of the interactive pipeline. It serves as an engine coordinating all the components mentioned above.

At a high level, it serves as an orchestrator of the overall simulation process. The simulator manages both the microphone agent, also referred to as the *agent*, and the various sound sources (speech and non-speech).

Although the room, representing the spatial and acoustic properties of the environment, and the microphone array have to be provided to the simulator at initialization, the latter can take care of creating the different sound sources. Once configured, the simulator can be interacted with. This process takes place in discrete time steps that each resemble the following execution:

- The audio objects (agent and sources) can first be relocated.

- The acoustic simulation is then performed by the Room module, itself using the RIR simulation library. The simulator ensures to propagate the positions and orientations of all elements as well as setting the right input signal of each source.
- Finally, acoustic features can be collected in multiple representations: raw multi-channel waveforms, STFT or ILD/IPD. The user can also access spatial data in convenient formats: distance from the microphone to a specific source, DoA, absolute position and orientation of the agent and sources, etc.

Spatial domain

The sound propagation libraries used in the simulation model three-dimensional scenes. Thus, we have built the rest of the pipeline to allow for full control of audio objects in the 3D space. Besides, as this thesis's downstream tasks predominantly involves planar problems, most of the implemented features focus on 2D movements and spatial measures. No artificial limitation prevents the use of our library for 3D problems.

Sources and Microphones Movement

Audio objects are of two kinds: sound sources and microphone arrays. Sound sources are modeled by a point in space with an orientation. Thus, a position and direction vector suffice to localize a source unambiguously. Microphone arrays are constituted of possibly several microphones. To ease their manipulation, the array provides an abstraction allowing the definition of a single position and orientation for the whole array. Also, the array object itself can be *moved* by the simulator while internally ensuring consistency between its microphone positions.

To enable modeling flexible interactive scenarios, the simulator provides a set of various movement primitives. The positions of all objects can be randomized to start from an arbitrary setup. Everything is placed so that all sources and microphones are within the borders of the room.

Apart from the random positioning of objects, the interface allows the agent to perform relative motions. Basic movements, such as moving forward by a given distance or rotating left or right by 90°, are relevant in the context of RL environments with a discrete action space. Besides, the `move_agent_polar(angle, distance)` method gives more control to perform relative movements. Finally, the user can place the microphone array and the sources at any arbitrary position.

An additional benefit of operating motions through the simulator is that it guarantees the correctness of positions at all times. If an arrival position lies at the room's exterior, an exception is raised, and the movement is not performed.

In conclusion, the simulator furnishes a convenient and safe interface for moving both the microphone arrays and the sound sources in the room. This facilitates the flexible implementation of numerous acoustic HRI use cases.

Simulation Process

Most downstream tasks leveraging the simulator involve iterations through a discrete time step simulation. More precisely, the typical workflow when using the simulator includes an initialization phase, during which the *Room*, microphone array, and *AudioSimulator* are created. [Listing 2.1](#) gives an overview of the corresponding code snippet.

Once the different components have been set up, the actual simulation process can start. The procedure involves positioning all audio objects in the room. This can be done directly by the simulator in a random way. Subsequently, each source is individually asked to provide a new sound sample (see [Section 2.3.2.2](#)). The actual sound propagation simulation can then happen and the embedded *Room* module returns the multi-channel audio signal received. Either the raw waveform or further processed time-frequency features can be produced. The user can also listen to the produced signal directly. In practice, those steps are abstracted and automated by the `step()` method. Lastly, the agent can be moved using the exposed displacement helpers presented in [Section 2.3.2.2](#). [Figure 2.11](#) illustrates this routine.

Duration control. By default, given n_s input signals of durations $(d_s^1, \dots, d_s^{n_s})$, the received signal at each microphone will last

$$d_{\text{rec}} = \left(\max_{i=1 \dots n_s} d_s^i \right) + T_{60} \quad (2.30)$$

seconds. After d_{rec} seconds, the energy of the received signal becomes negligible. In practice, the simulator allows for artificially reducing the time of the simulation. This may happen by first shortening the input signals to a given duration d_s^{lim} , thus leading to having $d_{\text{rec}} = d_s^{\text{lim}} + T_{60}$. Alternatively, the resulting audio can be trimmed to any desired duration. The duration control feature gives a fine-grained control of the computational time.

Feature Extraction

Observing the simulator's state represents an essential feature set of our library. Potential downstream usages may require different kinds of monitoring.

Spatial information. First, because the user does not always dictate the absolute position of each element, gathering information about the current layout can be useful. The raw positions of both the agent and the sources are directly accessible. Also, relative data such as the DoA or the distance from the agent to a given source can be extracted. This egocentric information can be used in multiple situations, such as training an RL agent or collecting a SSL dataset.

Audio data. Recording acoustic information is the main purpose of the simulator. Following the same principles of flexibility and convenience, audio data has been made accessible in various formats at different stages of the simulation. [Figure 2.9](#) outlines those steps visually. First and foremost, each source exposes its current raw signal. This is how the simulator fetches the audio signals from the different sources before forwarding them to the RIR simulation library. Once the simulation has been conducted, both the RIRs and the audio signal received at each

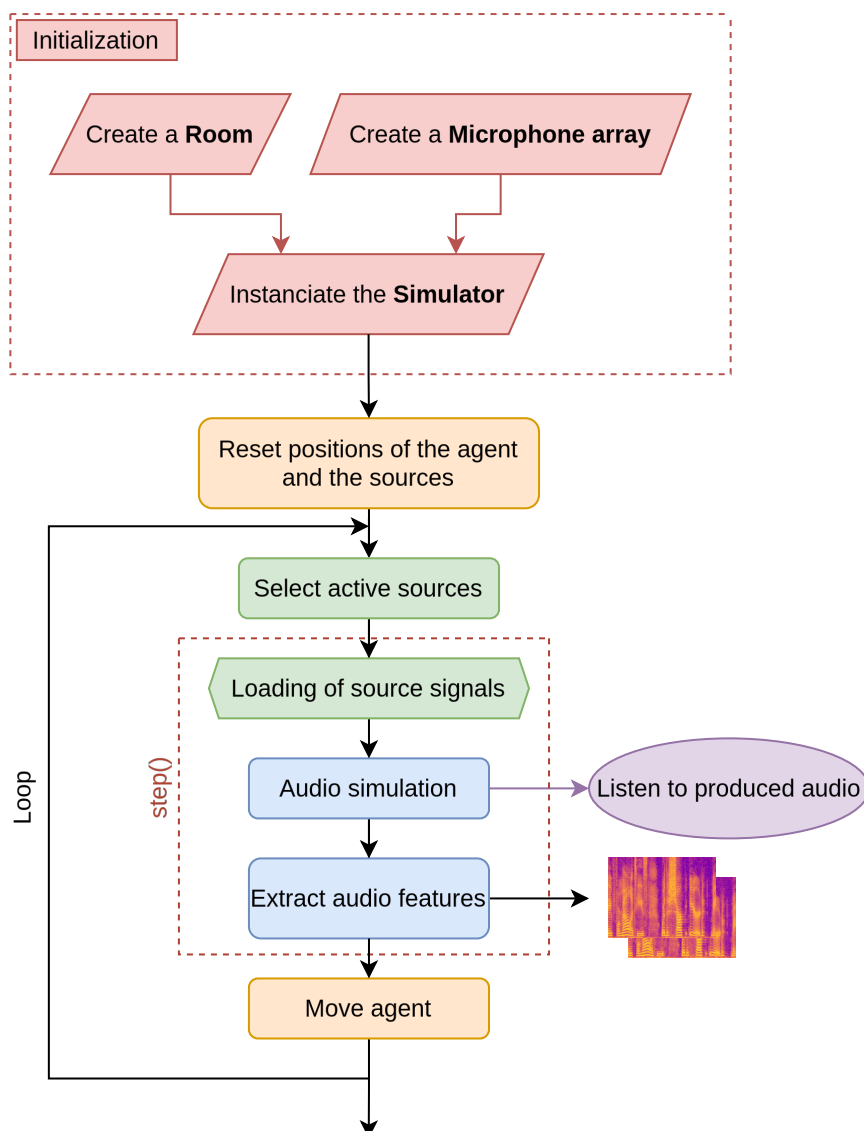


Figure 2.11 – This figure illustrates the typical simulator execution flow. After an initialization phase, the simulator proceeds in discrete steps, during which microphones and sources may move.

microphone become accessible. In addition to the raw multi-channel listened audio provided by the Room module, the simulator proposes further audio processing tools. Thus, the received acoustic data's STFT can be computed and recovered for direct use in a neural network or any method operating in the time-frequency plane. Finally, another function has been added to calculate the signal's ILD and IPD given a pair of microphones.

All those features partly remove the downstream user's post-processing burden, allowing for the most direct and practical usage possible.

Visualization

For development purposes, it may come in handy to graphically render the simulator's state. The pipeline includes a basic yet efficient way of visualizing the different objects in the room. Each available microphone array can be shown in the room, according to its geometry. The

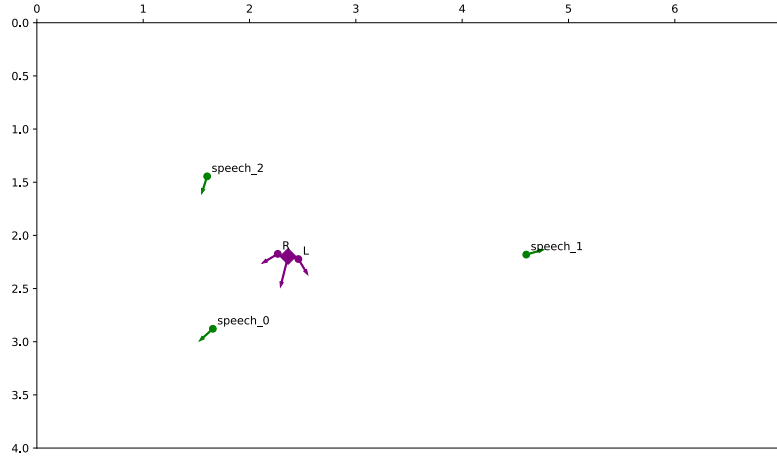


Figure 2.12 – Screenshot of the simulator renderer.

orientation of microphones and directional sources is also displayed. Figure 2.12 provides a demonstration of the renderer.

2.3.3 Simulation of Dynamic Scenarios

The regular `step()` method of the simulator, presented in Section 2.3.2 allows to update the simulator and to generate new received signals. At each call of this function, each audio source reloads an entirely new signal and the simulation is performed subsequently. When asked to refresh their signal, regular speech sources draw a distinct sample from the clean speech corpus and output the corresponding signal. The latter usually consists of a complete sentence pronounced by a speaker and lasts several seconds. The duration of the resulting simulated audio d_{rec} will differ from step to step. This process suits use cases where each *step* is independent of the others.

Continuous sources. To allow for more realistic modeling, we introduce continuous speech sources. The simulation still happens in discrete steps during which all audio objects remain static. However, each segment now has a short, fixed, and pre-determined duration d_{step} . Each active speech source in the room will progressively deliver the speech sentences from the corpus in chunks of approximately d_{step} seconds. Once a sentence has been exhausted, the source automatically loads a new one and starts yielding it.

Bootstrapping reverberation. Although the simulator exposes this abstracted pseudo-continuous interaction framework, the audio propagation is performed independently at each step. Consequently, as presented in Section 2.2.1.2, the early part of the received audio correspond to the direct path the source signal uses, and the reverberation’s effect remains invisible. When considering short steps, where d_{step} might be even lower than T_{60} , naively extracting and returning the first d_{step} seconds of the simulation would completely reduce

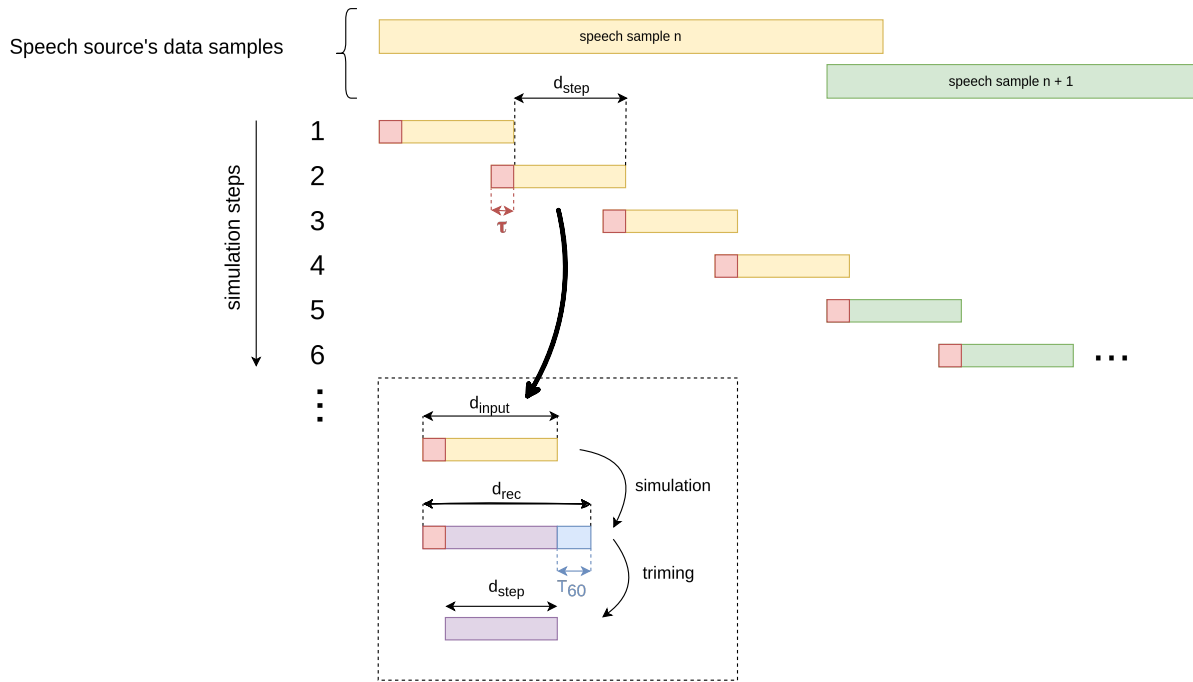


Figure 2.13 – Illustration of the pseudo-continuous simulation process. It renders the signal recorded by each sensor across time, accounting for the movement of sources and microphones. Reverberation bootstrapping ensures late reverberations are included in the short rendered audio chunks.

the consequences of reverberation. As such, each source loads longer audio chunks of $d_{input} = \tau + d_{step}$ seconds, leading to simulated received signals of $d_{rec} = \tau + d_{step} + T_{60}$. The reverberation time T_{60} is a sane default value for τ . Finally, the simulator trims the first τ and last T_{60} seconds from the simulated signal to return the requested d_{step} . The overall process is depicted in Figure 2.13.

Ultimately, the simulator can model dynamic interaction scenarios leveraging a static RIR simulation library. Thanks to our bootstrapping technique for reverberation, one may employ short simulation steps between which all objects in the room can be moved freely. RL environments represent discrete-time phenomena with Markov Decision Processes (MDPs) and figure as possible use cases for this framework.

2.3.4 Benchmarking and Performance Analysis

This section provides a short breakdown of the computation load of the simulator. As seen previously, a typical simulation workflow consists of the following two phases:

- Generation of the RIRs using the ISM-simulation library,
- Computation of the listened signal by convolving the source signals with the computed RIRs.

These two steps account for the majority of the overall computation time. Both are affected by the number of sources and microphones.

On the one hand, the simulation library handles RIR computation. It is provided with the room properties and the positions of each source and microphone. The result of this step

$(n_m, n_s, L_{\text{RIR}})$ -shape array containing the RIRs for each source-microphone pair. The performance of the ISM algorithm's implementation itself is independent of our control. It plays a significant role in the overall simulator performance.

On the other hand, once the RIRs have been generated for a given geometrical configuration, the simulator convolves each input signal with the corresponding RIR for a given microphone. The obtained results are then summed according to Equation (2.7). Hence, this step requires computing $n_m \times n_s$ convolutions. The time complexity of each convolution depends on the length of the RIR (L_{RIR}). To speed up this step, we perform the convolution in the Fourier domain instead of in the time domain (Section 8.7 of Oppenheim et al. [59]). This leverages the aforementioned convolution theorem (Equation (2.17)) and adapts Equation (2.4) as follows:

$$\begin{aligned} x[n] &= (h_r * s)[n] \\ &= \mathcal{F}^{-1}[\mathcal{F}(h_r) \times \mathcal{F}(s)]. \end{aligned} \quad (2.31)$$

In the Fourier domain, the convolution turns into a simple product and is thus less computationally expensive. The cost of computing the forward and inverse Fourier transforms is worth it overall, especially for longer signals. The complexity of the naive convolution, operated in the time domain, is $O(L_s \times L_{\text{RIR}})$ where L_s and L_{RIR} are the respective lengths of the clean speech signal and the RIR. By contrast, the total complexity of the FFT-convolution approach is $O(L \log(L))$.

We experiment with a binaural array and four sources randomly placed in a room. The reverberation time T_{60} is set to 500ms. The simulator renders the multichannel audio signal received by each microphone. These signals result from the sound emitted by the four sources and the reverberation effects encoded in the RIR. For 100 steps, the sources will load a new speech sample from the *LibriSpeech* corpus [81]. Each sentence is trimmed to last 6 seconds to enhance reproducibility. This experiment is repeated for the two supported RIR back-ends. Apart from which library performs the acoustic simulation, every parameter remains the same. We profile the simulator's execution in both cases to study the relative importance of the different computation steps. The *py-spy* [109] and *vprof* [110] utilities are employed to perform this profiling.

Figure 2.14 displays the resulting flame graphs. They illustrate the relative amount of time spent in each part of the code. We restricted the view to the `step` function, which hosts the simulation code of interest. This excludes the time spent initializing modules and libraries and loading the dataset. These representations do not display the difference in absolute time between the two runs but only how time is spent in the different parts of the process. To account for this, Table 2.1 gives the absolute durations involved. T_{sim} measures the total time spent in the `step` function. We notice that *gpuRIR* can compute the 100 sets of 8 RIRs considerably faster than *Pyroomacoustics*. The RIR simulation took 3.69 and 109 seconds, respectively (denoted as t_{RIR}). The flame graph clearly shows that this step is the principal bottleneck when running the simulator with the *Pyroomacoustics*. Conversely, when using *gpuRIR*, t_{RIR} becomes negligible compared to the other computation steps involved. t_{FFT} denotes the duration of the 8 convolutions between the 6s long clean source signals and the generated

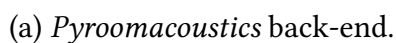


Figure 2.14 – Profiling results for both available RIR simulation back-ends.

Table 2.1 – Relative performance of the simulator back-end RIR libraries.

	T_{sim} (s, (%))	t_{RIR} (s, (%))	t_{FFT} (s, (%))	t_{STFT} (s, (%))
Pyroomacoustics	124 (100%)	109 (87.9%)	11.62 (9.37%)	2.28 (1.84%)
gpuRIR	21.69 (100%)	3.69 (17.01%)	14.56 (67.1%)	2.41 (11.1%)

RIRs. t_{STFT} measures the time taken by the computation of the STFT from the generated listened signals.

This simple experiment highlights how significantly the choice of each component can matter in the overall simulator performance. Our framework’s ability to support both *Pyroomacoustics* and *gpuRIR* back-ends grants it great flexibility. The high performance of the *gpuRIR* back-end allowed us to generate considerable datasets, which have been key to our subsequent research efforts in localization and navigation. In future works, it could be relevant to explore other strategies for speeding up the convolution step by operating the FFT-convolution operation on the GPU.

2.3.5 Conclusion

This section has presented our custom implementation of a complex acoustic pipeline. The solution has been designed from scratch while naturally incorporating existing open-source building blocks. This significant engineering effort is a central contribution of this thesis. It aims to allow researchers to test algorithms in diverse acoustic scenarios. The API has been optimized to be as complete as possible while remaining user-friendly. The code base is entirely written in Python to be directly integrated into new or existing research code bases. Nevertheless, performance has been considered, and efficient specialized libraries have been used for the key computational features.

While the library has been enhanced and completed throughout the research project, it could be extended. New features could later be added thanks to a very modular architecture. Most notably, more visualization features could be added. For instance, the current plotting features are based on the *Matplotlib* library [111] and remain limited. They only offer a rudimentary step-by-step preview of the room. Ultimately, the simulator could offer a more pleasant dynamic visualization of the movement of sources and sensors. Furthermore, the handling of dynamic scenes follows a simple step-based approach that discretizes the movement of objects over time. The addition of modern dynamic simulation techniques ([91], [52], [102]) could make this pipeline even more relevant to robotics. Finally, the simulator’s scope has been delimited to auditory cues and sound processing. The incorporation of a 3D visual rendering engine could transform the existing platform into a true multi-modal platform for a more complete simulation. Such an endeavor would, however, require a significant additional engineering effort. The entire code base of *Acoustix* is freely available under the GPLv3 license¹.

2.4 Conclusion

This chapter lays the foundation of the acoustics paradigm leveraged in this thesis. It introduced key theoretical notions in acoustics, signal processing, and simulation. This framework is essential to the experimental scientific work conducted in the rest of this project. After presenting the relevant background for understanding how sound propagates in reverberant

¹<https://github.com/GaetanLepage/acoustix>

environments, we gave an overview of the existing techniques used to model this paradigm. The literature comprises diverse methods that address this problem with various tradeoffs. In this regard, this chapter attempts to bridge the gap between the physical phenomenon of sound propagation and effective computational solutions for simulating it. Next, a survey of the available software implementations was conducted, highlighting the efforts of a vibrant research community.

Finally, we presented this thesis's first contribution: a complex and feature-rich pipeline for simulating diverse acoustic scenarios. We focused on developing a user-friendly toolbox, specifically targeting researchers. It aims to be comprehensive and complete while allowing for a user-friendly experience. This significant engineering effort resulted in the experimental setup for developing and testing learning algorithms with flexibility. As highlighted in the introduction of this chapter, simulation has proven to be greatly useful in science, and especially in the fields of robotics and statistical learning.

Chapter 3

Sound Source Localization

Auditory perception is a fundamental aspect of social robotics. In their interactions, humans and robots convey information through speech for effective and natural communication. As such, a robot must, *inter alia*, be able to listen and transcribe human speech. This is achieved by ASR systems. Besides simply hearing, a robot needs to be able to localize the active speaker. SSL is a central challenge in speech processing. Although several formulations of this problem coexist, its essence consists in inferring the position of one or more active sources solely from the audio signal received by a microphone array. Our interest in exploring SSL is two-fold. First, we aim to study how robots achieve this spatial aspect of auditory perception. Our second motivation is to design effective models to investigate more specific tasks later. Hence, along with [Chapter 2](#), this chapter will form the foundation for our following contributions.

First, this chapter provides an overview of the state-of-the-art in SSL. It depicts the main trends of research in this domain. Diverse and advanced approaches have successfully been applied to this challenging acoustic problem. Also, we discuss the relevance of the SSL task for the robotics research area and highlight essential works made in this field. We then present our methodological and experimental work on this topic. Single-source and multi-source statements of the localization problem have been explored in distinct ways. The proposed solutions have been tested and implemented in the simulator introduced in the previous chapter.

3.1 Background

3.1.1 Original Problem

Sound Source Localization (SSL) is part of the classic challenges in artificial audio processing. This challenge requires identifying the relative position of one or several sound sources leveraging an audition device, typically a microphone array. Localizing sound sources is fundamental and can serve many purposes across various fields.

Although initially targeted to replicate human-like auditory perception, it has progressively found more diverse applications. For instance, SSL enables hands-free computer-human interaction, improving teleconferencing systems and advancing autonomous vehicles. Overall, the ability to localize sound enriches machines' capabilities. For instance, in robotics, SSL aids navigation, situational awareness, and human-robot interaction by directing robots to

key auditory cues in their environment. Similarly, it helps amplify sounds from a specific direction in hearing aids, improving the user's experience in noisy environments. SSL plays a vital role in speech enhancement and recognition. It enhances speech intelligibility in noisy environments by isolating and amplifying signals from specific directions. This is critical in applications like teleconferencing and hearing aids [112]. In entertainment and Augmented Reality (AR), SSL contributes to the user's immersion by dynamically adjusting the audio environment based on the user's spatial orientation and movement [113], [114]. In security systems, it can help identify the origin of specific sound events. The detection and localization of such suspicious sounds can be achieved by combining audio and visual information [115]. This allows for more robust detection and tracking of sound-emitting targets. Interestingly, SSL is also used in autonomous vehicle systems. It facilitates environmental awareness and safety in those complex environments. Here, too, leveraging audio, video, and information for radar sensors improves autonomous vehicles' overall capabilities to detect important external elements quickly. Sun et al. [116] have demonstrated the effectiveness of using microphone arrays and machine learning algorithms to accurately localize such sounds, thereby improving the vehicle's decision-making capabilities. They specifically focus on the detection and localization of emergency vehicles. Using audio enables the gathering of crucial early spatial information.

The research on SSL has evolved significantly, starting with classical signal processing methods. Most of these techniques rely on geometric and physical principles, such as TDoA or beamforming. While such methods have contributed to significant progress, they remain limited in complex and noisy real-world environments. The multiplicity of sources is another obstacle to traditional SSL methods. As in several similar application fields, the rise of deep learning has quickly shown impressive results. Modern, data-driven approaches began leveraging neural networks to model complex acoustic environments, outperforming traditional methods in robustness and accuracy. Recent advances focus on integrating SSL with multi-modal systems, such as robotics and autonomous vehicles, to achieve real-time localization in dynamic and complex settings. Nevertheless, the fundamental concepts at the base of the more classical approaches remain considerably relevant today. Indeed, they can help design feature extractors or pre-processing techniques to further boost the performance of deep-learning-based localizers.

This introduction explores the evolution of SSL, starting with classical signal-processing approaches and their foundational principles. It then transitions to deep learning methods, highlighting key advances, architectures, and datasets. Finally, the focus shifts to SSL in robotics, where unique challenges and application-specific solutions are discussed, demonstrating the technology's real-world impact. Notably, numerous dedicated researchers have explored this research area over several years, publishing thousands of academic articles. Hence, this introduction does not aim to provide an exhaustive survey of the many methods and approaches addressing SSL. Conversely, our objective here is to give an overview of the field and share its significant trends. Alongside the multiple references cited in the following paragraphs, one may refer to Grumiaux et al. [69] for a more detailed picture of SSL at the age of deep learning. Notably, the active localization techniques dealing with moving sources and/or microphones will be presented in [Chapter 4](#) in addition to our contribution to the topic.

Before providing details on the various methods proposed to address SSL, we devote the following section to describing the variability in problem formulation and experimental settings. Chronologically, the first steps were developed in simple settings, and as progress was made, scenarios of increasing complexity were proposed. However, the next section is not structured chronologically but by properties (simulated vs real data, single vs multi-source) for clarity.

3.1.2 Variations in the Sound Source Localization Task

Localizing sound sources is a vague objective, and the exact formulation of the problem varies broadly across the literature.

From simulated to real-world environments First and foremost, the targeted acoustic environment can be of different natures. Some methods are solely tested in a simulation where the fidelity to the real world is inherently imperfect. As motivated in [Chapter 2](#), simulation provides several benefits. In particular, it offers a cost-effective data source that is particularly relevant when using data-based approaches. Naturally, the ability for a method to be applied in real-world conditions is more appealing. Conversely, techniques only tested in a simulator will most likely perform subpar when deployed in the physical world.

Single or multiple sources Furthermore, some works aim to handle multiple sources, while others are limited to a single one. This distinction significantly influences the complexity of the task, the choice of algorithms, and the expected performance. Single-source localization is the most straightforward setting, assuming that precisely one sound source is active. It served as the foundation for many initial developments in the field. Thanks to its simplified formulation, single-source localization allows for robust and performant trackers that can achieve high accuracy in ideal conditions. Those solutions can leverage the single-source assumption to provide a unique source position estimate. On the other hand, handling multiple sources inherently makes the localization problem more difficult. It addresses the challenging scenarios where several sources might be active simultaneously. This introduces additional complexities, such as separating overlapping signals, distinguishing sources from one another, and determining the number of active sources. Methods in this category often employ advanced signal processing or deep learning techniques to handle these challenges, relying on spatial and spectral differences between sources. Multi-source localization is crucial for speech separation, auditory scene analysis, and collaborative robotics applications. Grumiaux et al. [69] insist on the difficulty of the multi-source problem. They also highlight the connections of multi-source localization with other acoustic problems such as speaker diarization (determining *who speaks when*) or speech source separation [117]. Indeed, those tasks are complementary as they can share information to enhance each one's results. Transitioning from single-source to multi-source localization encompasses several challenges. First, the overlapping signals can create ambiguities in determining which spectral features belong to which source. Second, as the number of sources increases, the method's performance and/or algorithmic complexity can be difficult to scale. For instance, the impact of noise and reverberation increases as the number of sources grows [118], [119]. Finally, estimating the number of sources may also be complex. Some methods assume this number to be known as a working hypothesis. Others develop tools to estimate how many sources are active at a given time. In

this context, the multi-source localization problem is not merely a straightforward regression task; somewhat, it resembles a hybrid challenge that combines detection and localization. Even for single-source methods, determining whether the source is active is a problem of its own. Some methods explicitly couple a Voice Activity Detector (VAD) system with their localization algorithm [120], [121]. This chapter will study both single-source and multi-source techniques for SSL.

Angular or absolute localization The actual result of an SSL method is another example of variations of this task. Indeed, while specific techniques estimate the relative position of each source from the microphone array, some focus on the sole DoA value. In their survey [69], Grumiaux et al. exhaustively illustrate the variety of choices regarding the end result of the localization pipeline. This distinction originates in the significant difficulty gap implied by predicting the source-microphone distance. The time, phase, or intensity differences can be exploited when several microphones are used to perform localization. They allow for the deduction of angular information on source localization as they are closely related to the DoA. Conversely, distance information is considerably more complex to gather. It is not directly correlated to a single cue. It heavily depends on the acoustic properties of the environment, such as noise, reverberation, or even temperature. Also, acoustic cues such as amplitude have a non-linear relationship with the distance and are thus harder to exploit. Distance estimation may furthermore need to rely on calibration, which hinders the flexibility and relevance of the SSL system. Moreover, when considering angular-only SSL systems, it is necessary to distinguish planar detectors, which predict the sole DoA value, from 3D ones, which also estimate the elevation angle. Although the DoA is the most informative angle, elevation becomes relevant in more complex scenes or when there is a significant height difference between the microphone array and the sources. The detection format also varies across the literature, with methods producing Cartesian coordinates and others expressing the positions in polar or spherical coordinates.

Intermittency and nature of the sources SSL solutions make different assumptions about the target sources. First, the source signals' content significantly affects the difficulty of the task. Initially, several approaches were restricted to localizing white-noise sources. Those have the particularity of having the same energy at each time and each frequency. Nevertheless, detecting more realistic speech sources has become the de facto framework for SSL. Some works still use white-noise data for training deep neural networks, but their solution is evaluated using speech signals [122], [123]. Similarly, assumptions about the continuity of the sources can vary across the literature. Dealing with intermittent sources is another difficulty that several approaches do not consider. In this regard, handling sources that may become inactive is linked with the capacity to localize a variable number of sources. Naturally, assuming a pre-defined number of constantly active sources is easier and more common across the SSL literature.

Event localization and detection The Sound Event Localization and Detection (SELD) task is an extension of the SSL problem. It involves jointly performing Sound Event Detection (SED) and localizing the detected events. This research problem has several real-world applications in surveillance systems, robot hearing, and environmental monitoring. Hence, the literature on SELD methods is highly relevant for localization. The Detection and Classification of

Acoustic Scenes and Events (DCASE) challenge, held annually, proposes various tasks ranging from pure localization to the more complex SELD. Kapka et al. [124], Phan et al. [125], [126], Adavane et al. [127] are examples of DCASE submissions for the SELD task.

Those fundamental differences are not the only ones. They highlight the diversity of existing approaches and show that the difficulty of SSL can vary significantly. Most importantly, we have seen that the SSL task does not have a unique and clear definition. It refers to a complex problem that can be tackled with various nuances and complexities.

3.1.3 Classical Approaches

SSL has long been a central problem in auditory processing, and foundational methods have emerged from signal processing principles. These classical approaches primarily relied on analyzing multichannel audio data to extract spatial information about sound sources, leveraging the physics of wave propagation [69].

TDoA-based methods are among the most widely used classical techniques for SSL. Estimating the time delay between sound arrivals at different microphones can infer the source's spatial position. A more detailed overview of the geometrical aspect of sound propagation has been given in [Section 2.2.1.5](#). The GCC-PHAT is a prominent algorithm in this category, known for its robustness in noisy environments [79]. The seminal work by Knapp et al. [79] describes the GCC framework and explores the application of the Phase Transform (PHAT) weighting to improve time-delay estimation in noisy and reverberant environments. GCC-PHAT remains a cornerstone technique in signal processing for sound source localization [74]. Alameda et al. [68] developed a geometric formulation of the localization problem along with an optimization algorithm. Their approach illustrates the relevance of time delays as powerful features to perform SSL. However, reverberation and closely spaced microphones often degrade TDoA accuracy. Also, a well-known issue of binaural systems is the front-back ambiguity [128]. It directly comes from the symmetry of such setups in the sagittal plane. A sound arriving from a direction in front of the listener produces similar Interaural Time Difference (ITD)s and ILDs as a sound from behind the listener. For sounds coming from the front and back, the paths to the left and right microphones (or ears) differ by the same amount of time, but from opposite directions. This symmetry means that the TDoA measurement alone cannot distinguish whether the source is in front or behind.

Beamforming approaches focus on spatial filtering by steering the microphone array to maximize the energy from a specific direction. Methods such as the Steered Response Power with Phase Transform (SRP-PHAT) are commonly employed to construct acoustic energy maps, identifying source directions as peaks in the map. Van Veen et al. [129] provide a comprehensive overview of beamforming techniques, discussing their applications in spatial filtering and signal enhancement. The book by Brandstein et al. [130] explores microphone array signal processing more in-depth. While effective in simple environments, beamforming techniques struggle with real-world conditions involving diffuse noise and multiple overlapping sources.

Subspace methods such as Multiple emitter location and signal parameter estimation (MUSIC) [131] and Estimation of Signal Parameters via Rotational Invariance Technique (ESPRIT) [132]

rely on the decomposition of the microphone covariance matrix to estimate signal and noise subspaces. MUSIC, in particular, identifies source directions by projecting steering vectors onto the noise subspace and detecting peaks in the resulting pseudo-spectrum. These methods are computationally intensive and can be sensitive to reverberation, but they provide high localization accuracy under controlled conditions.

Probabilistic methods model the spatial distribution of sound sources using generative frameworks like Gaussian Mixture Models (GMMs) [133], [134]. These approaches combine statistical inference with signal sparsity in the time-frequency domain, providing robust SSL performance in scenarios with multiple sources. Extensions to these models include Gaussian mixture regression for single and multi-source localization, highlighting their adaptability [123], [135], [136]. In his PhD thesis [137], Michael I. Mandel proposes a model-based expectation-maximization (EM) approach for sound source separation and localization. This system can handle challenging scenarios involving reverberation and the presence of an adversarial *masker* sound source.

Despite their utility, classical SSL methods exhibit several limitations. Furthermore, many classical techniques fail to generalize across varying microphone array configurations and acoustic environments. Those shortcomings have catalyzed the development of data-driven techniques, particularly those leveraging deep learning. These modern methods excel in handling complex, nonlinear relationships in audio data, offering greater robustness in diverse and dynamic environments. The evolution of SSL from classical to deep learning approaches marks a significant leap in addressing real-world challenges.

3.1.4 Deep Learning Methods for SSL

The advent of deep learning has brought transformative changes to SSL, enabling robust performance in real-world conditions involving noise, reverberation, and multi-source scenarios. Unlike classical methods that rely on handcrafted features and simplified models, deep learning techniques harness large datasets and performant neural network architectures to learn complex spatial and spectral relationships directly from audio data. Today, the vast majority of research efforts on SSL leverage deep-learning-based solutions.

Let us first describe the typical workflow process of such approaches. Deep-learning-based SSL systems typically involve three primary components. First, **input feature extraction** consists of computing acoustic features from the raw audio data. Secondly, the most crucial step is **feeding the pre-processed input data into a DNN**. While approaches differ, their task generally consists of mapping the sound signal to the sound source(s) location(s). The network is often trained in a supervised fashion on a collected offline dataset containing numerous pairs of sound recordings and source positions. After training, the model is supposed to be able to predict the sound source location from unheard recordings resulting from new situations. The third and final aspect of the workflow consists of choosing the right **output strategy**. Grumiaux et al. [69] highlight the wide variety of possible choices in this matter. As noted previously, not all methods share the same capacities (Section 3.1.2). Some can handle the detection of multiple sources [138], [118], [134], while others are limited to single-source scenarios [139], [140], [141].

Input Data

The *Wav2Vec* method initially proposed by Schneider et al. [142] and refined by Baevski et al. [143] directly learns from raw audio data in a self-supervised fashion. This work demonstrates that deep neural networks can directly learn helpful representations of audio signals, given enough data. A few works have trained DNNs to perform SSL from the waveform domain [117], [144], [145], [146], [147]. Although conceptually elegant, such raw-audio-based approaches remain scarce in the literature. Spectral representations are significantly more popular. Most notably, the STFT is used to compute acoustic signals' magnitude, power, and phase spectra. For example, Chakrabarty et al. [141] have used the phase spectrogram to localize multiple speakers. Additionally, interaural (or interchannel) features are commonly used across the literature, such as the aforementioned ILD, IPD, and ITD [122], [141], [148], [149]. Please refer to [Section 2.2.1.5](#) for more details on such features. Also, the acoustic representations introduced by the classical localization methods remain popular in the deep learning era. For instance, Diaz-Guerra [150], [151] proposes to feed SRP-PHAT power maps to a Convolutional Neural Network (CNN) for performing localization. In [152], Boztas has explored wavelet transforms for pre-processing the raw audio data.

Ambisonics is a format for representing acoustic signals as a spherical harmonic decomposition of the sound field. This multi-channel representation is agnostic to the microphone array configuration and encodes the spatial properties of a sound field [69]. Those properties are relevant in the context of SSL. Applying the STFT to an ambisonic signal can express it in the time-frequency domain. Adavanne et al. [153], [127] employ spectrograms of first-order ambisonics to localize and track multiple sound events. Perotin et al. have conducted a series of works on ambisonics and its application to SSL [70], [139], [154], [155]. In [156], Varanasi et al. derive novel spherical harmonic magnitude and phase features for DoA estimation. These features allow for the separate estimation of the elevation and azimuth, lowering the method's computational cost. Compared to first-order Ambisonics (FOA), which are more common in the literature, the proposed format uses higher-order spherical harmonics (up to the third order), granting an enhanced spatial resolution.

Dataset Collection

Most modern solutions adopt a supervised approach to the SSL task. They thus require gathering numerous data samples from which they can learn. The two main approaches for data gathering consist of acoustic simulation and recording in real environments. The former is naturally a more cost-effective solution and scales significantly better with the amount of collected data. As presented in [Chapter 2](#), a large ecosystem of acoustic simulation environments exists. They are constantly improved to reach higher levels of fidelity and accuracy. A typical data generator for SSL is the association of a bank of RIRs computed by simulation software and a set of clean speech signals from an existing corpus. Speech signals are then convolved with the clean recordings to obtain simulated listened-to signals. The ground-truth source and microphone positions are known from the start, and no further labeling work is required. Generating a dataset from an acoustic simulator does not require recording equipment and allows for collecting arbitrarily large amounts of data [51]. Nonetheless, the real-world performance of networks trained on such datasets is generally lower than in simu-

lation. Transferring those methods to the physical world is an important research area. For instance, some works, such as Chakrabaty et al. [141], [157], Xiao et al. [158], and Grumiaux et al. [159], have tested their system on real-world recordings after an initial training phase on synthetic data. Commonly, modern SSL research combines both data sources. Inexpensive synthetic datasets are often used to train deep neural networks. Once trained, the performance of the obtained systems is evaluated in real scenarios using microphone arrays or microphone-equipped robotic heads [122], [160], [161]. Also, the simulation of moving sources remains an obstacle for simulation software. Publicly available acoustic datasets have been used to train or evaluate SSL methods [162], [163]. Finally, the DCASE challenge [164] has historically integrated SELD and pure localization tasks. It provides the datasets that all participants can use to train and evaluate the performance of their approaches.

Network Architecture

Grumiaux et al. [69] discuss the various popular choices regarding neural network architectures. The network topology design is a fundamental property of deep-learning-based SSL systems.

The **feed-forward neural network** is the simplest form of DNNs. It was also the first deep neural architecture used to perform SSL [165], [166]. These networks typically map preprocessed spatial audio features, such as IPD, ILD, or Generalized Cross-Correlation (GCC) [158], to the DoA of a sound source. In [158], a single-hidden-layer Multi-Layer Perceptron (MLP) is trained on GCC-PHAT-based features. Takeda et al. [167] use a DNN to learn from the eigenvectors of the recorded signal interchannel correlation matrix. The feed-forward networks' simplicity and efficiency make them suitable for single-source localization in controlled environments. However, their lack of temporal modeling capabilities limits their performance in dynamic scenarios or multi-source settings.

Convolutional neural networks have also been employed widely for SSL. As seen previously, it is typical for the recorded signal to first be transformed into the time-frequency domain. The Fourier representations of temporal signals share several properties with images. Most notably, they are often represented as multi-channel images expressed in the time-frequency plane. As such, the acoustic community has leveraged the vast computer vision literature that has designed numerous deep neural networks for processing images. CNNs have shown promising results for SSL. The inherent ability of such networks to process an arbitrary number of channels allows for the leverage of recordings from several microphones. The convolutional kernels are responsible for combining this information. Hirvonen et al. [140] are among the first to apply a CNN to SSL. Multiple works have followed, adopting a similar approach and trying to learn from different types of input features [127], [141], [157], [168]. In [157], Chakrabarty et al. empirically show that the optimal number of convolutional layers for a microphone array with M microphones is $M - 1$. Diaz-Guerra et al. [151] demonstrate the relevance of icosahedral CNNs for DoA estimation. This architecture has the advantage of being equivariant to the 60 rotational symmetries of the icosahedron. The method achieves state-of-the-art results, even in highly reverberant environments. The authors attribute this success to using a model that encodes the properties of the problem in its architecture.

Primarily used in the Natural Language Processing (NLP) community, **Recurrent Neural Networks (RNNs)** are a popular choice for processing sequential data and time series in general. They can inherently propagate information along a sequence to leverage global context to perform a task. The principal advantage of those architectures is their capacity to model temporal phenomena, which can help localize sound sources in more complex settings. The main RNN architectures are the Long-Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) designs. Some works, such as [105], [153], [124] and [169] have used a combination of a CNN and a RNN, referred to as a Convolutional Recurrent Neural Network (CRNN) for SSL.

Vaswani et al. [1] famously introduced the **transformer architecture**, which has become the de facto neural network design for numerous complex tasks. Attention-based neural networks were first employed by the NLP community to replace LSTMss [170]. They share with RNNss the ability to propagate information along a sequence of tokens, which can then be used to make a decision. In vision tasks, the transformer architecture has been successful as an alternative to the well-established convolutional design [34]. Some approaches have employed transformers for SSL as well. Phan et al. [125], [126] have coupled a CRNN architecture with the self-attention mechanism. Experiments by Grumiaux et al. [171] and He et al. [13] concluded that self-attention could be used to improve a baseline CRNN network.

Other architectures have also been successfully used to achieve localization. Encoder-decoder neural networks, for example, are designed to learn a lower-dimensional compressed representation of some input data. Various forms of this architecture served in SSL methods. Le Moing et al. [161] employed an encoder-decoder-style network to predict 2D Cartesian coordinates of multiple sound sources. Variational auto-encoders [172] and U-net architectures [117] are other examples of encoder-decoder networks used for SSL.

Output Format

As previously explained in [Section 3.1.2](#), the output formats of SSL detectors vary considerably across methods. On the one hand, not all solutions estimate the same values. Some systems are limited to DoA estimation, while others can additionally predict the distance. On the other hand, the coordinate system used to express the detections is not always the same. The vast majority of works compute the source's position with respect to the microphone array. Computing an absolute position would require additional knowledge about the environment, the microphones' positions, and choosing a global frame for reference. Nonetheless, different relative coordinate systems are used. Choosing spherical or polar systems consists of predicting a DoA value and, optionally, the elevation angle and/or the distance to the source. Alternatively, other works choose to write the estimated detections in Cartesian coordinates. The latter choice can involve predicting the distance, too. Adavanne et al. [153], [127] instead normalize their network's outputs so that estimated coordinates are restricted to the unit sphere. Additionally, some works can be further distinguished from the usual regression formulation of SSL. Specific approaches have framed the localization problem as a classification task where the system is expected to select a region of space instead of producing one or more scalar values [149], [158].

3.1.5 Sound Source Localization in Robotics

Although SSL has been studied as a self-contained problem, it certainly has many notable downstream applications. Among those, robotics is a significant use case of SSL algorithms. Indeed, perception is an essential building block of a social robotics platform. Human abilities in this regard are robust and efficient. We can detect, locate, extract, and recognize multiple sound events in complex scenes. Research in acoustics has investigated ways to replicate such skills in automatic systems. Robotics is a major application field for these acoustic challenges. More specifically, social robotics focuses on designing agents that are able to interact with humans. Besides exploiting visual features, which have been investigated in the computer vision domain, leveraging audio cues can provide valuable information for a social robot. Naturally, such an agent will use language as the primary means of communication with humans and thus needs to extract the meaning of its interlocutors' speech. Yet, sound information may have additional use cases besides ASR. For instance, human-robot interaction can be enhanced by having the agent adjust its gaze and look at the person it interacts with. A robot that can locate other sound sources accurately can also adjust its navigation policy to take advantage of this knowledge. Robot navigation is likewise complex and often relies on multi-modal perception. LIDAR, or depth information, allows the robot to localize itself and other potentially moving subjects in the environment. Visual information has also been used to perform SSL [173]. Regarding robotics, Naranag et al. [174] have combined a sound source localizer with a visual Simultaneous Localization And Mapping (SLAM) system for performing auditory-aware navigation. Using both modalities helped improve overall performance, including the SSL accuracy itself. In [175], Trifa et al. discuss the relevance of combining audio and visual cues for human-robot interaction tasks. While solely limited to audio features, their experiments involve performing sound source localization on a physical robotic platform. Li et al. [63] demonstrate a capable SSL solution tested on the NAO robot. Gala et al. [176] manage to predict both the orientation and distance to the source by rotating a self-rotating binaural microphone array. Introducing movement is a common technique to eliminate front-back ambiguity. In [177], Sewtz et al. present an audio segmentation system. It assigns speech phases to specific speakers in multi-party indoor environments. The authors insist that, contrary to vision-based systems, sound source localization does not necessitate any movement from the robot. This property helps with robot acceptance from humans. Argentieri et al. [178] provide an in-depth review of the state of the art as of 2014. Naturally, their survey has not discussed more recent works applying modern deep-learning-based techniques. For instance, Nguyen et al. [122] designed a neural network to map audio features recorded by a binaural humanoid robot to the relative source direction. The model is trained in a supervised fashion using white noise. It is then tested in a realistic setup where the task is to localize speech sources. Boztas [152] trains a deep neural network to perform SSL with a four-microphone array integrated on the Nao robot's head. Several neural network architectures are compared (MLP, CNN, LSTM). They evaluate the proposed method by having a TurtleBot3 mobile robot carrying a sound source around the fixed Nao robot.

Robotics often brings additional challenges to the SSL task. Indeed, a robotic platform implies dealing with several constraints, mainly caused by interacting with the real world. Realistic environments are dynamic, reverberant, and noisy. They involve intermittent, moving, and

concurrent sources. Furthermore, to be relevant, a robotic system must operate in real-time. Contrary to some offline techniques, which can rely on computationally expensive techniques, SSL methods are considerably constrained. Additionally, SSL is generally the first block of multi-step acoustic pipelines. For instance, Asano et al. [179] combine a localization block, the results of which are used to separate the speech signal from ambient noise. An online ASR system finally processes the cleaned audio. Nakadai et al. designed advanced systems for localization on actual robotics systems [180], [181]. Argentieri et al. [178] differentiate two categories of robotic acoustic frameworks. On the one hand, binaural setups try to model human hearing. Experimentation platforms involve a robotic head with one microphone on each side. The head-related transfer function (HRTF) models how the physical head between the two microphones impacts the recorded signals. It can be measured in an anechoic environment [182], [183] or simulated [184]. On the other hand, efforts have been made to use more than two microphones. Li et al. [77] estimate the transfer function from recorded signals and use this information to localize a sound source with a real robotic head. On the other hand, array processing involves using several receivers arranged in more complex geometries [68], [185]. They leverage the redundancy of the spatial information across the multiple recorded channels.

Deep learning methods have also been used in robotics to perform SSL. Nguyen et al. [122] have collected a dataset to train a CNN for localizing and facing a sound source with a humanoid robot head. SSL can also be employed to enhance the ASR performance of a robotic system. Dávila-Chacón et al. [186] designed such a system that infers the DoA to rotate a robotic head to maximize the speech recognition score. After adjusting the head orientation, their algorithm selects the most appropriate channel to provide to the ASR system. They measure a 50% improvement in the recognition performance thanks to ideally orienting the humanoid head with respect to the speech source. Interestingly, the optimal angle does not correspond to the sound entering one of the robot's microphones perpendicularly. Instead, the angle maximizing the reverberation of the sound wave on the artificial pinna provides the best condition for accurate ASR.

3.1.6 Conclusion

As depicted in this condensed overview, multiple and diverse research efforts have been focused on solving the SSL task. Although this challenge has been explored extensively for decades, the ecosystem remains vibrant, and new solutions are constantly proposed. The field's evolution demonstrates valuable progress as methods handle more and more complex variations of the problem. Classical methods like TDoA, beamforming, and subspace algorithms have provided foundational approaches, particularly in controlled settings. They also leveraged foundational physical and statistical characteristics of the recorded signals. Researchers have gained a deep understanding of the underlying mechanism that could be used to infer sources' positions. Then, as in most information-processing fields, deep learning methods have permitted significant advances. They allowed pushing the boundaries of existing SSL systems by leveraging large volumes of training data. Many limitations of the classical methods were circumvented thanks to the expressiveness of deep neural networks. Current research efforts try to optimize and enhance those data-based approaches by making

them more efficient, robust, and performant. The design of network architectures and pre-processing pipelines is an example of amelioration axes for such systems. Despite its utility, SSL remains a complex problem due to real-world constraints. Deep-learning-based SSL methods often require large, annotated datasets and face difficulties in generalizing across unseen environments and array geometries. Issues such as scalability in multi-source settings and real-time processing constraints further underline the need for ongoing innovation. Also, robotics is a vibrant application of SSL systems, putting them in challenging environments. In conclusion, SSL research continues to evolve, balancing the precision of classical methods with the adaptability and robustness of modern deep learning techniques. As advancements in hardware, algorithms, and datasets continue, SSL systems are poised to play an increasingly integral role in enabling intelligent, perceptive machines to interact seamlessly with their acoustic environments.

3.2 Single-source localization

As a first experiment, we investigate the single-source localization problem. The goal is to design and train a deep learning model that can precisely localize a source in a reverberant environment. This project involves programming a data collection pipeline from the earlier developed simulator. Generating custom synthetic datasets has allowed us to test our approach in various conditions. This section details the adopted deep-learning-based approach and the technical choices made.

3.2.1 Problem Statement

A robotic agent is evolving in a reverberant room. A single speech source is also present in the environment. The task consists of determining the relative position of a unique sound source (Figure 3.1). Although the focus will be directed towards methods that predict solely the DoA, solutions that estimate the distance to the source have been evaluated.

As seen in Section 3.1.5, multi-modal information can be leveraged to perform SSL in a robotics context. However, this chapter will focus on the exclusive use of audio information. This choice is more representative of the classical formulation of the SSL problem, and although simpler to formulate, it constitutes a challenging task.

Furthermore, we expect to leverage our solution in dynamic, active interaction scenarios. In a real-time context, we expect to reduce the latency of our localization system to improve the solution's overall responsiveness. As a consequence, only a short recording should suffice to accomplish an accurate localization of the source. This work is motivated by human-robot interaction and thus attempts to detect speech sources. The framework aims to model a reverberant environment where a human speaker is located. The robot is asked to estimate the person's position even if the geometric situation is challenging.

The objective of this work is not to establish a state-of-the-art solution for SSL. Conversely, a proof of concept for a complete localization pipeline is proposed. We intend to study the

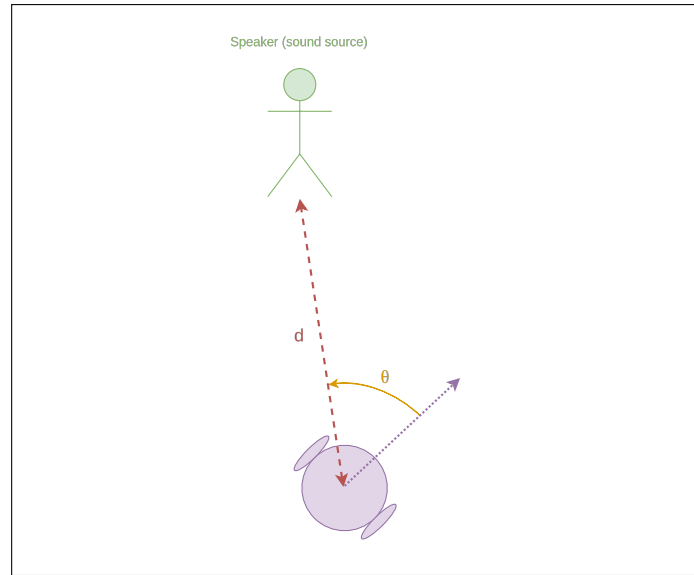


Figure 3.1 – Schema of the SSL task.

various performance-determining factors of a deep-learning-based SSL system. Solving this task will provide a base building block for integrating into our more complex RL pipeline, as presented in [Chapter 5](#).

3.2.2 Method

3.2.2.1 Custom Dataset for SSL

This study aimed to adapt state-of-the-art SSL methods to diverse, challenging setups. The audio simulator presented in [Chapter 2](#) has been leveraged to generate various synthetic datasets for experimentation.

Although public SSL datasets have been shared publicly by the community, we have chosen to work within our artificial HRI environment for later reuse of this method. For the sake of completeness, here are some examples of other relevant datasets. He et al. [187] have proposed the Sound Source Localization for Robots (SSLR) dataset using the Pepper robot equipped with a four-microphone array. Furthermore, the third task of the DCASE challenge proposes a yearly competition around designing the best SSL system. The target dataset for the 2024 edition of DCASE was STARSS23 [188], introduced at the NeurIPS 2023 conference.

Training and test samples are generated independently according to the following process. An omnidirectional speech source, our acoustic model for a speaking human, is placed randomly in the room. A multi-microphone array is also positioned randomly in the room. Apart from ensuring that the source and microphones are not too close to the walls, no constraint is set on their location. Several published SSL methods are only tested with restricted positions for the sources. Such limitations can be inherent to the physical experimental setup. Using a simulator allows for maximizing the diversity of environment configurations. Once the source

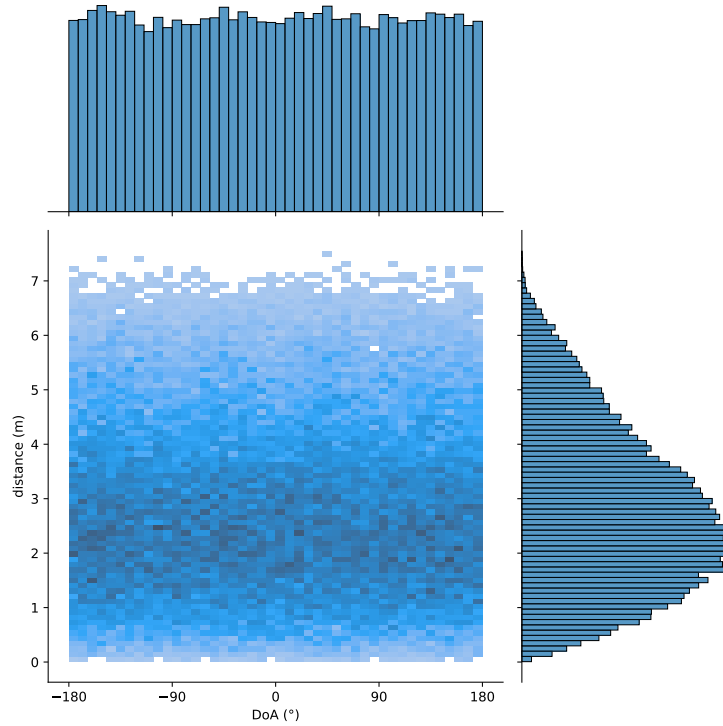


Figure 3.2 – Statistics of ground-truth label pairs (θ, D) in the generated dataset. Microphone and source positions are randomly sampled within a rectangular room, resulting in a uniform distribution of DoA values. Source-to-array distances range from 0 to 7 meters, with values around 2-3 meters being the most frequent.

and microphones are positioned, this configuration's RIR is computed. Then, a random sample is drawn from the LibriSpeech [81] dataset. An arbitrary 1s chunk is extracted from the speech recording and set as the source input signal. By convolving this input signal with the RIR, we obtain the simulated signal recorded by each microphone. Finally, the multi-channel complex STFT is computed from the waveform and saved on disk. For each sample, we save the localization ground-truth information, additional metadata, and the acoustic observation. Most notably, the DoA value θ and the source-array distance D are included. This process is repeated to obtain 100,000 distinct samples, which will later be split between training, validation, and testing. The final datasets weigh from 26 to 50GB, depending on the number of microphones in the array. Figure 3.2 outlines the repartition of the generated samples regarding source-array relative positions.

3.2.2.2 Microphone Arrays

Several microphone arrays have been experimented with in this study. Leveraging multiple microphones to form an array is essential. Geometric information is extracted from the differences between the signals each sensor receives. The room's acoustic properties and the array's layout significantly impact the characteristics of the generated audio signals. The resulting patterns in the collected data are thus heavily affected by these choices.

We present the following microphone array configurations that have been tested:

- A **binaural** array comprises two microphones placed a few centimeters apart. This setup constitutes the most studied robotic SSL framework in the literature. This layout was primarily considered because of a humanoid robotic head equipped with two microphones on each side.
- We have also proposed a **three-microphone** design in a V-shaped arrangement.
- A **square** array of four microphones has been implemented too.
- Finally, the **Uniform Linear Array (ULA)** configuration is available, supporting a configurable number of microphones.

Their implementation has been integrated into our simulator (see [Section 2.3.2.2](#)).

The number of microphones plays an essential role in the SSL performance. As an illustrative example, when having a binaural microphone in an anechoic environment, i.e., where the effects of reverberation can be neglected, there exists a fundamental limit: It is theoretically impossible to distinguish between two possible locations for the source. This phenomenon, known as front-back ambiguity, was presented earlier in this chapter [Section 3.1.3](#). The front-back ambiguity can be cleared by introducing relative movement or an additional microphone in the array.

Importantly, all our microphone arrays are deprived of any physical incarnation. No real material constitutes the actual array. In a more realistic setup, the presence of a robotic head between two microphones can be modeled using an HRTF.

Each microphone's *polar pattern* also stands out as an essential characteristic of the array. This property describes which incoming capture directions are amplified or dampened by the sensor. Each use of a microphone can benefit from appropriate directionality. For instance, when recording a singer or speaker's voice, one can afford to point the receiver towards the source and have it ignore the unwanted sounds coming from other directions.

The *omnidirectional* pattern is the simpler one to think of. All directions are given equal importance. In contrast, the cardioid and its variants (super-cardioid, hyper-cardioid, ultra-cardioid, etc.) weigh non-uniformly each angle of incidence and thus privilege some directions above others. [Figure 3.3](#) shows the receptive field of the most common microphone patterns.

In the context of SSL, a non-homogeneous pattern brings extra angular information that a neural network may be able to exploit. We have thus tested different configurations in our experiments.

3.2.2.3 Audio Pre-Processing

Motivated by their popularity in the literature, we focus on spectral representations of audio data. The simulator allows the extraction of such representations directly from the generated signals. The observations are stored in the dataset as multichannel complex STFTs. These complex tensors are not fed directly into the neural network but are further processed. The role of this step is to convert the complex spectral observation to a real-valued tensor. The neural network is expected to extract the relevant localization information from the obtained repre-

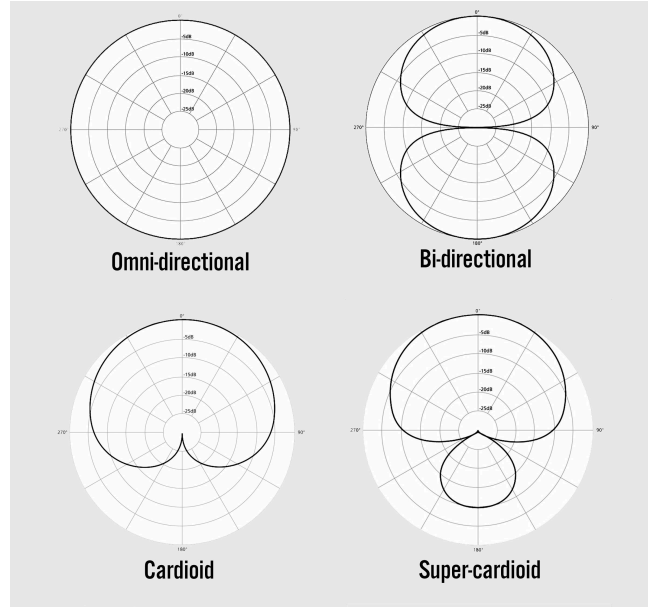


Figure 3.3 – Illustration of the most common microphone polar patterns [189].

sensation. This section explores the importance of choosing the input features fed into the network. Several methods have been tested and compared. The corresponding experimental section (Section 3.2.3.4) summarizes our findings regarding their respective performance.

Adaptations of 2D convolutions to complex tensors do exist and have already been used in the SSL literature. Krause et al. [190] present this variation along with its benefits (Section II.B). However, regular 2D convolutions have been employed in the present work, and the complex-valued STFT needed to be converted to real values. To achieve this, two schemes were compared:

- On the one hand, both the real and imaginary parts of the complex data can populate the two real resulting matrices:

$$\begin{aligned} \varphi_{\text{Cart}} : \quad \mathbb{C}^{F \times T} &\longrightarrow \mathbb{R}^{2 \times F \times T} \\ Z &\longmapsto (\Re(Z), \Im(Z)). \end{aligned} \quad (3.1)$$

This form will be referred to as the Cartesian projection.

- The other method consists in using the polar form of the Fourier representation:

$$\begin{aligned} \varphi_{\text{polar}} : \quad \mathbb{C}^{F \times T} &\longrightarrow \mathbb{R}^{2 \times F \times T} \\ Z &\longmapsto (|Z|, \arg(Z)). \end{aligned} \quad (3.2)$$

In both cases, a C -channel STFT (C, F, T) complex tensor translates to a $(2C, F, T)$ real one.

Besides raw STFT values, interaural features, presented in Section 2.2.1.4, have been widely used in the SSL literature [122], [166], [191]. Binaural representations have been explicitly designed to highlight geometric information relevant to localization. Hence, and for the sake

of exhaustivity, those cues have also been tested. This comparison employs a binaural array which allows for trivial computation of the ILD and IPD from the two STFT arrays. Notably, the number of resulting channels in the processed data remains two. The interaural tensor $\mathcal{J} \in \mathbb{R}^{C \times F \times T}$, with $C = 2$, amounts to:

$$\mathcal{J} = \begin{pmatrix} \text{ILD}(m_1, m_2) \\ \text{IPD}(m_1, m_2) \end{pmatrix}. \quad (3.3)$$

Both ILD and IPD take real values, which does not lead to doubling the number of channels. When dealing with arrays having $C > 2$ microphones, we compute the interaural features for successive and overlapping microphone pairs. For an array with microphones $\{m_1, \dots, m_k\}$, the interaural features is expressed as:

$$\forall i \in \llbracket 1, C \rrbracket,$$

$$\mathcal{J}[i] = \begin{cases} \text{IPD}(m_i, m_{(i+1) \bmod C}) & \text{if } i \text{ even;} \\ \text{ILD}(m_i, m_{(i+1) \bmod C}) & \text{if } i \text{ odd.} \end{cases} \quad (3.4)$$

When using a single interaural feature, and not both ILD and IPD, the coefficients of $\mathcal{J}$ become:

$$\mathcal{J}[i] = \text{IPD}(m_i, m_{(i+1) \bmod C}). \quad (3.5)$$

An ablation study was conducted to measure the impact of pre-processing methods on SSL performance (Section 3.2.3.4).

3.2.2.4 Neural Network Architecture

As demonstrated in Section 3.1.4, deep neural networks are flexible and effective building blocks for an SSL solution. We focused in this work on a simple architecture that takes some representation of the listened audio signal as its single input. At the other end, this network is trained to infer the DoA value θ and optionally the distance D from the single speech source in the room. Our model is trained in a supervised fashion using some custom datasets presented in Section 3.2.2.1.

The architecture, depicted in Figure 3.4, consists of five convolutional blocks. Each encompasses a 2D convolution layer, batch normalization, and a Rectified Linear Unit (ReLU) operator. The convolutional filters operate in the time-frequency plane. The dimension of the multi-channel image progressively shrinks along the network. The convolutional feature extractor ends with an adaptive max-pooling operation, which reduces the input tensor from a (C, F, T) shape to a C -dimensional vector. At this stage of the network, the spatial dimensions F and T have been reduced to 30 and 6, respectively, while the number of channels C has increased to 256. The convolutional backbone is followed by a 3-layer MLP in charge of regressing the computed features to the final expected values. Each fully-connected hidden layer is followed by a ReLU and a dropout operation. The output neurons are trained to predict the sine and cosine of the DoA and, optionally, the distance to the source D .

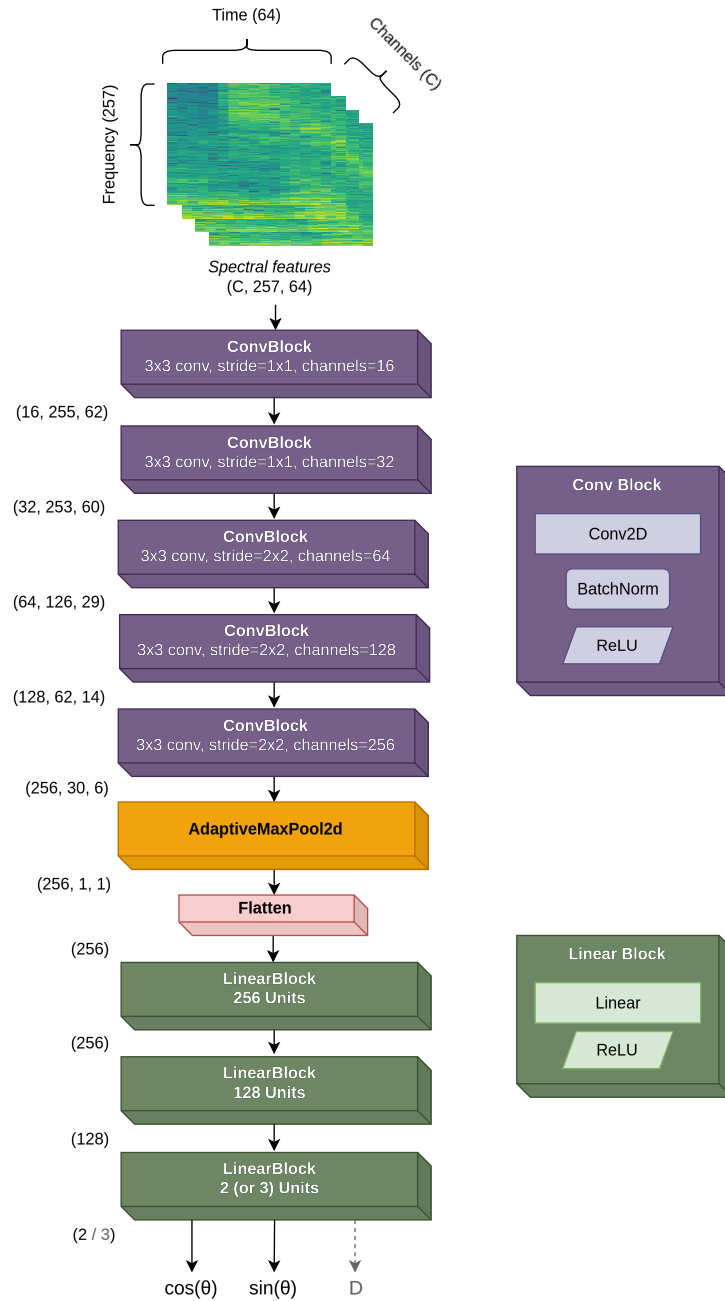


Figure 3.4 – Deep convolutional network architecture for SSL.

3.2.2.5 Loss Function

This single-source SSL task boils down to a low-dimensional regression problem. The network is designed to eventually predict a scalar value for the DoA and optionally an extra value for the source-microphone distance.

Angular loss. While the distance case is straightforward, the DoA estimation should be cautiously handled. Indeed, the DoA lies in the $[-\pi, \pi]$ periodic interval. For instance, if the ground truth is -3.1 radians, then values close to either $-\pi$ or π would be accurate predictions. A naive Mean Squared Error (MSE) loss would wrongly penalize estimations close to

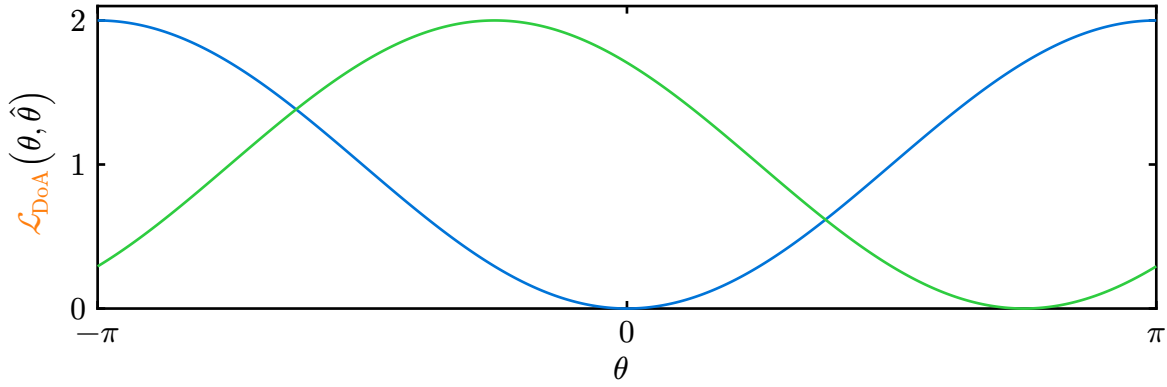


Figure 3.5 – Angular loss plots for $\hat{\theta} = 0$ and $\hat{\theta} = \frac{3\pi}{4}$. The x axis corresponds to the predicted DoA value $\hat{\theta}$.

$+\pi$. We adopt a periodic loss for the DoA to account for this specificity. Also, the network does not directly predict the DoA value θ , but its sine and cosine instead.

Let $\hat{\theta} = (\hat{\theta}_1, \dots, \hat{\theta}_n)$ be the set of DoA angles predicted by the network and $\theta = (\theta_1, \dots, \theta_n)$ the corresponding ground truth values. The loss function is expressed as

$$\mathcal{L}_{\text{DoA}}(\hat{\theta}, \theta) = \frac{1}{n} \sum_{i=1}^n [1 - (\sin(\theta_i) \sin(\hat{\theta}_i) + \cos(\theta_i) \cos(\hat{\theta}_i))]. \quad (3.6)$$

Mathematically, we can write $\mathcal{L}_{\text{DoA}}(\hat{\theta}, \theta) = \frac{1}{n} \sum_{i=1}^n [1 - \cos(\theta_i - \hat{\theta}_i)]$, but the implementation specifically uses Equation (3.6) as the network's output neurons ($\cos \hat{\theta}_i$ and $\sin \hat{\theta}_i$) need to appear explicitly. Figure 3.5 plots the value of $\mathcal{L}_{\text{DoA}}(\cdot, \hat{\theta})$ for different values of $\hat{\theta}$. We use this loss function to train the neural network to output accurate DoA values without suffering from boundary effects.

Distance loss. When the model additionally estimates the distance to the source, the natural MSE loss is used to supervise the relevant output neuron:

$$\mathcal{L}_{\text{dist}}(\hat{d}, d) = \frac{1}{n} \sum_{i=1}^n \|\hat{D}_i - D_i\|_2^2, \quad (3.7)$$

where $D = (D_1, \dots, D_n)$ is the set of predicted distances and $\hat{D} = (\hat{D}_1, \dots, \hat{D}_n)$ the ground truth data.

The total loss then becomes

$$\mathcal{L} = \mathcal{L}_{\text{DoA}}(\hat{\theta}, \theta) + \kappa \mathcal{L}_{\text{dist}}(\hat{D}, D), \quad (3.8)$$

where κ balances the relative importance of the distance loss in the final result.

3.2.2.6 Training Strategy

Training deep neural networks involves determining relevant values for multiple hyperparameters. The network architecture plays a crucial role and has already been discussed in

Section 3.2.2.4. Similarly, the rest of the parameters have been set empirically, leveraging their impact on the final performance.

The model is trained in a supervised fashion on the synthetic datasets generated by the audio simulator (see **Section 3.2.2.1**). The training set consists of 80k samples. 72k elements are used for training, while 8k are reserved for validation. Besides, the remaining 20k samples constitute a test dataset and serve to evaluate the model's performance.

The training employs a batch size of 250 items for $T_{\max} = 100$ epochs. A learning rate scheduler helps stabilize the training and further improve the final results. Cosine annealing, proposed by Loshchilov and Hutter [192], decays the learning rate according to the following scheme:

$$\eta_t = \eta_{\min} + \frac{\eta_0 - \eta_{\min}}{2} \left(1 + \cos \left(\frac{T_{\text{cur}}}{T_{\max}} \pi \right) \right), \quad (3.9)$$

where η_t is the learning rate at epoch t , η_0 is the initial learning rate, T_{cur} is the current epoch. The minimum learning rate $\eta_{\min}$ has been set to 10^{-5} and is reached at the very end of the training. A base learning rate of $\eta_0 = 10^{-3}$ has been shown to ensure rapid convergence without suffering from instability issues. Regarding the optimizer, we have chosen to use Adam [193]. Training runs have been performed on an Nvidia RTX A6000 GPU and have lasted approximately one hour.

3.2.3 Experiments

This section presents a collection of experimental results on SSL performance. This study aims to understand the influence of specific parameters on the difficulty of the SSL task. For this purpose, a baseline solution and scenario are described and will serve as a reference point for the tested variations. First, the general testing methodology and metrics will be presented. The actual experimental results, including quantitative results, will follow.

3.2.3.1 Metrics

In this first formulation of the SSL problem, each situation includes exactly one source to localize.

DoA metric. Naturally, the performance of the method is characterized by how far the estimate $\hat{\theta}$ lies from the real DoA value θ . As the network predicts the sine and cosine of θ , the DoA value is first computed following:

$$\begin{aligned} \hat{\theta} &= \text{atan2} \left(\sin(\hat{\theta}), \cos(\hat{\theta}) \right) \\ &= \arg \left(\cos(\hat{\theta}) + i \sin(\hat{\theta}) \right). \end{aligned} \quad (3.10)$$

We then compute the average ℓ^1 angular distance d between $\hat{\theta}$ and θ . The following expression for d accounts for the periodicity of the angular interval $[-\pi, \pi]$:

$$\begin{aligned} d : [-\pi, \pi]^2 &\longrightarrow [0, 1] \\ (\theta_1, \theta_2) &\longmapsto \pi - \left| |\theta_2 - \theta_1| - \pi \right|. \end{aligned} \quad (3.11)$$

This distance is used to define the Mean Absolute Error (MAE) metric that quantifies the performance of DoA estimation:

$$\text{MAE}_\theta = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} d(\hat{\theta}_i, \theta_i), \quad (3.12)$$

where n_{test} counts the number of samples in the test set.

Source-array distance metric. When predicting the source-array distance, the MAE is also used, here between predicted $\hat{D}$ values and ground truth D :

$$\text{MAE}_D = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} |\hat{D}_i - D_i|. \quad (3.13)$$

The choice of the MAE as performance criteria has the advantage of being expressed in length units. For clarity reasons, the values for this metric will be displayed in centimeters (cm).

3.2.3.2 Base Solution and General Methodology

This study investigates how different parameters influence the performance of our SSL pipeline. A baseline must be defined to have a reference for comparing experimental settings. The starting setting involves a binaural array with two cardioid microphones for the agent. Microphones are two centimeters apart. The room's reverberation level (T_{60}) is set to 500ms. Interaural features (ILD + IPD) have been chosen for this baseline. By default, only the DoA is predicted, not the distance. The default number of epochs is set to 100. This setup is not the best-performing combination, but it provides a credible formulation to start with. These parameters will be individually changed across the following study.

Furthermore, the following settings are kept fixed across the experiments. We use a single 4×7 meter room. Learning parameters, such as the batch size, the learning rate, and the number of epochs, remain constant across most experiments. Also, the network architecture is the same for each run. Naturally, the number of input channels is adapted to the shape of the input data. The number of epochs has been exceptionally increased to reach proper convergence when necessary. Finally, the network training for each individual configuration was repeated several times to ensure that the obtained results were repeatable. The reported performance scores correspond to the most successful training run.

Table 3.1 – SSL performance for different microphone arrays.

Array	#mics	microphone pattern	MAE _{θ} (°) ↓
Binaural	2	omnidirectional	46.13
	2	cardioid	17.32
Triangle	3	omnidirectional	5.07
	3	cardioid	5.11
Square	4	omnidirectional	3.8

3.2.3.3 Microphone Arrays and Layouts

SSL relies on extracting spatial information from audio signals. Hence, as discussed in [Chapter 2](#) and in the present chapter, the use of arrays of multiple microphones is essential for this task. SSL methods aim to leverage the microphones' phase and level differences. The number of microphones in the array is an essential characteristic of the localizer. In the physical world, available hardware can limit the size and layout of microphone arrays. This work employs a simulator, so different configurations could easily be tested. We selected three array layouts: binaural, triangle, and square. They have, respectively, two, three, and four microphones. Furthermore, both microphone patterns were tested for the binaural and triangle arrays. [Table 3.1](#) shows the obtained SSL performance with each array. The binaural array allows for an accurate localization as long as the pattern is directional. Indeed, when using omnidirectional microphones, the performance drops significantly to 46°. The asymmetry of the cardioid pattern helps to discriminate the direction of arrival. Sound wavefronts coming from the back of a directional microphone are strongly attenuated. This fundamentally injects additional spatial information into the recorded signal. Besides the pattern influence, we have noticed that adding microphones substantially improves performance too. Interestingly, the difference between an omnidirectional and a directional microphone pattern diminishes when dealing with three or more microphone arrays. Our understanding is that the third microphone provides enough extra information to compensate for each microphone's lack of directionality. The four-microphone array scores the lowest MAE.

Inter-microphone distance is another fundamental characteristic of a microphone array. We have experimented with the influence of the microphone spacing in a binaural array on performance. [Table 3.2](#) displays the results of this study. Increasing the distance between microphones appears to be detrimental to the localization accuracy. When microphones are further apart, the aliasing phenomenon becomes problematic and deteriorates the relationship between the recorded signal and the source positions. Indeed, if the spacing amounts to d meters, all frequencies above $f_0 = \frac{c}{2d}$ Hz cannot be adequately distinguished. For $d = 2$ cm, this maximum frequency is 8.575 kHz, while most of human speech's frequency content is below 5 kHz [194]. Further lowering the distance to 1cm does not appear to bring additional benefits.

Table 3.2 – Reverberation impact on SSL performance.

microphone spacing (cm)	MAE_θ (°) ↓
1	17.71
2	17.28
4	18.75
6	21.86
8	22.58

Table 3.3 – SSL performance depending on the input features.

	#channels	#epochs	MAE_θ (°) ↓
Interaural (ILD/IPD)	2	100	17.21
STFT (Cartesian)	4	200	17.05
STFT (polar)	4	200	25.7
ILD only	1	100	38.27
IPD only	1	100	22.74
STFT magnitude only	2	100	31.17
STFT phase only	2	100	27.97

3.2.3.4 Impact of Input Signal Representation

To measure the impact of the input representation, we run the training process with each encoding choice (see [Section 3.2.2.3](#)). The number of channels depends on the selected method and varies from 1 to 4. The network architecture’s first convolutional layer is naturally scaled accordingly.

[Table 3.3](#) summarizes the method’s performance using different input features. We report the corresponding channel dimension of the resulting tensor for each set of cues. On the one hand, both complex-to-real STFT polar and Cartesian mappings yield substantially different results. Indeed, despite having the same underlying data, those two representations do not offer the network the same performance. The polar projection proved to be harder to learn from. Also, both Cartesian and polar projections lead to more unstable training than the interaural features. The Cartesian features perform the same as the interaural ones in training but lead to a less robust convergence on the validation set. Despite the final performance being comparable, training our neural network on interaural features is significantly more repeatable and robust. Hence, the ILD IPD combination is shown to generalize better and have stronger stability. [Figure 3.6](#) illustrates the training dynamics for the three types of features. The discrepancy between training and validation for the Cartesian projection of the STFT appears clearly. We trained the network for 100 epochs in all three cases. The number of epochs was later increased from 100 to 200 to attempt reaching a more stable training regime. The MAE_θ reported in [Table 3.3](#) results from this longer training time. However, the loss behavior was equally unstable, and the network achieved competitive metrics at the end of

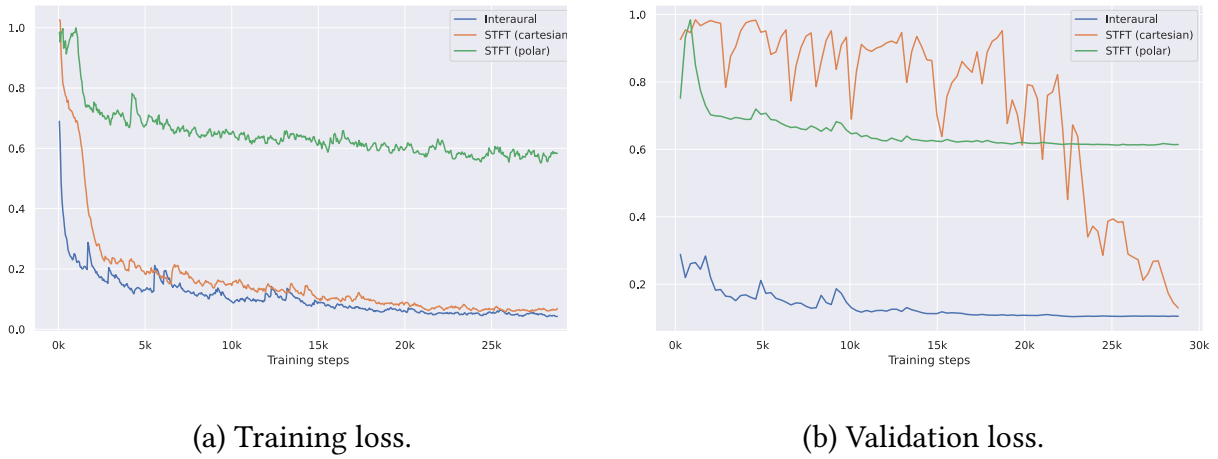


Figure 3.6 – Evolution of the training and validation DoA loss $\mathcal{L}_{\text{DoA}}$ for different input features.

the training. This contrasts with the interaural features experiments, where strong accuracy is already reached after a few epochs. Normalization could be a potential explanation for these inconsistencies. No additional pre-processing was applied in this study. Additional normalization schemes might boost the network’s performance, generalization, and stability. Stoller [195] presents two strategies for spectrogram normalization.

Furthermore, we have explored the relative importance of the individual sub-features for each category. The network was retrained with more granular features. The results are presented in the bottom half of Table 3.3. Regarding interaural features, neither ILD nor IPD appears to be fully redundant. Each sub-feature performs worse than the ILD-IPD combo referred to as *Interaural features* above. Yet, IPD allows for a lower MAE_θ compared to using ILD only. This observation is reasonable as IPD is directly correlated to the TDoA and thus the DoA. Besides, regarding the sub-features of the polar STFT representation, we notice a low drop in performance when discarding the magnitude information. While the complete polar STFT features allow for an MAE_θ of 25.7° , using the sole phase spectrogram only worsens it by 2.3° .

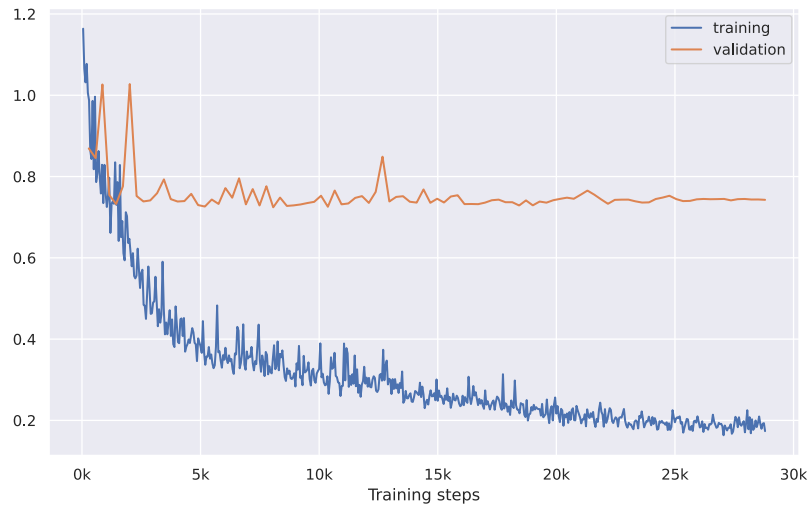
In conclusion, we have shown that it is critical to carefully choose a pre-processing strategy when training a deep neural network for SSL. While the information content is theoretically the same across different bijective transformations of the complex STFT performance, some can be harder to learn from. Our experimental results suggest that interaural features offer the best performance and stability.

3.2.3.5 Distance Estimation

DoA estimation has historically been the focus of the SSL literature. Indeed, predicting the distance to a sound source is known to be a significantly more challenging problem [69]. As the proposed architecture and methodology also support distance estimation, some experimental attempts were made with the combined objective (Equation (3.8)).

Table 3.4 – SSL performance when estimating the source-microphone distance.

Mic. array	Distance prediction	MAE_{θ} (°) ↓	MAE_D (cm) ↓
Binaural array	No	17.51	-
	Yes	21.99	88.53
Triangle array	No	5.17	-
	Yes	8.49	74.44

Figure 3.7 – Evolution of the mean absolute error on the distance estimation MAE_D during training.

Our results have confirmed that predicting the distance was less accessible for our neural network. Table 3.4 shows the combined localization performance for two microphone arrays (binaural and triangle). The obtained results are not satisfying, as the mean absolute error in distance estimation (MAE_D) is over 70cm in the best-case scenario. Also, control experiments were included in Table 3.4 to highlight the negative influence of distance estimation on the DoA error.

Finally, a closer observation of the training behavior suggests an overfitting phenomenon. Indeed, the network achieves a significantly lower error on distance estimation on the training set than on the test set. When using the triangle array, the MAE_D reaches 17cm on the training set, while it stagnates at 74cm on the validation and test datasets. As discussed in Section 3.1, methods that can accurately estimate the distance to the source often rely on additional temporal information or even controlled movement of the microphone array. It appears that the proposed CNN-based method presented in this work cannot solve this task. In Chapter 4, we leverage robot movement to accumulate localization information over time and accurately estimate the source’s position.

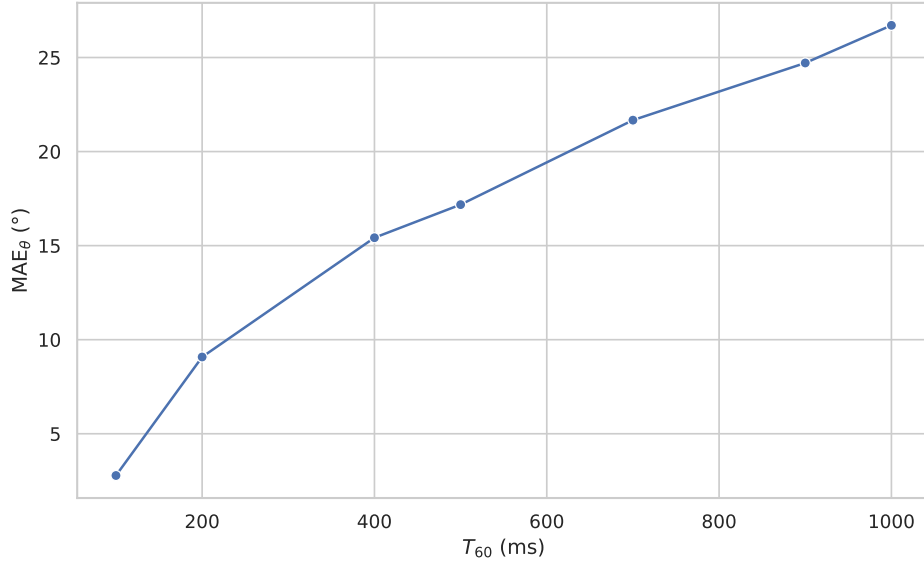


Figure 3.8 – Reverberation impact on SSL performance.

3.2.3.6 Reverberation

SSL methods leverage the inter-channel differences in the time-frequency input data to infer the source position. As discussed in [Section 2.2.1.4](#), these interaural features are theoretically sufficient to infer a single sound source’s DoA. This result does not hold in a reverberant environment where sound reflections deteriorate the direct relationship between the recorded waveform and the direction of arrival. Performing SSL in reverberant environments remains a core challenge for the acoustic research community. Thanks to our simulation library, we generated datasets from different reverberation settings. A neural network was trained from scratch on these individual datasets before being evaluated on the corresponding test set. Experimental results are summarized in [Table 3.5](#). Naturally, our method achieves very accurate localization in low-reverberation scenarios. These results demonstrate the challenge of operating SSL in reverberant environments. Notably, there is a 10x factor between performance at $T_{60} = 100\text{ms}$ and the performance at $T_{60} = 1\text{s}$. [Figure 3.8](#) illustrates the quasi-linear relation between the MAE_{θ} and the reverberation level T_{60} . Although performance certainly suffers from an increase in the reverberation time T_{60} , we noticed that the proposed method remained robust to reverberation. Even in highly reverberant scenarios, the MAE remains inferior to 30° . This shows that the network successfully filters out the direct path information from the recorded signal’s early and late reverberation artifacts. Furthermore, we demonstrate that a simple CNN architecture can perform accurate localization in challenging reverberation conditions.

3.2.4 Conclusion

Single-source localization is a fundamental problem of acoustics and plays a central role in robotics applications. In this first section, we have developed a deep neural network sound source localizer. Our solution has been designed and trained from scratch and is inspired by the

Table 3.5 – Reverberation impact on SSL performance.

T_{60}	MAE_θ (°) ↓
100ms	2.78
200ms	9.08
400ms	15.42
500ms	17.18
700ms	21.67
900ms	24.71
1s	26.71

vast deep-learning-based SSL literature. Original datasets were synthesized using our audio simulation library. They allowed us to train and evaluate the proposed network architecture on various scenarios. An extensive experimental campaign assessed the relative importance of different parameters in the localization performance. Additional questions could be further investigated. For instance, this work has not studied the impact of adversarial noise sources. Although accounting for acoustic reverberation is essential to properly model the physical world, noise sources should ideally also be included. While our simulator handles several sources and can use various noise signals as inputs, we have not been able to dedicate enough time to its inclusion in the SSL pipeline. This project was essential to better understand the SSL problem. Also, the deep neural network trained for this occasion serves as a feature extractor in our RL pipeline for acoustic-based navigation (see [Chapter 5](#)). The rest of this chapter will study the more complex case of multi-source localization.

3.3 Multi-Source Localization

Complex human-robot interaction environments often imply a varying number of sound sources. Hence, at a given time, the room may be completely silent. On the other hand, multiple concurrent sources of different kinds can be active simultaneously. The current section presents our investigation of a multi-source localization framework that will give our acoustic agent additional flexibility. We showcase a deep neural network implemented and trained on a challenging custom dataset, collected using our simulator. This approach complements the initial investigation on single-source localization.

3.3.1 Method

He et al. conducted a solid line of work on the multi-source SSL problem, focusing on the associated robotics challenges [13], [147], [187]. Their approach shows strong performance in challenging real-world scenarios. For this reason, several aspects of the methodology from [13] inspired the work presented in this section.

3.3.1.1 Dataset Generation and Pre-Processing

The dataset generation process remains essentially identical to that presented in [Section 3.2.2.1](#). All samples are independent and identically distributed. The positions of both the microphone array and the sources are randomly sampled. This section will focus on the necessary additions to support the multi-source setting. Also, most choices have been made to follow the methodology from He et al. [13].

Microphone array. To effectively localize multiple sound sources, we choose to use a four-microphone array. The $n_m = 4$ omnidirectional sensors are arranged in a 2cm wide square.

Audio processing has been kept the same, except for the sample duration. The latter now amounts to approximately 360ms as 16 STFT frames participate in each input of the model.

The microphone array and n_s speech sources get randomly positioned in the room. Such a choice has led to challenging samples where multiple targets share very similar DoA angles from the agent's point of view. The resulting RIRs are computed to account for the room's reverberation properties. Then, each source outputs a clean speech signal randomly chosen from the *LibriSpeech* [81] dataset. The simulator computes the resulting listened signals at each microphone of the array. Such signals last around 10 seconds.

Source-wise simulation and late mixing. In practice, the sources are not placed simultaneously in the room. Instead, they only get enabled individually to perform n_s distinct single-source simulations. This offers more possibilities as explained below.

First, let us start by leveraging the following property. The signal s_k recorded by the k -th microphone expresses simply as the sum of the signals $s_{i,k}$ providing from each source:

$$s_k[t] = \sum_{i=1}^{n_s} s_{i,k}[t], \quad (3.14)$$

where $s_{i,k}$ denotes the signal coming from the i -th source as received by microphone k . The STFT being a linear operator, this equality holds in the feature space:

$$S_k[t, f] = \sum_{i=1}^n S_{i,k}[t, f], \quad (3.15)$$

where S_k and $S_{i,k}$ are the STFT of respectively s_k and $s_{i,k}$. Hence, instead of saving the final features $S = (S_1, \dots, S_{n_m})$, we save each source-specific encoding $S_i = (S_{i,1}, \dots, S_{i,n_m})$ individually.

This overhead brings complexity to the data collection process but allows for a significant increase in flexibility. Indeed, the number of sources plays a prominent role in performance, and quantifying this influence has required experimenting with this parameter. Disposing of the audio features relating to each individual source allows for choosing how many sources should be active when loading each data sample from the disk. One solely has to sample a set $\{i_1, i_2, \dots, i_n\}$ of sources to enable and sum the relevant source-specific features $S_{i_1}, \dots, S_{i_n}$. The impact of the number of active sources is further studied in [Section 3.3.2](#).

Sampling frequency. The method was designed to operate with audio signals sampled at 48 kHz, which does not match the 16 kHz sample rate of the LibriSpeech [81] dataset, which provides the simulator with clean speech utterances. To account for this, the simulation of the audio signals received by each microphone in the array is performed at the native 16 kHz sampling rate. The generated signals are then up-sampled to 48 kHz using the Fourier method. More specifically, the signal's FFT is zero-padded. This provides an ideal antialiasing filter at the cost of assuming the signal to be periodic.

STFT representation of audio signals. As discussed in Section 2.2.1.4, several choices can be made regarding data representation. Although we have generated different datasets, the format used in the majority was the Short-Term Fourier Transform. Thus, the STFT is computed from the complete up-sampled simulated signal captured by each of the four microphones. For this, we employ a Hann window of length 2048, with a 50% overlap. We also apply band-pass filtering, removing frequencies below 100Hz and above 8kHz. The consequent STFT counts 337 frequency bins.

Audio chunking. We finally extract at most five short chunks of 320ms (i.e., 16 frames) from the global STFTs. This duration constitutes a tradeoff between detection latency and performance. The longer the method is offered to listen, the more accurate the results will be. However, in a dynamic robotics context, which we ultimately target, we cannot afford to use long audio sequences to infer the source positions.

Minimal energy criteria. We aim to prevent the inclusion of samples where one of the target sources is not active enough for the recording duration.

Given its STFT $S \in \mathbb{C}^{T \times F}$, the average energy² of a real-valued signal, expressed in decibels (dB), is defined as

$$E(S)_{\text{dB}} = \frac{1}{TF} \sum_{t=1}^T \sum_{f=1}^F 20 \log_{10} |S[t, f]|. \quad (3.16)$$

We reject the chunks of the simulated samples where, for at least one microphone, the energy of the selected fragment is too low compared to the average energy of the entire simulated signal. Let

- S_k the STFT of the signal received by microphone k .
- $\tilde{S}_k$ the STFT of the considered chunk, i.e., a slice of S_k .

The energy criteria δ_{energy} defining a valid sample expresses as

$$\delta_{\text{energy}}(\tau_E) = \bigwedge_{k=1}^4 \left[E(\tilde{S}_k)_{\text{dB}} > E(S_k)_{\text{dB}} - \tau_E \right]. \quad (3.17)$$

where τ_E has been set to 10dB in our primary dataset. The average energy of a given chunk can be at most 10dB lower than that of the entire signal. In practice, around 40% of the generated chunks are rejected.

²Strictly speaking, this quantity is dimensionally equivalent to a spectral energy density. For clarity, we will further refer to it as *energy*.

The STFT of each multi-channel 360ms segment provides the dataset's final training samples. Besides each input sample, the relevant ground truth information gets saved for supervising the learning process and computing performance metrics. It comprises all the necessary geometric information about the microphone array and sources (positions, orientations, relative distance, and angle of incidence). One million of such sample pairs constitute the core training and test datasets (800k and 200k samples, respectively). The total audio duration of the data approximates 47 hours.

3.3.1.2 Direction of Arrival Representation

The objective of the SSL task is to predict the DoA of the sound sources. Hence, the number of predictions outputted by an SSL method can differ from situation to situation. We therefore decided to use a representation of this information that is agnostic to the number of sources. Such a property is of great interest when training a deep neural network. The model can then have a fixed output while still being able to handle a varying number of sources. The latter will be further denoted n_s . The set of DoA values will be noted $\Theta = (\theta_1, \dots, \theta_{n_s})$.

Spatial Spectrum

The solution in question has been introduced by He et al. [187] and entails estimating the spatial spectrum. The spatial spectrum is a real-valued function of the DoA ($\phi : [-\pi, \pi] \rightarrow \mathbb{R}$). We discretize this continuous function by encoding the spectra in a d -dimensional real vector o :

$$o \in [0, 1]^d. \quad (3.18)$$

We denote ϕ_i the angle value corresponding to the i -th index of o :

$$\begin{aligned} \phi : [1, d] &\longrightarrow [-\pi, \pi] \\ i &\longmapsto \phi_i. \end{aligned} \quad (3.19)$$

We naturally have

- $\phi_1 = -\pi$,
- $\phi_{\lfloor \frac{d}{2} \rfloor} \simeq 0$,
- $\phi_d = \pi$.

We choose $d = 360$, corresponding to a 1° resolution. Higher spectrum numerical values indicate the presence of a source at this location. Those angles, being DoAs, are relative to the microphone array's orientation. A peak at 0° designates the presence of a source in front of the microphones.

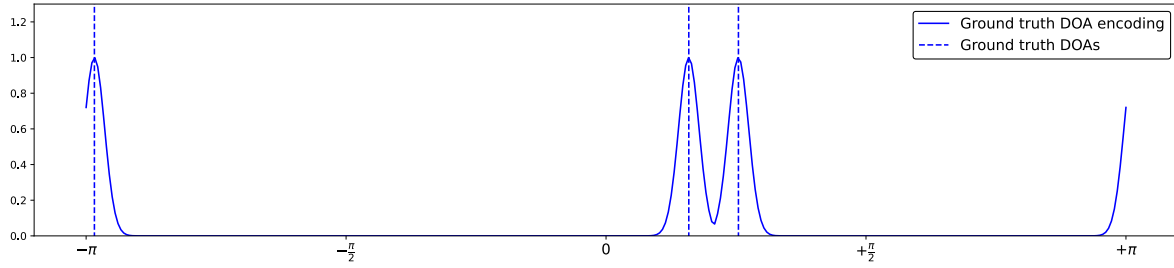


Figure 3.9 – DoA spectrum encoding of three sources.

Multi-Source DoA Encoding

The dataset contains the DoA values for each sample. We need to convert this list of scalar angular values to our spatial spectrum encoding format so that we can use it as a regression target. Numerous methods could be employed to achieve this. A first solution to this problem could be placing a pseudo-Dirac at the exact location of the sources.

$$o(\Theta)_i := \begin{cases} 1 & \text{if } \exists \theta \in \Theta \mid \phi_i = \theta \\ 0 & \text{otherwise,} \end{cases} \quad (3.20)$$

Instead, a more robust regression target is introduced. It consists in combining n_s unnormalized Gaussians centered at each DoA angles:

$$o(\Theta)_i := \begin{cases} \max_{\theta \in \Theta} \left\{ e^{-\frac{d(\phi_i, \theta)^2}{\sigma^2}} \right\} & \text{if } n_s > 0 \\ 0 & \text{otherwise,} \end{cases} \quad (3.21)$$

where d is the symmetric angular distance introduced in Section 3.2.3.1 (Equation (3.11)). We chose to set $\sigma = 5^\circ$. Figure 3.9 shows an example of the DOA encoding scheme for a situation with two sources.

Detection Decoding

The employed DoA encoding presented in Section 3.3.1.2 presents several advantages. Namely, thanks to its flexibility, it allows for the representation of a variable number of sources. Also, it enables the framing of the multi-source SSL problem as a simple regression task. However, to extract the set of actual DoA values, one has to explicitly process the obtained spatial spectra. This is achieved by detecting the peaks in the network output. The index of local maxima higher than a threshold ξ serve as the DoA predictions:

$$\hat{y}(\hat{o}, \xi) = \left\{ \phi_i : \hat{o}_i > \xi \text{ and } \hat{o}_i = \max_{\substack{j \in \llbracket 1, d \rrbracket, \\ d(\phi_i, \phi_j) < \sigma_n}} \hat{o}_j, \quad i \in \llbracket 1, d \rrbracket \right\}. \quad (3.22)$$

For this process to succeed, the neighborhood threshold σ_n must be defined carefully. If too low, some high-frequency noise in the spatial spectrum could lead to several false-positive angle detections. On the other hand, a large value of σ_n might cause two close peaks to be

wrongly identified as a single one, thus missing a positive detection. We have found $\sigma_n = 8^\circ$ to be a satisfying value.

When the number z of active sources is known, Equation (3.22) can be adapted as:

$$\hat{y}(\hat{o}; z) = \left\{ \phi_i : \text{among the } z \text{ greatest } \hat{o}_i = \max_{\substack{j \in \llbracket 1, d \rrbracket, \\ d(\phi_i, \phi_j) < \sigma_n}} \hat{o}_j, \quad i \in \llbracket 1, d \rrbracket \right\}. \quad (3.23)$$

The z highest peaks are used as the predicted angles.

3.3.1.3 Neural Network Architecture

The implemented neural network for multi-source localization is inspired by the one proposed by He et al. [13]. It aims to process multi-channel audio data and extract the angular positions of the speech sources. The model's input is the multi-channel signal's STFT representation. The STFT of a signal is a complex-valued matrix of size $F \times T$. We then split the real and imaginary values to form two distinct matrices. Each one of the M microphones leads to a $(2, F, T)$ -shape real-valued tensor. Its shape is noted (C, F, T) , where C is the number of channels, i.e., twice the number of microphones in the array.

The architecture draws inspiration from vision models by employing 2D convolution. As discussed in Section 3.1.4, using the image-like time-frequency representation of audio signals allows applying techniques proven to perform well on conventional image data. Figure 3.10 depicts the proposed network architecture and details the inner layout of the convolutional and residual building blocks.

3.3.2 Experiments

3.3.2.1 Metrics

To evaluate the performance of the proposed SSL method, a total of four metrics are computed. Each pair of metrics targets a different task.

Known number of sources. In this first task, the number of active sources is assumed to be known. Equation (3.23) is then used to perform the decoding of the neural network output. This process ensures that the number of detections matches the number of ground-truth sources and allows for the computation of the following two metrics.

- The *MAE* gives an idea about the angular distance between detected and correct angles:

$$\text{MAE} = \frac{\sum_i \sum_{j=1}^{z_i} d(\hat{\phi}_{ij}, \phi_{ij})}{\sum_i z_i}, \quad (3.24)$$

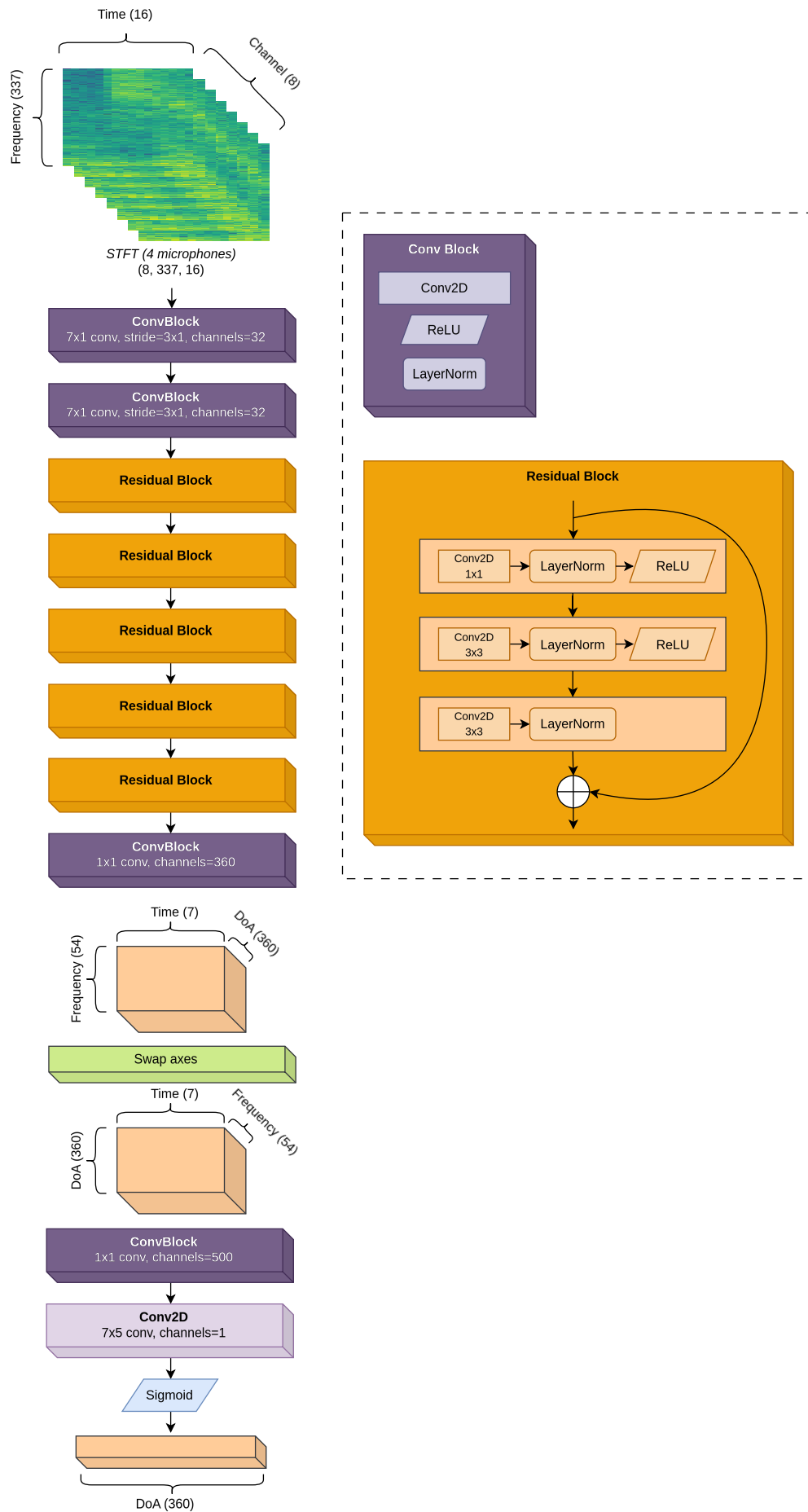


Figure 3.10 – Deep neural network architecture for multi-source SSL.

where z_i is the known number of active sources in sample i and d refers to the symmetric angular distance defined in Equation (3.11). The MAE will most often be expressed in degrees for convenience.

The *Accuracy* (ACC) constitutes the second metric for this framework and, given an error threshold E_a , provides the proportion of correctly localized sources:

$$\text{ACC} = \frac{\sum_i \sum_{j=1}^{z_i} \mathbb{1}_{d(\hat{\phi}_{ij}, \phi_{ij}) < E_a}}{\sum_i z_i}. \quad (3.25)$$

Unknown number of sources. The second task also evaluates the model's ability to accurately predict an unknown number of DoA values. Such a setup resembles a conventional single-class detection problem in the computer vision literature. In this case, the matching between ground truth DoA angles $y_i = \{\phi_{ij} : j = 1, \dots, z_i\}$ and the predictions $\hat{y}_i = \{\phi_{ik} : k = 1, \dots, \hat{z}_i\}$ extracted by applying Equation (3.22) might be incomplete, i.e. $\hat{z}_i \neq z_i$.

The following function denotes a positive match:

$$m(\hat{\phi}_{ik}, \phi_{ij}) = \begin{cases} 1 & \text{if } d(\hat{\phi}_{ik}, \phi_{ij}) < E_a \text{ and } k = \underset{k' \in \{1, \dots, \hat{z}_i\}}{\operatorname{argmin}} d(\hat{\phi}_{ik'}, \phi_{ij}) \\ 0 & \text{otherwise,} \end{cases} \quad (3.26)$$

We can then introduce the two metrics used in this framework, *Precision* and *Recall*:

$$\text{Precision} = \frac{\sum_i \sum_{j=1}^{z_i} \sum_{k=1}^{\hat{z}_i} m(\hat{\phi}_{ik}, \phi_{ij})}{\sum_i \hat{z}_i}, \quad (3.27)$$

$$\text{Recall} = \frac{\sum_i \sum_{j=1}^{z_i} \sum_{k=1}^{\hat{z}_i} m(\hat{\phi}_{ik}, \phi_{ij})}{\sum_i z_i}. \quad (3.28)$$

As an important note, those two scenarios are used to grasp the overall performance of a given model. The method stays the same in both cases as solely the extraction of the prediction employs either Equation (3.22) or Equation (3.23).

3.3.2.2 Loss and Convergence

Loss function. The objective used by He et al. in [187] along their DoA encoding is a simple MSE loss between the ground truth DoA representation and the output vector provided by the neural network:

$$\mathcal{L}(\hat{o}, o) = \|\hat{o} - o\|_2^2. \quad (3.29)$$

Sub-Optimal Convergence

Local minimum. Several experiments were conducted to identify working hyperparameters for the proposed SSL method. An interesting observation has been the impact of the learning

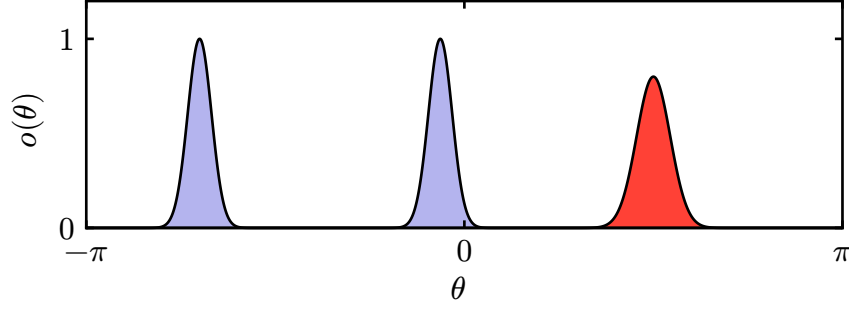


Figure 3.11 – Illustrative example of a poor network prediction (in red) and the corresponding ground truth spectrum (in violet).

rate and batch size combination. Theoretically, a larger batch size leads to more accurate gradients and thus, more sensible parameter updates. This also allows for an increase in the learning rate and a reduction in overall training steps. In most cases, the acceleration hardware and its inherently finite memory capacity dictate the limit for the maximum usable batch size. However, in our situation, a different restraining factor has been observed. There exists a trivial local optimum for the SSL task defined as a DoA spatial spectrum regression. Indeed, because of the relative sparsity of the ground truth encoding, a method outputting a plain zero spectrum achieves a comparatively low loss. More precisely, the loss for the samples would approximately equal the area under n_s distinct Gaussians. Some limited mass placed randomly on the $[-\pi, \pi]$ interval would likely not overlap with the ground-truth Gaussians. The loss would most often approach its upper bound:

$$0 \leq \mathcal{L}(\mathbf{o}, \hat{\mathbf{o}}) < \mathcal{L}(\mathbf{o}, 0) + \mathcal{L}(\hat{\mathbf{o}}, 0). \quad (3.30)$$

From this observation, the strategy of enforcing $\hat{\mathbf{o}} = 0$ ensures the loss will never exceed $\mathcal{L}(\mathbf{o}, 0)$. To prevent this phenomenon from occurring, both the batch size and the learning rate needed to be carefully chosen. To empirically illustrate this behavior, we monitor in Figure 3.12 the ℓ^2 norm $\|\mathbf{o}\|_2^2$ of the network output, defined by:

$$\|\mathbf{o}\|_2^2 = \frac{1}{d} \sum_{i=1}^d o_i^2, \quad (3.31)$$

along a successful training process.

We can distinguish two distinct phases:

- First, the network exploits a trivial local optimum, which entails predicting a null output. Both the loss and the output norm reach stable values.
- Subsequently, from the 50,000th step onward, the model escapes this plateau and successfully learns to solve the regression task.

When using too important batch sizes or too aggressive learning rates, the model indefinitely stagnates, keeping predicting zeros. Keskar et al. [196] have documented the adverse effects that large batch sizes could have on generalization performance.

Identifying, characterizing, and overcoming this shortcoming was essential in developing this model.

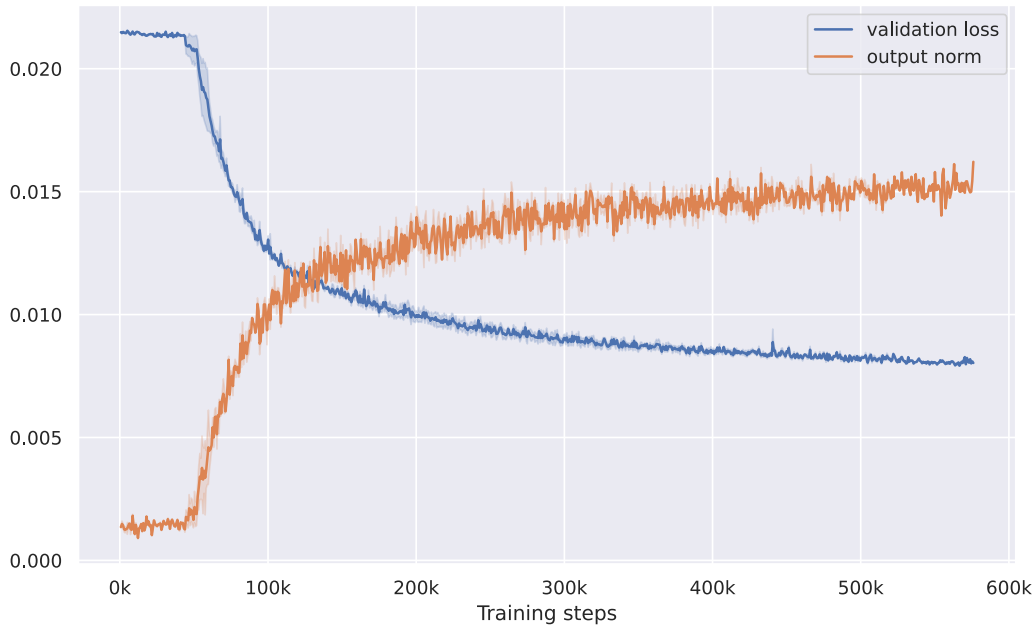


Figure 3.12 – Evolution of the norm of the network output $\|o\|_2^2$ (orange) and loss (blue) during training.

3.3.2.3 Performance Evaluation

Impact of the number of sources. As explained in [Section 3.3.1.1](#), the dataset allows for dynamically selecting a subset of 0 to 4 sources at runtime. This feature has allowed us to experiment with the impact of how many sources are present in the room simultaneously.

Training frameworks. On the one hand, the two following training setups can be compared:

- *Scenario A* is the setup proposed in [13] with the following repartition of samples:
 - 0 sources: 20%,
 - 1 source: 40%,
 - 2 sources: 30%,
 - 3 sources: 5%,
 - 4 sources: 5%.
- *Scenario B* uniformly chooses a number of sources between one and four for each sample. Thus, it is more challenging as at least one source is always present in the room, and significantly more samples present 3 or 4 sources.

As no artificial noise is added to the speech sources' signals, the training dataset in *scenario A* brings exactly 160k identical samples, whose observation tensor is null. Once the network successfully learns that it should output a zero-vector for those trivial samples, they will not contribute to increasing or lowering the detection scores.

Furthermore, the more sources are simultaneously present in the room, the more challenging it becomes to localize them properly. [Table 3.6](#) displays the final performance of models trained on datasets corresponding to each scenario.

Table 3.6 – SSL performance when trained with different number of sources.

	Dataset A	Dataset B
MAE (°) ↓	9.13	14.05
Accuracy (%) ↑	71.36	61.76
Precision (%) ↑	80.98	76.96
Recall (%) ↑	69.26	58.53

Table 3.7 – SSL performance depending on the number of active sources.

	MAE (°) ↓	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑
1 source	2.59	88.58	87.70	88.70
2 sources	7.26	70.78	79.56	68.07
3 sources	15.89	58.18	74.99	54.73
4 sources	21.95	50.21	72.08	46.22
Scenario A (0-4 sources)	9.13	71.36	80.99	69.26
Scenario B (1-4 sources)	15.24	60.52	76.73	57.36

One should note that both training and test datasets are different. The goal of this experiment is to highlight the consequential impact that the problem formulation can have on performance. Notably, it compares both aforementioned scenarios. The rest of the experiments have been conducted with respect to *Scenario A*, following the same distribution of source numbers as He et al. used in [13].

Evaluation frameworks. We have evaluated a given network in various scenarios to understand the impact of the number of concurrent sources in the room on performance. The network has been trained according to the *scenario A* presented above. [Table 3.7](#) summarizes the performance on the aforementioned model in different evaluation scenarios. We observe that the fewer the number of active sources, the higher the performance. Even when evaluating the model in the hardest scenario, where four sources are always active at the same time, MAE remains lower than 22°. However, accuracy and recall suffer in those challenging situations.

3.3.2.4 ϵ -Loss

We propose an original modification of the loss function. The motivation comes from the observation that the target DoA spatial spectrum is sparse (see [Figure 3.9](#) for instance). This causes the active part of the spectrum to have a limited impact on the gradients. As seen previously, we use a simple MSE loss for the cost function. Hence, predicting high activation will be heavily penalized as it will statistically correspond to false positives. However, predicting overall low values will not lead to high loss values as the ground truth spectrogram is primarily

Table 3.8 – Performance of the SSL model trained with the ε -loss.

	MAE (°) ↓	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑
$\emptyset$	9.36	70.56	81.04	68.36
$\varepsilon = 0.1$	8.17	71.68	67.86	70.62
$\varepsilon = 0.2$	8.29	71.02	70.36	69.86
$\varepsilon = 0.4$	8.13	71.99	75.96	70.28
$\varepsilon = 0.6$	8.32	71.60	76.94	69.88
$\varepsilon = 1.0$	8.49	71.38	76.87	69.61

flat. This leads to the sub-optimal convergence phenomenon described in Section 3.3.2.2. The naive MSE loss does not prioritize the supervision in the active region of the spectrogram. We have attempted to circumvent this by more aggressively penalizing the sections of the spatial spectrum where sources are effectively present. To circumvent this phenomenon, we introduce the following ε -loss:

$$\mathcal{L}_\varepsilon(\hat{o}_i, o_i) = \frac{1}{d} \sum_{i=1}^d (o_i + \varepsilon)(\hat{o}_i - o_i)^2. \quad (3.32)$$

The damping term $(o_i + \varepsilon)$ reduces the penalization of false positive peaks in the estimated spectrogram. In a region of the spectrum where no sources are effectively present, i.e., where $o \approx 0$, the loss value will be bounded by ε .

Table 3.8 summarizes the performance of our model after being trained with the ε -loss. More precisely, we compare different values of ε to better measure its influence on performance. A baseline corresponding to the MSE loss is also included for comparison. This ablation study suggests that a value of $\varepsilon = 0.4$ improves the MAE and accuracy scores at the expense of losing some precision points. A value closer to 0.1 slightly boosts the recall, too, but comes with an important dip in precision. Also, this choice has an impact on the training dynamics. The ε loss seems to shorten the initial stagnating phase of the training process, where the validation loss sees no improvement. When training with the MSE, the loss only starts improving after 50 to 60k steps, while this number falls to 16k steps when using the ε loss. Unfortunately, the behavior of the ε loss remains unclear in some aspects. Overall, its benefits are not satisfying enough to be included in the final method. Notably, its impact on precision is detrimental to the overall performance.

3.3.2.5 Normalization

Background

Various schemes of normalization have been used in Deep Neural Networks. They address the phenomenon of *internal covariate shift*, which appears as architectures deepen. This problem comes from the change in the distribution of each layer’s inputs during training. Such a drift causes the non-linear activation functions to saturate and harms the learning process.

Normalization also attempts to reduce the effects of mismatch between the training and test dataset distributions.

Batch Normalization (BatchNorm), proposed by Ioffe et al. [197], has gathered significant success, especially in the computer vision community. It entails normalizing the activations within each mini-batch using that mini-batch's own mean and variance. Acting as a form of regularizer, this process stabilizes learning by ensuring that the values entering all layers do not deviate too significantly. The data will be distributed according to a standard normal distribution. The *Batch Normalization Transform* algorithm is expressed as such:

$$y_i = \gamma \left[\frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \right] + \beta, \quad (3.33)$$

where

- x_i is an individual entry in the mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$,
- $\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i$ is the mini-batch mean,
- $\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$ is the mini-batch variance,
- ε is a constant ensuring numerical stability,
- γ and β are learnable parameters.

To be able to perform inference on single samples, i.e., without disposing of an entire mini-batch, substitution statistics are used in place of $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$. Indeed, during training, the BatchNorm layer will keep updating a running mean and variance to be used at evaluation time.

Layer Normalization (LayerNorm) (Ba et al. [198]) follows the same principle but chooses to normalize each sample individually by computing statistics across the features' dimensions. **Figure 3.13** displays the differences between both schemes. Historically, LayerNorm has been most commonly employed in Natural Language Processing.

$$y_{l,i} = \gamma \left[\frac{x_{l,i} - \mu_l}{\sqrt{\sigma_l^2 + \varepsilon}} \right] + \beta, \quad (3.34)$$

where

- $x_{l,i}$ is an individual hidden unit in the l -th layer's inputs $X = \{x_{l,1}, \dots, x_{l,H}\}$,
- $\mu_l = \frac{1}{H} \sum_{i=1}^H x_{l,i}$ is the mean,
- $\sigma_l^2 = \frac{1}{H} \sum_{i=1}^H (x_{l,i} - \mu_l)^2$ is the variance,
- ε is a constant ensuring numerical stability,
- γ and β are learnable parameters.

Those two methods have proven effective in training deep neural network architectures. Ren et al. [200] develop a unified view of the BatchNorm and LayerNorm schemes. Furthermore, they proposed a novel addition to better handle sparsity and achieved better results in various downstream tasks. Similarly, the *PowerNorm* scheme, introduced by Shen et al. [201], attempts to circumvent the identified weaknesses of the existing normalization schemes when applied

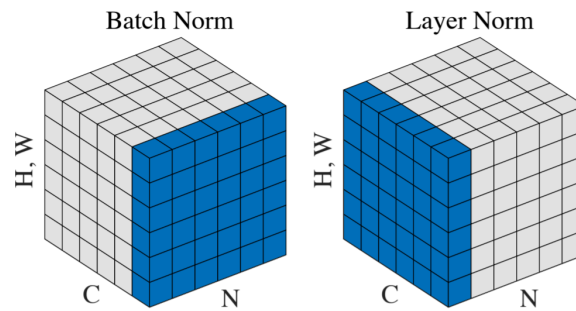


Figure 3.13 – A visual comparison of Batch and Layer Normalization, adapted from [199].

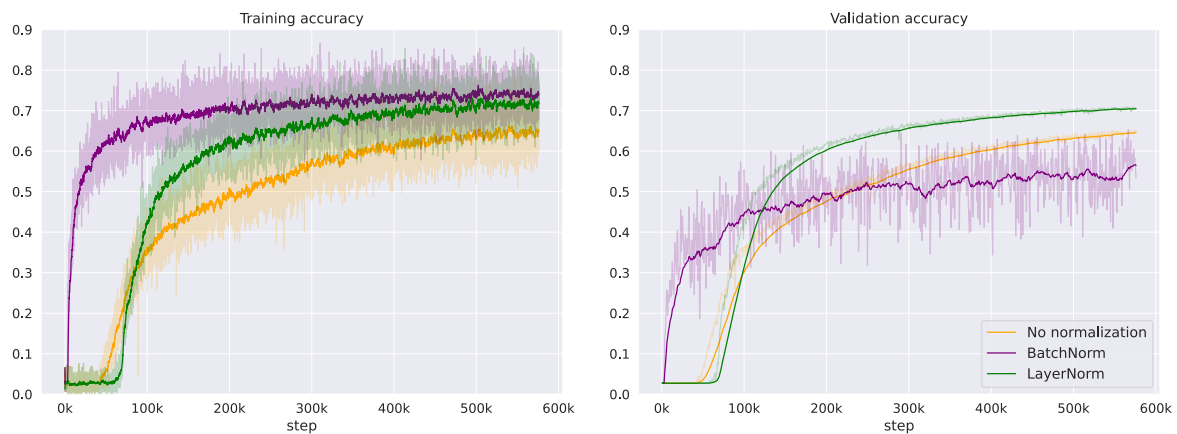


Figure 3.14 – Training and validation accuracies during training for different normalization schemes.

to the transformer architecture. Those works further demonstrate the importance of normalization in deep neural networks.

Experiments

Although He et al. chose to use BatchNorm in their work, our final architecture employs the more flexible LayerNorm. The choice of the normalization scheme ended up being crucial to achieving good performance. We observed that the latter yielded the same stabilization benefits during training while removing the dependence on the batch size.

Figure 3.14 illustrates the evolution of the training and validation accuracy during training for different normalization schemes.

This specific metric clearly exposes the differences between those three choices but the other metrics behave similarly. Both normalization techniques bring additional stability and performance to the training process. However, significant differences arise when looking at the validation metrics. When run in evaluation mode, i.e., using the running statistics gathered during training, the network trained with BatchNorm performs poorly compared to training. This would suggest that the saved means and averages do not adequately account for the differences between the training and validation sets.

Table 3.9 – Final performance of SSL for different normalization/evaluation schemes.

Normal- ization layer	eval mode	Batch size	MAE (°) ↓	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑
Batch- Norm	False	1	42.10	26.03	12.39	51.43
		50	9.32	73.00	83.70	70.61
		100	9.11	73.43	84.26	70.96
		200	9.07	73.66	84.71	71.19
		500	8.95	73.76	84.78	71.35
	True	-	29.58	53.45	45.37	61.00
Layer- Norm	True	-	9.37	70.35	80.21	68.26

Interestingly, evaluating this network’s performance while forcing the batch normalization to use the training strategy avoids facing this issue. Indeed, using the current validation batch statistics instead of those gathered at training time provides results that are on par with the training performance. This constitutes an essential limitation of batch normalization in this case, as the evaluation thus needs to be performed in a batched manner. Table 3.9 displays the batch size’s influence on the network’s performance trained with BatchNorm.

Those results depict the positive role played by larger batch sizes for evaluation in *training mode*. Although in a purely synthetic benchmark, this does not constitute an important drawback, it will become one as soon as the model will be asked to perform one-shot inference. In our robotics context, the developed SSL solution would have to be able to be used in a real-world scenario where an entire batch of observations is not available for inference.

This is what motivated the enhancement of the model using other normalization schemes. As explained in Section 3.3.2.5, LayerNorm does not encompass this behavioral distinction between training and evaluation. Table 3.9 also compares the final performance of the layer normalization strategy.

Overall, LayerNorm and BatchNorm offer comparable performance. However, the model trained with LayerNorm behaves consistently when used in evaluation. For those reasons, we have preferred this approach over the original one.

3.3.2.6 Exploring How Context Length Matters

The choice of the signal duration used for training the localizer has some importance. A trade-off needs to be made between the system’s reactivity and detection performance. Naturally, disposing of longer sequences of input audio is suspected to lead to higher metrics values. On the other hand, restricting the snippet length even further might hinder the robustness of the results. Also, when available, a pre-trained method should be able to leverage longer segments of audio to refine its prediction.

To evaluate those assumptions quantitatively, we train our neural network on different context lengths. Each training dataset has been generated from audio recordings of a given fixed duration, translating into STFT tensors of T_{train} frames. As presented in Section 3.3.1.1, the baseline duration of used recordings amounts to roughly 320ms of audio ($T_{\text{train}} = 16$). Here, training on shorter durations has also been attempted. We test the models obtained from each value of T_{train} on samples of equal or larger durations by performing subsequent forward passes. This process will be referred to as *sequence processing*.

Sequence processing. The main idea of sequence processing is splitting the longer input audio into M chunks of T_{train} frames to be processed individually by the neural network. M output DoA spectra are thus obtained and must be aggregated. We average those signals to obtain a single vector:

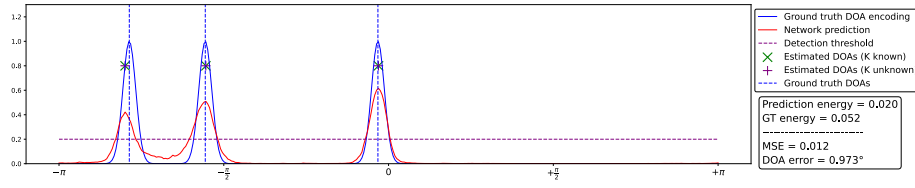
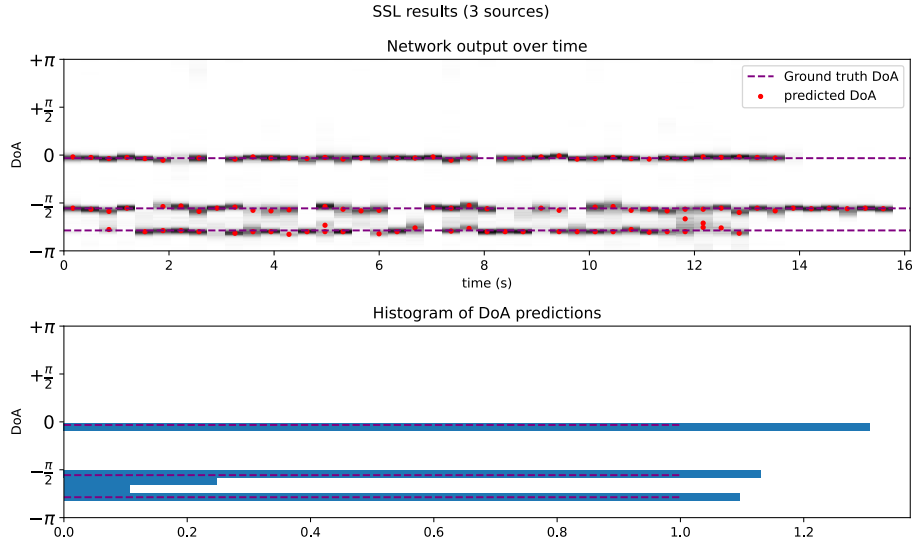
$$\hat{o} = \frac{1}{M} \sum_{i=1}^M \hat{o}_k \in [0, 1]^d. \quad (3.35)$$

The flexibility of the DoA spatial spectrum encoding permits the former combination without the need of additional steps. The detection algorithm can then be applied on the average output.

Sequence dataset generation. To evaluate the performance of the obtained models, a new dataset $\mathcal{D}_{\text{full}}$ is generated. Instead of saving 16 frames long individual STFT chunks, we record the features for recordings of several seconds. To generate each sample, each active source outputs one recorded sentence from the *LibriSpeech* [81] dataset. STFTs of the multi-channel signals received by the microphone array from each source are saved independently. Disposing of features corresponding to several seconds of simulation allows for performing SSL on context windows of varying lengths.

Figure 3.15 illustrates the sequence processing workflow on a single example. Here, around 16 seconds of continuous speech produced by three distinct static sources get recorded by the microphone array. The latter also stands at a fixed position in the room. Figure 3.15a depicts the averaged DoA spectrum $\hat{o}$ along with the corresponding overall predictions. On this specific example, the averaging process successfully aggregates the angular information and allows for the accurate localization of all three sources. More precisely, the top part of Figure 3.15b displays the network output at each time step. The gray-scale patches represent the individual estimated DoA spectra $\hat{o}_k$. The resulting predicted angles are highlighted by the red dots. Finally, the histogram of predictions characterizes the distribution of detections along the process.

Presenting the SSL results as such highlights the strengths and weaknesses of the proposed approach. Even for a single frame, the estimated DoA spectrum allows for precise predictions. Very few false positives are observed, as confirmed by the several quantitative experiments conducted. However, individual sources are sometimes missed, maybe because they were not active enough at this specific time. This drawback gets offset by leveraging the overall consistency of the method over a longer time.

(a) Averaged DoA spectrum $\hat{o}$.

(b) Network output and extracted detections over time (top) and histogram of predictions (bottom).

Figure 3.15 – Example of a sequence processing result.

In order to further characterize this behavior, we have executed an exhaustive performance evaluation of the sequence processing workflow. Table 3.10 summarizes the results from the conducted experiments. Leveraging the aforementioned generated dataset $\mathcal{D}_{\text{full}}$, containing full audio recordings, it was possible to evaluate multiple models on different context lengths.

For evaluation durations d_{eval} from 21ms to 320ms, we compare models trained on different sub-factors of T_{eval} frames performing *sequence processing*. A network trained on STFTs of T_{train} frames gets evaluated $N_{\text{pass}} = \frac{T_{\text{eval}}}{T_{\text{train}}}$ times using the *sequence processing* method. The results unsurprisingly suggest that directly training a model on T_{eval} frames will always yield better results than averaging inference results of a model trained on fewer frames.

However, *sequence processing* still holds value by allowing a given pre-trained network to leverage lengthier recordings. The base model trained on $T_{\text{train}} = 16$ samples has been evaluated on signals ranging from 320ms, its base context length, up to more than 10s. The last row of Table 3.10 comes from an experiment where the entire recordings were provided to the network. As many forward passes as needed were performed for each sample to exploit the complete data. It appears clearly that the longer the agent is able to hear, the better its

Table 3.10 – SSL performance depending on the input duration.

Settings			Metrics			
T_{eval} (d_{eval})	T_{train}	N_{pass}	MAE (°) ↓	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑
2 (21ms)	2	1	29.31	39.11	58.83	38.04
4 (64ms)	4	1	21.28	50.73	68.83	49.37
	2	2	24.51	47.34	68.87	46.47
8 (149ms)	8	1	12.84	62.05	75.33	61.18
	4	2	15.13	58.76	75.10	57.21
	2	4	19.76	54.40	76.18	52.95
16 (320ms)	16	1	8.71	72.99	83.35	71.73
	8	2	9.92	69.51	82.26	67.76
	4	4	13.85	63.99	82.84	62.15
	2	8	19.56	58.86	83.64	56.92
32 (661ms)	16	2	6.20	80.15	89.46	78.03
64 (1.34s)	16	4	5.43	83.64	92.88	80.83
512 (10.9s)	16	32	4.18	86.83	95.48	83.93
full	16	-	4.14	86.35	95.25	83.54

localization performance becomes. In this way, we account for missed detections and enhance the robustness of the system.

The base context window of 16 STFT frames amounts to approximately 320ms, a fairly short period. During this interval, one or more speech sources could be inactive as the energy criteria $\delta_{\text{energy}}(\tau_E)$ is not enforced on this specific data set. This sole difference in the data generation process explains the gap in performance between this experiment and the evaluation on the normal dataset reported in Section 3.3.2.3 (see Table 3.7 for example).

3.3.2.7 Ablation Study on Sources' Angular Proximity

The decoding process, presented in Section 3.3.1.2, involves extracting the local maxima of the predicted DoA spectrum. The abscissas of the resulting peaks are considered as the final angle values. As such, our method is expected to be challenged by samples involving sources with close DoA values.

Let $\Delta\theta_{\min}$ be the angle difference between the two closest sources in terms of their DoA:

$$\Delta\theta_{\min} = \min_{\substack{i,j \in [1, n_s] \\ i \neq j}} d(\theta_i, \theta_j), \quad (3.36)$$

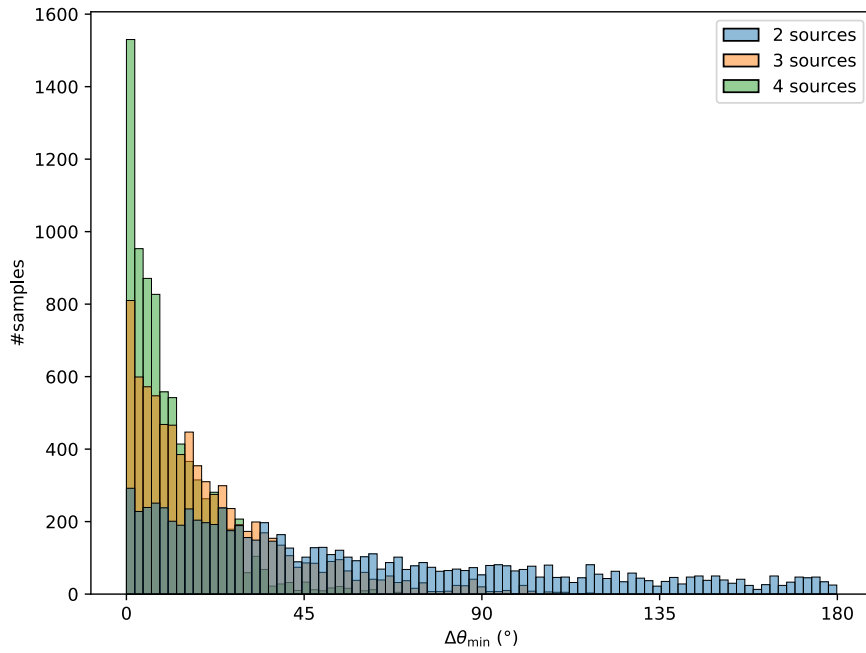


Figure 3.16 – Distribution of $\Delta\theta_{\min}$ for different numbers of active sources.

where $(\theta_i)_{i=1\dots n_s}$ are the real DoA values for this sample and d is the angle distance introduced in Equation (3.11). It should be noted that this quantity can only be defined for samples encompassing at least two sources.

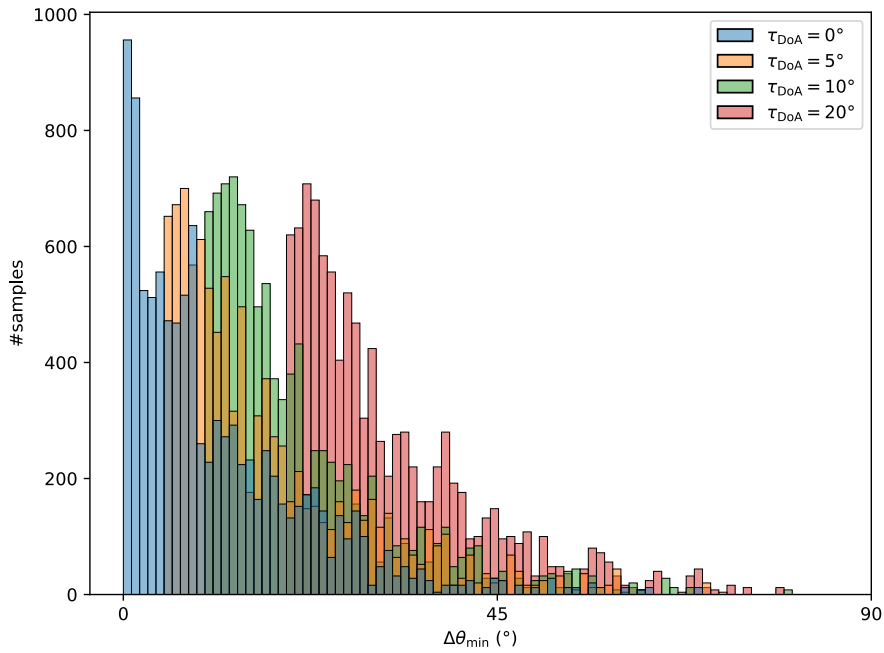
From a single original dataset, generated with $n_s = 4$ sources active in each of the 8000 samples, $\Delta\theta_{\min}$ is evaluated in three cases. In the first scenario, all four sources remain active, and the four corresponding DoA values are used to compute $\Delta\theta_{\min}$. On the other two, only two (respectively three) random sources are enabled simultaneously in every sample. Figure 3.16 depicts the distribution of $\Delta\theta_{\min}$ depending on this number of active sources. Naturally, when only two sources are present concurrently, high values of $\Delta\theta_{\min}$ remain likely. Yet, increasing the number of sources tends to decrease their likelihood, and the minimum DoA gap more often reaches lower values. Hence, the correlation between the number of sources and the difficulty of the SSL task highlighted in Section 3.3.2.3 might be caused by two underlying reasons. On the one hand, the model is expected to extract each speaker’s location from the mixture of speech signals that constitute its input. An increase in the number of sources inherently hardens this task. On the other hand, low $\Delta\theta_{\min}$ samples also become more frequent, which could contribute to hindering proper localization by itself.

To empirically study the impact of $\Delta\theta_{\min}$ on the SSL performance, the model trained on a regular dataset has been evaluated on specific test cases. Each test dataset ensures that it respects a lower bound τ_{DoA} such that $\Delta\theta_{\min} \geq \tau_{\text{DoA}}$ for all samples. Also, the number of sources is fixed to $n_s = 4$ to best isolate the influence of $\Delta\theta_{\min}$ on the results.

Figure 3.17 plots the distribution of $\Delta\theta_{\min}$ of all four test datasets.

Table 3.11 – SSL performance for different values of τ_{DoA} .

τ_{DoA} (°)	MAE (°) ↓	Accuracy (%) ↑	Precision (%) ↑	Recall (%) ↑
0	20.05	50.55	71.29	46.11
5	20.85	51.08	72.17	50.45
10	21.81	52.64	72.10	52.82
20	21.56	57.17	71.68	57.38

Figure 3.17 – Distribution of $\Delta\theta_{\min}$ for different values of τ_{DoA} .

The pre-trained model’s performance on each scenario is summarized in Table 3.11. Although MAE and Precision do not show to be meaningfully affected by $\Delta\theta_{\min}$, Accuracy and Recall improve by 6.5 and 11 points, respectively, across this range of scenarios. This observation confirms that samples with very low $\Delta\theta_{\min}$ constitute more difficult cases.

3.3.3 Discussion

Overall, we have bootstrapped a complete multi-source SSL pipeline by drawing crucial inspiration from the work of He et al. in [187], [138], and [13]. This method was adapted and partly enhanced to provide convincing localization results on challenging datasets collected thanks to the simulator presented in Chapter 2.

The motivation for operating this large implementation effort was not to compete with state-of-the-art SSL solutions. Conversely, the objective was to develop a flexible and effective platform that could be leveraged to achieve demanding dynamic robotic tasks.

The main advantage of the obtained method lies in its ability to handle a various number of sources. This contrasts with the approach presented in [Section 3.2](#) where only a single source can be localized simultaneously. Also, the network correctly handles cases where no speech sources are active in the environment. Given this module, an agent would be able to act depending on the detected speech activity. Furthermore, the short context window of 363ms on which the network has been trained allows for using our model in real-time situations. It thus permits a decision policy to act based on the localization results.

3.3.3.1 Limitations

Our framework still suffers from shortcomings. On the one hand, the obtained metric values, even though respectable, do not line up with other works. Indeed, although precise, as it reached more than 83% precision scores, the network sometimes misses some sources. Such a shortcoming motivated our investigation of sequence processing, in which we leverage multiple consecutive observations to enhance the method's performance. Adding adversarial noise sources in the room, employing white noise and, more realistically, sound from musical instruments or television would constitute an interesting extension of this work. It would thus be interesting to study the impact of noise on this approach.

3.4 Conclusion

This chapter exposes our investigations of the SSL task. Implementing different methods for this classic acoustic challenge constituted a significant engineering effort. The empirical aspect of training deep neural networks leads to numerous experiments being conducted. This is essential to guarantee the reproducibility of the results, which is not granted when stochastic approaches are used. Our approach was not motivated by beating state-of-the-art SSL techniques. Conversely, sensible yet straightforward and robust models were designed and thoroughly tested to explore the intricacies of the localization task. The acoustic simulator was leveraged to synthesize diverse datasets to study the limits of our solution. An initial CNN model handles single-source localization in reverberant environments. It can handle multi-channel spectral data recorded by microphone arrays of different structures. Next, a deeper architecture was implemented to tackle the more challenging problem of localizing several speech sources. We adapted a multi-source localization method from recent works in the domain. It was trained on large synthesized datasets generated for this purpose. Our contribution includes an extensive experimental campaign to evaluate the performance of the proposed methods. It also entails the complete pipeline to generate the datasets, train, and evaluate the network architectures. This framework, combined with our acoustic simulator, may help the research community to easily and efficiently iterate on new ideas to improve SSL methods. It could alleviate future research efforts from going through an expensive engineering task. Finally, both developed models were used as building blocks for this thesis's other contributions (see [Chapter 4](#) and [Chapter 5](#)).

Yet, the proposed methods have some limitations. Our models have been solely evaluated on synthetic data. Testing the model's performance on real recordings would be a valuable addition. Furthermore, embedding the localizers on a physical robotic platform would provide additional insight into their robustness to real-world constraints. An additional study of localization performance in the presence of adversarial noise sources was initially planned. Due to insufficient time, such experiments could not be satisfactorily conducted. To conclude, SSL is a diverse and challenging acoustic task that has benefited from extensive research effort. Nevertheless, many amelioration paths are still worth pursuing, especially regarding their application in social robotics.

Chapter 4

Active Sound Source Localization

Chapter 3 addressed the challenge of SSL in static environments, where both the robot and the sources remained fixed. While this formulation is useful for controlled experimentation, it does not adequately represent real-world scenarios encountered in social robotics. In such contexts, robots operate in dynamic spaces and must leverage movement to refine their perception of surrounding sound sources.

This chapter introduces the problem of Active Sound Source Localization (ASSL), in which the agent actively moves through the environment and uses its accumulated observations to localize sound sources—without actively controlling its trajectories. Unlike passive SSL, ASSL benefits from temporal and spatial integration, allowing the robot to resolve ambiguities and gather more informative observations over time. This dynamic setup is more representative of practical robotic applications, including speaker tracking and human-robot interaction.

ASSL presents new challenges: observations are temporally distributed, perception varies with position and orientation, and fusing noisy cues requires careful design. In this work, we propose a modular learning-based approach to address these issues. The method builds upon the static SSL model introduced earlier, augmenting it with a mechanism to aggregate directional cues into egocentric 2D maps. We explore two fusion strategies: a simple averaging method and a more advanced deep learning model trained to produce clean spatial heatmaps. These maps are then post-processed to extract the estimated source positions.

The rest of the chapter is organized as follows. Section 4.1 review related work on ASSL, highlighting its challenges and the range of approaches explored in the literature. Section 4.2 formally defines the problem addressed in this chapter. Section 4.3 presents our proposed method, detailing the full localization pipeline, including the generation and aggregation of egocentric maps and the detection strategy. Section 4.4 reports experimental results and ablation studies, analyzing the influence of key design choices. Finally, Section 4.5 concludes the chapter and discusses possible extensions and future research directions.

4.1 Background

The problem of SSL has been extensively studied in both the robotics and acoustic signal processing communities. While their research overlaps significantly, the two domains approach the problem from different perspectives. In signal processing, SSL is typically framed in terms of static microphone arrays and occasionally moving sound sources — a scenario often referred to as sound source tracking. In contrast, the robotics perspective introduces additional considerations due to the physical embodiment of robots. Even in the static case, robot-specific challenges arise, such as the need to acoustically model the material and geometry of the robot’s head or body (see Skaf et al. [202] for example). [Section 3.1.5](#) provides an overview of SSL in robotics, focusing on these specific challenges.

ASSL is a natural extension of SSL in the context of robotics, where the agent is not static but mobile. The ASSL problem does not follow a single, well-defined formulation; rather, its definition varies across studies and application domains. When the robot is capable of movement, the problem space expands significantly, and different research communities may adopt distinct perspectives. In all cases, the robot’s motion influences its acoustic perception — either by affecting the signals it receives or by enabling additional observations over time. This mobility introduces both challenges and opportunities for improved localization. A key factor distinguishing ASSL approaches is how the robot’s motion is handled: in some cases, it is explicitly optimized to enhance localization performance; in others, motion arises from external constraints or tasks, and the localization system must adapt accordingly. Wightman et al. [128] experimented on human listeners and identified movement as an effective way of dealing with front-back confusions in binaural hearing. Naturally, researchers have explored how a robot’s movements might help improve localization performance. Regardless of the formulation, mobility plays a central role in designing and evaluating localization strategies.

Nakadai et al. [203] designed a complex robotic system to perform *active audition*. This work consisted of a physical robotic prototype equipped with microphones and cameras for sensing the environment. The *active audition* task aims to estimate the DoA of the present sources accurately. To achieve this objective, both vision and audio signals are processed to enhance the robustness of the prediction. Also, the robot’s head movement automatically adjusts to face the currently active source. Visual data plays an essential role in refining the DoA estimation. The authors use the framework of Epipolar Geometry to compute an angle from both the camera images and the spectra of the signals received by left and right external microphones. Thanks to this angle, a directional band-pass filter is constructed, which permits dampening of noise sounds produced by the robot’s motors. This method circumvents the use of HRTF, which is challenging to access in real-world scenarios. They test this pipeline within a relatively simple experimental scenario. Two loudspeakers play a monotone sound at a given, distinct frequency. They operate sequentially, ensuring that only one source is active at any given time. The robotic head is expected to progressively face the active source so that it lands in its cameras’ field of view. This work was among the first attempts to perform SSL in a dynamic context. However, it presents a few limitations. On the one hand, the considered setting remains very simple. No authentic speech signals are employed, and the sources remain fixed. On the other hand, contrary to what this chapter will present, the agent does not actually

move in the room; only its head does. Finally, their system does not estimate the distance to the active source.

More recently, Nguyen et al. conducted a series of works [204], [205], [206] that tackle motion planning for robot auditions. They designed different probabilistic algorithms that attempt to localize an eventually moving sound source from a mobile robot. In [204], the authors proposed a mixture Kalman filter (MKF) to model the system's evolution. While angular SSL methods yield satisfying results, estimating the robot-source distance has proven significantly harder. Hence, this study focuses on fusing movement information with DoA estimation to predict the distance to the source. The state vector consists of the absolute position of the robot, the sound source, and the source's activity (whether it is active or not). The provided theoretical derivation of the MKF allows the estimation to be updated. The experiments illustrate the importance of the robot's movements in achieving successful localization. Indeed, at the beginning of the trajectories, the estimation suffers from the front-back ambiguity. As the robot moves and accumulates information, the accuracy of the prediction increases. Although the proposed method outperformed some existing solutions, the robot's trajectory was not optimized. In [205], the authors improved their framework by proposing a long-term motion planning algorithm to localize a sound source. They introduced a Monte Carlo tree search (MCTS) method to compute the optimal robot trajectory. The optimization objective is to reduce the entropy of the belief in the source localization. Shannon's entropy measures uncertainty and drives the exploration of the MCTS. This framework allows the motion planning algorithm to run on a predefined, limited computational budget. Experimental results show that the MCTS planning successfully minimizes the entropy during the trajectory compared to greedy or random algorithms. Furthermore, the average estimation error also sees an improvement using this approach. This pipeline has been subsequently extended in [206]. First, the MKF formulation is extended to handle intermittent sound sources better and to be more robust to erroneous measurements of sound activity and DoA. In addition to the entropy objective introduced in [205], the standard deviation of the estimated belief on the source location may now also be used to compute the optimal trajectory. Both criteria are compared in a thorough experimental study and are shown to successfully reduce the source location estimation error.

In the same period, Bustamante et al. presented a similar pipeline in a collection of articles [49], [207], [208], [209]. In their initial paper [207], they introduced a three-stage strategy to combine a mobile robot control scheme and the associated source location estimator. The first stage consists of the short-term detection of azimuth and activity. This step solely leverages the most recent binaural features. Their DoA estimator is a maximum likelihood estimator that directly takes the DFT of the listened signal as input. Both the single-source and multi-source cases are handled. The latter is addressed by an EM algorithm that iteratively performs source separation (E-step) and source localization (M-step). Conversely, the second stage aggregates the data over time and combines it with past motor commands. This provides a progressively finer estimate of the source's relative location. Finally, the third stage is computing the following control command by implementing a feedback loop. The first two phases were tested in an anechoic room. A technical solution for the third stage is provided in [208]. They model the source localization's posterior distribution as a Gaussian mixture and use a Kalman filter to predict the next state posterior pdf. The control commands are computed by solving an

optimization problem that maximizes the mutual information between the predicted sensor position and the sound source's measurement. This ensures that the sensor moves in a way that yields the most informative measurement at the next time step. This method was evaluated both in simulation and on a physical system. The complete pipeline is further detailed and tested in [209]. To enhance their three-staged approach, Bustamante et al. [49] extended the control command optimization to a multi-step ahead process. The optimization problem now consists of determining the sequence of controls that minimizes a cost function at each iteration. This objective directly incorporates the expected uncertainty on the source's position over multiple future steps. Overall, Bustamante's work demonstrates another approach to ASSL. On the one hand, it aggregates DoA estimations and robot movement over time to localize a sound source in 2D relative to the agent. On the other hand, it provides a controlling scheme for the agent that optimizes the location performance in the long term.

In addition to the works mentioned above, we subsequently mention other relevant references. Roman et al. [210] designed a binaural tracking system to localize several moving sources. They use a probabilistic model to predict the sources' locations from interaural cues (ILD and IPD). The solution's tracking capability relies on an Hidden Markov Model (HMM) formulation. It ensures that sources are tracked across time and imposes continuity constraints on the detections. The challenge of reverberant environments has not been considered in the methodology. However, the authors do test their system in the presence of reverberation. Performance is shown to suffer from increasing the T_{60} . Working on the robustness of localization systems to reverberation is a key area for future improvement. Kneip et al. [211] claim that combining rotation and translation movements from a robot significantly enhances localization performance. Argentieri et al.'s survey on SSL in robotics [178] includes additional references to works in ASSL. Yet, the authors conclude that *active audition* would benefit from additional research effort. The BINAural Active Audition for Humanoid Robots (BINAHR) project was a French-Japanese collaboration on robotic auditory perception. It allowed for several publications on active audition (see Markovic et al. [212] and Portello et al. [213], for example). In [176], Gala et al. derive a mathematical model for predicting both the DoA and distance to a single static sound source solely from the ITD. They achieve this by using a robotic head that can rotate. It can thus leverage the recorded ITD at different angular positions. Adavanne et al. [153] handle multiple sound sources' localization, detection, and tracking. The framework is tested in both anechoic and reverberant environments. Their CRNN architecture successfully aggregates information over time, allowing for strong tracking performance.

In conclusion, active SSL has been a topic of interest in the robotic literature. Indeed, static localization approaches suffer from limitations such as the front-back ambiguity or distance non-observability. The literature contains successful examples of ASSL approaches. Most existing strategies rely on probabilistic and information-theoretic frameworks and derive analytical schemes. No deep-learning-based solutions have been identified. This chapter introduces a novel framework for ASSL leveraging an original DNN architecture.

4.2 Problem Formulation

As presented in [Section 4.1](#), experimenting with SSL in mobile robotic settings presents challenges and opportunities. In this work, we introduce and explore an ASSL problem, motivated by the extension of the static SSL methods developed in [Chapter 3](#) to more realistic situations. Indeed, estimating both distance and DoA from a single recording has been shown to be a challenging version of the SSL problem. Grumiaux et al. [69] insist on this difficulty in their survey by highlighting the relative scarcity of the literature in this specific area. Neither the single-source nor the multi-source localizer introduced in the previous chapter supports accurate distance prediction. Theoretically, though, aggregating solely angular information accumulated over time by a mobile robot could allow for predicting the actual location of sources in a room.

More precisely, a robotic agent moves in a room with one or several human speakers. We adopt a step-based representation in which the robot performs movements at discrete time steps. Each movement consists of a rotation of angle θ_t followed by a translation of distance d_t in the new direction. The robot’s movement at time step t is thus written as $\delta_t = (d_t, \theta_t)$. Its trajectory is assumed to be determined by an external policy that should not be affected. Thus, the developed method aims to localize each source’s relative position over short time windows. To emulate realistic timing constraints, each step duration is limited to a few hundred milliseconds. The method is provided with the recorded signal at each microphone of the agent corresponding to this time frame.

Horizon. A key distinction from the static SSL formulation lies in the temporal dimension of the input. In ASSL, the agent gathers multiple observations over time as it moves through the environment, rather than relying on a single auditory snapshot. While the movement policy itself is externally defined and not optimized, the agent’s relative displacements are available to the localization module. This sequential structure enables the model to accumulate spatial cues and refine its estimate of source positions. The number of steps over which these observations are integrated is referred to as the horizon H . Notably, only relative movement information may be leveraged to perform the task, as the absolute agent position remains unknown. This formulation is simpler than active control schemes that explicitly optimize motion to reduce localization uncertainty, such as those developed by Bustamante et al. [208], [209], but it retains the key benefit of leveraging egocentric movement for improved perception. In our setup, sound sources represent human speakers positioned arbitrarily within the room. Their positions are assumed to be static for an entire episode of H steps. This simplifying assumption is necessary for the development of our current work. Future work may have to consider more complex source behaviors.

```

1: function ACTIVE-SSL( $[M_{t-H}, \dots, M_{t-1}]$ ,  $[\delta_{t-H}, \dots, \delta_{t-1}]$ , STFT $_t$ )
2:   ▷  $[M_{t-H}, \dots, M_{t-1}]$ :  $H$  previous DoA maps
3:   ▷  $[\delta_{t-H}, \dots, \delta_{t-1}]$ :  $H$  previous relative movements.
4:   ▷ STFT $_t$ : The STFT of the multi-channel audio received by the microphone array.
5:
6:   ▷ Run the static SSL model on the current received audio
7:    $o_t \leftarrow \text{SSL}(\text{STFT}_t)$ 
8:   ▷ Compute the DoA map from the spectrum
9:    $M_t \leftarrow \text{DoA-MAP}(o_t)$ 
10:
11:  ▷ Transpose all maps to the current frame
12:  for  $t'$  in  $\llbracket t - H, t \rrbracket$  do
13:    ▷ Compute resulting displacement from past step  $t'$  to current step  $t$ 
14:     $\Delta_{t' \rightarrow t} \leftarrow \text{COMBINE}(\delta_{t-H}, \dots, \delta_{t'-1})$ 
15:    ▷ Project the DoA map to the current robot frame
16:     $\tilde{M}_{t'} \leftarrow \text{SHIFT}(M_{t'}, \Delta_{t' \rightarrow t})$ 
17:
18:  ▷ Aggregate all shifted maps
19:   $\mathcal{M}_t \leftarrow \Psi(\tilde{M}_{t-H}, \dots, \tilde{M}_{t-1})$ 
20:
21:  ▷ Extract detections
22:   $\hat{X}_t = [(x_1, y_1), \dots, (x_K, y_K)] \leftarrow \text{CLUSTER}(\mathcal{M}_t)$ 
23:
24:  return  $\hat{X}_t$ 

```

Listing 4.1 – Active-SSL algorithm.

4.3 Method

4.3.1 Pipeline Overview

We first explore leveraging the previously developed multi-source static localizer to tackle the ASSL problem. The method's central concept is building and refining a 2D egocentric map that encodes the relative positions of each source. This map is built to model a heatmap of the sources' presence in the robot's surroundings.

To build such a map, we start by running the SSL model (Section 3.3), which provides an estimated DoA spectrum. This detection is then transformed into a *DoA map*, projecting the one-dimensional localization result to an egocentric 2D map containing the same information. Then, this map is combined with the ones from previous steps after having been transposed to the current robot frame. Different ways of operating this aggregation are proposed. Finally, the 2D relative positions of the sources are extracted from this estimated egocentric map.

The overall procedure for performing one step of active-SSL is illustrated in Figure 4.1 and described in Listing 4.1. The individual steps of the process will be detailed in the following sections.

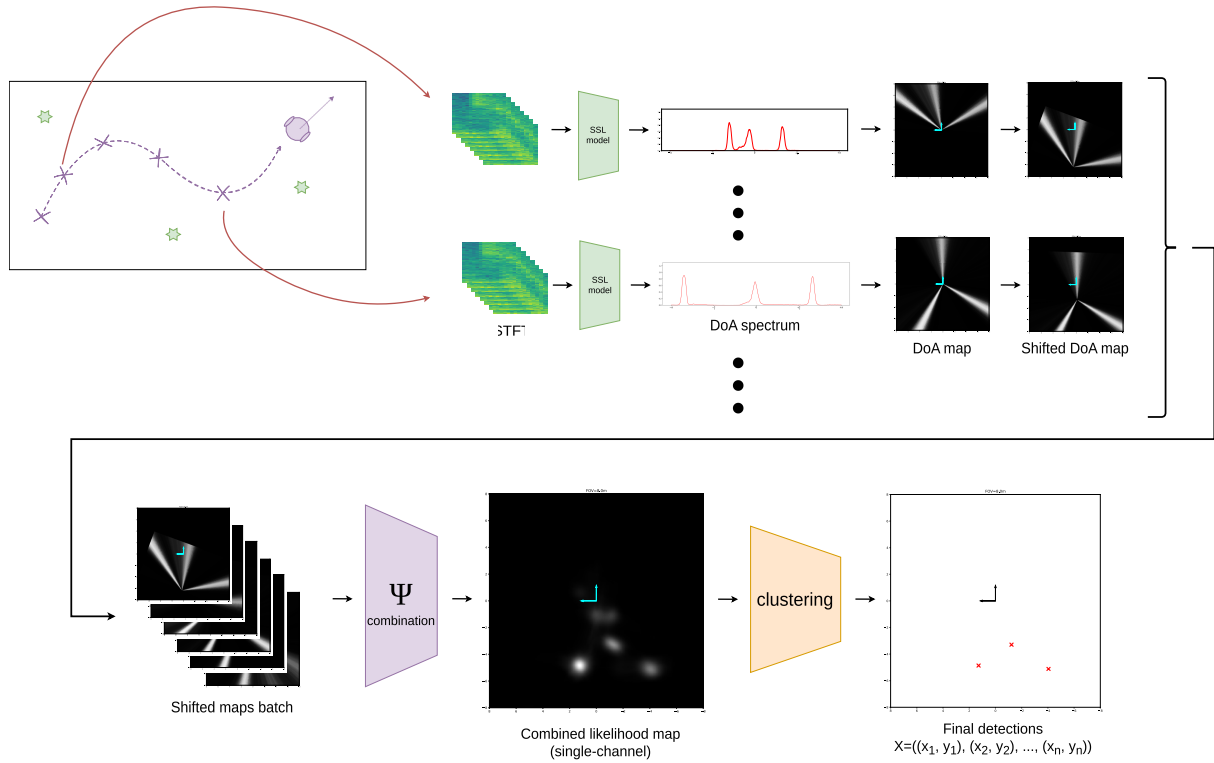


Figure 4.1 – Active-SSL pipeline.

4.3.2 Egocentric SSL Maps

At each step, a DoA map M_t is built from the DoA spectrum for the corresponding audio recording (line 9 in Listing 4.1). This spectrum solely provides angular information. At this stage, no distance knowledge has been gathered yet. The choice was made to not explicitly extract a set of detections from the spectrum, but to preserve the raw 360-long vector. Generating the DoA map M_t involves projecting this spectrum on the egocentric 2D space. All maps have a resolution of p pixels and are thus represented by $p \times p$ matrices. They represent a $L \times L$ square area around the robot.

First, each point $\mathbf{p} = (x, y) \in [-\frac{L}{2}, \frac{L}{2}]^2$ in the robot frame is mapped to the corresponding DoA value $\theta(\mathbf{p}) \in [-\pi, \pi]$. This permits associating all pixels $(i, j) \in \llbracket 1, p \rrbracket^2$ of the map to the corresponding angle $\theta_{i,j}$. Then, the associated DoA spectrum value can be recovered by computing the index $k_{i,j} \in \llbracket 1, d \rrbracket$ corresponding to the angle $\theta_{i,j}$:

$$M_{t(i,j)} = o[k_{i,j}] \quad \forall (i, j) \in \llbracket 1, p \rrbracket^2. \quad (4.1)$$

Hence, a pixel's intensity equals the DoA spectrum's value at the corresponding angle.

Figure 4.2 presents an example of such a DoA map along with the originating spectrum. By construction, the value of this 2D function remains constant along lines where $\theta = \arctan\left(\frac{x}{y}\right)$ is constant. This leads to a mixture of cone shapes originating at the center of the egocentric map.

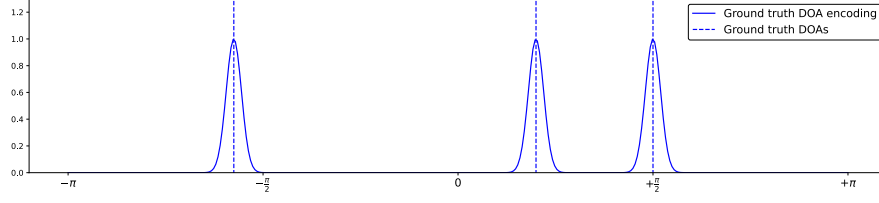
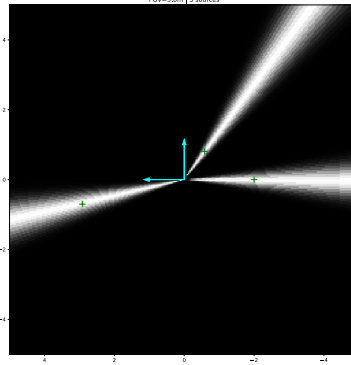
(a) DoA spectrum o_t .(b) Corresponding DoA map M_t .

Figure 4.2 – Example of a DoA map.

Shifting. At each step, the method gets the DoA maps $(M_{t-H+1}, \dots, M_{t-1})$ from the $H - 1$ previous steps. All these maps need to be *shifted* from their original frame $\mathcal{F}_{t'}$ to the current robot frame $\mathcal{F}_t$ to be combined with the current DoA map M_t . This operation uses the relative movement $\Delta_{t_1 \rightarrow t_2}$ between two steps t_1 and t_2 . It encodes the position and orientation of the previous map in the current frame $\mathcal{F}_t$. This movement is computed from the robot movement at each step $\delta_{t'}$, with $t' \in \llbracket t_1, t_2 \rrbracket$. The result of this operation will be denoted as $\tilde{M}_{t'}$. Figure 4.3 provides an example of a DoA map along with its shifted version. Of course, we have $\tilde{M}_t = M_t$ as the current map does not need to be shifted. This corresponds to lines 12-16 of Listing 4.1. The Field of View (FoV) parameter L has to be chosen carefully. If L is too small, information gathered during earlier steps may fall outside the current map and become unusable for aggregation. Conversely, a very large M may dilute spatial resolution or introduce irrelevant areas into the heatmap estimation. Its impact will be studied in more detail in a later section.

4.3.3 Aggregation Strategies

Once the collection of $H - 1$ previous DoA maps has been correctly shifted to $\mathcal{F}_t$, they will be *combined*, along with the current map M_t . The intuition behind this approach for ASSL lies in combining those step-maps into a single aggregate 2D heatmap that estimates the positions of each source around the robot.

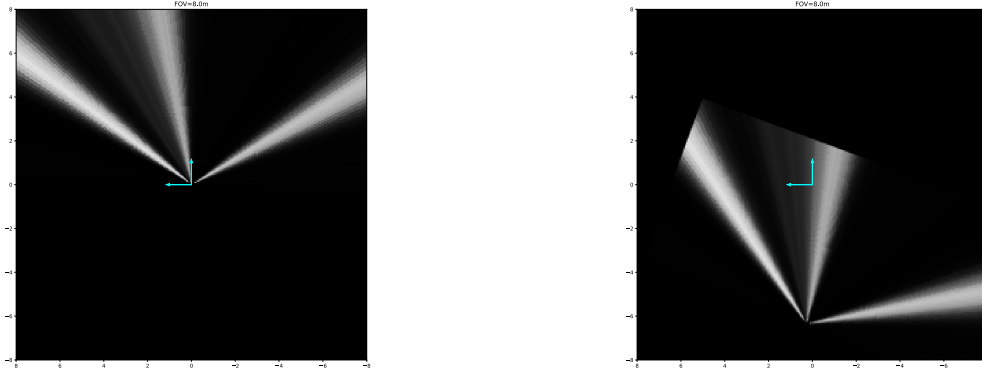
(a) Local DoA map M_t , expressed in $\mathcal{F}_t$.(b) Same map shifted to $\mathcal{F}_{t'}$.

Figure 4.3 – Shifting process for local DoA maps.

Such a mapping is defined as:

$$\begin{aligned} \Psi : \quad & \mathbb{R}^{H \times p \times p} \longrightarrow \mathbb{R}^{p \times p} \\ & \tilde{\mathcal{M}}_t = (\tilde{M}_{t-H+1}, \dots, \tilde{M}_t) \mapsto \mathcal{M}_t, \end{aligned} \quad (4.2)$$

where $\mathcal{M}_t$ denotes the obtained estimate for the likelihood.

Designing a reliable process to aggregate the maps is crucial. Indeed, the resulting $\mathcal{M}_t$ likelihood will be used to extract the positions of each source. The cleaner the combined map is, the easier the detection task becomes. This section proposes two methods for performing map blending.

4.3.3.1 Deterministic Averaging

On the one hand, a naive approach has been attempted. It simply entails averaging the maps:

$$\Psi_{\text{avg}}(\tilde{M}_{t-H+1}, \dots, \tilde{M}_t) := \frac{1}{H} \sum_{t'=0}^{H-1} \tilde{M}_{t-t'}. \quad (4.3)$$

As all maps are correctly expressed in the same local frame $\mathcal{F}_t$, a given pixel in each one corresponds to the same position.

The idea of averaging DoA maps means that each time step provides equal information about the presence of sources. Thus, the intersection of the different cones will have the highest scores, which appears reasonable. However, in some trajectories, averaging leads to artifacts that harden the clustering task performed to extract the final detection results. The performance of this blending approach is discussed further in [Section 4.4.3.3](#).

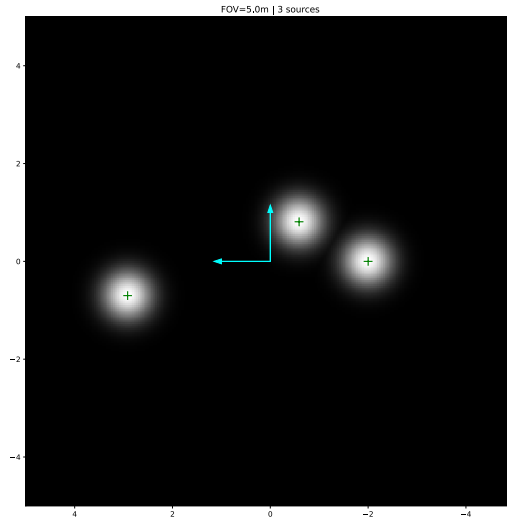


Figure 4.4 – Ground truth encoding $\mathcal{M}_t^*$ of the localization map.

4.3.3.2 Deep Neural Network

Motivation

Although averaging DoA maps is an explainable and straightforward method for blending, we have developed a more advanced technique involving a deep neural network. Indeed, an oracle knowing the absolute positions of each source could be used to generate an ideal version of the estimated 2D heatmap $\mathcal{M}_t^*$. As we use simulated data, such an oracle can easily be implemented. This statement leads to the definition of the blending process as a regression task where a neural network $\Psi^{\text{DNN}(\theta)}$ is trained to blend real DoA maps in the ideal estimate $\mathcal{M}_t^*$.

Ground Truth Encoding

The target map $\mathcal{M}_t^*$ is computed from a set of ground truth sources positions $\mathcal{X}_s = \left((x_1, y_1), \dots, (x_{n_s}, y_{n_s}) \right)$ expressed in the current robot frame $\mathcal{F}_t$.

We have chosen a strategy similar to the spectrum encoding introduced for multi-source SSL presented in [Section 3.3.1](#). Each source is modeled by a 2D Gaussian blob centered at its position. The envelope of those Gaussians leads to the continuous target map:

$$\begin{aligned} \overline{\mathcal{M}}_t^* : \left[-\frac{L}{2}, \frac{L}{2} \right]^2 &\rightarrow [0, 1] \\ X = (x, y) &\mapsto \max_{X_s = (x_s, y_s) \in \mathcal{X}_s} \left\{ e^{-\frac{\|X - X_s\|^2}{\sigma^2}} \right\}, \end{aligned} \quad (4.4)$$

where σ affects the spread of each blob and has been set to 0.5m. This continuous function is discretized in the final $p \times p$ matrix $\mathcal{M}_t^*$. [Figure 4.4](#) shows the ground truth value for our previous example.

Hence, we expect to achieve a finer filtering result using a more advanced blending method such as a DNN. The network should remove all noise in the input maps and underline the regions where sources are expected.

Architecture

The neural network $\Psi_{\text{DNN}(\theta)}$ takes the H shifted DoA maps $(\tilde{M}_{t-H+1}, \dots, \tilde{M}_t)$ as a single H -channel image $\tilde{M}_t$. It outputs a single-channel image $\mathcal{M}_t$, which is expected to approximate the target map $\mathcal{M}_t^*$.

Conventional architectures from the computer vision literature are considered because the model targets image-like information. Although several solutions could successfully solve this problem, one must pay attention to the network's receptive field. Indeed, designs purely based on small convolutional kernels would not be able to capture global information. In this specific task, the intersecting rays present in each DoA map must be considered from a sufficiently large scale. What could appear locally as an intersection of rays, indicating the presence of a source, could be noisy artifacts caused by an earlier crossing. This hypothesis may explain the failure of networks tested in experiments with receptive fields that are too small.

To account for this problem, we propose a custom U-net architecture adapted from Ronneberger et al. [214]. This type of design suits the present task of image-to-image mapping well. More specifically, it allows for multi-scale processing, thus allowing for leveraging larger patterns. We opt for a four-stage layout (Figure 4.5) within which the $(8, 256, 256)$ input gets progressively downsampled to a $(512, 32, 32)$ latent representation. The second half of the network scales the data back to the $p \times p$ resolution, outputting a single-channel map. Skip connections permit incorporating data from various stages of the downscaling process into the upsampling process. They explain the interesting multi-scale properties offered by the U-net architecture.

Each encoder block comprises two convolutional layers and a ReLU activation function. The first 2D convolution doubles the number of channels, while the second preserves its input dimension. Those layers are followed by a 2D max-pooling operation (corresponding to downward arrows in Figure 4.5). The latter downscales the images by a factor of two. Regarding the decoding stage, transpose convolutions, also known as deconvolutions [215] (upward arrows), upscale the data. The resulting higher-resolution images are concatenated with the corresponding descending tensor from the downsampling stage before being fed into two more consecutive convolutional layers. The process is repeated until the original dimension of the image is recovered.

No architectural element explicitly bounds the network output to a given numerical range. Thus, when using the network at inference, the output is normalized to ensure that all pixel values lie in the $[0, 1]$ interval.

Training Strategy

The network is trained in a supervised manner on a synthetic dataset. The latter was generated by simulating H -step-long random trajectories in the simulator (Section 2.3). The static SSL model is called at each step and outputs an estimation of the DoA spectrum o_t . The latter are

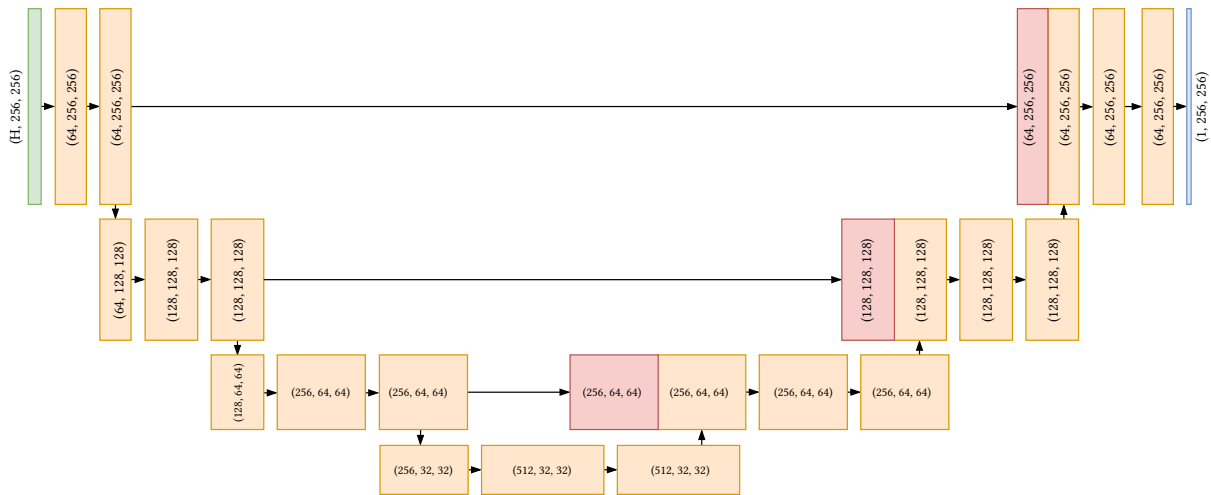


Figure 4.5 – Deep neural network architecture for 2D localization map aggregation.

converted to DoA maps M_t , which are finally shifted to the frame $\mathcal{F}_H$, corresponding to the last robot position in the considered trajectory. We collect 10,000 such trajectories using this process, totaling 80k individual positions. Each trajectory represents a training sample. The dataset generation process will be detailed in [Section 4.4.2](#).

The loss function used is simply the MSE between the predicted and expected likelihood maps, given by

$$\mathcal{L}(\mathcal{M}_t, \mathcal{M}_t^*,) = \frac{1}{p^2} \|\mathcal{M}_t - \mathcal{M}_t^*\|_F^2, \quad (4.5)$$

where $\mathcal{M}_t = \Psi^{\text{DNN}(\theta)}(\tilde{\mathcal{M}}_t)$ is the output of the network and $\mathcal{M}_t^*$ is the target map.

We train the network using the Adam [193] optimizer with a 10^{-4} learning rate. The training process uses mini-batches of 100 samples and lasts for 20 epochs. 20% of the complete training set is reserved as a validation set to ensure the model does not overfit.

4.3.4 Clustering for Detection Extraction

The detection process occurs once the aggregation function Ψ has been used to generate a single-channel 2D map. The core principle of this step lies in extracting a set of source positions from the heatmap. Naturally, the detection performance is highly correlated with the quality of the provided maps. This task resembles the DoA spectrum post-processing performed for multi-source SSL (Section 3.3.1.2). Indeed, local maxima in the likelihood 2D maps are expected to encode potential sources' positions. However, the simple algorithm implemented there remains impractical as it is limited to one-dimensional data (DoA spectra). Conversely, the ASSL heatmaps are two-dimensional.

Therefore, we choose to formulate this decoding task as a clustering problem. First, the coordinates of all pixels with a value higher than a pre-determined threshold τ are extracted. This step results in a 2D point cloud that can be seen as a collection of samples from a spatial probability distribution of the source presence. The clustering step aims to extract the modes

of this estimated distribution. To achieve this objective, we directly employ a state-of-the-art clustering algorithm. This set of 2D points is fed into the DBSCAN algorithm, introduced by Ester et al. [216]. DBSCAN takes two parameters: ε defining the radius of a neighborhood and m_p the minimum number of samples in a neighborhood for a point to be considered as a *core* point. A distance compatible with the input samples also has to be specified. In this case, as we deal with points in the plane, we use the conventional Euclidean distance. DBSCAN categorizes all input points into three groups: *core* points, *reachable* points, and *outliers*. Outliers are considered noise and do not belong to any cluster, while core points form the connected groups of samples, i.e., the clusters. A key advantage of DBSCAN is that it does not require the number of clusters to be specified a priori—a limitation common in other clustering algorithms such as k -means. DBSCAN itself does not define a concept of center for clusters. Here, we define the cluster center as the point with the highest value in the aggregated likelihood map. One should note that the actual values of each point in the heat map only impact the center search. Prior clustering happens without access to the estimated likelihood values and solely operates on the points' proximity to determine the clusters. Each cluster is interpreted as one source. One significant advantage of such a clustering formulation is that the proposed method can detect a variable number of sources.

4.4 Experiments

This section presents the main experiments conducted to assess the proposed solution's performance for ASSL.

4.4.1 Metrics

Overall, for our agent, the goal of the ASSL task, as defined in this chapter, is to extract the sound sources' positions after H arbitrary steps in the environment. The detection is only performed at step H , where the method outputs a list of coordinates relative to the robot's position. In this aspect, the ASSL problem can be compared to the object detection task in computer vision [217]. This problem involves finding the classes, positions, and sizes of semantic objects present in an image. In our formulation, only the position of the sources should be inferred. Bounding boxes and classes are not expected. Yet, object detection's precision and recall are natural metrics that can be adapted to evaluate our method's performance. A detection is considered correct if it falls within a δ -meter radius of the ground truth. For a detection to be considered valid, its estimated position must be closer than δ meters from the ground truth. The following function thus characterizes a correct/incorrect detection:

$$m(\hat{X}_k^i, X_j^i) = \begin{cases} 1 & \text{if } \|\hat{X}_{i,k} - X_{i,k}\|_2 < \delta \\ & \text{and } k = \underset{k' \in \{1, \dots, z_i\}}{\operatorname{argmin}} \|\hat{X}_{i,k'} - X_{i,k'}\|_2 \\ 0 & \text{otherwise,} \end{cases} \quad (4.6)$$

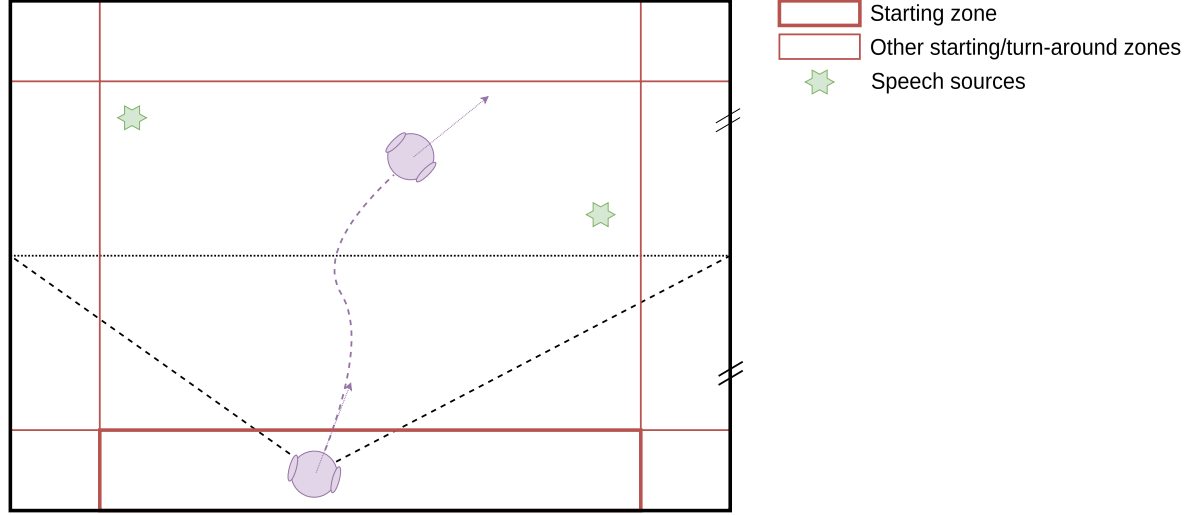


Figure 4.6 – Agent at the start and intermediary positions for a generated trajectory. The initial orientation is sampled in the cone delimited by dotted lines.

where $\hat{X}_{i,k} = (\hat{x}_{i,k}, \hat{y}_{i,k})$ is the estimated position of the k -th detected source in sample i and $X_{i,j}$ is the ground truth position of the j -th real source in this sample. z_i denotes the number of real sources in sample i while $\hat{z}_i$ is the number of detections. *Precision* and *recall* definitions remain the same as for the previously introduced static SSL task:

$$\text{Precision} = \frac{\sum_i \sum_{j=1}^{z_i} \sum_{k=1}^{\hat{z}_i} m(\hat{X}_{i,k}, X_{i,k})}{\sum_i \hat{z}_i}, \quad (4.7)$$

$$\text{Recall} = \frac{\sum_i \sum_{j=1}^{z_i} \sum_{k=1}^{\hat{z}_i} m(\hat{X}_{i,k}, X_{i,k})}{\sum_i z_i}. \quad (4.8)$$

4.4.2 Dataset Collection

To design, run, and evaluate our method on the ASSL task, we generated a synthetic dataset using the simulator presented in Chapter 2. Specifically, we leverage our simulator's ability to model *dynamic* discrete-time environments (see [Section 2.3.3](#)). The latter consists of a repository of independent H -step trajectories.

Acoustic objects and movement policy. First, a random number of speech sources z_i is sampled uniformly between one and four. Those z_i sources get randomly positioned in the reverberant room. The reverberation time T_{60} has been set to 0.5s, and the room measures 4×7 meters. We pick a random starting side (top, bottom, left, or right) and randomly place the agent equipped with a four-microphone array in a 50cm strip along the corresponding wall. The agent aims in a random direction, yet it is ensured to turn its back to the wall it starts from. The range of possible initial orientations can be observed on [Figure 4.6](#).

Regarding the movement policy, at each step, the new orientation $\theta_t + 1$ of the agent is sampled from the following normal distribution of mean θ_t and variance σ_θ^2 :

$$\theta_{t+1} \sim \mathcal{N}(\theta_t, \sigma_\theta^2). \quad (4.9)$$

In practice, we use a value of 0.1 radians ($\approx 5.7^\circ$) for σ_θ . The agent then moves forward in this new direction by a distance of 50cm.

A special case occurs when the robot is less than 50cm from a wall. Instead, its orientation is sampled according to the initialization process to turn its back to this wall.

Data gathering. The collected datasets will be used for two distinct purposes: evaluating the global ASSL pipeline and training the $\Psi_{\text{DNN}(\theta)}$ combination operator. Hence, exhaustive geometric and acoustic data are gathered. For each step, the audio signal received by the agent is fed to the SSL network to collect the estimated DoA spectrum $\hat{o}_t$. The oracle spectrum o_t also gets saved for further comparisons. Also, the absolute positions of the agent and the relative source locations are saved at every step. Finally, the robot's relative movements are recorded to perform the map shifting operation later. Local DoA maps have not been generated at this point, but all the necessary information for their synthesis is available. This choice allows for experimenting with the relevant hyperparameters, such as the FoV L and pixel resolution p .

4.4.3 Performance Study

Both the ASSL task and the proposed method admit variants and parameters. In this section, an in-depth exploration of specific settings is conducted.

4.4.3.1 Likelihood Cutoff

To feed the aggregated heatmaps to the clustering algorithm, a set of points must first be extracted from the final 2D map $\mathcal{M}_t$. When building its clusters, DBSCAN does not consider the pixel values and only relies on the distance between the provided points. Thus, the set of filtered coordinates should be carefully chosen.

We adopt a simple thresholding approach, selecting all points whose values are greater than a given target $\delta_{\min}$. This parameter should be high enough to let the clusters surface. If it is too low, the resulting point cloud is fully connected, leading DBSCAN to find a single cluster. Conversely, increasing $\delta_{\min}$ too much will result in local peaks being wholly filtered out, which will produce a missed detection.

Figure 4.7 shows a given aggregated map after filtering with different values of $\delta_{\min}$. The top row depicts the map obtained from the averaging aggregation (Ψ_{avg}) while the bottom one exposes the neural network output ($\Psi_{\text{DNN}(\theta)}$). In this example, both blending strategies have provided a solid result that is not particularly challenging to cluster. While the neural network could directly yield distinct blobs, the obtained averaged map has been shown to be more impacted by the thresholding. Indeed, values of $\delta_{\min}$ that are too low do not manage to disconnect the various clusters and would link to a single prediction from the detection pipeline. On the contrary, the network output suffers from too aggressive filtering as the blob with the lowest intensity eventually disappears for $\delta_{\min} \geq 0.8$.

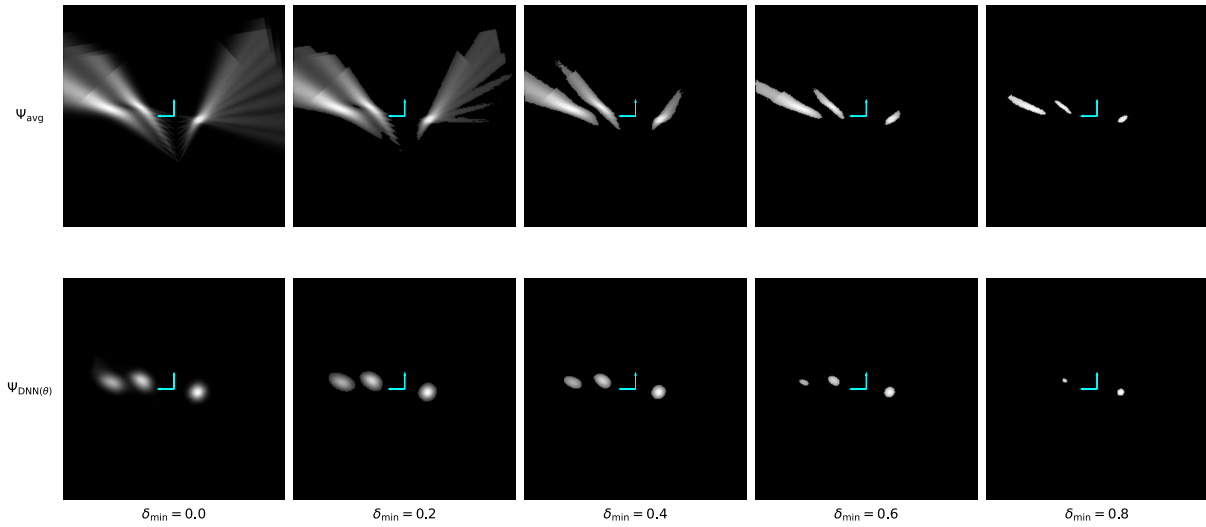


Figure 4.7 – Effect of the clipping threshold on the aggregated maps.

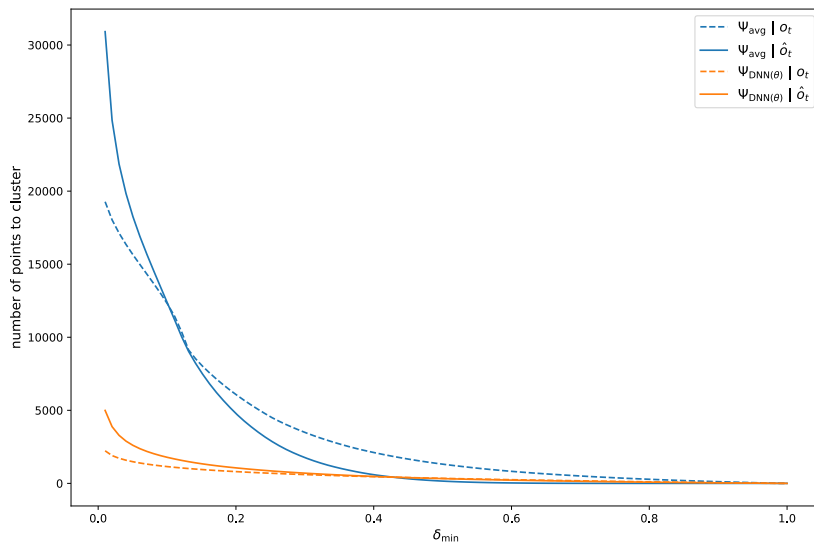
Table 4.1 gathers the result of an experimental campaign on the impact of $\delta_{\min}$ on the final ASSL performance. All four combinations of blending methods and DoA spectrum providers have been tested to ensure completeness. It shows that a single optimal value for $\delta_{\min}$ does not exist. As both the aggregation process and the source DoA data significantly impact $\mathcal{M}_t$, the threshold needs to be chosen accordingly. Thus, for each scenario, we identify the best precision-recall tradeoff and select the corresponding $\delta_{\min}$ value. Those pairs are underlined in **Table 4.1** and do not always coincide with the highest values of individual metrics (highlighted in bold). Those optimal $\delta_{\min}$ values will be used in later experiments to extract the best performance from each method.

Finally, another factor might be brought into consideration: computational cost. DBSCAN’s time complexity is $O(n \log n)$ when the data layout is favorable and $O(n^2)$ in the worst-case [216] (n denotes the number of points). In the use made of this algorithm here, the number of provided points is highly impacted by the choice of $\delta_{\min}$. For instance, the value of 0 is not tested. This case indeed often translates to none of the pixels getting filtered. For a resolution of $p = 256$, the number of points forwarded to DBSCAN amounts to $p^2 = 65,536$. Also, the algorithm output is a single cluster containing all points.

Figure 4.8 plots the number of points remaining after the filtering process. Blue lines correspond to using the naive averaging strategy to aggregate the maps and orange lines relate to the use of the neural network. Also, dashed plots allow differentiation of which DoA spectra have been employed from the start. This shows that the inferred likelihood map should be as sparse as possible, with peak values approaching 1. Those properties allow for better separability and fewer points being fed into the clustering algorithm.

Table 4.1 – Comparison of the ASSL performance for different clipping threshold ($\delta_{\min}$) values.

Aggregation method	$\delta_{\min}$	Estimated DoA spectrum $\hat{o}_t$		Ground truth DoA spectrum o_t	
		Precision (%) $\uparrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	Recall (%) $\uparrow$
Ψ_{avg}	0.1	78.0	32.43	94.92	37.38
	0.2	69.25	40.05	84.25	45.58
	0.3	<u>72.33</u>	46.60	91.73	54.85
	0.4	75.71	42.09	94.18	64.19
	0.5	79.46	26.42	95.62	72.91
	0.6	84.08	10.99	<u>96.02</u>	77.70
	0.7	81.82	2.12	96.39	74.48
	0.8	-	0.0	96.23	68.20
	0.9	-	0.0	95.53	54.50
$\Psi_{\text{DNN}(\theta)}$	0.1	75.02	47.27	99.48	67.92
	0.2	75.59	51.67	99.30	72.91
	0.3	77.27	53.91	99.39	76.91
	0.4	80.03	54.61	99.46	80.29
	0.5	82.62	54.14	99.63	83.59
	0.6	<u>86.05</u>	<u>53.28</u>	99.59	86.26
	0.7	87.89	51.00	99.56	88.50
	0.8	89.73	47.66	<u>99.74</u>	90.54
	0.9	90.01	42.09	99.91	90.11

Figure 4.8 – Number of points remaining after the filtering operation with respect to $\delta_{\min}$.

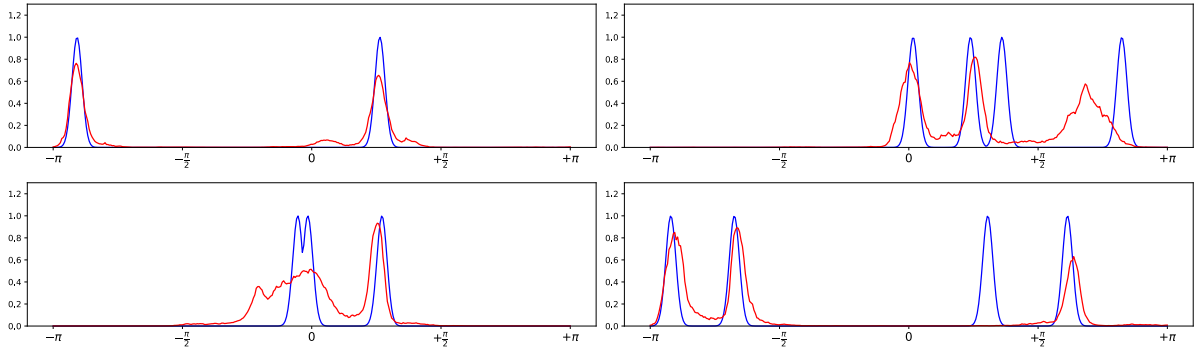


Figure 4.9 – Comparison of ground-truth (blue) and predicted (red) DoA spectra obtained from our trained model.

4.4.3.2 Impact of the Upstream SSL Model

The upstream static SSL model features are a core part of the ASSL pipeline. The quality of DoA spectra it provides plays a significant role in the final performance of the method. Hence, two scenarios are compared to isolate the behavior of the ASSL method itself. On the one hand, the neural network implemented and trained in [Section 3.3](#) predicts the DoA spectra $\hat{o}_t$ from the listened audio. This scenario is the most realistic as it can be used directly in a real-world scenario and does not rely on an oracle. On the other hand, the ASSL framework is also evaluated directly using the ground truth spectra o_t . Here, the potential of our method can be explored under ideal conditions.

[Figure 4.9](#) shows instances of DoA spectra. Although the SSL model correctly infers most peaks, failed detections can still be observed across the trajectory dataset. Most detection failures consist of false negatives in which the network outputs either a too-low peak or no activation at all. In those cases, the averaged maps might still include local maxima in the correct locations, but those often get filtered when thresholding the final map. Samples where several sources stand particularly close with respect to DoA also represent challenging cases.

From a performance point of view, [Table 4.1](#), for instance, highlights an important gap between using ground-truth spectra and predicted ones. For instance, the obtained recall on real data, peaking at around 55%, clearly appears to be a weakness of the proposed pipeline. The leading cause lies in the shortcomings of the angular localization method. Nonetheless, leveraging the static model across multiple distinct agent positions still permits recovery from partial misses and provides precise 2D localization. This observation further validates the relevance of the proposed approach.

4.4.3.3 Comparison of Blending Methods

Two alternatives have been compared for the map blending operation: naive averaging Ψ_{avg} and learned $\Psi_{\text{DNN}(\theta)}$ (see [Section 4.3.3](#)). The former serves as a baseline, offering the advantage of being explainable and straightforward, while the latter aims to provide superior performance.

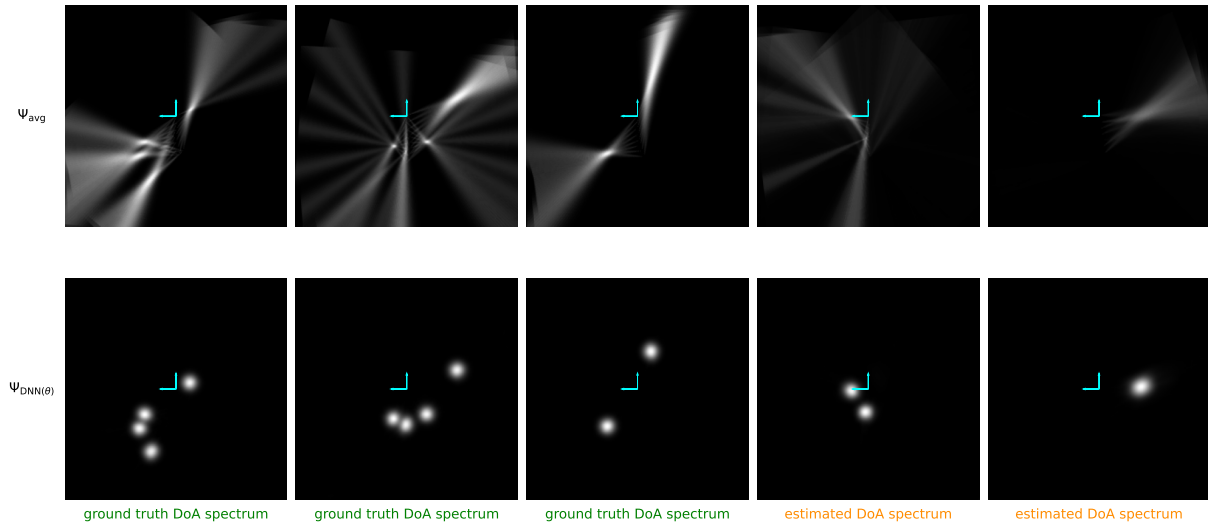


Figure 4.10 – Visual comparison of the two aggregation methods (Ψ_{avg} top and $\Psi_{\text{DNN}(\theta)}$ bottom). Each column represents an example from the dataset. The first three use the ground-truth DoA spectrum, while the last two use the one estimated by the pre-trained SSL network.

Table 4.2 – Final performance of both aggregation methods.

Aggregation method	Estimated DoA spectrum $\hat{o}_t$		Ground truth DoA spectrum o_t	
	Precision (%) $\uparrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	Recall (%) $\uparrow$
Ψ_{avg}	72.33	46.60	96.02	77.70
$\Psi_{\text{DNN}(\theta)}$	86.05	53.28	99.74	90.54

Figure 4.10 discloses a selection of 2D heatmaps produced by the two proposed methods. Naturally, when significantly precise DoA spectra are extracted at each step, even the naive averaging method suffices for accurately estimating the 2D heatmap. However, when more imperfect and challenging DoA maps are considered, the neural network shows a greater capacity to ignore the noise and provide a sharp likelihood estimation. Also, as $\Psi_{\text{DNN}(\theta)}$ has been trained with localized 2D Gaussian blobs as targets, it has learned to properly filter the unnecessary parts of the original cones. Its output successfully concentrates on the actual position of the sources. The network facilitates clustering by precisely separating and isolating the different local peaks in the map. This decreases the sensitivity to the hyperparameters of DBSCAN.

Table 4.2 summarizes the best performance achieved by each aggregation strategy. In particular, the most efficient value of the $\delta_{\min}$ parameter has been used. Unsurprisingly, employing the neural network offers a tangible advantage over simply averaging the DoA maps. Those results confirm the qualitative observations made above. When provided with the ground-truth DoA spectra, $\Psi_{\text{DNN}(\theta)}$ achieves an almost perfect precision. However, the recall score slightly lags, with a value of 90.54%. The few missed detections involve situations in which at least one of the sources remains strictly in front of or behind the agent during the entire

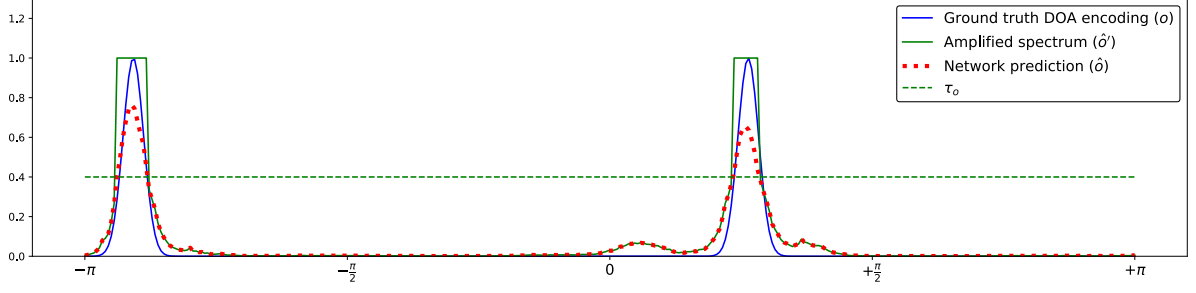


Figure 4.11 – Illustration of the DoA spectrum transformation. All values of $\hat{o}$ above the threshold τ_o get pushed to 1.

trajectory. The indirect triangulation phenomenon leveraged by our method becomes almost infeasible, and the distance cannot be accurately estimated. Nonetheless, even in challenging cases where no clear cone intersection can be visually distinguished, the network sometimes manages to perform correct detections by relying on the prior it has learned during training. During training, the network learns the statistical distributions of sound sources. This can therefore help it to guess where a source can be, even when the actual map appears to be lacking the necessary information.

In summary, the proposed deep neural architecture has been shown to be a robust and powerful method for performing the aggregation step of the ASSL pipeline.

4.4.3.4 A Tentative for DoA Spectrum Amplification

As seen in [Section 4.4.3.2](#), the ASSL process works significantly better when provided with the ground-truth DoA spectra instead of using the pre-trained SSL model. One reason for this decrease in performance is that the peaks present in those estimated heatmaps are lower. This does not necessarily impact the static SSL metrics as any local maximum above the detection threshold ξ would be counted as a DoA prediction (see [Section 3.3.1.2](#)). However, this threshold does not intervene in the ASSL pipeline, and the DoA spectrum is directly converted into a 2D DoA map. When the deep neural network $\Psi_{\text{DNN}(\theta)}$ is used to combine the maps, its output is normalized to the $[0, 1]$ range. Although this would help amplify localization heatmaps that are too dim, it cannot compensate for relative differences caused by uneven peaks in the DoA spectra.

To account for this phenomenon, we have attempted to artificially adjust the spectra with an amplification trick. Its concept remains simple: It simply consists of clipping all the portions of a DoA spectrum higher than a given threshold τ_o to one. In other words, a DoA spectrum $\hat{o}$ is transformed in the following way:

$$\hat{o}' = \max(\hat{o}, 1_{\hat{o} > \tau_o}). \quad (4.10)$$

[Figure 4.11](#) displays this peak amplification process on an arbitrary example. In [Figure 4.12](#), one can see the consequence of this process on the DoA maps. Each row corresponds to a different value for the threshold. Qualitatively, a too-high τ_o will lead to lower peaks being

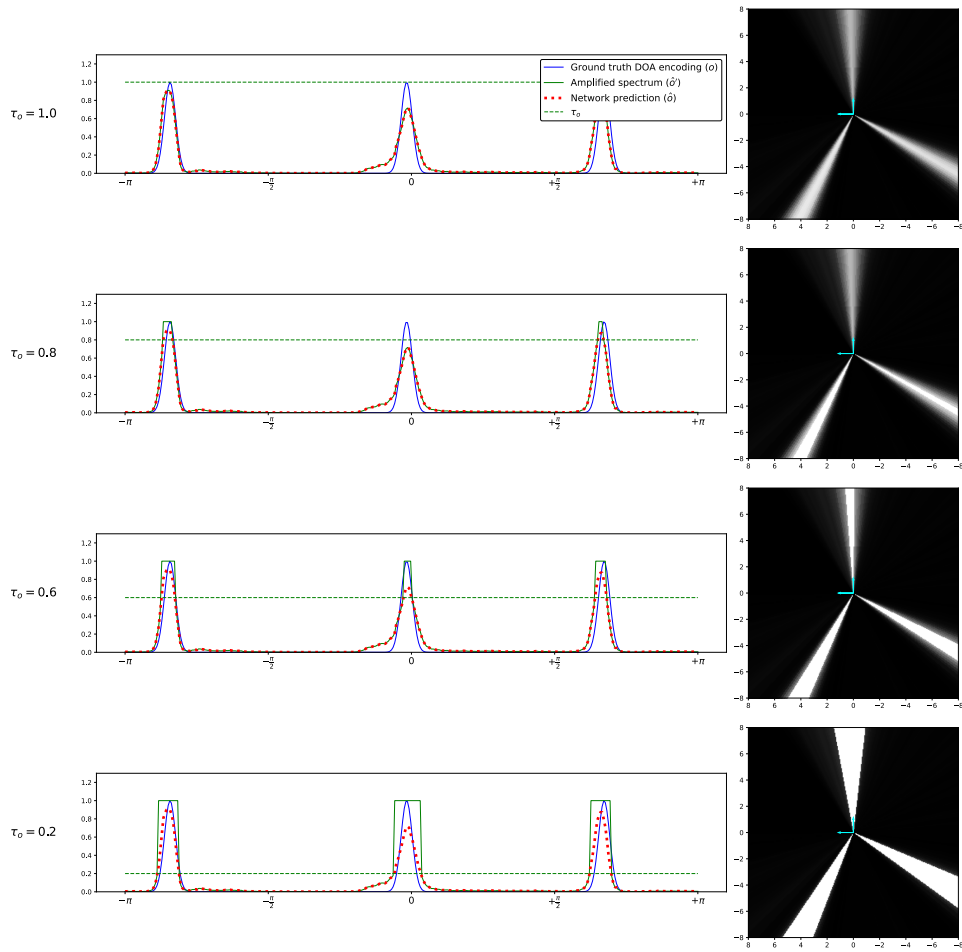


Figure 4.12 – Effect of DoA spectrum amplification on the local maps.

left unamplified. However, decreasing this parameter causes bigger, oversaturated cones that lose their localization information.

Our experiments show that DoA spectrum thresholding fails to bring tangible benefits (see [Table 4.3](#)). Our $\Psi_{\text{DNN}(\theta)}$ network has been retrained on a dataset of maps corresponding to each τ_o value to obtain those results. The Ψ_{avg} technique, however, does not require any form of training. Performance is often best for $\tau_o = 1$. Employing this process when using the ground-truth DoA spectra was not expected to bring any additional performance, as all peaks precisely maximize at 1.0 and thus do not need to be any further amplified. In contrast, when using the DoA spectra estimated by the SSL model combined with the neural network for map blending, DoA amplification slightly boosts the overall performance. Conversely, the naive averaging approach does not benefit from more saturated spectra.

Finally, the marginal and situational advantages of clipping DoA spectra are insufficient to justify including them in the final method. However, this tentative result further illustrates the pipeline’s sensitivity to the quality and processing of the input DoA information.

Table 4.3 – Comparison of the ASSL performance for different DoA threshold (τ_o) values.

aggregation method	τ_o	Estimated DoA spectrum $\hat{o}_t$		Ground truth DoA spectrum o_t	
		Precision (%) $\uparrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	Recall (%) $\uparrow$
Ψ_{avg}	0.2	55.40	28.78	54.40	36.87
	0.4	66.00	39.03	61.43	45.47
	0.6	67.31	43.97	68.59	53.32
	0.8	71.09	45.86	77.72	62.03
	1.0	72.33	46.60	96.02	77.70
$\Psi_{\text{DNN}(\theta)}$	0.2	83.27	50.02	99.07	88.30
	0.4	86.34	49.86	99.43	88.89
	0.6	86.60	46.68	99.57	90.46
	0.8	84.89	44.33	99.70	90.18
	1.0	83.70	43.74	99.74	90.69

Table 4.4 – Evolution of localization performance for different horizon values.

Aggregation method	Horizon	Estimated DoA spectrum $\hat{o}_t$		Ground truth DoA spectrum o_t	
		Precision (%) $\uparrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	Recall (%) $\uparrow$
Ψ_{avg}	1	12.28	4.83	9.03	3.61
	2	39.19	17.86	62.26	45.47
	4	55.50	35.06	88.15	67.45
	6	65.56	43.42	94.29	73.89
	8	72.33	46.60	96.02	77.70
$\Psi_{\text{DNN}(\theta)}$	1	37.90	20.65	34.83	25.52
	2	55.59	29.88	98.57	78.45
	4	75.72	41.26	99.91	89.32
	6	82.18	49.08	99.28	86.14
	8	86.03	52.73	99.74	90.81

4.4.3.5 Horizon

The ASSL task, as defined in this work, requires the agent to provide an estimate of the sources' locations after a fixed number of H steps. The proposed method shows promising performance when $H = 8$ consecutive maps can be combined and analyzed. To further explore the informative content of each step, we perform an ablation study on the horizon parameter. The base dataset gathers trajectories of $H_0 = 8$ steps and is the only one used in this experiment. To train or evaluate our method on a shorter horizon H' , we ignore the first $H - H'$ steps. The final frame remains the same across all the experiments.

Table 4.5 – Comparison of the ASSL performance for different FoV values.

FoV (m)	Estimated DoA spectrum $\hat{o}_t$		Ground truth DoA spectrum o_t	
	Precision (%) $\uparrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	Recall (%) $\uparrow$
2	38.99	19.12	49.43	22.30
4	72.26	36.51	91.10	49.43
8	85.84	49.98	98.94	84.02
16	85.36	52.65	99.74	90.50

Table 4.4 shows the final performance of the pipeline for different horizons. The neural network $\Psi_{\text{DNN}(\theta)}$ is retrained for each horizon to get the best possible performance. As anticipated, the highest detection scores are achieved when using all eight steps. Besides granting better absolute performance, the neural network demonstrates a greater robustness to lower horizons. The performance drop is less pronounced than for the averaging approach.

4.4.3.6 Visual Encoding

The choice to model the 2D localization problem with heatmaps involves exploring hyperparameters related to this 2D representation. As DoA maps are generated from the projection of DoA spectra, we can define the output domain without prior constraints. Two parameters influence the synthesized maps: the FoV L and the pixel resolution p .

Field of View. The FoV (L) determines the size of the range covered by the egocentric map, which is an $L \times L$ square centered around the robot agent.

Its value must be chosen diligently as it bounds the information available once the shifting and aggregation have occurred at the final position. One has to consider the maximum distance $d_{\max}$ traveled by the robot at each step and the horizon H . If the characteristic value $H \times d_{\max}$ significantly overpasses $\frac{L}{2}$, information from the oldest steps might be at least partly out of scope and thus useless. In practice, there is no particular reason to keep L small, and choosing it greater than the room’s dimensions ensures that most of the available knowledge is captured in the shifted maps $\tilde{M}_t$. To quantify this parameter’s impact, the neural network is trained on various FoVs, ranging from 2 to 16 meters. Results are summarized in **Table 4.5**, confirming the abovementioned expectation overall. Performance indeed suffers from a reduction of the FoV. The value of 16 meters is used in the rest of our experiments as it provides the most favorable conditions.

Pixel resolution. As opposed to the FoV, pixel resolution stands solely as a representation hyperparameter and does not fundamentally change the maps’ informative content. Naturally, a higher resolution would limit any loss caused by the spatial discretization process. During our experiments, we noticed that imprecision would arise within the map-shifting process when using a too-low resolution. The latter is performed directly on the discrete heatmap thanks to the *OpenCV* [218] software library. On the other hand, increasing the resolution induces a larger image fed into the neural network. As our U-net architecture is fully convolutional, the number of parameters remains identical when the input size changes. However,

Table 4.6 – Comparison of the ASSL performance for different pixel resolutions.

pixel resolution	Estimated DoA spectrum $\hat{o}_t$		Ground truth DoA spectrum o_t	
	Precision (%) $\uparrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	Recall (%) $\uparrow$
64	85.05	45.35	97.14	8.01
96	94.92	68.24	99.45	71.61
128	86.34	50.88	99.54	85.67
256	85.41	52.65	99.61	90.73

the computational cost is still impacted by such modifications. We have once more trained the neural network on maps of different resolutions between 64 and 256 pixels. The results, presented in Table 4.6, suggest that a finer resolution helps with the localization process. Although this parameter does not strongly impact precision, the recall is shown to be sensitive to pixel resolution. Training time grows from 5 minutes when using $p = 64$ to 30 minutes for $p = 256$. Inference time scales similarly, ranging from 30s to 2min 15s for the largest maps. As those constraints remain acceptable for real-world use cases, the most favorable resolution ($p = 256$) is used in the rest of our experiments.

4.5 Conclusion and Discussion

This chapter presents a custom approach for solving an SSL task in a dynamic multi-source context.

The proposed approach generally breaks the ASSL task into several steps for which a specific solution is presented. Although this allows for a high degree of explainability, an end-to-end solution could be explored. Regarding the deep neural network trained to perform map aggregation, the target maps are defined artificially. One could envisage directly learning to predict the source’s positions from a batch of shifted DoA maps. No clustering algorithm would thus be needed. Additionally, a deep learning approach could be directly fed with the DoA spectra instead of projecting them first as cones in 2D maps. Both tracks might allow pushing detection performance by eliminating hand-crafting middle steps. However, they would necessitate additional adaptations, such as the ability to localize a variable number of sources.

The presented solution achieves convincing performance on this task. Naturally, the precision of the final position estimation highly depends on the quality of the provided DoA spectra. Also, the agent trajectory and the relative position of sources significantly impact the detection. For instance, cases where, on the one hand, the trajectory remains relatively straight and, on the other hand, the source lies on this line happen to be challenging. Indeed, the relative movement of the agent does not bring additional angular information. This makes the implicit triangulation process fail almost certainly. Also, when sources are very close to the agent, the produced aggregated map is difficult to cluster correctly.

More generally, the chosen formulation for the ASSL problem remains simplistic. Several works have addressed more complex variations of this task. Performing source localization and providing an optimized movement policy constitutes a more complete and relevant problem. Also, including a multimodal approach to this framework would constitute a noticeable improvement to this audio-based solution. Indeed, most social robots incorporate audio and vision sensors, allowing for broader perceptual capabilities. Exploiting visual information could certainly strengthen the agent's ability to localize speakers despite imperfect observations.

Chapter 5

Deep Reinforcement Learning for Sound-Driven Navigation

After having explored different acoustic challenges, this chapter will push further toward interactive robotics scenarios. While supervised learning has been shown to be very effective for solving many robotic tasks, Reinforcement Learning (RL) constitutes an alternative framework for specific problems. Its fundamental formulation targets interactive environments where an agent learns through trial and error. We will use the modern Deep Reinforcement Learning (DRL) paradigm to train an agent to navigate autonomously from auditory cues. First, the general framework of RL will be introduced, along with how deep neural networks are leveraged as function approximators in this context. Secondly, we will present a problem of perceptually motivated acoustic-based robot navigation and relevant literature. This chapter also exposes a proof of concept of a complete pipeline to address this challenging task. Finally, experimental results will demonstrate the effectiveness of the proposed approach.

5.1 Introduction to Reinforcement Learning

5.1.1 History

RL originates from two formerly distinct fields. On the one hand, psychology researchers have attempted to understand how humans and animals could learn. The American psychologist Edward Lee Thorndike laid out foundational work on animal learning and behavior. In his 1911 book *Animal Intelligence: Experimental studies* [219], Thorndike presented a collection of experiments involving animals solving different tasks. He wanted to understand the core principles and mechanisms allowing the subjects to adapt to the problem they were facing. Observing animal reactions and abilities, he inferred the fundamentals of behaviorism and the trial-and-error theory. For instance, he put hungry cats in cages, and to escape and reach food placed outside, they had to solve a puzzle. He noticed that the animals did not overcome the difficulties through insight or understanding but through repeated trial and error. He noticed that successful behaviors were reinforced, leading to quicker escapes, while unsuccessful behaviors were abandoned. From these observations, Thorndike developed the *Law of Effect*, which states that behaviors that lead to satisfying outcomes are more likely to happen again.

On the contrary, actions leading to unpleasant consequences are discouraged, and their frequency decreases. This law has two essential aspects. On the one hand, trial-and-error learning is *selectional*, as the subject tries different alternatives to identify the optimal one. On the other hand, it is *associative* as the selected decisions are associated with particular situations. This principle stands as a core ingredient of modern theories of learning and behavior modification. The idea of learning progressively and gradually from substantial experience contrasts with the theories stating that animals learn from higher-level reasoning, similar to humans. In 1927, Ivan Pavlov detailed the concept of trial-and-error in *Conditioned reflexes: An investigation of the physiological activity of the cerebral cortex* [220]. Pavlov described reinforcement as the strengthening of a pattern of behavior due to an animal receiving a stimulus—a reinforcer—in an appropriate temporal relationship with another stimulus or with a response [221]. Overall, psychology, by observing animal behaviors, has provided the intuition behind the formulation of RL as a framework to solve complex decision problems.

On the other hand, RL has been preceded by the field of optimal control. Its objective is to design a controller for a dynamic system that should minimize some cost function. Richard Bellman has conducted essential work on this problem, notably by introducing dynamic programming [222] and the foundational Bellman equation. Dynamic programming is the most general and feasible solution to optimal control problems but it suffers from limitations. For instance, when the number of dimensions of the involved control spaces grows too large, they suffer from the curse of dimensionality, which Bellman himself describes. He is also at the origin of the discrete stochastic version of the optimal control problem, called Markov Decision Process (MDP) [223]. Those concepts served as the foundations of modern reinforcement learning theory and algorithms.

Temporal-difference (TD) learning can be seen as another building block of the RL field. TD learning methods involve leveraging the difference between successive estimates of a given quantity. Although first proposed by Arthur Samuel (1959) [224] and Hyman Minsky (1961) [225], the idea was not widely adopted or implemented at the time. In 1972, Harry Klopf [226] combined TD learning and trial-and-error in its theory of *heterostasis*. Klopf's theory was pursued further by Richard Sutton. For instance, in 1988, Sutton used TD learning as a stand-alone prediction method [227]. He also extended the principle of TD learning by inventing the TD(λ) approach. This extension bridges the gap between Monte Carlo methods (which wait until the end of an episode to update values) and one-step TD learning, providing a more flexible framework for RL. Chris Watkins is responsible for a major breakthrough in RL by having introduced Q-learning (1989) [228]. This algorithm constitutes a simple solution to the optimization of an MDP. Its tabular approach consists of learning the expected future rewards for taking a particular action in a given state. The definitive proof for the Q-learning algorithm [229] ensures its almost certain convergence to the optimal action-values.

Although this short introduction is far from exhaustive, it helps place more recent advances in context. RL has evolved significantly since its early days, driven by both conceptual innovations and computational advances, which have enabled its application in an increasing range of domains. The book *Reinforcement Learning: An Introduction* [221] by Sutton and Barto is one of the field's most recognized resources. It provides both a deep look at the theoretical grounds of RL and a wide overview of modern algorithms. The original 1998 edition was revisited in

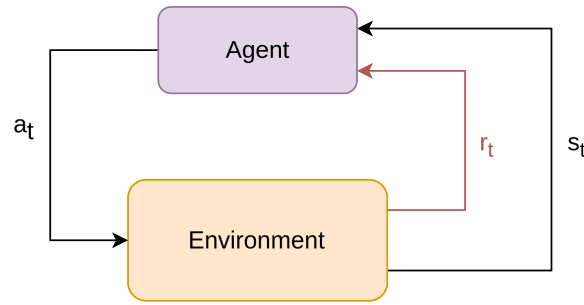


Figure 5.1 – Reinforcement Learning framework.

2018 to reflect the important progress made during this period. The present section draws substantial inspiration from this resource.

5.1.2 Core Notions

RL encompasses various techniques for solving stochastic sequential decision problems. It leverages trial-and-error learning by making an agent evolve in its environment while rewarding it according to its performance. This feedback loop constitutes the reinforcement aspect and permits the agent to self-improve. At each time step t , the agent observes the current state s_t of the environment. It then has to select an action so as to maximize the future accumulated rewards. In response to this action, the environment communicates the reward signal r_t to the agent and transitions to a new state s_{t+1} (Figure 5.1). This section briefly introduces the main concepts and notations required to formalize this process further and the relevant RL algorithms.

Markov Decision Process (MDP). The stochastic sequential decision problems are modeled using the MDP framework [221]. It is based on the work of Andrey Markov in the early 20th century about stochastic processes. Notably, the Markov process defines a process in which future states only depend on the current state, but not on the sequence that preceded it. Bellman later extended Markov processes to add a decision aspect, introducing the MDP [222]. An MDP consists of a tuple $\langle \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle$.

- $\mathcal{S}$: The state space defines the set of all attainable states for the problem. Both discrete (finite or not) and continuous spaces are valid in this context.
- $\mathcal{A}$: The action space is the mathematical space in which the agent's actions are defined. It may be discrete or continuous, and either bounded or unbounded. It can also be multi-dimensional, in which case it is typically expressed as the Cartesian product of simpler subspaces.
- P defines the one-step dynamics of the environment. It denotes the probability that the agent will transition to state s' while in state s and taking action a . $P(s_{t+1} = s' \mid s_t = s, a_t = a)$ denotes the *transition probabilities*.
- r : The reward real-valued function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ maps each state-action pair to its reward $r(s, a, s')$. In some environments, the reward might solely depend on the next state: $r(s')$. The reward may also be non-deterministic, in which case we take its expectation.
- γ : The discount factor in $[0, 1)$ dampens the impact of future rewards.

Expected Return Maximization. The agent seeks to maximize the cumulative discounted rewards it collects over time. This objective is formalized as the return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (5.1)$$

Here, r_{t+k+1} is the reward obtained after taking action a_{t+k} in state s_{t+k} , leading to state s_{t+k+1} . Discounting models the idea that immediate rewards are more valuable than distant ones, and helps prioritize short-term gains while still accounting for long-term outcomes. It also ensures that the return remains bounded as long as the reward function is bounded too. The agent's goal is to learn a policy π that maximizes the expected return $\mathbb{E}_{\pi}[G_t]$, averaged over trajectories it generates through interaction with the environment. γ denotes the previously introduced MDP discount factor.

Policy. The policy denotes a function or algorithm that maps each state to a probability distribution over the action space.

$$\begin{aligned} \pi : \mathcal{S} &\longrightarrow \mathcal{P}(\mathcal{A}) \\ s &\longmapsto \pi(\cdot \mid s). \end{aligned} \quad (5.2)$$

As a distribution over the action space, the policy can take various forms. For finite MDPs, i.e., when both the action and state spaces are finite, the policy is a $|\mathcal{S}| \times |\mathcal{A}|$ matrix:

$$\left(P(a_t = a_i \mid s_t = s_j) \right)_{\substack{1 \leq i \leq |\mathcal{A}|, \\ 1 \leq j \leq |\mathcal{S}|}}. \quad (5.3)$$

When dealing with continuous action spaces, most methods use classical probability distributions for which they learn the parameters. Finding a policy amounts to learning a mapping from the state space to the distribution's parameter space. A common choice in modern RL is the Gaussian distribution, where the policy assigns a mean vector and covariance matrix to each state. A probabilistic policy can be used in two ways. During reinforcement learning algorithms' training, the action is often selected by sampling the policy $a \sim \pi(\cdot \mid s)$. This ensures exploration during training. At test time, the agent picks the optimal action, i.e., the one with the highest probability:

$$a^* = \operatorname{argmax}_{a \in \mathcal{A}} \pi(a \mid s). \quad (5.4)$$

Value function. The value function is a fundamental quantity in RL. Given a policy π , it measures the *quality* of being in a specific state and following this policy. It is the expectation of future gains starting from the state s :

$$\begin{aligned} V^{\pi}(s) &:= \mathbb{E}_{\pi}[G_t \mid s_t = s] \\ &= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s \right]. \end{aligned} \quad (5.5)$$

Naturally, this quantity depends on the considered policy π . The value function is central to RL. It quantifies the long-term desirability of being in a particular state under a given policy. A state with high value is one from which the agent expects to collect large cumulative rewards in the future. This notion underlies many RL algorithms, especially those that evaluate and improve policies by estimating how good it is to visit certain states.

Q-function. The action-value function or Q-function quantifies the expected return for taking action a in state s , and thereafter following policy π :

$$\begin{aligned} Q^\pi(s, a) &:= \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a] \\ &= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]. \end{aligned} \quad (5.6)$$

The Q-function extends the concept of value by also conditioning on the action taken in the current state. This provides a more fine-grained assessment of decision quality, as it reflects not just the quality of a state but also the quality of a specific action within that state. Q-functions are particularly useful in algorithms that seek to learn optimal policies directly from action-value estimates, such as Q-learning [228] and many actor-critic methods [230]. Together, the value function and Q-function offer complementary perspectives: the former evaluates states globally, while the latter provides localized guidance for action selection because we can define a policy based on it: $a^* = \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a)$.

These foundational concepts – the MDP formalism, return maximization, policies, and value functions – form the theoretical basis for modern reinforcement learning algorithms. The following section examines how these ideas scale through deep function approximation and lead to robust learning systems capable of solving complex, high-dimensional tasks.

5.1.3 Deep Reinforcement Learning

Although RL has existed since the 1970s, it has experienced a more recent and considerable surge in popularity [231]. As supervised learning and the machine learning field in general, RL has benefited from the possibilities offered by deep neural networks. Traditionally, RL leveraged simpler methods such as tabular methods [227], [228], linear function approximators, or shallow neural networks [232], [233]. The main problem of those approaches for policy modeling is their limited capacity to handle larger state and action spaces. Hence, RL has been mainly limited to low-dimensional problems [231]. As MDPs become more intricate, the memory, computation, and, more importantly, sample complexities grow significantly. DNNs offer a capable scaling method and have been successfully used as function approximators [231], [234].

The first applications of artificial neural networks to RL occurred in the 1990s. For instance, the popular REINFORCE algorithm by Williams [235] employed a shallow network that is optimized using gradient descent. However, the major success awaited deep learning's boom of the 2010s. Mnih et al. [4] have famously demonstrated the capacity of those more modern models. They combined Watkins' Q-value algorithm [228] with a deep convolutional neural network. To showcase the capabilities of their method, the authors tackled seven Atari 2600

games from the Arcade Learning Environment [236]. Performance surpassed existing benchmarks and, notably, human performance in six games. This work acted as a foundation for the entire RL domain and started the highly dynamic era of DRL. Since then, larger and larger networks have been combined with advances made in algorithmic aspects.

Nevertheless, scaling DRL models up requires significant amounts of training data. This translates into the need for environments that can provide tens of thousands of interactions at a tractable cost. Mnih et al. used around 10 million frames of each game to train their Deep Q Learning (DQN) agents. The OpenAI Five [7] project consists of training an agent to play the game Dota 2. In total, their agent has played for an approximate duration of 180 years. Similarly, when applying DRL to autonomous driving, collecting a high amount of interactive experience is necessary to achieve decent performance. Bansal et al. [237] have used a dataset of 30 million samples to train their policy. The following paragraphs explore two successful application domains of DRL.

Games. Board or video games are suitable candidates for DRL techniques. Video games are intrinsically operated virtually; board games can easily be transposed to computer programs. This convenient property allows for scalable training data generation. Contrary to applications in robotics, they do not require any special transfer techniques to a real target environment. Also, some games are, by nature, easy, constrained problems in which achieving an optimal policy is sometimes easily feasible. As such, they have been used as toy examples or benchmark tasks to evaluate RL algorithms. The aforementioned Atari benchmark [4] provides RL environments of varying difficulty. Regarding board games, the *AlphaGo* [5] and *AlphaZero* [238] models from DeepMind have shown superhuman performance on the challenging game of Go. While *AlphaGo* was first trained on human expert games and only later fine-tuned using RL, *AlphaZero* exclusively leveraged self-play to learn the policy. This is a clear example of how DRL can solve complex tasks by exploiting massive amount of interactions with the environment.

Reinforcement Learning from Human Feedback (RLHF). Finally, RLHF helps turning LLMs into conversational agents [239], [240], [241]. At first, large-scale auto-regressive models such as GPT-3 [242] are trained on massive datasets in a supervised manner. Then, human agents interact with the resulting *chat bot* and express their preferences between several answers generated by the chatbot. These preference expressions are used to generate a reward signal. The initial model then gets fine-tuned using a DRL algorithm such as PPO [243] to account for the obtained feedback.

5.1.4 Reinforcement Learning for Robotics

By its very nature, robotics has long been a central application area for RL. Robotics tasks typically involve sequential decision-making under uncertainty, making them a natural fit for RL methods. Additionally, because robots are embodied agents that physically interact with the world, learning through trial and error has emerged as a compelling and biologically inspired strategy. Historically, robotics has been approached primarily through the lens of control theory, employing classical methods such as Proportional-Integral-Derivative (PID) controllers, Linear-quadratic regulator (LQR), and Model Predictive Control (MPC) [244],

[245], [246], [247]. These approaches, while successful in many domains, require hand-engineered models of system dynamics and generally cannot scale to high-dimensional, multimodal inputs or adapt flexibly to unmodeled environments.

Prior to the rise of deep learning, classic RL algorithms had been applied to robotics, often with limited success due to issues like poor sample efficiency and sensitivity to hyperparameters. Kober et al. [248] provide an overview of such early efforts, noting several cases in which RL enabled robots to acquire useful behaviors while highlighting the substantial manual engineering required. The advent of DRL, which integrates deep neural networks as function approximators with RL algorithms, has significantly expanded the potential of learning-based control in robotics. DRL makes it possible to learn policies directly from raw, high-dimensional sensor inputs, such as images, and has demonstrated remarkable success in domains like manipulation, locomotion, and navigation [249], [250].

Since the first notorious DRL successes, interest in applying these methods to robotics has grown substantially. However, integrating DRL into real-world robotic systems remains challenging. As Ibarz et al. emphasize [251], real-world deployment introduces issues that are absent from simulated environments. They include sensor noise, hardware wear, safety constraints, and asynchronous execution, all of which can degrade learning stability and policy performance. The authors document practical lessons learned from deploying DRL on real robots, highlighting strategies such as combining offline and online learning, incorporating demonstrations, and designing robust reward functions.

Numerous studies have explored the application of DRL to robotic navigation and social interaction. Akalin et al. [8] provide a comprehensive survey of reinforcement learning methods in social robotics, emphasizing reward design, communication modalities, and interaction dynamics. Pikuli et al. [252], for instance, investigate the use of RL for social robot navigation, where the dynamics of human-robot interaction are too complex to model explicitly. Their study emphasizes the importance of design choices such as observation space, action representation, and reward shaping, and provides practical guidance for ensuring policy convergence. Similarly, Lathuilière et al. [253] demonstrate how reinforcement learning can be used for gaze control in human-robot interaction, learning audio-visual attention policies through a combination of simulation and real-world fine-tuning.

At the same time, researchers have reflected on the broader capabilities and limitations of deep learning in robotics. Sünderhauf et al. [254] argue that while deep learning (DL) has enabled impressive progress in perception and control, robotics poses unique challenges, such as embodiment, uncertainty estimation, and sim-to-real transfer, that are not fully addressed by traditional DL paradigms. They advocate for a tighter integration between data-driven learning and model-based reasoning.

To help assess DRL algorithms in real robotic platforms, Mahmood et al. [255] benchmark several popular DRL methods (e.g., Trust Region Policy Optimization (TRPO), PPO, soft actor-critic (SAC), deep deterministic policy gradient (DDPG)) on a set of standardized real-world tasks. Their results show that while some algorithms perform well out-of-the-box, all are highly sensitive to hyperparameter choices, and performance often varies unpredictably

across tasks. This reinforces the importance of careful environment design and thorough empirical evaluation when applying DRL in robotics.

Overall, deep reinforcement learning offers powerful tools for enabling robot autonomous behavior. However, deploying these methods effectively requires addressing both fundamental algorithmic challenges and the complex realities of real-world robotic systems.

5.1.5 Policy Gradient Algorithms

Early reinforcement learning algorithms primarily focused on value-based methods, where the agent learns to estimate a value function that guides action selection. A notable example is Q-learning [228], which iteratively approximates the Q-function (as defined in Equation (5.6)) and derives a policy by acting greedily with respect to it.

In contrast, policy gradient methods aim to optimize the policy directly by maximizing a differentiable objective function with respect to the policy parameters. Rather than relying on value estimates to induce a policy, these methods treat the policy itself as a parameterized function π_θ and compute gradients of the expected return with respect to θ . This policy often takes the form of a deep neural network whose weights are θ . The objective is typically to maximize the expected return $J(\theta)$, defined as:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \\ &= \sum_{s \in \mathcal{S}} d_{\pi_\theta}(s) V_\pi(s) \\ &= \sum_{s \in \mathcal{S}} d_{\pi_\theta}(s) \pi_\theta(a | s) Q_{\pi_\theta}(s, a), \end{aligned} \tag{5.7}$$

This decomposition expresses the expected return as a weighted sum over states and actions, where $d_{\pi_\theta}(s)$ is the stationary state distribution under policy π_θ . $R(\tau)$ denotes the return of the trajectory τ , also referred to as the cumulative reward. The final line expands the value function using the policy and Q-function, which will be instrumental in deriving the policy gradient. Most policy gradient algorithms rely on maximizing this objective using gradient ascent algorithms. This process results in an optimal set of weights θ^* leading to the highest return. However, computing the gradient of J directly can be complex. The policy gradient theorem offers a tractable expression for this gradient that avoids the need to differentiate through the environment's dynamics.

Several policy gradient algorithms have been developed and successfully used in conjunction with deep neural networks. Our main focus will be on the PPO algorithm [243], but other notable methods include DDPG [256], SAC [230] and TRPO [257]. A key advantage of policy gradient approaches over value-based methods like Q-learning is their ability to handle both discrete and continuous action spaces naturally.

In the next section, we present the Policy Gradient Theorem, which underpins all such algorithms and provides the theoretical foundation for computing policy updates via gradient ascent.

5.1.5.1 Policy Gradient Theorem

Sutton & Barto [221] present the Policy Gradient theorem as a fundamental result enabling policy gradient algorithms. It provides a tractable expression for the gradient of the expected return under a differentiable stochastic policy. Let $\pi_\theta(a \mid s)$ be a differentiable policy and $J(\theta)$ the expected return (Equation (5.7)). Then:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d_{\pi_\theta}, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a \mid s) Q_{\pi_\theta}(s, a)] \quad (5.8)$$

Proof. (finite-horizon case)

Adapted from Sutton and Barto [221].

We aim to compute the gradient of the expected return $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$, where $\tau = (s_0, a_0, r_0, \dots, s_T)$ is a trajectory generated by the policy π_θ and $R(\tau)$ its return. Using the log-derivative trick, we write:

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log p(\tau; \theta)]. \end{aligned} \quad (5.9)$$

The trajectory probability under policy π_θ is:

$$p(\tau; \theta) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t), \quad (5.10)$$

where $\rho(s_0)$ is the initial state distribution, and $P(s_{t+1} \mid s_t, a_t)$ are the MDP's transition dynamics. As they are both independent of θ , the log of the trajectory probability simplifies to:

$$\log p(\tau; \theta) = \sum_{t=0}^{T-1} \log \pi_\theta(a_t \mid s_t) + \text{const}. \quad (5.11)$$

Taking the gradient:

$$\nabla_\theta \log p(\tau; \theta) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t). \quad (5.12)$$

We substitute back into the gradient of the objective:

$$\begin{aligned} \nabla_\theta J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right] \\ &= \sum_{t=0}^{T-1} \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a_t \mid s_t) R(\tau)] \end{aligned} \quad (5.13)$$

Since the return $R(\tau)$ is constant for all time steps, we can reduce variance by using the time-dependent return $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ starting at time t , as proposed in REINFORCE [235]:

$$\nabla_{\theta} J(\theta) = \sum_{t=0}^{T-1} \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t] \quad (5.14)$$

This is the REINFORCE estimator, which gives an unbiased estimate of the policy gradient [235]. Finally, this expectation can be rewritten in terms of the stationary distribution $d_{\pi_{\theta}}(s)$, yielding the result:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d_{\pi_{\theta}}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)] \quad (5.15)$$

■

This result shows that policy gradients can be estimated using samples from the environment without requiring gradients through the environment's dynamics. In practice, $Q^{\pi_{\theta}}(s, a)$ is often replaced by the empirical return G_t , an advantage estimate, or a separately learned value function

5.1.5.2 Advantage Estimation

In the policy gradient expression given by the theorem (Equation (5.8)), the expected return is weighted by the Q-function. However, estimating $Q^{\pi}(s, a)$ directly can be challenging in practice due to its high variance, especially when using sample-based estimates. To address this, many policy gradient algorithms rely on advantage functions, which capture how much better (or worse) an action is compared to the average action at a given state.

Advantage function. The advantage function is defined as:

$$A^{\pi}(s_t, a_t) := Q^{\pi}(s_t, a_t) - V^{\pi}(s_t), \quad (5.16)$$

This formulation provides a more targeted signal for learning: instead of reinforcing actions that are merely good in absolute terms, it favors those that are better than expected. The use of advantage estimates has become standard in modern actor-critic algorithms.

Estimation. The estimation of the advantage function was addressed by Schulman et al. [258]. To estimate the advantage function, we may consider the following class of estimators $\hat{A}_t^{(k)}$:

$$\begin{aligned} \hat{A}_t^{(1)} &:= \delta_t^V &= r_t + \gamma V(s_{t+1}) - V(s_t) \\ \hat{A}_t^{(2)} &:= \delta_t^V + \gamma \delta_{t+1}^V &= r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t) \\ \dots &:= \dots &= \dots \\ \hat{A}_t^{(\infty)} &:= \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}^V &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots - V(s_t), \end{aligned} \quad (5.17)$$

where δ_t^V estimates the TD error. While all those quantities approximate A_t , they offer different tradeoffs. Indeed, as $V(s_t)$ is not the exact value function $V^{\pi, \gamma}$ for this policy, the estimator is biased. However, this bias decreases when $k \rightarrow +\infty$ as the term $V(s_{t+k})$ becomes increasingly dampened. $V(s_t)$ remains constant among the class of estimators and thus does not affect the relative bias of $\hat{A}_t^{(k)}$. Although being asymptotically unbiased, $\hat{A}_t^{(\infty)}$ has a high variance. The variance increases with k , as each additional term r_{t+k} introduces more uncertainty into the estimator.

Generalized Advantage Estimation (GAE). Schulman et al. [258] proposed a novel estimation method by combining all the estimators $\hat{A}_t^{(k)}$ into a single one. More precisely, the GAE is an exponentially weighted average of the aforementioned k -step estimators.

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} := (1 - \lambda) \left(\hat{A}_t^{(1)} + \hat{A}_t^{(2)} + \hat{A}_t^{(3)} + \dots \right), \quad (5.18)$$

where γ is the MDP's discount factor and λ is a hyperparameter that controls the estimator's bias-variance tradeoff. When $\lambda = 0$, GAE reduces to a one-step temporal difference estimate (similar to TD(0)), which has low variance but may be biased due to relying heavily on the learned value function. When $\lambda = 1$, it approximates a Monte Carlo return by summing full future rewards, yielding an unbiased but high-variance estimate. By tuning λ , one can interpolate between these extremes, making GAE a versatile and effective estimator.

It can be shown that Equation (5.18) simplifies to:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\lambda \gamma)^k \delta_{t+k}^V. \quad (5.19)$$

Therefore, the GAE estimator is the exponentially-decayed sum of rewards. In practice, it has become a standard component in modern actor-critic methods, such as PPO [243] and A3C [259].

5.1.5.3 The Proximal Policy Optimization (PPO) Algorithm

PPO is a widely adopted reinforcement learning algorithm known for its balance between implementation simplicity and robust performance. It has been effectively applied in various domains, including robotics and autonomous navigation [260]. For instance, PPO has been utilized to train mobile robots for safe navigation in complex environments, demonstrating improved stability and obstacle avoidance capabilities. Additionally, PPO has been employed in multi-robot systems to optimize path planning and reduce navigation time through effective conflict resolution strategies.

TRPO. Schulman et al. [257] introduced the concept of trust region policy optimization. They observed that overly aggressive policy updates were causing instabilities in training Policy Gradient (PG) algorithms. The TRPO algorithm seeks to constrain the policy to a *trust region*, preventing it from evolving too drastically at each step. The notion of distance for policy updates is the Kullback-Leibler (KL) divergence. The TRPO objective is to maximize the

expected advantage while keeping the KL divergence between the old and new policy below a certain threshold. This translates to maximizing:

$$L^{\text{TRPO}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \quad (5.20)$$

subject to

$$\hat{\mathbb{E}}_t \left[\text{KL} \left[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t), \right] \right] \leq \xi, \quad (5.21)$$

for a given threshold ξ . $\hat{A}_t$ is the advantage estimate at time step t . This objective can be rewritten using a penalty term. It boils down to maximizing:

$$\hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL} \left[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_\theta(\cdot | s_t) \right] \right] \quad (5.22)$$

for some coefficient β .

In practice, TRPO approximates the KL constraint with its second-order Taylor expansion. This requires computing the Fisher Information Matrix, translating into a quadratic optimization problem. Because it is a constrained problem, first-order optimizers like Adam or stochastic gradient descent (SGD) cannot be used, and the objective is thus optimized using conjugate gradient optimization. These practical limitations make TRPO training computationally expensive.

PPO. In [243], Schulman et al. iterate on TRPO by proposing the PPO algorithm, which solves most of the original algorithm's shortcomings. Its main innovation is replacing the KL divergence constraint with a clipped surrogate objective, which prevents too-large policy updates without second-order optimization. This choice slightly loosens the constraint that TRPO imposes but significantly decreases the optimization's computational complexity. Bick [261] offers a comprehensive and accessible dissection of PPO, bridging theoretical foundations with practical implementation details.

The PPO objective combines three components: a clipped policy loss, a value function loss, and an optional entropy bonus. The clipped policy loss is defined as:

$$L_t^{\text{CLIP}}(\theta) = \min \left[\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right], \quad (5.23)$$

where $\rho_t(\theta)$ is the ratio between the old and new policy for the state s_t :

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (5.24)$$

The clipping function prevents large updates when the new policy diverges too far from the old one, thus preserving stability. This constitutes the primary innovation of PPO over TRPO

as it enforces a constraint on the policy update size without the need to deal with an explicitly constrained optimization problem.

The second component of the loss is the value function loss, typically a squared error between the predicted value and the empirical return:

$$L_t^{\text{VF}}(\theta) = [V_\theta(s_t) - R_t]^2, \quad (5.25)$$

where $V_\theta(s_t)$ is the predicted state value and R_t is the estimated return. R_t can be derived via the aforementioned GAE estimator.

Finally, the PPO loss entails an entropy bonus, which encourages exploration by maximizing the policy's entropy:

$$S[\pi_\theta](s_t) = \sum_{a \in \mathcal{A}} \pi_\theta(a | s_t) \log[\pi_\theta(a | s_t)]. \quad (5.26)$$

The final PPO loss can finally be expressed as:

$$L_t^{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t \left[L_t^{\text{CLIP}}(\theta) - c_V L_t^{\text{VF}}(\theta) + c_S S[\pi_\theta](s_t) \right], \quad (5.27)$$

where c_V and c_S are coefficients that weight the value function loss and entropy bonus, respectively, relative to the policy loss. These are treated as hyperparameters. This formulation is the most common in the literature and software implementations of PPO. Although often denoted as the PPO *loss*, this function quantifies the objective and should thus be maximized.

Actor and Critic Networks. As in other actor-critic DRL algorithms, PPO leverages two neural networks during training.

- The **critic** estimates the value function. It is trained in a supervised manner during the PPO optimization step.
- The **actor**, given an observation from the environment, outputs a probability distribution over the action space. Its role is to implement the actual policy dictating the agent's behavior.

In practice, these networks may share some weights.

Algorithm. To optimize the PPO objective, the algorithm involves iteratively sampling training data by running the policy in the environment and training the actor and critic networks. Each iteration starts by freezing the neural network parameters θ and collecting a set of trajectories $\mathcal{T}$ by running the current policy π_θ in the environment. This sampling phase ends with the computation of the advantages $\hat{A}_t$ with the GAE method presented in Section 5.1.5.2, and the returns R_t . The second part of the iteration is where the learning occurs. The actor and critic networks are optimized by maximizing the overall PPO objective L^{PPO} (Equation (5.27)). This is most often done with a stochastic gradient optimizer such as Adam [193] or SGD. Listing 5.1 gives the pseudo-code for the PPO algorithm.

```

1: function PPO( $n_{\text{iter}}, n_{\text{epochs}}, b_{\text{size}}$ )
2:    $\triangleright n_{\text{iter}}$ : The number of PPO iterations
3:    $\triangleright n_{\text{epochs}}$ : The number of training epochs done in each iteration
4:    $\triangleright b_{\text{size}}$ : The number of samples (transitions)
5:
6:   for  $i$  in  $1 \dots n_{\text{iter}}$  do
7:     Initialize the network parameters  $\theta$  randomly.
8:     Collect a set of trajectories  $\mathcal{T} = \{\tau_k\}$  by running policy  $\pi_\theta$  in the environment.
9:     Compute advantage estimates  $\hat{A}_t$  using GAE based on the current value function  $V_\theta$ .
10:    Compute returns  $\hat{R}_t = V_{\pi_\theta}(s_t) - \hat{A}_t$ 
11:
12:     $\triangleright$  Optimize the actor and critic networks
13:    Split  $\mathcal{T}$  in mini-batches of  $b_{\text{size}}$  transitions each.
14:    for  $k$  in  $1 \dots n_{\text{epochs}}$  do
15:      for each mini-batch in  $\mathcal{T}$  do
16:        Update the actor's and critic's parameters by maximizing the PPO objective
17:         $L^{\text{PPO}}(\theta)$  using a stochastic optimizer (e.g., Adam, SGD).
18:
19:  return  $\pi_\theta$ 

```

Listing 5.1 – Simplified PPO algorithm. It iterates between a data sampling phase where trajectories are collected by running the agent in the environment, and a learning phase where the neural network is optimized according to the PPO objective L^{PPO} .

5.2 Sound-Driven Robot Navigation: Problem Statement

5.2.1 Motivation

Audio perception is an essential pillar of social robotics. Achieving realistic human-robot interactions is conditioned on the ability to hear and understand the content of people's speech. ASR techniques have significantly progressed, mainly thanks to recent advances in deep learning [262]. However, some real-world settings remain challenging, and ASR systems can struggle to extract the speech content properly. Most solutions are trained on a clean speech dataset. The literature has studied the impact of noise and reverberation on ASR performance. Wang et al. [263] explored different strategies to account for the challenges present in reverberant and noisy environments.

In this chapter, we propose to study this issue in the robotic context. We aim to develop algorithms that improve robots' hearing ability in real-world interaction environments. To tackle this issue, we target the design of navigation policies. An agent will be placed in a reverberant environment with an active speech source. Its goal is to position itself optimally to maximize the ASR performance. Our approach solely relies on the positioning of the robot. The ASR algorithm that we use is a pre-existing DNN-based solution. Our motivation relies on the observation that the microphone's position and orientation relative to the target source in a complex acoustic environment significantly impact the ASR performance. This insight results from an experimental study conducted within our simulated environment. Other metrics exist to quantify auditory perception and could have been alternatives to the ASR performance. For

instance, the Perceptual Evaluation of Speech Quality (PESQ) score, introduced by Rix et al. [264], predicts the perceived audio quality of speech. It primarily targets human perception by measuring how a degraded signal compares to its original counterpart. ASR performance is likely correlated to the PESQ score, but directly quantifies how well the robot understands human speech. In this sense, PESQ would have been a more indirect proxy for robotic auditory perception.

We use DRL as a learning framework to tackle this navigation task. Different solutions have been investigated in the literature for designing navigation policies. MPC [247] is a widely used mathematical framework in robotics. It is an advanced control strategy used to determine the optimal control inputs for a robot by predicting future behavior over a finite time horizon. While MPC has been used extensively for navigation and motion planning in robotics [265], [266], it is not quite suited for handling acoustic cues directly. On the one hand, the Word Error Rate (WER) provides an informative reward signal and can thus be exploited by an RL algorithm. On the other hand, the combination of DNN and RL in the form of DRL allows processing high-dimensional representations for the MDP observations.

5.2.2 Background

The problem of perceptually-motivated audio-based navigation has not been studied much prior. Nevertheless, this section will highlight some notable works that have already proven the relevance of this area of research. Also, they provide advanced solutions to problems similar to the one tackled in this thesis.

Magassouba et al. [267] proposed a novel framework for robotic audio-based navigation, *AuralServo*. Although inspired by ASSL pipelines similar to those presented in Chapter 4, their approach differs significantly. No explicit SSL block is defined in their framework. Indeed, the robot controls are directly inferred from the aural perception. This system is architected around a direct feedback loop, allowing for low computational cost and, thus, response times. Their contribution is a theoretical scheme for commanding a robot using auditory features and real-world experiments with audio-based control tasks. Those tasks include automatic gaze adjustment to face the active speaker and a navigation task based on ILD. The latter comprises following a sound source indoors, solely relying on audio perception. Regarding the proposed methodology, the process involves computing relevant auditory features such as the ILD and IPD. These quantities are then interpreted geometrically to be linked to the task objective, thus defining an interaction matrix $\mathbf{J}_s$ satisfying:

$$\dot{\mathbf{s}} = \mathbf{J}_s \mathbf{u}, \quad (5.28)$$

where $\dot{\mathbf{s}}$ is the variation of the feature set and $\mathbf{u}$ denotes the control signal. $\mathbf{u}$ is then inferred so as to minimize the error $e(t)$ to the target state $\mathbf{s}^*$:

$$e(t) = \mathbf{s}(t) - \mathbf{s}^*. \quad (5.29)$$

The paper provides a detailed theoretical derivation of the auditory features, interaction matrices, and solving schemes. This work thus adopts a feedback loop strategy to control the robot and does not involve any learning algorithm. Finally, *AuralServo* provides an interesting

formulation for active robotic navigation problems and an analytical solution that could be demonstrated in real-world experiments.

Furthermore, a research group from UT Austin, led by Kirsten Grauman, has conducted pioneering work in perceptually-motivated robotic navigation and achieved impressive results. In their paper *SoundSpaces* [48], Chen et al. define and solve the task of audio-visual navigation towards a sound source in complex environments. Their contribution is twofold. On the one hand, they introduce *SoundSpaces*, a dataset that extends existing realistic 3D environments with simulated audio renderings. The environment uses the *Habitat* simulator [268] alongside the *Matterport3D* [269] and *Replica* [270] datasets that it includes. This simulator provides only visual cues from its 3D representation of indoor spaces. Chen et al. leveraged a geometrical acoustic method for room acoustic simulations consisting of bidirectional path tracing [52] to add auditory information to the simulator. On the other hand, they designed a DRL agent that can navigate these challenging environments. Its neural network architecture receives the RGB and depth frames from the virtual camera, the STFT of the listened signal, and the relative displacement vector pointing from the agent to the goal. Following feature-specific, then shared layers are two heads modelling the actor and the critic, respectively. The authors trained the system to navigate to a source in previously unknown environments. Audio is shown to improve navigation performance substantially. Mixing audio and visual information allows the agent to extract more spatial knowledge from its environment and, thus, navigate more efficiently towards the source. This study has also further demonstrated the capacity of neural networks to exploit the reverberation phenomenon to achieve a navigation task. This proposal illustrates how DRL can be used with multi-modal agents to solve complex navigation tasks.

Grauman's team has expanded on their framework in a follow-up paper by Majumder et al. called *Move2Hear* [10]. The task tackled here consists of navigating a complex 3D environment with the motivation to enhance audio perception. More precisely, the agent's objective is to adjust its position with respect to several sound sources to perform optimal audio separation. The agent starts at a random position in a 3D scene from the *Habitat* simulator. One is designated as the target source among the active sound sources in the environment. The performance obtained on the audio separation task is the primary reward signal available for learning the policy. Besides, the agent is also rewarded for reducing the geodesic distance to the target source. The neural network that implements it is split into two main blocks. The first takes the STFT of the binaural signal listened to by the agent and the identifier of the target speech. It is trained to output the isolated target speech signal's STFT and thus performs the actual source separation. The second component of the agent is the active audio-visual controller. It is responsible for implementing the navigation policy. A common feature extraction backbone uses GRUs [271] to perform the core perception work. It is followed by two heads implementing the actor and the critic, optimized by the PPO algorithm. The policy comprises two sub-policies: one for improving the separation quality when relatively close to the target source, and an audio-visual navigation policy to get closer to it. Both networks are trained in a cyclic pattern, ensuring continuous and synchronous overall performance improvement. A convincing experimental study is also conducted to demonstrate the approach's effectiveness. They propose two different benchmarks. In the *near-target* task, the agent starts relatively close to the target sound source. Here, it has to adjust its position to optimize for the

separation score. In the *far-target* setting, the agent is initially placed further from the source of interest and has to navigate the environment to eventually get close to it. This task variant targets leveraging audio-visual information to plan the shortest possible trajectory. Overall, the *Move2Hear* framework successfully applies DRL to a robotic navigation problem where the objective is motivated by audio perception. This work does not specifically optimize for the ASR performance. Also, the proposed method makes use of visual information.

5.2.3 Problem Formulation

We adopt a DRL approach to design such a navigation policy. This policy will be modeled by a deep neural network trained in a simulated environment. We develop a complete pipeline for solving the perceptually motivated audio-based navigation task. The problem is cast as a sequential decision process, aligning naturally with the RL framework. This work's original contribution includes implementing the RL environment, agents, algorithm, and evaluation setup.

Also, our solution tackles a challenging framework where only audio data can be used to perceive the environment. The agent has neither visual cues nor direct spatial information, such as absolute or relative positions. To address this limitation, we leverage the algorithms and knowledge obtained from our study of SSL.

In the following paragraphs, we will formalize the task we plan to solve. The core novelty of this problem is to use the ASR performance as the reward signal. First, we will introduce the relevant metric and its relation to speech recognition. Second, we will present the RL environment and the justification for our different choices.

WER metric for ASR. The ASR task involves transcribing the speech content from an audio recording. It can be done using pre-recorded samples or in real-time from an audio stream. The WER metric measures the performance of ASR systems. It is computed as follows:

$$\text{WER} = \frac{s + d + i}{n}, \quad (5.30)$$

where s is the number of substitutions, d is the number of deletions, and i is the number of insertions needed to transform the true sentence into the predicted text. n counts the total number of words in the ground truth transcript. The WER quantifies the difference between the original and transcribed text. The total number of errors $s + d + i$ is also called the Levenshtein or edit distance, which Vladimir Levenshtein proposed in 1965 [272]. Its default formulation considers comparing two strings at the character level. Lets consider two strings a and b (of length $|a| = n + 1$ and $|b| = m + 1$ respectively).

The Levenshtein distance between a and b is computed as:

$$\text{lev}(a, b) = \begin{cases} |a| & \text{if } m = 0 \\ |b| & \text{if } n = 0 \\ \text{lev}(a_{1..n}, b_{1..m}) & \text{if } a_0 = b_0 \\ 1 + \min \begin{cases} \text{lev}(a_{1..n}, b) \\ \text{lev}(a, b_{1..m}) \\ \text{lev}(a_{1..n}, b_{1..m}) \end{cases} & \text{otherwise,} \end{cases} \quad (5.31)$$

where a_0 is the first character of a and $a_{1..n}$ is the string a without its first character. It counts the number of edits to turn string a into string b . As such, it is bounded by the length of the longest string:

$$0 \leq \text{lev}(a, b) \leq \max(|a|, |b|). \quad (5.32)$$

The WER score uses words instead of characters as the fundamental tokens.

Running of an episode. An episode starts with the agent and one or several sources randomly placed in the room. One source is considered the target. This means that the ASR output will be evaluated against the actual transcript pronounced by this source. Other sources might also be added and can act as adversarial sources. The process is sequential and discrete. At each time step, the agent will be offered to listen to the surrounding audio for a fixed duration (inferior to one second). The robot will then be asked to select a movement action to enhance its relative position to the target source. The simulator then applies this movement, after which the agent receives both a new observation and a reward signal computed from the WER of its new position. We choose to fix a maximum number of 32 steps per episode. This environment thus has a finite horizon. No specific event occurs at the final step. The reward of the last state is computed using the same scheme as for the intermediary steps. The following section will formalize the definition of the RL environment.

5.2.3.1 Environment Definition

The RL environment corresponds to the MDP formulation and implementation for the task. In this section, we present those characteristics and the motivations that led to this final specification.

POMDP. The original MDP model lacks expressivity to model our environment accurately. Instead, we rely on the POMDP framework, which does not allow direct observation of the state. The agent can probe its environment through observations distinct from the underlying states. This choice decouples the environment logic from the sensory observation available to the agent. A POMDP can be described as a tuple $\langle \mathcal{S}, \mathcal{A}, P, \mathcal{R}, \Omega, O, \gamma \rangle$ where:

- $\mathcal{S}, \mathcal{A}, P, \mathcal{R}$ and γ describe the underlying MDP
- Ω is the observation space; and
- $O : \mathcal{S} \times \mathcal{A} \rightarrow \Pi(\Omega)$ is the *observability function* which maps all state-action pairs to a probability distribution over the observation space Ω . $O(s' \mid a, o)$ denotes the probability of making observation o given that the agent took action a and landed in state s' .

This definition and its notations have been borrowed from Leslie Kaelbling et al. [273]. She and her research group have conducted groundbreaking work related to the comprehension and use of POMDPs, particularly in robotics. From her original work on this topic, *Acting Optimally in Partially Observable Stochastic Domains* [274] in 1994 Kaelbling has published several papers exploring how to efficiently solve partially observable environments problems [275], [276], [277], [278], [279]. Those works are not strictly limited to the RL domain. More recently, Azizzadenesheli et al. [280] investigated how policy gradient algorithms could be adapted to POMDPs.

State and action spaces. We chose a spatially discrete setting for modeling the environment. The agent evolves along a virtual $n_x \times n_y$ grid spanning the entire room. Also, its orientation is restricted to be aligned with the grid, i.e., being either *up*, *down*, *left*, or *right*. The discrete state space $\mathcal{S}$ can then be expressed as follows:

$$\mathcal{S} = \left\{ (x_i, y_j, \alpha) \mid (i, j, \alpha) \in \llbracket 1, n_x \rrbracket \times \llbracket 1, n_y \rrbracket \times \left\{ 0, \frac{\pi}{2}, \pi, \frac{3\pi}{2} \right\} \right\}, \quad (5.33)$$

where x_i and y_j denote the 2D position of the agent and α its orientation with respect to the room global frame. The position and orientation restrictions are enforced by the following action space $\mathcal{A}$:

$$\mathcal{A} = \{\text{STAY}, \text{FORWARD}, \text{TURN_LEFT}, \text{TURN_RIGHT}\}. \quad (5.34)$$

- STAY is the neutral action where the agent does not move away from its current location.
- FORWARD means moving by a distance of d_{forward} meters in the current direction α .

The step size d_{forward} must correspond to the distance δ_{grid} between two adjacent grid points.

- TURN_LEFT and TURN_RIGHT correspond to a quarter-turn rotation. In this case, only the orientation of the agent changes, not its position.

A learned policy for this problem will be a probabilistic distribution $\pi(\cdot \mid \cdot) : \mathcal{A} \times \Omega \rightarrow [0, 1]$ over this finite set of four actions.

Although our audio simulator would have permitted a continuous formulation, choosing a discrete setting provides important benefits. The most crucial one of them is the ability to pre-compute the reward. Indeed, having a discrete state space allows for caching the reward function for all possible states. As will be explained later in [Section 5.3.2](#), our reward function is very computationally expensive. On the other hand, having a finite action space enables working with equally finite policies $\pi(a \mid s) = \pi_a(s)$. This choice permits the use of RL algorithms modeling the Q-value, such as DQN methods [4]. Also, such a framing is both computationally and conceptually simpler. Zhu et al. [281] survey the role of action spaces in RL and its applications. They notably explore the differences between continuous, discrete, and discrete-continuous hybrid action spaces.

Observation space. The agent's perception of its environment is limited to the audio signal received by its microphones. It does not know its current position. This acoustic information consists of a one-second-long recorded signal mapped to the time-frequency domain. Practically, the observations are (C, F, T) real tensors where C is the number of output channels,

F is the number of frequency bins, and T is the number of temporal indices. Formally, the observation space is $\Omega = \mathbb{R}^{C \times F \times T}$. The agent aims to learn a mapping $s \rightarrow \pi(\cdot | s)$ from this space to the probabilities over actions. This represents the essence of the audio-based navigation task: learning to decide how to move depending on what is heard. The observability function translates the listening process. Sound sources speak continuously. At each step, the robot listens for τ_{step} seconds while remaining immobile. The random nature of the speech content delivered by each source accounts for the position-agnostic stochasticity of the observation function. The rest of the process consists of the sound propagation and reverberation simulation. By nature, there is no analytical closed-form expression of O . It is directly embedded in the environment implementation.

Reward function. The main originality of our approach lies in the perceptually motivated objective. The agent should be trained to navigate to the optimal location regarding the ASR performance. Naturally, the reward function should be a decreasing function, $f : [0, 1] \rightarrow \mathbb{R}$, of the WER metric so that the highest reward would correspond to the lowest possible WER. For now, we assume having access to an oracle cost function $C : \mathcal{S} \rightarrow [0, 1]$ that maps each possible state to a penalty that should be minimized. It quantifies how undesirable it is to be in this specific state. Although we explored alternative formulations for the cost function C , the primary cost used in the task is C_{WER} , which represents the estimated average Word Error Rate (WER) that the ASR system would produce if the agent were located at that specific position. The practical implementation of this mapping will be discussed later in [Section 5.3.2](#). The reward function can then be written as follows:

$$r_t = \begin{cases} -\mu_W & \text{if the movement is invalid} \\ f(C_t) - \mu_m \mathbb{1}(a_t = \text{FORWARD}) & \text{otherwise,} \end{cases} \quad (5.35)$$

where:

- $C_t = C(s_t)$ is the cost value of the state s_t ;
- μ_W is a positive scalar that can be adjusted according to f 's magnitude; It allows penalizing movements that would lead the robot to collide with a wall. When the policy samples such an impossible action, the environment ignores it, the agent remains immobile for this step, and a fixed reward of $-\mu_W$ is returned. This can only occur for the forward action.
- μ_m is a movement penalty that disincentivizes the agent from moving uselessly. It encourages the policy to remain static when it can, while solely moving when it serves a meaningful purpose. Combined with the standard cost reward $f(C_t)$, it entices the agent to position itself optimally as efficiently as possible.

Environment initialization When an episode starts, the initial state is drawn randomly according to the probability distribution $\rho : \mathcal{S} \rightarrow [0, 1]$. In our environment, we randomly draw the agent's starting position and orientation when an episode begins. Therefore, the initial state distribution is the uniform distribution over the state space: $s_0 \sim \rho = \mathcal{U}(\mathcal{S})$.

Transition dynamics. Finally, we must provide the transition dynamics to fully define our MDP. More precisely, we refer to the conditional probability $P(s' | s, a)$ of being in the state s' when coming from the state s and performing action a . Those dynamics are implemented by the simulator along with the observability function O . In our case, they are completely

deterministic due to their purely geometric nature. Indeed, each action leads to a predictable change of the agent's position (x_a, y_a, α_a) . The transition dynamics can be expressed as the following deterministic function t , which maps a state-action pair to the next state:

$$t : \mathcal{S} \times \mathcal{A} \longrightarrow \mathcal{S}$$

$$((x_a, y_a, \alpha_a), a) \mapsto \begin{cases} (x_a + d \cos(\alpha_a), y_a + d \sin(\alpha_a), \alpha_a) & \text{if } a = \text{FORWARD} \\ (x_a, y_a, (\alpha_a + \frac{\pi}{2})[2\pi]) & \text{if } a = \text{TURN_LEFT} \\ (x_a, y_a, (\alpha_a - \frac{\pi}{2})[2\pi]) & \text{if } a = \text{TURN_RIGHT} \\ (x_a, y_a, \alpha_a) & \text{if } a = \text{STAY} \\ & | (a = \text{FORWARD} \wedge a \text{ is invalid}). \end{cases} \quad (5.36)$$

The collisions are detected before execution so that the environment can deny the action.

Finally, the environment's horizon is finite. No event can terminate the environment before this time limit. Therefore, each episode lasts a fixed number of steps, denoted as the horizon T .

5.2.3.2 Alternative Continuous Formulation

A spatially discrete solution was proposed, where the state and action spaces are finite. Such a choice grants various benefits, such as an easier implementation and the possibility of using RL methods that cannot handle continuous spaces.

Alternative POMDP. On the other hand, a more general continuous formulation can be described. It would offer a more realistic modeling of the real-world problem. Indeed, restricting a real robot to move exclusively on a grid is not sensible. Alternatively, one could change the previously defined POMDP. First, the state space would become $\mathcal{S}_c = [0, L_x] \times [0, L_y] \times [0, 2\pi] \subset \mathbb{R}^3$. Several choices are possible regarding the action space. Although it would be possible to employ the original finite action space $\mathcal{A}$, this solution would still restrict the set of reachable positions to a same-size 2D grid. It would be rotated and translated accordingly to the initial position. A more sensible choice would be sampling a distance and a rotation angle to parametrize a straight translation. This would amount to the continuous action space $\mathcal{A}_c = [0, d_{\max}] \times [0, 2\pi]$. Entirely freeform movements would still not be possible, but this new space would allow for more expressive trajectories. The current implementation of the simulator allows for easily extending the existing discrete environment to continuous variants. The distinctions exposed here only impact the spatial properties of the MDP. Audio aspects of the environment may remain the same.

Reward computation. The reward computation is another obstacle to handling the problem's continuous form. Currently, it is fully pre-computed and cached using the WER maps solution. Keeping this paradigm is possible by interpolating the grid-evaluated reward function to compute the reward for arbitrary positions.

Continuous audio simulation. Besides, the proposed framework assumes that movements are instantaneous and that listening happens strictly statically. Accounting for a continuously moving agent would contribute to improving the model's realism further. As the MDP model remains fundamentally sequential, the agent would still make step-based decisions. A hybrid solution entails performing the audio simulation along a trajectory instead of assuming the

position to be fixed. Although our simulator does not support such a feature, the underlying *gpuRIR* authors explain how it could be implemented (see section 3.4 of Diaz-Guerra et al. [12]).

Algorithmic implications. The PPO algorithm employed in this work is fully compatible with continuous state and action spaces. Instead of predicting the probability p_a of each discrete action, the actor neural network outputs the parameters $(\mu, \sigma I)$ of a $\dim(\mathcal{A})$ -dimensional normal distribution. Petrazzini et al. [282] alternatively propose to use the Beta distribution. Compared to a Gaussian distribution, its primary benefit is having a finite support that naturally fits bounded action spaces. They also found the Beta distribution to outperform the Gaussian one.

In conclusion, the simple discrete formulation of the sound-driven navigation problem can be generalized to better represent real-world robotic scenarios.

5.3 Proposed Approach

5.3.1 Acoustic Pipeline

The abstract environment defined previously in Section 5.2.3.1 is implemented in practice thanks to our simulator Section 2.3. More precisely, its capacity to operate in discrete time steps makes it particularly convenient for use in an RL framework. The specific implementation details of the simulator’s dynamic features have been described in Section 2.3.3. It allows us to place our microphone array and the target source at the beginning of each episode. The RL environment wraps the simulator’s step function and moves the agent according to the action selected from the policy. The simulator loads the next chunk of audio and sets it as the source’s input. The source clean speech recordings are sampled from the *LibriSpeech* [81] corpus. When transitioning from one step to the next, the source keeps playing the same recording to improve realism. After loading the source’s input signal, the simulator refreshes the cached RIRs if necessary, i.e., if the agent has moved, and computes the next chunk of recorded audio. Finally, the resulting multi-channel signal is further processed into an interaural spectral representation. Specifically, we use the IPD and ILD features. A three-microphone array is used to model the agent’s sensors. Each microphone has a cardioid pattern. Therefore, the observations materialize as $(6, F, T)$ multi-channel spectral features.

The final environment is Gym-compliant. OpenAI *Gym* [283] was created to define the standard way of interacting with RL environments. *Gymnasium* [284] by Towers et al. has since replaced it and was used in this project. This precaution ensures compatibility with the rest of the RL ecosystem and would allow other researchers to use the designed environment.

5.3.2 WER as a Reward Signal

This section presents the design and implementation details of the WER cost function (C_{WER}). The reward signal introduced previously expects an oracle to provide an estimate of the WER score for each possible state. This is achieved by pre-computing an average WER for every

position on the grid, and possibly every orientation of the agent. The obtained WER cost function is given by:

$$C_{\text{WER}} : \mathcal{S} \rightarrow \mathbb{R}_+$$

$$(\mathbf{x}_a, \alpha_a) \mapsto \mathbb{E}_{(v,t) \in \mathcal{D}} \left[\frac{1}{100} \text{WER} \left(\underbrace{\text{ASR}_\psi [\text{listened}(v, \mathbf{x}_a, \alpha_a, \mathbf{x}_s)]}_{\text{predicted transcript } \hat{t}}, t \right) \right], \quad (5.37)$$

where:

- v and t are clean speech recordings, respectively, and their associated transcripts are drawn from a corpus $\mathcal{D}$.
- $\text{listened}(v, \mathbf{x}_a, \alpha_a, \mathbf{x}_s)$ is the signal recorded by the agent's primary microphone when it is located at $\mathbf{x}_a$ and oriented by α_a while the speech source, located at $\mathbf{x}_s$, plays the recording v . This operator encompasses the reverberation properties of the room.
- ASR_ψ denotes a deep ASR model parametrized by parameters φ . Given a recorded signal, it outputs a prediction for the transcript $\hat{t}$.
- $\text{WER}(\hat{t}, t)$ is the WER (in %) between predicted transcript $\hat{t}$ and ground truth transcript t .

Although the array may comprise multiple microphones, ASR measurements rely solely on a single one — namely, the microphone that is aligned with the agent's forward-facing direction. Eventual techniques to combine the signal from multiple microphones are out of the scope of this work. The following paragraphs discuss the C_{WER} 's computing and caching implementation.

5.3.2.1 ASR Frameworks

Several ASR frameworks have been made available by industrial and academic actors. Kaldi [285] is one of the most complete and established open-source projects for speech recognition. The C++ code base includes various algorithms and helpers to process speech and perform speech-related tasks. It includes feature extraction mechanisms, decoding algorithms, seq2seq models for end-to-end training, and support for traditional HMM-GMM and modern DNN-based approaches, making it a versatile toolkit for research—and production-grade ASR systems. PyKaldi [286] offers a Python wrapper that allows users to interact with the Kaldi library easily.

Vosk [287] is another open-source, popular speech recognition toolkit. It aims to be a user-friendly product that can be employed in real-world use cases. Bindings for several programming languages (Rust, Java, Go...). Besides its open-source programs, professional licenses for Vosk can be purchased and provide additional features.

SpeechBrain [288] is a more recent library based on the widely used PyTorch [289] DL framework. It grants convenient implementations of novel deep neural networks for speech recognition. Its objective is to grant research and industrial actors an all-in-one speech toolkit. It has been used as a building block in various research works [290], [291].

Table 5.1 – Comparison of three ASR models provided by *SpeechBrain*.

Name	Model		Performance	
	Language model	Acoustic model	Samples/s $\uparrow$	WER (%) $\downarrow$
asr-crdnn-rnnlm	RNNLM	CRNN	2.84	1.82
asr-crdnn-transformerlm	Transformer Model (LM)	CRNN	0.24	1.16
asr-transformer-transformerlm	Transformer LM	Transformer	0.75	0.05

Our final implementation uses the *SpeechBrain* library, which has allowed us to choose from a substantial pool of state-of-the-art pre-trained models. The specific pipeline that was used in this work involves three components:

- The **tokenizer** that transforms each word into one or more tokens. This model was trained on the *LibriSpeech* dataset [81].
- The **neural language model** represents the dynamics of language. Here, we have chosen a Recurrent Neural Network Language Model (RNNLM) [292] provided by the *SpeechBrain* library. This architecture applies the successful RNN architecture to language modeling. In this case, the recurrent units act on word sequences. It comprises an embedding layer that maps individual words to 128-dimensional vectors. These vectors are then fed to the RNN, which is followed by fully connected layers. *SpeechBrain* also provides other architectures, such as a *TransformerLM*, based on the widespread Transformer architecture [1].
- Finally, the **acoustic model** performs the actual task of speech recognition by mapping audio features to tokens. It employs a CRNN architecture for the encoder, which maps audio features (e.g. STFT or Mel-Frequency Cepstral Coefficients (MFCC)) to tokens. *SpeechBrain* applies an additional Connectionist Temporal Classification (CTC) [293] loss to the encoder. The CTC cost function allows the training of recurrent architectures to perform speech recognition without requiring prior alignment between the input and target sequences. Alternatively, *SpeechBrain* ships a Transformer-based encoder-decoder that also uses the CTC training strategy.

To choose the best model for our use case, we empirically compared three models provided by the *SpeechBrain* toolkit. We evaluated them on the *LibriSpeech* training set, which contains 25,539 samples ranging from 3 to 16 seconds. Each model’s performance is measured through the WER. Table 5.1 summarizes the benchmarking results. All three models provide distinct tradeoffs between inference speed and performance. asr-crdnn-rnnlm has the fastest measured sample rate. The transformer architecture allows the other two models to reach sub-1% WER, but at the expense of being 2 to 7 times slower. asr-crdnn-transformerlm has the lowest throughput while providing worse performance than asr-transformer-transformerlm. As the absolute ASR performance is not particularly relevant for this project, we adopted the asr-crdnn-rnnlm model to benefit from its higher throughput.

5.3.2.2 Computation of WER Maps

ASR setup. We compute the WER score using the *jiwer* [294] library. To compute the minimum edit distance, it wraps the fast C++ matching library *RapidFuzz* [295]. The calculation of the metric has a negligible impact on runtime performance. However, running the *SpeechBrain* ASR model is highly computationally expensive. When the model is run in inference mode to evaluate its performance on 100 samples from the *LibriSpeech* dataset, approximately 96% of the total runtime is spent on the speech recognition process. On the contrary, less than 1% is used for computing the WER. We perform this test on an RTX A6000 NVIDIA GPU that gets fully utilized by this decoding task. The *RapidFuzz* library runs directly on the CPU. Notably, the decoding process remains slow: It takes approximately 2s to process a single 16s sentence at 16kHz.

Motivation. The setup mentioned above allows the computation of WER scores on full simulated audio recordings. However, the proposed RL environment involves short steps of 1s, during which the agent is assumed to be immobile and gathers audio data. Computing the WER on such a small snippet would not make sense. A complete sentence is the minimum necessary for the WER to have meaning. Additionally, as with every metric, this indicator is supposed to be averaged over a significant number of samples to purposefully assess the performance of the evaluated method. Here, the cost $C_{\text{WER}}(s)$ is expected to provide an estimate for the average ASR performance for a given state s . For those reasons, the oracle cannot work in real-time and needs to rely on prior information.

WER maps. To solve this issue, we introduce the WER map abstraction. The core idea of WER maps is to pre-compute an average WER score for each attainable state of the MDP and cache it for later use. More precisely, a microphone is positioned sequentially in each cell of the 2D grid spanning the room. If the microphone is not omnidirectional, the agent’s orientation will also impact the received signal and, eventually, the recognition performance. In this case, all four cardinal directions must be evaluated. The WER map materializes as a 2D matrix for an omnidirectional microphone and as a 3D tensor otherwise.

Listing 5.2 describes the algorithm used to compute those WER maps. It implements [Equation \(5.37\)](#). The computation time of the WER map algorithm can quickly grow to several hours. Its time complexity grows in the order of:

$$O\left(\left\lceil \frac{L_x}{\delta_{\text{grid}}} \right\rceil \times \left\lceil \frac{L_y}{\delta_{\text{grid}}} \right\rceil \times |\mathcal{D}|\right). \quad (5.38)$$

Hence, increasing the spatial resolution by lowering δ_{grid} increases the computing time quadratically. Moreover, if the *LibriSpeech* corpus (28,549 samples) were used as dataset $\mathcal{D}$, computing a single WER map would require over a year. Naturally, distributing this algorithm over several nodes would help reduce the overall walltime and would be an interesting extension to our current implementation. Finally, directional maps require four times more time to compute as the process has to be repeated for each possible agent orientation. To balance spatial resolution and statistical significance, we settle on using $|\mathcal{D}| = 40$ recordings and $\delta_{\text{grid}} = 50\text{cm}$. With the 4×7 meter room used in this study, this resolution translates to

```

1: function COMPUTE-WER-MAP( $\delta_{\text{grid}}$ ,  $(L_x, L_y)$ ,  $\mathcal{D}$ , directional)
2:    $\triangleright \delta_{\text{grid}}$ : spatial resolution
3:    $\triangleright (L_x, L_y)$ : dimensions of the room
4:    $\triangleright \mathcal{D}$ : dataset of  $n_{\text{samples}}$  clean speech samples and their associated transcripts
5:    $\triangleright$  directional (boolean): whether the microphone is directional or omnidirectional
6:
7:    $\triangleright$  Initialize the agent positions
8:    $\mathcal{X} \leftarrow \left\{ i\delta_{\text{grid}} \mid i \in \left[ \left\lfloor 0, \left\lfloor \frac{L_x}{\delta_{\text{grid}}} \right\rfloor \right\rfloor \right] \right\}$ 
9:
10:   $\mathcal{Y} \leftarrow \left\{ j\delta_{\text{grid}} \mid j \in \left[ \left\lfloor 0, \left\lfloor \frac{L_y}{\delta_{\text{grid}}} \right\rfloor \right\rfloor \right] \right\}$ 
11:
12:  if directional then
13:     $\Theta \leftarrow \left\{ 0, \frac{\pi}{2}, \pi, \frac{3\pi}{2} \right\}$ 
14:     $Z \leftarrow \mathcal{X} \times \mathcal{Y} \times \Theta$ 
15:  else
16:     $Z \leftarrow \mathcal{X} \times \mathcal{Y}$ 
17:
18:   $\triangleright$  Initialize the 2D/3D WER map tensor
19:   $\mathcal{W} \leftarrow \mathbf{0}_{\mathbb{R}^{|\mathcal{X}| \times |\mathcal{Y}| \times |\Theta|}}$ 
20:
21:   $\triangleright$  Loop through positions
22:  for  $z \in Z$  do
23:    MOVE-MICROPHONE( $z$ )
24:
25:     $\triangleright$  Loop through speech samples
26:    for  $(x_i, t) \in \mathcal{D}$  do
27:
28:       $\triangleright$  Call the simulator and get the received audio signal
29:       $x_r \leftarrow \text{SIMULATE-AUDIO}(x_i)$ 
30:
31:       $\triangleright$  Transcribe the recording
32:       $\hat{t} \leftarrow \text{ASR}_{\psi}(x_r)$ 
33:       $\mathcal{W}[z] \leftarrow \mathcal{W}[z] + \text{WER}(t, \hat{t})$ 
34:
35:   $\mathcal{W} \leftarrow \frac{1}{|\mathcal{D}|} \mathcal{W}$ 
36:  return  $\mathcal{W}$ 

```

Listing 5.2 – WER map computation algorithm.

maps of dimension $14 \times 8 (\times 4)$. Directional maps take approximately 17 hours to compute. A WER map needs to be computed for each source position considered.

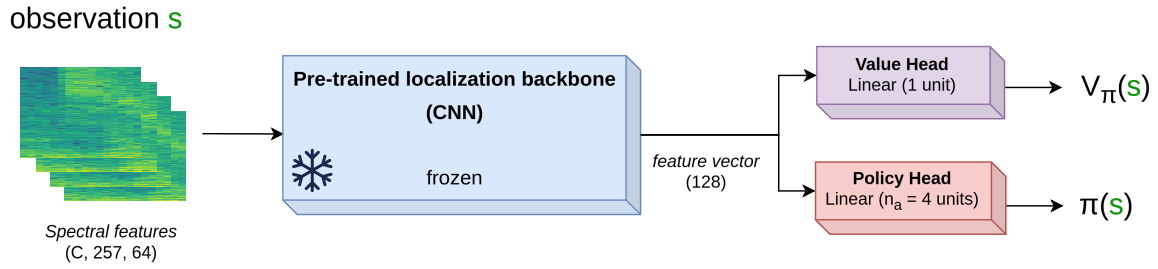


Figure 5.2 – Neural network architecture for the DRL agent. The feature extractor is the backbone of a pre-trained SSL model. The actor and critic-specific approximators are implemented as downstream MLP heads.

5.3.3 Deep Neural Agent

The multiple recent successes of DRL in solving various tasks originate consequently in the use of DNN as function approximators (Section 5.1.3). We propose a custom architecture for the neural network implementing the RL agent. Figure 5.2 gives a schematic view of its core components. The choice of PPO as a training algorithm requires defining two models: the actor and the critic (Section 5.1.5.3). We have designed a common backbone between those two systems, allowing them to share a significant amount of parameters. This feature extractor is followed by two heads implemented as MLP.

The main difficulty of the sound-driven navigation problem lies in the agent’s ability to map sound cues to spatial information. The partial observability aspect of the environment prevents the agent from directly and transparently observing either its own or the source’s position. Therefore, we hypothesize that the agent will implicitly acquire localization capabilities while learning the RL navigation task. We propose to leverage a pre-trained deep sound-source localizer to bootstrap this capability in the agent’s initial weights. Specifically, we use the model introduced in the third chapter, trained to localize a single speech source randomly located in a reverberant room. The architectures and the pre-trained weights are directly transferred to build the deep neural agent. Figure 3.4 provides a more detailed representation of the feature extractor’s architecture. Its final regression layer is removed. The obtained model outputs 128-dimensional feature vectors fed into the actor and critic heads. The feature extractor’s weights are kept frozen during the entire PPO training process. Section 5.4.4 discusses different strategies regarding backbone initialization.

5.3.4 PPO Implementation and Training Strategy

Environment. While the environment is defined initially as a POMDP, we apply the standard PPO algorithm, adopting a classical MDP formulation. The environment for the navigation task is parametrized with a horizon T of 32 steps. This value allows the agent to navigate the room in its entire span several times per episode. The value of the discount factor γ is fixed at 0.99. Furthermore, the incremental moving distance d_{forward} is set to 50cm to match the chosen grid spatial resolution δ_{grid} .

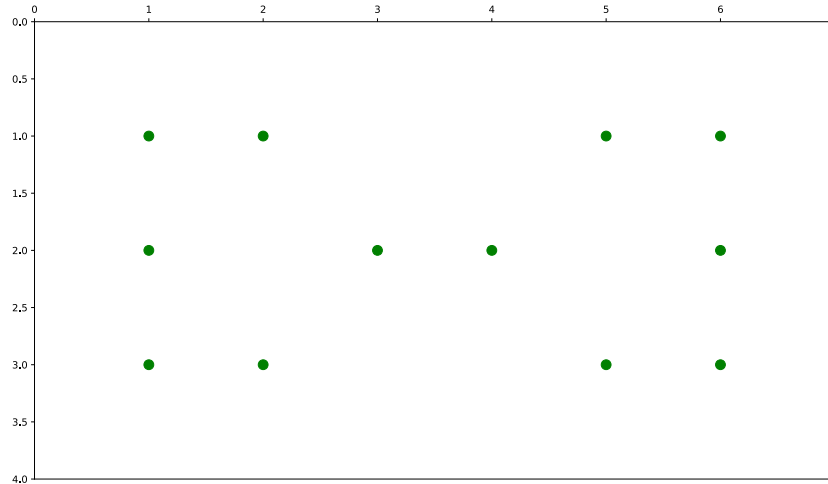


Figure 5.3 – Distribution of the 12 possible source positions.

Source positions. As previously mentioned, computing a WER map is computationally intensive. Also, a map needs to be computed for each possible source position. The final formulation for the environment involves $n_{\text{source pos}} = 12$ possible starting positions deterministically spread across the room. This choice is a tradeoff between the computational cost of WER maps caching and the diversity and difficulty of the environment. Figure 5.3 shows the chosen distribution of source positions across the room.

Implementation details and hyperparameters. Although PPO has been successful at solving many complex RL problems, it remains highly dependent on its hyperparameter values and implementation details. In [255], Mahmood et al. study the sensitivity of RL algorithms to their hyperparameters. Mahmood et al. mention that PPO, among other state-of-the-art algorithms, is highly sensitive to its hyperparameter values [255]. It thus requires careful fine-tuning on each new environment where it is tested. Our experience corroborates those observations. Extensive experimental campaigns were required to isolate a satisfying set of hyperparameter values. We underscore how essential – and nontrivial – hyperparameter tuning can be. Table 5.2 summarizes the final hyperparameter values used for training our agent with PPO.

In addition to careful hyperparameter tuning, a selection of implementation details was necessary to train the agent successfully. Engstrom et al. [296] explicitly studied the “code-level optimizations” of the TRPO and PPO algorithms. This work formalized the community’s shared impression that PPO’s promised performance was subject to subtle implementation details. Huang et al. have also contributed to this practical investigation by publishing *The 37 Implementation Details of Proximal Policy Optimization* [297]. We incorporate several of these optimizations in our custom PPO implementation. While the employed implementation details are not exhaustively listed here, the following are notable examples. Most notably, value loss clipping effectively helped stabilize the critic’s optimization.

Table 5.2 – Final hyperparameter values used to train the PPO algorithm.

Type	Name	Description	Value
Environment	T	Horizon: number of steps per episode	32
	$n_{\text{source pos}}$	Number of possible source positions	12
	γ	Reward discount factor	0.99
	δ_{grid}	Grid spatial resolution	0.5m
General	n_{iter}	Number of PPO iterations	100
	n_{steps}	Number of transitions per batch	2000
Loss	λ_{GAE}	Bias–variance trade-off parameter for the GAE	0.95
	ε	Clipping threshold for the clipped loss L_t^{CLIP}	0.2
	ε_V	Clipping threshold for the clipped value loss $L_t^{\text{VF, CLIPPED}}$	0.1
	c_V	Loss value coefficient	0.5
	c_S	Entropy coefficient	10^{-2}
Learning	mini-batch size	Number of samples (transitions) in each mini-batch	500
	n_{epochs}	Number of training epochs per iteration	64
	Optimizer	Used for optimizing the actor and critic networks	Adam
	η	Learning rate	10^{-3}

The unclipped value loss function, introduced in Equation (5.25) is replaced by:

$$L_t^{\text{VF, clipped}}(\theta) := \max \left[\underbrace{(V_\theta(s_t) - R_t)^2}_{\text{unclipped loss}}, (V^{\text{clipped}}(s_t) - R_t)^2 \right], \quad (5.39)$$

where $V^{\text{clipped}}(s_t)$ is the clipped value prediction:

$$V^{\text{clipped}}(s_t) = V_{\theta_{\text{old}}}(s_t) + \text{clip}[V_\theta(s_t) - V_{\theta_{\text{old}}}(s_t), -\varepsilon_V, \varepsilon_V]. \quad (5.40)$$

Here, $V_{\theta_{\text{old}}}$ is the value network from the previous iteration, and ε_V controls the clipping range of the value update. This new loss is inspired by the clipped loss L_t^{CLIP} used to optimize the actor network. It prevents large updates to the value function while still allowing it to improve when it's confident. Our experiments showed that using this clipped formulation of the value loss significantly helps stabilize training. Furthermore, the final training process also progressively decays the learning rate during training. This is achieved using a cosine annealing scheduler (see Section 3.2.2.6, for example).

Reward. The reward design was fundamental in achieving reliable training of the agent. The general formulation of the reward function was given by Equation (5.35). Its final parameter values have been tuned empirically to ensure proper training dynamics and the convergence to satisfying policies. This process is later discussed in the Section 5.4.2, which also gives the final expression of the reward function.

5.4 Experiments

Following the design and implementation of the complete RL pipeline presented in the previous section, we now conduct an experimental study of its capabilities and performance.

5.4.1 ASR Performance in a Reverberant Room

First and foremost, we investigate how ASR performance is impacted by reverberation. This study also explores the importance of the agent’s position relative to the source, especially in highly reverberant environments.

ASR performance and reverberation. Several WER cost maps were computed to visualize what the reward signal would be used to supervise the agent training. [Figure 5.4](#) provides high resolution ($\delta_{\text{grid}} = 10\text{cm}$) omnidirectional maps for different reverberation levels. It must be noted that the obtained WER values are entirely dependent on the ASR model ASR_{ψ} used to approximate C_{WER} ([Equation \(5.37\)](#)). It comes out that the microphone position does not affect ASR_{ψ} performance when reverberation is low. For $T_{60} = 100\text{ms}$ and $T_{60} = 300\text{ms}$, the ASR model achieves performance similar to what was measured in the clean-speech evaluation (see [Table 5.1](#)). However, from $T_{60} = 500\text{ms}$, reverberation starts to impact the WER more, decreasing the transcript’s fidelity globally. Furthermore, the microphone’s relative position from the source notably affects the mean WER. This phenomenon amplifies as T_{60} reaches 800ms. At such reverberation levels, the ASR model can no longer provide reliable transcripts unless the agent is very close to the source. Importantly, the model used in this study, `asr-crdnn-rnnlm-librispeech`, is trained exclusively on clean signal samples. Reverberant conditions are therefore outside of its training distribution. We do not report results for more advanced ASR models, particularly those trained on reverberant or noisy data, which may perform better in such conditions. Lastly, although informative, these sampled WER cost maps remain noisy and do not provide a smooth reward signal.

This study confirms the relevance of developing a navigation policy that improves the agent’s auditory perception. More specifically, we focus on the $T_{60} = 500\text{ms}$ case, which grants an interesting framework for this navigation task.

Cost Map Directionality The map is considered omnidirectional when the cost function is invariant in the agent orientation α_a . In the case of the WER cost C_{WER} , this is when an omnidirectional microphone is used to record signals at each position on the grid. When a directional microphone is used, separate recordings are made for all four cardinal orientations of the agent. Directional maps add more challenge to the navigation task as the agent must optimize both its position x_a and orientation α_a to achieve the highest performance. [Figure 5.5](#) displays an example of directional WER cost map. Each subfigure corresponds to a given agent orientation. For a given agent position, the WER cost can vary significantly depending on whether the agent is facing the source. The following experiments investigate the agent’s performance when trained with directional and omnidirectional costs.

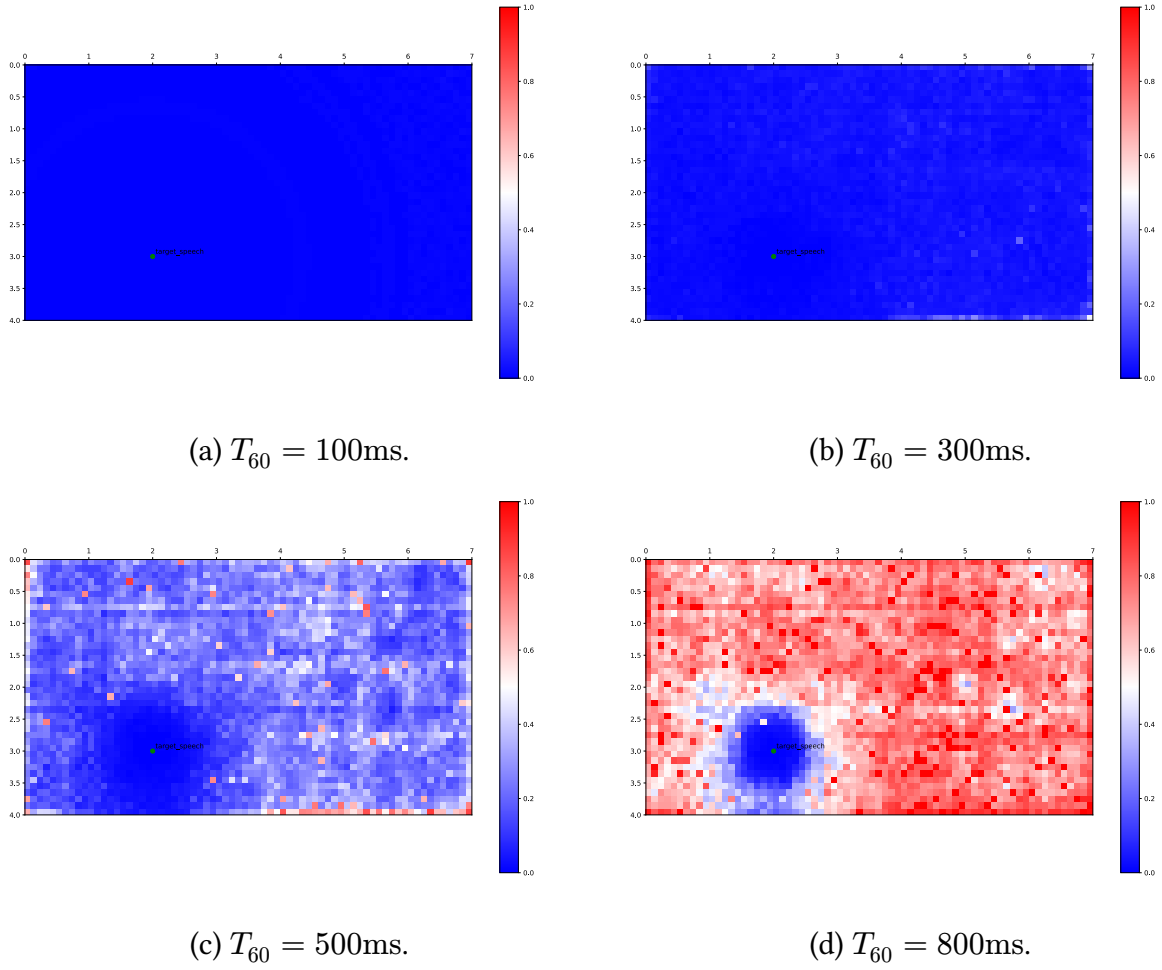


Figure 5.4 – WER cost maps for different reverberation levels. They were computed by averaging the WER obtained for a selection of recorded samples at each position (see Equation (5.37) and Listing 5.2).

5.4.2 Reward Design

Reward design is critical to training DRL models. Contrary to supervised learning, where clear ground truth labels are available for all training samples, the RL training process relies only on the reward signal. It is responsible for conveying information on the quality of the action sampled from the policy. More specifically, framing a novel RL task from scratch and developing an appropriate method for solving it necessitates meticulously adjusting the reward shape. The following explains the key design decisions regarding our reward.

Exponential scaling. The base cost value is a scalar value between 0 and 1 (see Section 5.2.3.1). For example, most experiments use the WER maps values C_{WER} . A custom function f is designed to derive a reward signal from the underlying cost. Most importantly, f must be a decreasing function of C_t . We choose an exponential base to ensure the reward smoothly decays as C_t decreases. More precisely, we introduce two scaling parameters η_C and ξ_C to control the final reward range:

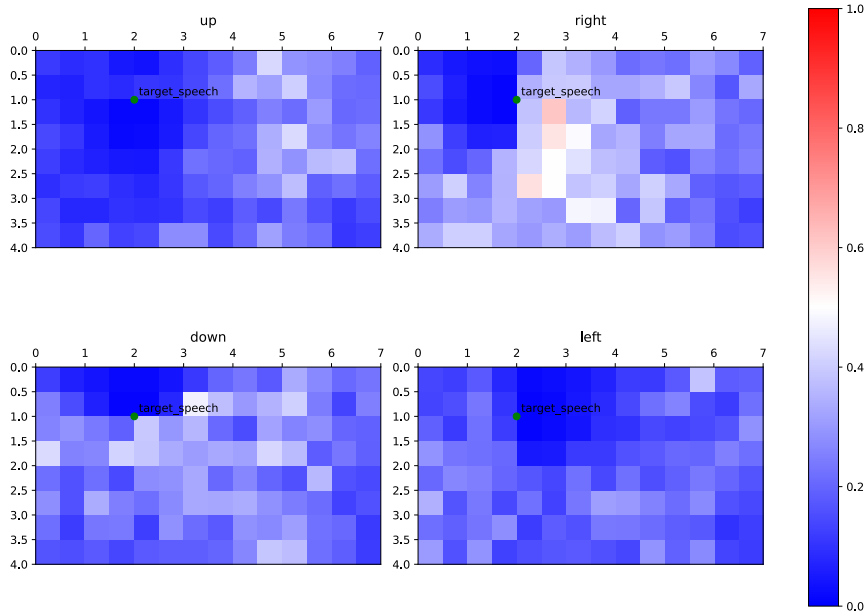


Figure 5.5 – Example of a directional WER cost map. The four sub-figures correspond to the four cardinal agent orientations. The average WER was computed from recordings at each position and cardinal orientation of a cardioid microphone. Hence, the obtained cost depends on both the agent’s position and orientation.

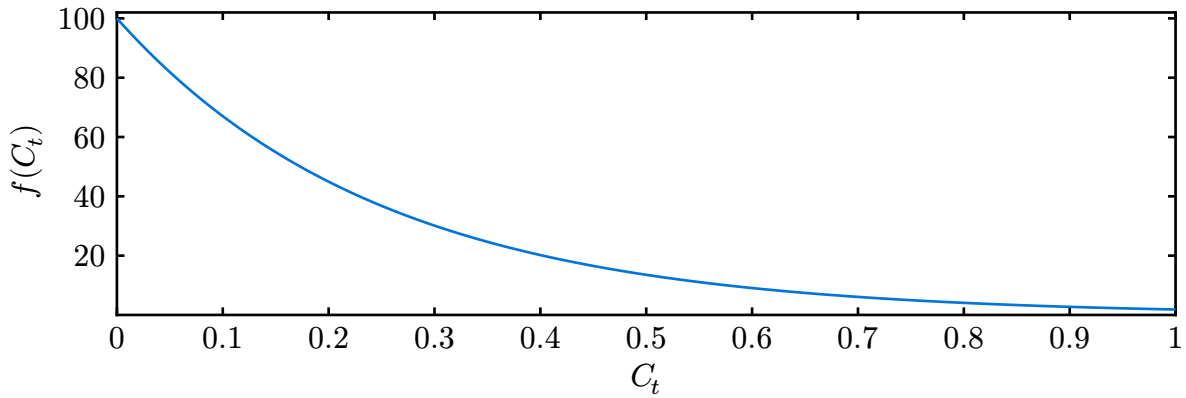


Figure 5.6 – Plot of the main reward component $\eta_C \exp[-\xi_C C_t]$ as a function of the cost C_t .

$$\begin{aligned} f : [0, 1] &\rightarrow \mathbb{R}_+^* \\ C_t &\mapsto \eta_C \exp[-\xi_C C_t]. \end{aligned} \quad (5.41)$$

Figure 5.6 plots the value of $f(C_t)$ in function of the cost value C_t . The final reward, introduced in Equation (5.35), becomes:

$$r_t = \begin{cases} -\mu_W & \text{if the agent tries to hit a wall} \\ \eta_C \exp[-\xi_C C_t] - \mu_m \mathbb{1}(a_t = \text{FORWARD}) & \text{otherwise,} \end{cases} \quad (5.42)$$

where

- η_C and ξ_C are the scaling factors for the exponential cost term;

- μ_m is the movement penalty;
- μ_W is the penalty for invalid movements. It corresponds to when the agent would hit a wall.

The values of these parameters were set after an empirical study of their impact. The following paragraphs detail these investigations.

Policy collapsing. Training a DRL agent is significantly more intricate than classically supervised neural networks. Training dynamics are complex, sensitive, and depend on multiple factors. For instance, the numerous hyperparameters in the PPO algorithm are pivotal. Some turned out to be crucial for achieving proper training. One example of the subtle instabilities encountered during our experimental study is the phenomenon of policy collapsing. Due to the significant expressivity of the feature vector extracted by the localizer backbone, it is easy to learn to move towards the source. Let us denote this specific policy π^* . Our initial experiments with the pre-trained backbone showed excellent performance, and the agent could consistently learn π^* . However, additional ablation studies demonstrated that setting the reward to a constant or random value did not change the learnt behavior, and the policy was still converging to π^* . We interpret that the loss for the value function L_t^{VF} (Equation (5.25)) and the entropy bonus $S[\pi_\theta](s_t)$ prevailed in the training dynamics. It was expected that instead of converging to π^* , the learnt policy π_θ would collapse to the static policy, denoted π_{still} . Indeed, in the absence of an informative reward signal and because of the penalties for hitting the room's walls (μ_W) and moving (μ_m), the agent would be expected to remain still.

Reward scaling. The reward design needed further elaboration to prevent this phenomenon and ensure the reward signal dictated the learning behavior. On the one hand, scaling up the reward permitted increasing its relative importance during training. It balances the reward weight in the overall loss values and its gradients. In practice, experimentally tuning the shaping coefficients led to choosing $\eta_C = 100$ and $\xi_C = 4$. On the other hand, scaling μ_m to 10 was also needed to balance its value properly concerning $f(C_t)$. Once these two parameters were adequately tuned, the experiments showed more exploitable results. Agents trained with the WER cost C_{WER} quickly learned the π^* policy, while agents trained with an uninformative reward learned π_{still} .

To illustrate the phenomenon of policy collapsing, we monitor the following two metrics:

- n_{forwards} counts the number of forward actions during an episode:

$$n_{\text{forwards}} := \sum_{t=1}^T \mathbb{1}(a_t = \text{FORWARD}). \quad (5.43)$$

- $D_F := \|\mathbf{x}_{a,T} - \mathbf{x}_s\|_2^2$ records the distance from the agent to the target source at the final step of each episode.

Figure 5.7 plots, along training, both metrics averaged on the $|\mathcal{T}|$ trajectories collected during PPO's sampling phase. These two metrics clearly translate the collapsing of each agent to a respective typical policy. The policy of the agent trained with the uninformative constant cost function converges to π_{still} , the static policy. On the contrary, the agent trained with the regular WER cost successfully learns to navigate to the source.

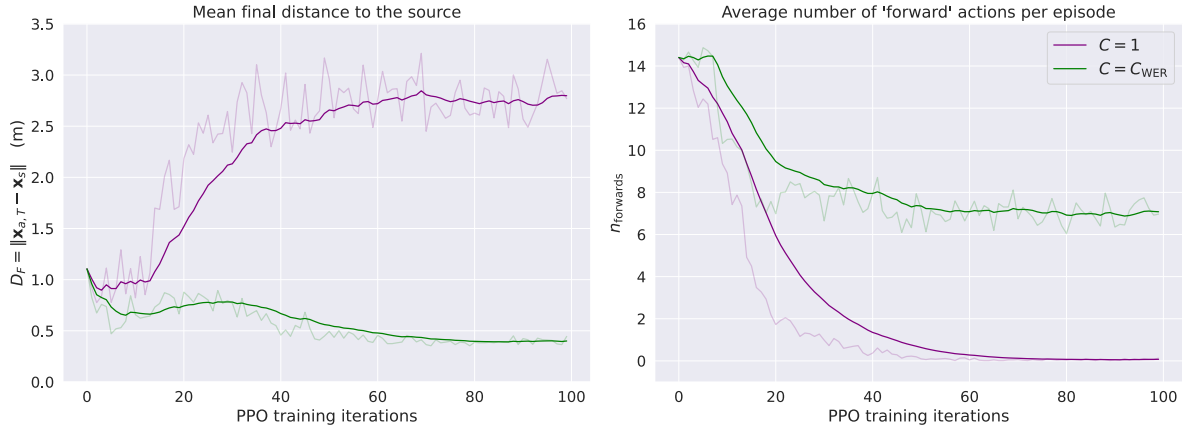


Figure 5.7 – Evolution of the training metrics demonstrating the phenomenon of policy collapsing. The behavior of two agents is monitored through the agent-to-source distance at the end of the episode D_F and the average number of FORWARD actions performed per episode. The first agent is trained with an uninformative constant (purple) reward while the second one is trained with the WER reward.

5.4.3 Agent Performance on the Navigation Task

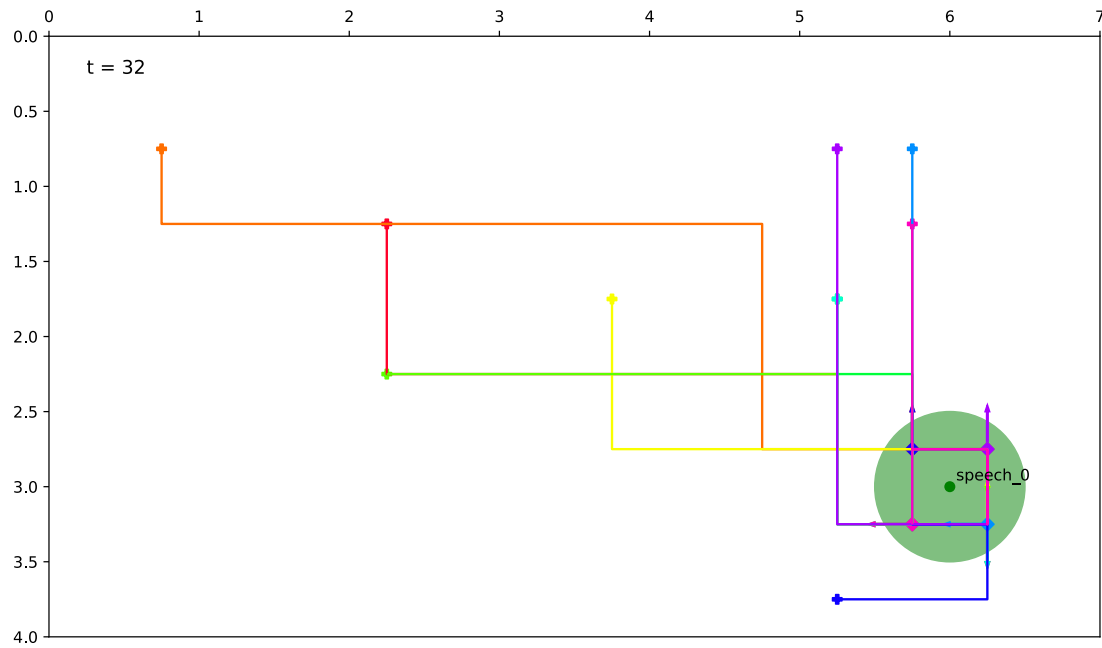
We report qualitative and quantitative results to evaluate the proposed method on the main navigation task. The goal is to assess how the model performs in the environment once trained with the PPO algorithm. The evaluation consists of running several episodes where the agent acts according to the learnt policy. We record various metrics to evaluate the navigation performance later. Furthermore, trajectories are also saved to assess the agent’s behavior qualitatively. Examples of trajectories are represented on [Figure 5.8](#). The agent can learn π^* , the policy of navigating to the source, thus minimizing the associated C_{WER} cost.

A quantitative study of the agent’s navigation performance is also conducted. After training, a series of n_{ep} episodes are executed to assess the performance of the RL pipeline. The agent greedily selects which action to take at each step according to [Equation \(5.4\)](#). This selection technique differs from the training phase, where the action is sampled according to the policy probability distribution π_θ .

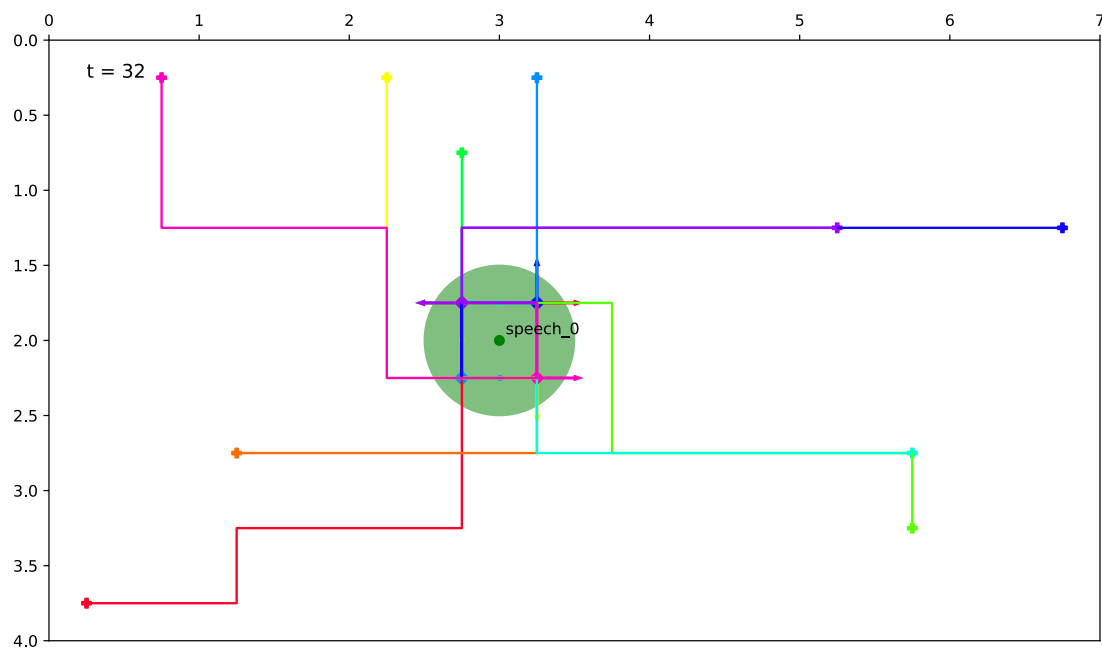
We define the primary performance metric for an episode as the cost value obtained at the final step. The obtained final costs are averaged over the n_{ep} episodes and give the *mean final cost (MFC)*:

$$\hat{C}_F = \frac{100}{n_{\text{ep}}} \sum_{i=1}^{n_{\text{ep}}} C(s_{i,T}), \quad (5.44)$$

where T is the environment horizon and s_T^i is the final state of the i -th test episode.



(a) Source position (6, 3).



(b) Source position (3, 2).

Figure 5.8 – Examples of agent trajectories for two distinct source positions. Each sub-figure represents 10 different trajectories corresponding to a random agent starting position. In every case, the agent successfully learns to navigate to the source, minimizing the WER.

When using the WER cost, it corresponds to the ASR performance after the agent is done moving. We convert the normalized cost to a percentage for more intuitive interpretation, especially in the case of C_{WER} . In addition to MFC, we report the undiscounted cumulated reward $\bar{R}$:

$$\bar{R} = \frac{1}{n_{\text{ep}}} \sum_{i=1}^{n_{\text{ep}}} \sum_{t=1}^T r_{i,t}, \quad (5.45)$$

where $r_{i,t}$ is the reward the agent received when transiting to state s_t during episode i . The performance of the trained policy (denoted π_θ) is compared to a selection of baseline deterministic policies:

- π_{random} , where the agent acts randomly, also disregarding the state value;
- $\pi_{\text{safe random}}$, which additionally avoids hitting the walls;
- π_{still} , where the agent remains static and always chooses the STAY action;
- $\pi_{\text{still orient}}$ where the agent never moves, but orients itself to face the source.

Furthermore, we test both our directional and omnidirectional formulations of the WER cost C_{WER} . π_θ is trained and evaluated on both environments separately. We use the reward function defined in Equation (5.42). Results are reported in Table 5.3. The number of test episodes is set to $n_{\text{ep}} = 1000$ to ensure statistical significance. Also, π_θ is trained $n_{\text{rep}} = 8$ times from scratch on each environment, and the evaluation metrics are averaged across these n_{rep} runs. π_{still} and $\pi_{\text{still orient}}$ have the same performance on the omnidirectional environment, where the agent’s orientation does not affect the cost value. In contrast, the MFC of $\pi_{\text{still orient}}$ is lower than that of π_{still} , with the directional cost as facing the source contributes to reducing the WER on average. The low cumulated reward achieved by π_{random} is caused by the agent repeatedly hitting the room walls and being penalized by the μ_W penalty. By contrast, $\pi_{\text{safe random}}$ achieves a similar mean final cost, but does not see its reward impacted by the wall hit penalty. The trained deep neural policy π_θ outperforms all baselines on the two environments. It significantly improves the MFC over other navigation strategies. Reducing the WER from around 20% to 5.69% in the omnidirectional case and 8.59% in the directional case would considerably help a real robot understand the human speaker.

5.4.4 Importance of Localization Feature Extraction

The agent neural network’s backbone is pre-trained on the supervised static SSL task. It outputs 128-dimensional feature vectors that are highly correlated with the source localization. To assess the impact of this choice, we conduct an ablation study where different initialization strategies are tested. In addition to our regular initialization strategy, we train an agent where the entire network is initialized from scratch, with no pre-training. All three agents are trained on the directional C_{WER} -based environment. We include an extra variation in which the backbone is pre-trained on the localization task, but whose weights are not frozen during the RL training phase. The directional WER cost is used to train and evaluate all agents. Table 5.4 reports the main evaluation metrics $\bar{R}$ and $\hat{C}_F$. The performance of $\pi_{\text{safe random}}$ is also reported for reference.

Table 5.3 – Benchmark of various policies’ performance on the navigation task. Both omnidirectional and directional WER cost environments have been tested. π_{random} samples actions randomly; $\pi_{\text{safe random}}$ does the same, but never hits the room’s walls; π_{still} has the robot remaining immobile; $\pi_{\text{still orient}}$ never has the agent moving but ensures it faces the source; and π_{θ} is our deep neural agent policy trained with PPO.

Policy	Omnidirectional cost		Directional cost	
	$\bar{R} \uparrow$	$\hat{C}_F (\%) \downarrow$	$\bar{R} \uparrow$	$\hat{C}_F (\%) \downarrow$
π_{random}	-25	21.16	-22	22.2
$\pi_{\text{safe random}}$	1420	20.99	1408	22.38
π_{still}	1481	21.13	1512	21.37
$\pi_{\text{still orient}}$	1495	20.87	1789	16.56
π_{θ}	2432	4.18	2302	8.01

Table 5.4 – Comparison of the agent’s performance for different feature extractor training strategies. In the first case, the model’s convolutional backbone is initialized randomly and trained from scratch by the PPO algorithm. The second network’s backbone is pre-trained, but its weights are not frozen. In the last case, the backbone is frozen during the RL training process.

Feature extractor training strategy	$\bar{R} \uparrow$	$\hat{C}_F (\%) \downarrow$
$\pi_{\text{safe random}}$	1408	22.38
No pretraining	1473	22.78
Pretraining + fine-tuning	1480	22.36
Pretraining + frozen	2302	8.01

First, training the backbone from scratch does not succeed. The end-to-end agent cannot learn directly from the raw acoustic observations. Pre-training the feature extractor on a supervised localization task appears to be crucial to solving the present navigation problem. Training the proposed architecture from scratch might be achievable, but would probably require more PPO iterations and a slower learning rate. Secondly, and more surprisingly, our attempt at fine-tuning the localizer’s backbone failed too. Even though the backbone’s weights were initialized from the pre-trained checkpoint, PPO could not learn a satisfying navigation policy. We hypothesize that the training hyperparameters that have been tuned for working with the frozen backbone cannot fine-tune the backbone stably. More precisely, the learning rate of 10^{-3} , coupled with PPO’s highly chaotic early training regime, is probably altering the pre-trained backbone weights before learning a working policy. A two-stage training process appears to be a plausible solution to this problem. At first, its goal would be to prevent perturbing the feature extractor while the actor and critic are stabilized. In a second time, it could be unfrozen and fine-tuned with a lower learning rate.

To conclude, this study confirms the relevance of pre-training the agent’s feature extractor. It permits rapid learning of a navigation policy by employing minimal MLP-style actor and critic networks that ingest the localization feature vectors.

5.4.5 Alternative Cost Maps

Before using WER cost maps to compute the reward, we conducted experiments with alternative cost maps. As highlighted in previous [Section 5.3.2.2](#), computing C_{WER} is very computationally intensive. Furthermore, the WER cost is noisy and has some artifacts. The present study introduces an alternative formulation for the cost function C . This *analytical cost* $\tilde{C}$ is a closed-form formula that directly maps a state $s \in \mathcal{S}$ to its normalized cost. It was motivated to provide a replacement for the WER cost maps, which are computationally heavy to create. We questioned whether this synthetic cost could permit the training of an effective policy that perform on the WER-based environment. The analytical cost is defined over the state space $\mathcal{S}$ as:

$$\begin{aligned} \tilde{C} : \quad \mathcal{S} &\longrightarrow \mathbb{R}_+ \\ (\mathbf{x}_a, \alpha_a) &\longmapsto D_{\text{source}}(\mathbf{x}_a, \mathbf{x}_s) + \eta \text{DoA}(\mathbf{x}_a, \alpha_a, \mathbf{x}_s) \end{aligned} \quad (5.46)$$

where:

- $\mathbf{x}_a$ and θ_a are respectively the position and orientation of the agent in the room’s frame;
- $\mathbf{x}_s$ is the source position and is fixed for a given episode;
- $D_{\text{source}}(\mathbf{x}_a, \mathbf{x}_s) = \|\mathbf{x}_s - \mathbf{x}_a\|_2^2$ is the source-array distance;
- $\text{DoA}(\mathbf{x}_a, \theta_a, \mathbf{x}_s)$ is the direction of arrival for this source-microphone positioning;
- η is a scaling factor, set to 1 in the conducted experiments.

This definition is naturally inspired by the shape of the WER maps (see [Figure 5.5](#) for an example). The obtained map is normalized to constrain its range in the $[0, 1]$ interval. Setting η to 0 gives the omnidirectional formulation of the analytical cost. [Figure 5.9](#) plots the analytical cost $\tilde{C}$ for the east orientation, i.e., where the agent is facing right in this figure’s frame. As $\tilde{C}$ has a closed-form definition, the resulting cost maps are considerably smoother and less noisy than WER maps.

To evaluate the impact of the cost function on the learned policy, we start by training the deep neural agent on both cost variants later to evaluate them on the same final target environment. Naturally, the target environment is based on the WER cost function. This study aims to see the benefits of using analytical maps for training the agent. [Table 5.5](#) gathers the quantitative results from this experimental campaign. For both omnidirectional and directional settings, we train a policy on the synthetic environment, using the $\tilde{C}$ cost, and another one on the C_{WER} -based environment. We then evaluate the performance of the agent trained with the analytical cost map in the target environment, using the C_{WER} cost function. Besides being considerably more efficient to compute, $\tilde{C}$ -based cost maps appear to help with the final WER performance in the omnidirectional case. Indeed, the policies trained to optimize the analytical cost yield a lower $\hat{C}_F$ cost than those directly trained with the target cost. Analytical maps provide a stronger reward signal and help achieve better policies thanks to their inherent

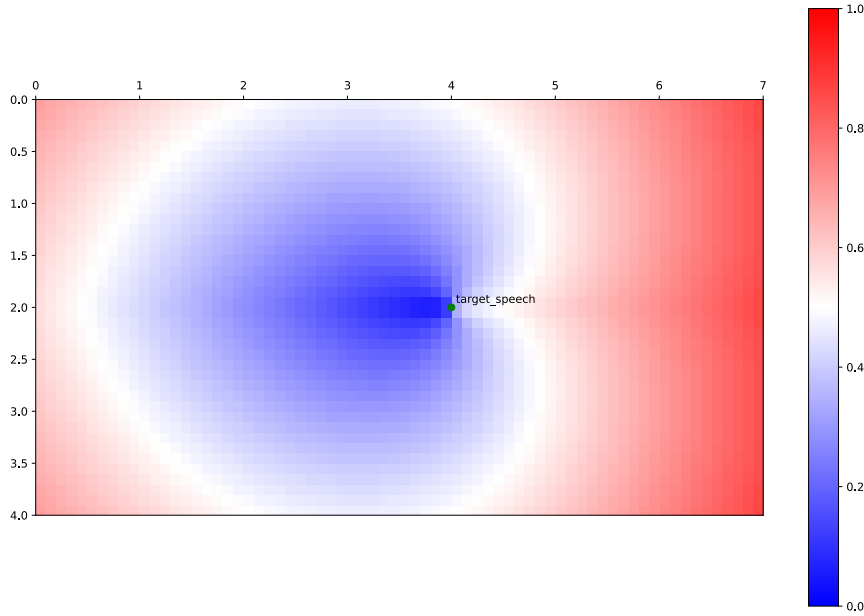


Figure 5.9 – Directional (east orientation) analytical cost map. Each pixel corresponds to the value of $\tilde{C}$ for a specific agent position. From this view, the agent’s orientation is kept constant as it faces the right direction. The cost is significantly lower when the agent is located close to the source and is facing it.

Table 5.5 – Navigation performance when using different cost functions. The agent is trained with the two variants of the cost C_{WER} and $\tilde{C}$. All policies are then evaluated on the same C_{WER} -based target environment.

Policy	Directionality	Training cost function	Performance synthetic environment		Performance C_{WER} environment	
			$\bar{R} \uparrow$	$\hat{C}_F (\%) \downarrow$	$\bar{R} \uparrow$	$\hat{C}_F (\%) \downarrow$
π_θ	omni.	C_{WER}	-	-	2432	4.18
		$\tilde{C}$	2033	6.47	2586	1.50
	dir.	C_{WER}	-	-	2302	8.01
		$\tilde{C}$	1234	22.53	1789	16.56

consistency and smoothness. However, training the agent on the directional analytical cost was not as successful. The performance in the training environment is inconsistent with the omnidirectional case. Although the underlying cost maps and thus optimal policies are different, the agent does not succeed in learning a robust policy. The analytical directional environment is more challenging as the zone yielding the lowest cost is smaller. Furthermore, if the agent is not facing the source, being at the proper position does not suffice. Besides performing poorly in the training environment, the obtained policy compares unfavorably to the one trained directly on the target environment. Therefore, transferring a policy trained using the directional analytical cost is not advantageous.

5.5 Conclusion

This chapter introduced a novel perceptually motivated navigation task for robotics, in which an agent equipped with a multi-microphone array must reposition itself to enhance its ability to understand human speech. The agent relies solely on auditory cues, without access to visual or spatial information, requiring it to build internal representations of space and develop source localization capabilities. To quantify perceptual quality, we employed the WER of an ASR system as the reward signal. Experimental evidence showed that small changes in position and orientation could significantly impact ASR performance in reverberant environments, motivating the task.

We proposed a rigorous MDP formulation for this navigation problem, fully integrated with the acoustic simulation tools presented earlier in this thesis. The WER-based cost maps were precomputed and cached to enable tractable training, despite the high computational cost of running ASR evaluations. A deep reinforcement learning agent, trained using the PPO algorithm, successfully learned to minimize the WER by navigating toward favorable positions. The neural agent incorporates a pre-trained source localization backbone, which encodes interaural acoustic features into a compact embedding, enabling the policy to leverage spatial cues. The learning process was supported by extensive hyperparameter tuning and careful reward shaping. Empirical results, both qualitative and quantitative, demonstrated that the learned policy consistently outperformed baseline strategies and led to significant reductions in WER.

While the proposed agent successfully solves the initial formulation of the task, several limitations remain. First, the current environment setup implicitly favors policies that navigate directly to the sound source. If an external source localizer were available, then this behavior could be replicated deterministically, reducing the need for reinforcement learning. However, this chapter provides the groundwork for tackling more complex and realistic scenarios where such shortcuts are no longer feasible. For instance, introducing occlusions or non-convex environments would challenge the agent to explore and reason over partial observations. The *Move2Hear* framework [10] exemplifies such complexity by coupling audio-visual perception with source separation and navigation. Within this broader context, the integration of pre-trained perception modules with DRL agents remains a promising and effective strategy.

Second, training with PPO was non-trivial. While PPO is often praised for its stability, we found it to be highly sensitive to implementation details and reward scaling. Careful tuning of hyperparameters, loss coefficients, and training dynamics was essential to obtain a working solution. Our experience echoes observations from prior work on the fragility of DRL algorithms when applied to new domains.

The complete DRL pipeline presented here — including the simulator interface, WER map generation, agent architecture, PPO training loop, and evaluation framework — was implemented from scratch. This engineering effort contributes a reproducible foundation for future work on audio-based navigation tasks. Several promising extensions lie ahead. Incorporating multiple sound sources, enabling dynamic environments with moving targets, or introducing higher-level behavioral goals could create more realistic and socially aware robotics scenarios.

In such contexts, deep reinforcement learning agents will need to combine multimodal perception, dynamic planning, and long-horizon reasoning — all of which are facilitated by the foundations established in this chapter.

Chapter 6

Conclusion

6.1 Summary of Contributions

This thesis explored how deep learning can support spatial auditory perception and sound-driven decision-making for robots operating in acoustically complex environments. Across simulation, perception, and control, we proposed a coherent pipeline in which audio-based models were trained, evaluated, and integrated into higher-level behaviors. Rather than treating sound as a supplementary cue, we placed it at the core of the robotic perception loop—asking how far a robot can go by relying on audio alone.

To support this investigation, we first developed a modular and extensible **acoustic simulation platform**, tailored for learning-based robotic experiments. Unlike existing tools focused on fixed environments or signal processing evaluation, our simulator was designed to generate dynamic, spatialized audio observations suitable for training and testing neural networks. It supported the construction of controlled experimental settings, enabling reproducible comparisons across tasks and architectures.

Using this foundation, we explored two formulations of the **Sound Source Localization (SSL) problem**. In the single-source case, we trained supervised deep models to estimate the DoA from short binaural audio segments. We studied the effects of microphone array configuration, spatial feature encoding, and training loss formulation. In the multi-source setting, we extended the problem to predict a continuous DoA spectrum capturing simultaneous speaker locations. We adapted and reimplemented existing architectures within our framework, and conducted extensive ablations on model performance under varying spatial and acoustic conditions. Together, these contributions provide a robust baseline for learning-based SSL in simulated environments.

We then shifted from passive to **active localization**, introducing a method for aggregating directional predictions over time as a robot moves. By accumulating egocentric DoA maps along a trajectory, we constructed spatial heatmaps that estimate source positions without relying on external localization data or privileged information. This approach demonstrated how movement itself can be leveraged as a perceptual strategy, allowing the robot to disambiguate uncertain cues and improve spatial awareness over time.

Finally, we proposed a **reinforcement learning approach for perceptually motivated navigation**, where a robot learns to move in order to improve downstream ASR performance.

We formalized the task as a partially observable Markov Decision Process (POMDP) and trained policies using the PPO algorithm. The agent relied only on audio input and used a pretrained SSL model as a feature extractor. Despite the minimal sensing setup, the learned policies reliably navigated toward acoustically favorable positions. This contribution shows that auditory perception can be treated not only as a sensory challenge but also as a control objective, where movement is learned to improve perception.

These contributions form a consistent and reproducible framework for investigating deep learning methods in auditory robotics. The tools, models, and experimental insights developed throughout this work lay the foundation for future research in sound-driven interaction, navigation, and decision-making.

6.2 Challenges and Limitations

While the contributions presented in this thesis provide a coherent and reproducible framework for studying audio-based perception and control, several limitations and challenges constrain the scope and generalizability of the results. These limitations arise from deliberate design choices, computational constraints, and broader research trade-offs that reflect the current state of deep learning for embodied AI.

Simulation-Only Evaluation

All experiments in this work were conducted within a custom acoustic simulation environment. Although this simulator was carefully engineered to model reverberation, spatialization, and microphone configurations with high realism, it inevitably abstracts away many factors present in real-world settings. Real acoustic environments involve a broader range of phenomena—such as non-uniform materials, background noise, microphone imperfections, and non-stationary reverberation—that are difficult to model or anticipate.

This reliance on simulation limits the empirical validation of the proposed models and policies. In particular, the active localization and reinforcement learning experiments assume perfect control over the agent’s movement and audio capture, and do not account for hardware-specific delays, sensor calibration errors, or ambient variability. Additionally, the presence of extra adversarial noise sources was not investigated in this thesis. While the simulation-based approach was essential for controlled experimentation and large-scale training, future work will need to address the sim-to-real transfer gap to confirm the applicability of these methods in physical robotic systems.

Task and Agent Constraints

Several simplifications were made in the formulation of the localization and navigation tasks. First, the SSL models assume that all sound sources are static during inference, with no modeling of speaker motion or interruption. This assumption facilitates tractable evaluation but limits applicability to dynamic environments. In the active localization pipeline, the agent moves in a discrete 2D grid with uniform steps and perfect self-location, without incorporating odometry drift, inertial feedback, or motion constraints typical of real robots.

The agent is modeled as a simple microphone array, without explicit embodiment or interaction with the environment beyond locomotion. Notably, Head-related transfer functions (HRTFs), body occlusion effects, and mechanical actuation noise are not accounted for in this work while being highly relevant to embodied auditory sensing. While this abstraction enabled a focused study on acoustic perception, it also means that the system does not yet reflect the full complexity of real-world sensing platforms.

The navigation task, in particular, is limited to a single, known room layout with a fixed number of possible source locations. Although this constraint was imposed to manage the computational cost of generating and storing WER maps, it narrows the diversity of encountered scenarios and may reduce the robustness of the learned policies.

Engineering and Algorithmic Challenges

From an engineering perspective, training deep reinforcement learning agents remained non-trivial. The PPO algorithm, while widely adopted, is known to be sensitive to hyperparameters and implementation details. Extensive empirical tuning was required to obtain stable training and meaningful policies. This sensitivity introduces reproducibility challenges and increases the barrier to broader adoption.

The design of the reward function, based on ASR performance using precomputed WER maps, provided a task-relevant optimization signal but also imposed practical limitations. Generating accurate WER maps for multiple source positions is computationally expensive and scales poorly with the number of concurrent speakers or room configurations. As a result, the learning environment was deliberately simplified, and the agent was trained with a frozen feature extractor rather than in an end-to-end fashion.

6.3 Prospective Research Directions

This thesis focused on the use of audio as a primary modality for spatial perception and navigation in robotics. By investigating learning-based localization and control in simulated environments, we explored how far a robot can go by “hearing” alone. Although this work covered simulation, learning, and control, several directions remain open for extending and applying this work. The following prospective directions reflect both short-term extensions and longer-term research trajectories.

Toward More Realistic and Transferable Simulation

One natural direction is to improve the realism and transferability of the simulated environments. While the current simulator supports detailed spatialization and reverberation, it lacks features such as dynamic scenes, continuous-time audio rendering, and realistic sensor noise. Incorporating human motion models, dynamic speech sources, and variable background conditions would allow the training of models that generalize more effectively. The addition of domain randomization techniques or hybrid simulation (combining measured and synthetic impulse responses) may also help bridge the sim-to-real gap. Ultimately, deploying trained

models on real robotic platforms in uncontrolled acoustic environments will be a key step toward validating their practical utility.

Embodied and Multimodal Audio Perception

Another promising direction lies in integrating embodiment and multimodal sensing. The current agent is modeled as a floating microphone array, abstracted away from physical structure. Introducing embodiment could enable richer and more grounded perceptual policies. This could be done through HRTF modeling, simulated actuation noise, or even proprioceptive feedback. Similarly, combining audio with visual input or depth sensing would allow for sensor fusion models that operate more robustly in ambiguous or noisy conditions. A particularly interesting challenge is to investigate how attention can be coordinated across modalities. The agent could be left to decide when to rely on audio, when to move, and when to switch sensing strategies.

Active Perception Beyond Localization

While this thesis focused primarily on localization and speech-driven navigation, the broader concept of active perception still offers many opportunities for exploration. Tasks such as speaker-following, audio-based exploration, or information-seeking behavior could be formulated within the same reinforcement learning framework. More complex objectives, such as reducing semantic uncertainty or improving conversational turn-taking, may also benefit from movement and auditory focus. They would bridge the existing gaps to higher-level HRI, which remain present in this work. Studying how agents can autonomously decide what to listen to and where to go to improve their understanding remains a key question for socially intelligent robots.

Model Efficiency and Generalization

Finally, future work could focus on improving the sample efficiency and generalization ability of learned policies. This includes investigating lightweight architectures for SSL, training with fewer labeled samples, and exploring transfer learning across environments or source types. While this thesis showed that pretrained SSL models can be reused effectively, more systematic studies are needed to understand how spatial knowledge generalizes across rooms, microphone arrays, and acoustic conditions. Meta-learning, continual learning, and hierarchical reinforcement learning could all contribute to more adaptable agents.

This work has shown that meaningful spatial understanding and behavior can emerge from sound alone. While many challenges remain, the results here provide a step toward more perceptive, autonomous, and communicative robots.

Bibliography

- [1] A. Vaswani *et al.*, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [2] A. Défossez *et al.*, “Moshi: a speech-text foundation model for real-time dialogue,” *arXiv preprint arXiv:2410.00037*, 2024.
- [3] H. Moravec, *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press, 1988.
- [4] V. Mnih *et al.*, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [5] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [6] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.
- [7] C. Berner *et al.*, “Dota 2 with Large Scale Deep Reinforcement Learning,” 2019.
- [8] N. Akalin and A. Loutfi, “Reinforcement Learning Approaches in Social Robotics,” *Sensors*, vol. 21, no. 4, p. 1292, 2021.
- [9] J. Raiman, S. Zhang, and F. Wolski, “Long-Term Planning and Situational Awareness in OpenAI Five,” *arXiv preprint arXiv:1912.06721*, 2019.
- [10] S. Majumder, Z. Al-Halah, and K. Grauman, “Move2Hear: Active Audio-Visual Source Separation,” *arXiv:2105.07142 [cs, eess]*, May 2021.
- [11] R. Scheibler, E. Bezzam, and I. Dokmanić, “Pyroomacoustics: A Python Package for Audio Room Simulation and Array Processing Algorithms,” *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 351–355, Apr. 2018.
- [12] D. Diaz-Guerra, A. Miguel, and J. R. Beltran, “gpuRIR: A python library for room impulse response simulation with GPU acceleration,” *Multimedia Tools and Applications*, vol. 80, no. 4, pp. 5653–5671, Feb. 2021.
- [13] W. He, P. Motlicek, and J.-M. Odobez, “Neural Network Adaptation and Data Augmentation for Multi-Speaker Direction-of-Arrival Estimation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 1303–1317, 2021.
- [14] D. Goldsman, R. E. Nance, and J. R. Wilson, “A brief history of simulation,” in *Proceedings of the 2009 Winter Simulation Conference (WSC)*, Austin, TX, USA: IEEE, Dec. 2009, pp. 310–313.

- [15] N. Metropolis, “The Beginning of the Monte Carlo Method,” in *Special Edition of Los Alamos Science*, 1987.
- [16] J. G. Charney, R. Fjörtoft, and J. Von NEUMANN, “Numerical Integration of the Barotropic Vorticity Equation,” *Tellus*, vol. 2, no. 4, pp. 237–254, 1950.
- [17] J. D. Durrant and J. A. McCammon, “Molecular dynamics simulations and drug discovery,” *BMC biology*, vol. 9, pp. 1–9, 2011.
- [18] T. Lavé, N. Parrott, H. Grimm, A. Fleury, and M. Reddy, “Challenges and opportunities with modelling and simulation in drug discovery and drug development,” *Xenobiotica*, vol. 37, no. 10–11, pp. 1295–1310, 2007.
- [19] A. E. Sallab, M. Abdou, E. Perot, and S. Yogamani, “Deep Reinforcement Learning framework for Autonomous Driving,” *arXiv preprint arXiv:1704.02532*, 2017.
- [20] B. Osiński *et al.*, “Simulation-Based Reinforcement Learning for Real-World Autonomous Driving,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, May 2020, pp. 6411–6418.
- [21] B. R. Kiran *et al.*, “Deep Reinforcement Learning for Autonomous Driving: A Survey,” *IEEE transactions on intelligent transportation systems*, vol. 23, no. 6, pp. 4909–4926, 2021.
- [22] F. Rosique, P. J. Navarro, C. Fernández, and A. Padilla, “A Systematic Review of Perception System and Simulators for Autonomous Vehicles Research,” *Sensors*, vol. 19, no. 3, p. 648, Jan. 2019.
- [23] J. Degraeve *et al.*, “Magnetic control of tokamak plasmas through deep reinforcement learning,” *Nature*, vol. 602, no. 7897, pp. 414–419, Feb. 2022.
- [24] J. Bradbury *et al.*, “JAX: composable transformations of Python+NumPy programs.” [Online]. Available: <http://github.com/google/jax>
- [25] J. Citrin *et al.*, “Torax: A fast and differentiable tokamak transport simulator in jax,” *arXiv preprint arXiv:2406.06718*, 2024.
- [26] O. Vinyals *et al.*, “Grandmaster level in StarCraft II using multi-agent reinforcement learning,” *nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [27] C. K. Liu and D. Negrut, “The Role of Physics-Based Simulators in Robotics,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. Volume4, 2021, pp. 35–58, May 2021.
- [28] J. Collins, S. Chand, A. Vanderkop, and D. Howard, “A Review of Physics Simulators for Robotic Applications,” *IEEE Access*, vol. 9, pp. 51416–51431, 2021.
- [29] H. Choi *et al.*, “On the use of simulation in robotics: Opportunities, challenges, and suggestions for moving forward,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 1, p. e1907856118, Jan. 2021.
- [30] H. R. Walke *et al.*, “BridgeData V2: A Dataset for Robot Learning at Scale,” in *Conference on Robot Learning*, 2023, pp. 1723–1736.

- [31] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, “D4RL: Datasets for Deep Data-Driven Reinforcement Learning,” *arXiv preprint arXiv:2004.07219*, 2020.
- [32] R. Rafailov *et al.*, “D5RL: Diverse Datasets for Data-Driven Deep Reinforcement Learning,” *arXiv preprint arXiv:2408.08441*, 2024.
- [33] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Vilamoura-Algarve, Portugal: IEEE, Oct. 2012, pp. 5026–5033.
- [34] A. Dosovitskiy, “CARLA: An Open Urban Driving Simulator.”
- [35] K. Greff *et al.*, “Kubric: A scalable dataset generator,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 3749–3761.
- [36] “Bullet Physics SDK, real-time collision detection and multi-physics simulation for VR, games, visual effects, robotics, machine learning, etc..” [Online]. Available: <https://github.com/bulletphysics/bullet3>
- [37] N. Robinson, B. Tidd, D. Campbell, D. Kulić, and P. Corke, “Robotic Vision for Human-Robot Interaction and Collaboration: A Survey and Systematic Review,” *ACM Transactions on Human-Robot Interaction*, vol. 12, no. 1, pp. 1–66, Mar. 2023.
- [38] A. Zacharaki, I. Kostavelis, A. Gasteratos, and I. Dokas, “Safety bounds in human robot interaction: A survey,” *Safety Science*, vol. 127, p. 104667, Jul. 2020.
- [39] S. W. Abeyruwan *et al.*, “i-Sim2Real: Reinforcement Learning of Robotic Policies in Tight Human-Robot Interaction Loops,” in *Proceedings of The 6th Conference on Robot Learning*, PMLR, Mar. 2023, pp. 212–224.
- [40] P. Kaur, Z. Liu, and W. Shi, “Simulators for mobile social robots: State-of-the-art and challenges,” in *2022 fifth international conference on connected and autonomous driving (MetroCAD)*, 2022, pp. 47–56.
- [41] Z. Sprague, R. Chandra, J. Holtz, and J. Biswas, “Socialgym 2.0: Simulator for multi-agent social robot navigation in shared human spaces,” *arXiv preprint arXiv:2303.05584*, 2023.
- [42] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey,” in *2020 IEEE symposium series on computational intelligence (SSCI)*, 2020, pp. 737–744.
- [43] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, May 2018, pp. 3803–3810.
- [44] J. Tan *et al.*, “Sim-to-Real: Learning Agile Locomotion For Quadruped Robots,” *arXiv preprint arXiv:1804.10332*, 2018.

- [45] F. B. P. Malavazi, R. Guyonneau, J.-B. Fasquel, S. Lagrange, and F. Mercier, "LiDAR-only based navigation algorithm for an autonomous agricultural robot," *Computers and Electronics in Agriculture*, vol. 154, pp. 71–79, Nov. 2018.
- [46] D. Hutabarat, M. Rivai, D. Purwanto, and H. Hutomo, "Lidar-based Obstacle Avoidance for the Autonomous Mobile Robot," in *2019 12th International Conference on Information & Communication Technology and System (ICTS)*, Jul. 2019, pp. 197–202.
- [47] L. Seminara, P. Gastaldo, S. J. Watt, K. F. Valyear, F. Zuher, and F. Mastrogiovanni, "Active Haptic Perception in Robots: A Review," *Frontiers in Neurorobotics*, vol. 13, Jul. 2019.
- [48] C. Chen *et al.*, "SoundSpaces: Audio-Visual Navigation in 3D Environments," *arXiv:1912.11474 [cs, eess]*, Aug. 2020.
- [49] G. Bustamante and P. Danes, "Multi-step-ahead information-based feedback control for active binaural localization," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vancouver, BC: IEEE, Sep. 2017, pp. 6729–6734.
- [50] C. Gan *et al.*, "ThreeDWorld: A Platform for Interactive Multi-Modal Physical Simulation," *arXiv preprint arXiv:2007.04954*, 2020.
- [51] P. Srivastava, A. Deleforge, A. Politis, and E. Vincent, "How to (virtually) train your speaker localizer," *arXiv preprint arXiv:2211.16958*, 2022.
- [52] C. Cao, Z. Ren, C. Schissler, D. Manocha, and K. Zhou, "Interactive sound propagation with bidirectional path tracing," *ACM Transactions on Graphics*, vol. 35, no. 6, pp. 1–11, Nov. 2016.
- [53] E. Vincent, T. Virtanen, and S. Gannot, *Audio source separation and speech enhancement*. Wiley, 2018.
- [54] S. Leglaive, R. Badeau, and G. Richard, "Multichannel Audio Source Separation With Probabilistic Reverberation Priors," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 12, pp. 2453–2465, Dec. 2016.
- [55] D. Di Carlo, "Echo-aware signal processing for audio scene analysis: The Call of Echo," 2020.
- [56] L. Savioja and U. P. Svensson, "Overview of geometrical room acoustic modeling techniques," *The Journal of the Acoustical Society of America*, vol. 138, no. 2, pp. 708–730, Aug. 2015.
- [57] W. C. Sabine, *Collected papers on acoustics*. Cambridge : Harvard University Press, 1922.
- [58] P. Srivastava, "Realism in virtually supervised learning for acoustic room characterization and sound source localization," 2023.
- [59] A. V. Oppenheim and R. W. Schaffer, *Discrete-time Signal Processing*. Prentice Hall, 1989.
- [60] Ronald Bracewell, *The Fourier Transform And Its Applications Bracewell*. 2000.

- [61] S. W. Smith, *The scientist and engineer's guide to digital signal processing*. USA: California Technical Publishing, 1997.
- [62] S. Gannot, D. Burshtein, and E. Weinstein, "Signal enhancement using beamforming and nonstationarity with applications to speech," *IEEE Transactions on Signal Processing*, vol. 49, no. 8, pp. 1614–1626, Aug. 2001.
- [63] X. Li, L. Girin, F. Badeig, and R. Horaud, "Reverberant sound localization with a robot head based on direct-path relative transfer function," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2016, pp. 2819–2826.
- [64] I. Cohen, "Relative transfer function identification using speech signals," *IEEE Transactions on Speech and Audio Processing*, vol. 12, no. 5, pp. 451–459, Sep. 2004.
- [65] S. Gannot, E. Vincent, S. Markovich-Golan, and A. Ozerov, "A Consolidated Perspective on Multimicrophone Speech Enhancement and Source Separation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 4, pp. 692–730, Apr. 2017.
- [66] N. D. Gaubitch and P. A. Naylor, "Analysis of the Dereverberation Performance of Microphone Arrays," 2005.
- [67] T. Nakatani, T. Yoshioka, K. Kinoshita, M. Miyoshi, and B.-H. Juang, "Blind speech dereverberation with multi-channel linear prediction based on short time fourier transform representation," in *2008 IEEE International Conference on Acoustics, Speech and Signal Processing*, Mar. 2008, pp. 85–88.
- [68] X. Alameda-Pineda and R. Horaud, "A Geometric Approach to Sound Source Localization from Time-Delay Estimates," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 6, pp. 1082–1095, Jun. 2014.
- [69] P.-A. Grumiaux, S. Kitić, L. Girin, and A. Guérin, "A survey of sound source localization with deep learning methods," *The Journal of the Acoustical Society of America*, vol. 152, no. 1, pp. 107–151, 2022.
- [70] L. Perotin, "Localisation et rehaussement de sources de parole au format Ambisonique," p. 149, 2019.
- [71] L. Parra and C. Spence, "Convolutional blind separation of non-stationary sources," *IEEE Transactions on Speech and Audio Processing*, vol. 8, no. 3, pp. 320–327, May 2000.
- [72] K. Imoto and N. Ono, "Spatial Cepstrum as a Spatial Feature Using a Distributed Microphone Array for Acoustic Scene Analysis," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, no. 6, pp. 1335–1343, Jun. 2017.
- [73] M. Zaunschirm, C. Schörkhuber, and R. Höldrich, "Binaural rendering of Ambisonic signals by head-related impulse response time alignment and a diffuseness constraint," *The Journal of the Acoustical Society of America*, vol. 143, no. 6, pp. 3616–3627, Jun. 2018.
- [74] T. Gustafsson, B. Rao, and M. Trivedi, "Source localization in reverberant environments: modeling and statistical analysis," *IEEE Transactions on Speech and Audio Processing*, vol. 11, no. 6, pp. 791–803, Nov. 2003.

- [75] L. Girin and S. Leglaive, “Fundamentals of Audio Processing (Lecture notes),” Université Grenoble Alpes, Grenoble Institute of Technology.
- [76] S. Markovich-Golan and S. Gannot, “Performance analysis of the covariance subtraction method for relative transfer function estimation and comparison to the covariance whitening method,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2015, pp. 544–548.
- [77] X. Li, L. Girin, R. Horaud, and S. Gannot, “Estimation of relative transfer function in the presence of stationary noise based on segmental power spectral density matrix subtraction,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2015, pp. 320–324.
- [78] M. I. Mandel and D. P. W. Ellis, “A Probability Model for Interaural Phase Difference,” 2006.
- [79] C. Knapp and G. Carter, “The generalized correlation method for estimation of time delay,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 4, pp. 320–327, Aug. 1976.
- [80] B. Uragun and R. Rajan, “The discrimination of interaural level difference sensitivity functions: development of a taxonomic data template for modelling,” *BMC Neuroscience*, vol. 14, no. 1, p. 114, Oct. 2013.
- [81] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2015, pp. 5206–5210.
- [82] M. Schroeder, “Natural Sounding Artificial Reverberation,” *Journal of The Audio Engineering Society*, Jul. 1962.
- [83] M. Kleiner, B.-I. Dalenbäck, and P. Svensson, “Auralization-An Overview,” *Journal of The Audio Engineering Society*, Nov. 1993.
- [84] T. Takala and J. Hahn, “Sound rendering,” *SIGGRAPH Comput. Graph.*, vol. 26, no. 2, pp. 211–220, Jul. 1992.
- [85] L. Savioja and N. Xiang, “Introduction to the Special Issue on Room Acoustic Modeling and Auralization,” *The Journal of the Acoustical Society of America*, vol. 145, no. 4, pp. 2597–2600, Apr. 2019.
- [86] R. P. Feynman, R. B. Leighton, and M. Sands, *The Feynman lectures on physics; New millennium ed.* New York, NY: Basic Books, 2010. [Online]. Available: <https://cds.cern.ch/record/1494701>
- [87] D. Botteldooren, “Acoustical finite-difference time-domain simulation in a quasi-Cartesian grid,” *The Journal of the Acoustical Society of America*, vol. 95, no. 5, pp. 2313–2319, May 1994.
- [88] S. Kirkup, *The Boundary Element Method in Acoustics*, vol. 8. 2007.

- [89] L. L. Thompson, "A review of finite-element methods for time-harmonic acoustics," *The Journal of the Acoustical Society of America*, vol. 119, no. 3, pp. 1315–1330, Mar. 2006.
- [90] N. Raghuvanshi, R. Narain, and M. C. Lin, "Efficient and Accurate Sound Propagation Using Adaptive Rectangular Decomposition," *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 5, pp. 789–801, 2009.
- [91] M. Rosen, K. W. Godin, and N. Raghuvanshi, "Interactive sound propagation for dynamic scenes using 2D wave simulation," p. 8, 2020.
- [92] M. Rosen, "themattrosen/Planeverb." Dec. 17, 2024.
- [93] J. Rindel, "Computer Simulation Techniques for Acoustical Design of Rooms," *Acoustics Australia / Australian Acoustical Society*, vol. 23, pp. 81–86, Jan. 1995.
- [94] S. Siltanen, T. Lokki, S. Kiminki, and L. Savioja, "The room acoustic rendering equation," *The Journal of the Acoustical Society of America*, vol. 122, no. 3, p. 1624, Sep. 2007.
- [95] L. Cremer, *Die wissenschaftlichen Grundlagen der Raumakustik: Geometrische Raumakustik*. S. Hirzel, 1948.
- [96] B. M. Gibbs and D. K. Jones, "A Simple Image Method for Calculating the Distribution of Sound Pressure Levels within an Enclosure," *Acta Acustica united with Acustica*, vol. 26, no. 1, pp. 24–32, Jan. 1972.
- [97] J. Borish, "Extension of the image model to arbitrary polyhedra," *The Journal of the Acoustical Society of America*, vol. 75, no. 6, pp. 1827–1836, Jun. 1984.
- [98] J. B. Allen and D. A. Berkley, "Image Method for Efficiently Simulating Small-room Acoustics," p. 8, 1979.
- [99] S. Schimmel, M. Muller, and N. Dillier, *A fast and accurate "shoebox" room acoustics simulator*. 2009.
- [100] "Wayverb - Ray tracer." Accessed: Sep. 03, 2024. [Online]. Available: https://reuk.github.io/wayverb/ray_tracer.html
- [101] A. Krokstad, U. P. Svensson, and S. Strøm, "The Early History of Ray Tracing in Acoustics," *Acoustics, Information, and Communication: Memorial Volume in Honor of Manfred R. Schroeder*. Springer International Publishing, Cham, pp. 15–31, 2015.
- [102] C. Schissler and D. Manocha, "Interactive Sound Propagation and Rendering for Large Multi-Source Scenes," *ACM Transactions on Graphics*, vol. 36, no. 4, p. 114, Jul. 2017.
- [103] Z. Tang, H.-Y. Meng, and D. Manocha, "Learning Acoustic Scattering Fields for Dynamic Interactive Sound Propagation," *arXiv:2010.04865 [cs, eess]*, Dec. 2020.
- [104] Z. Zhang, K. Nakadai, H. Nakajima, and N. Sumida, "Acoustic Simulation in Dynamic Environments for Robot Audition," in *2019 27th European Signal Processing Conference (EUSIPCO)*, Sep. 2019, pp. 1–5.

- [105] Y. Cao, T. Iqbal, Q. Kong, Y. Zhong, W. Wang, and M. D. Plumbley, "Event-Independent Network for Polyphonic Sound Event Localization and Detection," *arXiv preprint arXiv:2010.00140*, 2020.
- [106] A. Rathnayake and W. K. Wanniarachchi, "Image Source Method Based Acoustic Simulation For 3-D Room Environment," *International Journal of Scientific & Technology Research*, vol. 8, pp. 222–228, Nov. 2019.
- [107] L. Savioja, V. Välimäki, and J. Smith, "Audio Signal Processing Using Graphics Processing Units," *Journal of The Audio Engineering Society - J AUDIO ENG SOC*, vol. 59, Jan. 2011.
- [108] Z.-h. Fu and J.-w. Li, "GPU-based image method for room impulse response calculation," *Multimedia Tools and Applications*, vol. 75, no. 9, pp. 5205–5221, May 2016.
- [109] B. Frederickson, "benfred/py-spy." Accessed: May 04, 2025. [Online]. Available: <https://github.com/benfred/py-spy>
- [110] N. Volynets, "nvdv/vprof." Accessed: May 04, 2025. [Online]. Available: <https://github.com/nvdv/vprof>
- [111] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [112] R. Varzandeh, K. Adiloğlu, S. Doclo, and V. Hohmann, "Exploiting Periodicity Features for Joint Detection and DOA Estimation of Speech Sources Using Convolutional Neural Networks," in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2020, pp. 566–570.
- [113] J. Sodnik, S. Tomazic, R. Grasset, A. Duenser, and M. Billingham, "Spatial sound localization in an augmented reality environment," in *Proceedings of the 20th conference of the computer-human interaction special interest group (CHISIG) of Australia on Computer-human interaction: design: activities, artefacts and environments - OZCHI '06*, Sydney, Australia: ACM Press, 2006, p. 111.
- [114] F. Keyrouz and K. Diepold, "Binaural Source Localization and Spatial Audio Reproduction for Telepresence Applications," *Presence: Teleoperators and Virtual Environments*, vol. 16, no. 5, pp. 509–522, Oct. 2007.
- [115] J. Stachurski, L. Netsch, and R. Cole, "Sound source localization for video surveillance camera," in *2013 10th IEEE International Conference on Advanced Video and Signal Based Surveillance*, Aug. 2013, pp. 93–98.
- [116] H. Sun, X. Liu, K. Xu, J. Miao, and Q. Luo, "Emergency Vehicles Audio Detection and Localization in Autonomous Driving," *arXiv preprint arXiv:2109.14797*, 2021.
- [117] T. Jenrungrot, V. Jayaram, S. Seitz, and I. Kemelmacher-Shlizerman, "The Cone of Silence: Speech Separation by Localization," *Advances in Neural Information Processing Systems*, vol. 33, pp. 20925–20938, 2020.

- [118] J. Woodruff and D. Wang, “Binaural Localization of Multiple Sources in Reverberant and Noisy Environments,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 20, no. 5, pp. 1503–1512, Jul. 2012.
- [119] S. Braun and I. Tashev, “Acoustic Localization Using Spatial Probability in Noisy and Reverberant Environments,” in *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, New Paltz, NY, USA: IEEE, Oct. 2019, pp. 353–357.
- [120] X. Li, R. Horaud, L. Girin, and S. Gannot, “Voice activity detection based on statistical likelihood ratio with adaptive thresholding,” in *2016 IEEE International Workshop on Acoustic Signal Enhancement (IWAENC)*, Sep. 2016, pp. 1–5.
- [121] D. Salvati, C. Drioli, and G. L. Foresti, “Localization and Tracking of an Acoustic Source using a Diagonal Unloading Beamforming and a Kalman Filter,” *arXiv preprint arXiv:1812.01521*, 2018.
- [122] Q. Nguyen, L. Girin, G. Bailly, F. Elisei, and N. Duc Canh, *Autonomous Sensorimotor Learning for Sound Source Localization by a Humanoid Robot*. 2018.
- [123] A. Deleforge, R. Horaud, Y. Y. Schechner, and L. Girin, “Co-Localization of Audio Sources in Images Using Binaural Features and Locally-Linear Regression,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 23, no. 4, pp. 718–731, Apr. 2015.
- [124] S. Kapka and M. Lewandowski, “Sound source detection, localization and classification using consecutive ensemble of CRNN models,” in *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019)*, 2019.
- [125] H. Phan, L. Pham, P. Koch, N. Q. K. Duong, I. McLoughlin, and A. Mertins, “Audio Event Detection and Localization with Multitask Regression Network,” 2020.
- [126] H. Phan, L. Pham, P. Koch, N. Q. Duong, I. McLoughlin, and A. Mertins, “On Multitask Loss Function for Audio Event Detection and Localization,” *arXiv preprint arXiv:2009.05527*, 2020.
- [127] S. Adavanne, A. Politis, J. Nikunen, and T. Virtanen, “Sound Event Localization and Detection of Overlapping Sources Using Convolutional Recurrent Neural Networks,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 1, pp. 34–48, Mar. 2019.
- [128] F. L. Wightman and D. J. Kistler, “Resolution of front-back ambiguity in spatial hearing by listener and source movement,” *The Journal of the Acoustical Society of America*, vol. 105, no. 5, pp. 2841–2853, May 1999.
- [129] B. Van Veen and K. Buckley, “Beamforming: a versatile approach to spatial filtering,” *IEEE ASSP Magazine*, vol. 5, no. 2, pp. 4–24, Apr. 1988.
- [130] M. Brandstein, D. Ward, A. Lacroix, and A. Venetsanopoulos, Eds., “Microphone Arrays: Signal Processing Techniques and Applications.” in *Digital Signal Processing*. Springer, Berlin, Heidelberg, 2001.

- [131] R. Schmidt, "Multiple emitter location and signal parameter estimation," *IEEE Transactions on Antennas and Propagation*, vol. 34, no. 3, pp. 276–280, Mar. 1986.
- [132] R. Roy and T. Kailath, "ESPRIT-estimation of signal parameters via rotational invariance techniques," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 7, pp. 984–995, Jul. 1989.
- [133] J. T. Flåm, J. Jaldén, and S. Chatterjee, "Gaussian mixture modeling for source localization," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2011, pp. 2604–2607.
- [134] A. Bross, B. Laufer-Goldshtein, and S. Gannot, "Multiple Speaker Localization using Mixture of Gaussian Model with Manifold-based Centroids," in *2020 28th European Signal Processing Conference (EUSIPCO)*, Amsterdam, Netherlands: IEEE, Jan. 2021, pp. 895–899.
- [135] A. Deleforge, F. Forbes, and R. Horaud, "Acoustic Space Learning for Sound-Source Separation and Localization on Binaural Manifolds," *International Journal of Neural Systems*, vol. 25, no. 1, p. 1440003, Feb. 2015.
- [136] M. I. Mandel, R. J. Weiss, and D. P. W. Ellis, "Model-Based Expectation-Maximization Source Separation and Localization," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 18, no. 2, pp. 382–394, Feb. 2010.
- [137] M. I. Mandel, "Binaural Model-Based Source Separation and Localization," 2010.
- [138] W. He, P. Motlicek, and J.-M. Odobez, "Joint Localization and Classification of Multiple Sound Sources Using a Multi-task Neural Network," in *Interspeech 2018*, ISCA, Sep. 2018, pp. 312–316.
- [139] L. Perotin, R. Serizel, E. Vincent, and A. Guérin, "CRNN-based Joint Azimuth and Elevation Localization with the Ambisonics Intensity Vector," in *2018 16th International Workshop on Acoustic Signal Enhancement (IWAENC)*, Sep. 2018, pp. 241–245.
- [140] T. Hirvonen, *Classification of Spatial Audio Location and Content Using Convolutional Neural Networks*, vol. 2. 2015.
- [141] S. Chakrabarty and E. A. P. Habets, "Broadband DOA estimation using Convolutional neural networks trained with noise signals," in *2017 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, Oct. 2017, pp. 136–140.
- [142] S. Schneider, A. Baeviski, R. Collobert, and M. Auli, "wav2vec: Unsupervised Pre-training for Speech Recognition," *arXiv preprint arXiv:1904.05862*, 2019.
- [143] A. Baeviski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," *Advances in neural information processing systems*, vol. 33, pp. 12449–12460, 2020.
- [144] J. M. Vera-Diaz, D. Pizarro, and J. Macias-Guarasa, "Towards End-to-End Acoustic Localization using Deep Learning: from Audio Signal to Source Position Coordinates," *Sensors*, vol. 18, no. 10, p. 3418, Oct. 2018.

- [145] D. Suvorov, G. Dong, and R. Zhukov, “Deep Residual Network for Sound Source Localization in the Time Domain,” *arXiv preprint arXiv:1808.06429*, 2018.
- [146] P. Vecchiotti, N. Ma, S. Squartini, and G. J. Brown, “End-to-end Binaural Sound Localisation from the Raw Waveform,” in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 451–455.
- [147] Y. He, N. Trigoni, and A. Markham, “SoundDet: Polyphonic Moving Sound Event Detection and Localization from Raw Waveform,” in *International Conference on Machine Learning*, 2021, pp. 4160–4170.
- [148] H. Viste and G. Evangelista, “Binaural source localization,” 2004.
- [149] R. Roden, N. Moritz, S. Gerlach, S. Weinzierl, and S. Goetze, “On sound source localization of speech signals using deep neural networks,” 2015.
- [150] D. Diaz-Guerra, A. Miguel, and J. R. Beltran, “Robust Sound Source Tracking Using SRP-PHAT and 3D Convolutional Neural Networks,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 300–311, 2021.
- [151] D. Diaz-Guerra, A. Miguel, and J. R. Beltran, “Direction of Arrival Estimation of Sound Sources Using Icosahedral CNNs,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 313–321, 2022.
- [152] G. Boztas, “Sound source localization for auditory perception of a humanoid robot using deep neural networks,” *Neural Computing and Applications*, vol. 35, no. 9, pp. 6801–6811, Mar. 2023.
- [153] S. Adavanne, A. Politis, and T. Virtanen, “Localization, Detection and Tracking of Multiple Moving Sound Sources with a Convolutional Recurrent Neural Network,” *arXiv preprint arXiv:1904.12769*, 2019.
- [154] L. Perotin, A. Défossez, E. Vincent, R. Serizel, and A. Guérin, “Regression Versus Classification for Neural Network Based Audio Source Localization,” in *2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, Oct. 2019, pp. 343–347.
- [155] L. Perotin, R. Serizel, E. Vincent, and A. Guérin, “CRNN-Based Multiple DoA Estimation Using Acoustic Intensity Features for Ambisonics Recordings,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 1, pp. 22–33, Mar. 2019.
- [156] V. Varanasi, H. Gupta, and R. M. Hegde, “A Deep Learning Framework for Robust DOA Estimation Using Spherical Harmonic Decomposition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 1248–1259, 2020.
- [157] S. Chakrabarty and E. A. P. Habets, “Multi-Speaker DOA Estimation Using Deep Convolutional Networks Trained With Noise Signals,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 13, no. 1, pp. 8–21, Mar. 2019.
- [158] X. Xiao, S. Zhao, X. Zhong, D. L. Jones, E. S. Chng, and H. Li, “A learning-based approach to direction of arrival estimation in noisy and reverberant environments,” in *2015 IEEE*

- International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2015, pp. 2814–2818.
- [159] P.-A. Grumiaux, S. Kitić, L. Girin, and A. Guérin, “Improved feature extraction for CRNN-based multiple sound source localization,” in *2021 29th European Signal Processing Conference (EUSIPCO)*, 2021, pp. 231–235.
 - [160] T. N. T. Nguyen, W.-S. Gan, R. Ranjan, and D. L. Jones, “Robust Source Counting and DOA Estimation Using Spatial Pseudo-Spectrum and Convolutional Neural Network,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2626–2637, 2020.
 - [161] G. Le Moing *et al.*, “Learning Multiple Sound Source 2D Localization,” in *2019 IEEE 21st International Workshop on Multimedia Signal Processing (MMSP)*, 2019, pp. 1–6.
 - [162] L. Cristoforetti *et al.*, “The DIRHA simulated corpus,” in *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC’14)*, N. Calzolari, K. Choukri, T. Declerck, H. Loftsson, B. Maegaard, J. Mariani, A. Moreno, J. Odijk, and S. Piperidis, Eds., Reykjavik, Iceland: European Language Resources Association (ELRA), May 2014, pp. 2629–2634.
 - [163] J. Thiemann and S. van de Par, “A multiple model high-resolution head-related impulse response database for aided and unaided ears,” *EURASIP Journal on Advances in Signal Processing*, vol. 2019, no. 1, p. 9, Feb. 2019.
 - [164] A. Mesaros, R. Serizel, T. Heittola, T. Virtanen, and M. D. Plumbley, “A decade of DCASE: Achievements, practices, evaluations and future challenges,” in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.
 - [165] H. Ling, “Direction of arrival estimation of humans with a small sensor array using an artificial neural network,” *Progress In Electromagnetics Research B*, vol. 27, Jan. 2011.
 - [166] K. Youssef, S. Argentieri, and J.-L. Zarader, “A learning-based approach to robust binaural sound localization,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Nov. 2013, pp. 2927–2932.
 - [167] R. Takeda and K. Komatani, “Sound source localization based on deep neural networks with directional activate function exploiting phase information,” in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Mar. 2016, pp. 405–409.
 - [168] T.-H. Tan, Y.-T. Lin, Y.-L. Chang, and M. Alkhaleefah, “Sound Source Localization Using a Convolutional Neural Network and Regression Model,” *Sensors (Basel, Switzerland)*, vol. 21, no. 23, p. 8031, Dec. 2021.
 - [169] D. Comminiello, M. Lella, S. Scardapane, and A. Uncini, “Quaternion Convolutional Neural Networks for Detection and Localization of 3D Sound Events,” in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2019, pp. 8533–8537.

- [170] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.”
- [171] P.-A. Grumiaux, S. Kitić, P. Srivastava, L. Girin, and A. Guérin, “SALADnet: Self-attentive multisource localization in the Ambisonics domain,” in *2021 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2021, pp. 336–340.
- [172] M. J. Bianco, S. Gannot, and P. Gerstoft, “Semi-supervised source localization with deep generative modeling,” in *2020 IEEE 30th International Workshop on Machine Learning for Signal Processing (MLSP)*, Sep. 2020, pp. 1–6.
- [173] S. Mo and P. Morgado, “A Closer Look at Weakly-Supervised Audio-Visual Source Localization,” 2022.
- [174] G. Narang, K. Nakamura, and K. Nakadai, “Auditory-aware navigation for mobile robots based on reflection-robust sound source localization and visual SLAM,” in *2014 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, Oct. 2014, pp. 4021–4026.
- [175] V. M. Trifa, A. Koene, J. Moren, and G. Cheng, “Real-time acoustic source localization in noisy environments for human-robot multimodal interaction,” in *RO-MAN 2007 - The 16th IEEE International Symposium on Robot and Human Interactive Communication*, Aug. 2007, pp. 393–398.
- [176] D. Gala, N. Lindsay, and L. Sun, “Realtime Active Sound Source Localization for Unmanned Ground Robots Using a Self-Rotational Bi-Microphone Array,” *Journal of Intelligent & Robotic Systems*, vol. 95, no. 3, pp. 935–954, Sep. 2019.
- [177] M. Sewtz, T. Bodenmüller, and R. Triebel, “Sound Source Localization for Robotic Application,” in *ICRA Workshop*, Paris, Frankreich, Jun. 2020.
- [178] S. Argentieri, P. Danès, and P. Souères, “A survey on sound source localization in robotics: From binaural to array processing methods,” *Computer Speech & Language*, vol. 34, no. 1, pp. 87–112, Nov. 2015.
- [179] F. Asano, M. Goto, K. Itou, and H. Asoh, “Real-time sound source localization and separation system and its application to automatic speech recognition,” in *7th European Conference on Speech Communication and Technology (Eurospeech 2001)*, ISCA, Sep. 2001, pp. 1013–1016.
- [180] K. Nakadai, K. Hidai, H. Okuno, and H. Kitano, “Real-time speaker localization and speech separation by audio-visual integration,” in *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No.02CH37292)*, May 2002, pp. 1043–1049.
- [181] K. Nakadai, D. Matsuura, H. Okuno, and H. Kitano, “Applying scattering theory to robot audition system: robust sound source localization and extraction,” in *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*, Oct. 2003, pp. 1147–1152.

- [182] V. Algazi, R. Duda, D. Thompson, and C. Avendano, "The CIPIC HRTF database," in *Proceedings of the 2001 IEEE Workshop on the Applications of Signal Processing to Audio and Acoustics (Cat. No.01TH8575)*, Oct. 2001, pp. 99–102.
- [183] H. Wierstorf, M. Geier, A. Raake, and S. Spors, "A Free Database of Head Related Impulse Response Measurements in the Horizontal Plane with Multiple Distances," Jan. 2011.
- [184] M. Otani and S. Ise, "Fast calculation system specialized for head-related transfer function based on boundary element method," *The Journal of the Acoustical Society of America*, vol. 119, no. 5, pp. 2589–2598, May 2006.
- [185] C. T. Ishi, J. Even, and N. Hagita, "Using multiple microphone arrays and reflections for 3D localization of sound sources," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Nov. 2013, pp. 3937–3942.
- [186] J. Dávila-Chacón, J. Liu, and S. Wermter, "Enhanced Robot Speech Recognition Using Biomimetic Binaural Sound Source Localization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 1, pp. 138–150, Jan. 2019.
- [187] W. He, P. Motlicek, and J.-M. Odobez, "Deep Neural Networks for Multiple Speaker Detection and Localization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, May 2018, pp. 74–79.
- [188] K. Shimada *et al.*, "STARSS23: An Audio-Visual Dataset of Spatial Recordings of Real Scenes with Spatiotemporal Annotations of Sound Events," 2023.
- [189] T. Stoddart, "A Beginner's Buying Guide to Microphones." Accessed: Jul. 22, 2024. [Online]. Available: <http://www.homestudioarchive.com/blog/2016/12/15/a-beginners-buying-guide-to-microphones>
- [190] D. Krause, A. Politis, and K. Kowalczyk, "Comparison of Convolution Types in CNN-based Feature Extraction for Sound Source Localization," in *2020 28th European Signal Processing Conference (EUSIPCO)*, Amsterdam, Netherlands: IEEE, Jan. 2021, pp. 820–824.
- [191] S. Sivasankaran, E. Vincent, and D. Foehr, "Keyword Based Speaker Localization: Localizing a Target Speaker in a Multi-speaker Environment," in *Interspeech 2018*, ISCA, Sep. 2018, pp. 2703–2707.
- [192] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," *arXiv preprint arXiv:1608.03983*, 2016.
- [193] D. P. Kingma, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [194] H. Hollien, D. Dew, and P. Philips, "Phonational Frequency Ranges of Adults," *Journal of Speech and Hearing Research*, vol. 14, no. 4, pp. 755–760, Dec. 1971.

- [195] D. Stoller, "Spectrogram input normalisation for neural networks." Accessed: Aug. 24, 2024. [Online]. Available: <https://dans.world/Spectrogram-input-normalisation-for-neural-networks/>
- [196] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima," *arXiv preprint arXiv:1609.04836*, 2016.
- [197] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *International conference on machine learning*, 2015, pp. 448–456.
- [198] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [199] Y. Wu and K. He, "Group Normalization," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 3–19.
- [200] M. Ren, R. Liao, R. Urtasun, F. H. Sinz, and R. S. Zemel, "Normalizing the Normalizers: Comparing and Extending Network Normalization Schemes," *arXiv preprint arXiv:1611.04520*, 2016.
- [201] S. Shen, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "PowerNorm: Rethinking Batch Normalization in Transformers," 2020.
- [202] A. Skaf and P. Danes, "Optimal positioning of a binaural sensor on a humanoid head for sound source localization," in *2011 11th IEEE-RAS International Conference on Humanoid Robots*, Bled, Slovenia: IEEE, Oct. 2011, pp. 165–170.
- [203] K. Nakadai, T. Lourens, H. G. Okuno, and H. Kitano, "Active audition for humanoid," in *AAAI/IAAI*, 2000, pp. 832–839.
- [204] v. Q. Nguyen, F. Colas, E. Vincent, and F. Charpillet, "Localizing an intermittent and moving sound source using a mobile robot," in *International Conference on Intelligent Robots and Systems (IROS)*, Deajeon, South Korea, Oct. 2016.
- [205] Q. Nguyen Van, F. Colas, E. Vincent, and F. Charpillet, "Long-term robot motion planning for active sound source localization with Monte Carlo tree search," in *HSCMA 2017 - Hands-free Speech Communication and Microphone Arrays*, San Francisco, United States, Mar. 2017.
- [206] v. Q. Nguyen, F. Colas, E. Vincent, and F. Charpillet, "Motion planning for robot audition," *Autonomous Robots*, vol. 43, no. 8, p. 2293, 2019.
- [207] G. Bustamante, A. Portello, and P. Danès, "A Three-Stage Framework to Active Source Localization from a Binaural Head," presented at the 40th IEEE International Conference on Acoustics, Speech and Signal Processing, Apr. 2015, p. 5.
- [208] G. Bustamante, P. Danés, T. Forgue, and A. Podlubne, "Towards information-based feedback control for binaural active localization," in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Mar. 2016, pp. 6325–6329.

- [209] G. Bustamante, P. Danès, T. Forgue, A. Podlubne, and J. Manhès, “An information based feedback control for audio-motor binaural localization,” *Autonomous Robots*, vol. 42, no. 2, pp. 477–490, Jun. 2017.
- [210] N. Roman and D. Wang, “Binaural Tracking of Multiple Moving Sources,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 16, no. 4, pp. 728–739, May 2008.
- [211] L. Kneip and C. Baumann, “Binaural model for artificial spatial sound localization based on interaural time delays and movements of the interaural axis,” *The Journal of the Acoustical Society of America*, vol. 124, no. 5, pp. 3108–3119, Nov. 2008.
- [212] I. Markovic, A. Portello, P. Danes, I. Petrovic, and S. Argentieri, “Active speaker localization with circular likelihoods and bootstrap filtering,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Nov. 2013, pp. 2914–2920.
- [213] A. Portello, P. Danès, and S. Argentieri, “Active binaural localization of intermittent moving sources in the presence of false measurements,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Oct. 2012, pp. 3294–3299.
- [214] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., Cham: Springer International Publishing, 2015, pp. 234–241.
- [215] M. D. Zeiler, D. Krishnan, G. W. Taylor, and R. Fergus, “Deconvolutional networks,” in *2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, San Francisco, CA, USA: IEEE, Jun. 2010, pp. 2528–2535.
- [216] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, in KDD’96. Portland, Oregon: AAAI Press, Aug. 1996, pp. 226–231.
- [217] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object Detection in 20 Years: A Survey,” *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, Mar. 2023.
- [218] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [219] E. L. Thorndike, *Animal intelligence: Experimental studies*. in *Animal intelligence: Experimental studies*. Lewiston, NY, US: Macmillan Press, 1911.
- [220] P. I. Pavlov (1927), “Conditioned reflexes: An investigation of the physiological activity of the cerebral cortex,” *Annals of Neurosciences*, vol. 17, no. 3, pp. 136–141, Jul. 2010.
- [221] R. S. Sutton and A. G. Barto, *Reinforcement learning: an introduction*, Second edition. in *Adaptive computation and machine learning series*. Cambridge, Massachusetts: The MIT Press, 2018.
- [222] R. Bellman, *Dynamic programming*. Princeton, NJ: Princeton Univ. Pr, 1957.

- [223] R. Bellman, "A Markovian Decision Process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.
- [224] A. L. Samuel, "Some Studies in Machine Learning Using the Game of Checkers," *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, Jul. 1959.
- [225] M. Minsky, "Steps toward Artificial Intelligence," *Proceedings of the IRE*, vol. 49, no. 1, pp. 8–30, Jan. 1961.
- [226] A. Klopff, "Brain Function and Adaptive Systems: A Heterostatic Theory," Mar. 1972.
- [227] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, no. 1, pp. 9–44, Aug. 1988.
- [228] C. Watkins, "Learning From Delayed Rewards," Jan. 1989.
- [229] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3, pp. 279–292, May 1992.
- [230] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," *International conference on machine learning*, pp. 1861–1870, 2018.
- [231] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep Reinforcement Learning: A Brief Survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [232] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, no. 5, pp. 834–846, Sep. 1983.
- [233] G. Tesauro, "Temporal Difference Learning and TD-Gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [234] X. Wang *et al.*, "Deep Reinforcement Learning: A Survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5064–5078, Apr. 2024.
- [235] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, May 1992.
- [236] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The Arcade Learning Environment: An Evaluation Platform for General Agents," *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, Jun. 2013.
- [237] M. Bansal, A. Krizhevsky, and A. Ogale, "Chauffeurnet: Learning to drive by imitating the best and synthesizing the worst," *arXiv preprint arXiv:1812.03079*, 2018.
- [238] D. Silver *et al.*, "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," *arXiv:1712.01815 [cs]*, Dec. 2017.
- [239] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27730–27744, 2022.

- [240] Y. Bai *et al.*, “Training a helpful and harmless assistant with reinforcement learning from human feedback,” *arXiv preprint arXiv:2204.05862*, 2022.
- [241] D. Guo *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [242] T. Brown *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [243] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347 [cs]*, Aug. 2017.
- [244] R. Kelly, V. S. Davila, and A. Loria, Eds., “Control of Robot Manipulators in Joint Space.” in *Advanced Textbooks in Control and Signal Processing*. Springer-Verlag London Limited, London, 2005.
- [245] K. Galal, *Modern Robotics: Mechanics, Planning, and Control*. 2017.
- [246] E. F. Camacho and C. Bordons, *Model Predictive control*. in *Advanced Textbooks in Control and Signal Processing*. London: Springer, 2007.
- [247] J. Rawlings and D. Mayne, *Model Predictive Control: Theory and Design*. 2009.
- [248] J. Kober, J. A. Bagnell, and J. Peters, “Reinforcement learning in robotics: A survey,” *Int. J. Rob. Res.*, vol. 32, no. 11, pp. 1238–1274, Sep. 2013.
- [249] O. M. Andrychowicz *et al.*, “Learning Dexterous In-Hand Manipulation,” *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.
- [250] D. Kalashnikov *et al.*, “Scalable Deep Reinforcement Learning for Vision-Based Robotic Manipulation,” in *Conference on robot learning*, 2018, pp. 651–673.
- [251] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, and S. Levine, “How to Train Your Robot with Deep Reinforcement Learning; Lessons We’ve Learned,” *The International Journal of Robotics Research*, vol. 40, no. 4, pp. 698–721, Apr. 2021.
- [252] D. Pikuli, J. Cosio, X. Alameda-Pineda, P.-B. Wieber, and T. Fraichard, “Navigating the Practical Pitfalls of Reinforcement Learning for Social Robot Navigation,” in *Robotics: Science and Systems (RSS) Workshop on Unsolved Problems in Social Robot Navigation*, Delft / Netherlands, Netherlands, Jul. 2024.
- [253] S. Lathuilière, B. Massé, P. Mesejo, and R. Horaud, “Neural network based reinforcement learning for audio–visual gaze control in human–robot interaction,” *Pattern Recognition Letters*, vol. 118, pp. 61–71, Feb. 2019.
- [254] N. Sünderhauf *et al.*, “The limits and potentials of deep learning for robotics,” *The International journal of robotics research*, vol. 37, no. 4–5, pp. 405–420, 2018.
- [255] A. R. Mahmood, D. Korenkevych, G. Vasan, W. Ma, and J. Bergstra, “Benchmarking Reinforcement Learning Algorithms on Real-World Robots,” in *Conference on robot learning*, 2018, pp. 561–591.

- [256] T. P. Lillicrap *et al.*, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.
- [257] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *International conference on machine learning*, 2015, pp. 1889–1897.
- [258] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [259] V. Mnih *et al.*, “Asynchronous methods for deep reinforcement learning,” in *International conference on machine learning*, 2016, pp. 1928–1937.
- [260] H. Taheri, S. R. Hosseini, and M. A. Nekoui, “Deep reinforcement learning with enhanced ppo for safe mobile robot navigation,” *arXiv preprint arXiv:2405.16266*, 2024.
- [261] D. Bick, “Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization,” 2021.
- [262] M. Malik, M. K. Malik, K. Mehmood, and I. Makhdoom, “Automatic speech recognition: a survey,” *Multimedia Tools and Applications*, vol. 80, no. 6, pp. 9411–9457, Mar. 2021.
- [263] H. Wang, A. Pandey, and D. Wang, “A systematic study of DNN based speech enhancement in reverberant and reverberant-noisy environments,” *Computer Speech & Language*, vol. 89, p. 101677, Jan. 2025.
- [264] A. Rix, J. Beerends, M. Hollier, and A. Hekstra, “Perceptual evaluation of speech quality (PESQ)-a new method for speech quality assessment of telephone networks and codecs,” in *2001 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No.01CH37221)*, May 2001, pp. 749–752.
- [265] J. L. Piovesan and H. G. Tanner, “Randomized model predictive control for robot navigation,” in *2009 IEEE International Conference on Robotics and Automation*, May 2009, pp. 94–99.
- [266] S. H. Arul, J. Jin Park, V. Prem, Y. Zhang, and D. Manocha, “Unconstrained Model Predictive Control for Robot Navigation under Uncertainty,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, May 2024, pp. 9321–9327.
- [267] A. Magassouba, N. Bertin, and F. Chaumette, “Aural Servo: Sensor-Based Control From Robot Audition,” *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 572–585, Jun. 2018.
- [268] M. Savva *et al.*, “Habitat: A Platform for Embodied AI Research,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9339–9347.
- [269] A. Chang *et al.*, “Matterport3D: Learning from RGB-D Data in Indoor Environments,” *arXiv preprint arXiv:1709.06158*, 2017.
- [270] J. Straub *et al.*, “The Replica Dataset: A Digital Replica of Indoor Spaces,” *arXiv preprint arXiv:1906.05797*, 2019.

- [271] K. Cho *et al.*, “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” *arXiv preprint arXiv:1406.1078*, 2014.
- [272] V. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Proceedings of the USSR Academy of Sciences*, vol. 163, no. 4, pp. 845–848, 1965.
- [273] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1, pp. 99–134, May 1998.
- [274] R. Cassandra, L. P. Kaelbling, and L. Littman, “Acting Optimally in Partially Observable Stochastic Domains,” 1994.
- [275] A. Cassandra, L. Kaelbling, and J. Kurien, “Acting under uncertainty: discrete Bayesian models for mobile-robot navigation,” in *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS '96*, Nov. 1996, pp. 963–972.
- [276] G. Theodorou and L. Kaelbling, “Approximate Planning in POMDPs with Macro-Actions,” in *Advances in Neural Information Processing Systems*, MIT Press, 2003.
- [277] E. Brunskill, L. Kaelbling, T. Lozano-Perez, and N. Roy, “Continuous-State POMDPs with Hybrid Dynamics,” 2008.
- [278] N. Meuleau, L. Peshkin, K.-E. Kim, and L. P. Kaelbling, “Learning Finite-State Controllers for Partially Observable Environments,” *arXiv preprint arXiv:1301.6721*, 2013.
- [279] N. Meuleau, K.-E. Kim, L. P. Kaelbling, and A. R. Cassandra, “Solving POMDPs by Searching the Space of Finite Policies,” *arXiv preprint arXiv:1301.6720*, 2013.
- [280] K. Azizzadenesheli, Y. Yue, and A. Anandkumar, “Policy Gradient in Partially Observable Environments: Approximation and Convergence,” *arXiv preprint arXiv:1810.07900*, 2018.
- [281] J. Zhu, F. Wu, and J. Zhao, “An Overview of the Action Space for Deep Reinforcement Learning,” in *2021 4th International Conference on Algorithms, Computing and Artificial Intelligence*, Sanya China: ACM, Dec. 2021, pp. 1–10.
- [282] I. G. Petrazzini and E. A. Antonelo, “Proximal Policy Optimization with Continuous Bounded Action Space via the Beta Distribution,” in *2021 IEEE symposium series on computational intelligence (SSCI)*, 2021, pp. 1–8.
- [283] G. Brockman *et al.*, “OpenAI Gym,” *arXiv:1606.01540 [cs]*, Jun. 2016.
- [284] M. Towers *et al.*, “Gymnasium: A Standard Interface for Reinforcement Learning Environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [285] D. Povey *et al.*, “The Kaldi Speech Recognition Toolkit,” p. 4.
- [286] D. Can, V. R. Martinez, P. Papadopoulos, and S. S. Narayanan, “Pykaldi: A Python Wrapper for Kaldi,” in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2018, pp. 5889–5893.

- [287] “VOSK Offline Speech Recognition API.”
- [288] M. Ravanelli *et al.*, “SpeechBrain: A General-Purpose Speech Toolkit,” *arXiv:2106.04624 [cs, eess]*, Jun. 2021.
- [289] J. Ansel *et al.*, “PyTorch 2: Faster Machine Learning Through Dynamic Python Byte-code Transformation and Graph Compilation,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*, ACM, Apr. 2024.
- [290] J. Zuluaga-Gomez, S. Ahmed, D. Visockas, and C. Subakan, “CommonAccent: Exploring Large Acoustic Pretrained Models for Accent Classification Based on Common Voice,” *arXiv preprint arXiv:2305.18283*, 2023.
- [291] P. Mousavi, L. Della Libera, J. Duret, A. Ploujnikov, C. Subakan, and M. Ravanelli, “DASB - Discrete Audio and Speech Benchmark,” *arXiv preprint arXiv:2406.14294*, 2024.
- [292] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, “Recurrent neural network based language model,” presented at the Proc. Interspeech 2010, 2010, pp. 1045–1048.
- [293] A. Graves and N. Jaitly, “Towards End-To-End Speech Recognition with Recurrent Neural Networks,” in *Proceedings of the 31st International Conference on Machine Learning*, PMLR, Jun. 2014, pp. 1764–1772.
- [294] N. Vaessen, “jitsi/jiwer.” Accessed: Sep. 26, 2024. [Online]. Available: <https://github.com/jitsi/jiwer>
- [295] M. Bachmann, “rapidfuzz/RapidFuzz: Release 3.8.1.” [Online]. Available: <https://doi.org/10.5281/zenodo.10938887>
- [296] L. Engstrom *et al.*, “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO,” *arXiv preprint arXiv:2005.12729*, 2020.
- [297] S. Huang, R. F. J. Dossa, A. Raffin, A. Kanervisto, and W. Wang, “The 37 Implementation Details of Proximal Policy Optimization,” in *ICLR Blog Track*, 2022.